

2026年6月北美芯片设计/验证题目汇总

根据同学回忆整理 · 137道真题含答案与解析

AMD · Ambarella · Apple · Black Sesame · ByteDance · Cisco · Nvidia · Omnivision · Rambus

目录

Digital Design

Q1. Digital Design: Verilog Macro Define Inside if-else
Q2. Digital Design: Clock Divider Progression & Clock Gating
Q3. Digital Design: Max Operating Frequency & Hold Violation
Q4. Digital Design: Dual Sequence Detector FSM (10111 & 10010)
Q5. Digital Design: 4-Input 1024-bit Sorting Network with CAS (Designer Question)
Q6. Digital Design: Saturating PE Unit a*b+c (Designer Question)
Q7. Digital Design: Async FIFO Gray Code SDC Constraint (Designer Question)
Q8. Digital Design: Divide-by-3 with Single Counter, 50% Duty Cycle (Designer Question)
Q9. Digital Design: Sequence Detector Optimization (Designer Question)
Q10. Digital Design: Serial Divisibility-by-5 FSM (Designer Question)
Q11. Digital Design: Single-Port vs Dual-Port SRAM Interface (Designer Question)
Q12. Digital Design: create_generated_clock SDC (Designer Question)
Q13. Digital Design: MSI / MESI / MOESI Cache Coherence Protocols (Designer Question)
Q14. Digital Design: Booth Algorithm for Multiplication (Designer Question)
Q15. Digital Design: Two-Stage Pipeline a*c + b*c with Power Optimization (Designer Question)
Q16. Digital Design: Synchronous FIFO RTL Implementation (Designer Question)
Q17. Digital Design: SRAM Configuration Power Analysis (Designer Question)
Q18. Digital Design: 4-bit x 4-bit Array Multiplier FA/HA Count (Designer Question)
Q19. Digital Design: Latchup Effect in CMOS (Designer Question)
Q20. Digital Design: Tristate Buffer Circuit and Applications (Designer Question)
Q21. Digital Design: Metastability and 2-FF Synchronizer (Designer Question)
Q22. Digital Design: Gray Code Binary Conversion & CDC Use Case (Designer Question)
Q23. Digital Design: Async FIFO Data Path Handling (Designer Question)
Q24. Digital Design: Setup Time, Hold Time & Violation Fixes (Designer Question)
Q25. Digital Design: Async FIFO Architecture (Designer Question)
Q26. Digital Design: Same Frequency, Different Phase -- Why CDC Still Needed (Designer Question)
Q27. Digital Design: FIFO with Non-Power-of-2 Depth (Designer Question)
Q28. Digital Design: FIFO Registered Output and Empty Flag Design (Designer Question)
Q29. Digital Design: AXI Valid-Ready Handshake FSM (Designer Question)
Q30. Digital Design: Latch vs Flip-Flop Differences (Designer Question)
Q31. Digital Design: Avoiding Unintended Latches in RTL (Designer Question)
Q32. Digital Design: Pipeline Waveform -- always @(posedge clk) A<=en?1:C; B<=A; C<=B;
Q33. Digital Design: CDC Multi-bit Transfer -- Why Not 2-FF Synchronizer?
Q34. Digital Design: 128-bit Priority Arbiter with Logic Depth Analysis (Designer Question)
Q35. Digital Design: Cache -- CPU Write, Set Mapping, PLRU (Designer Question)
Q36. Digital Design: Packet Stream Interface Signals
Q37. Digital Design: Setup/Hold Timing Analysis and Checker
Q38. Digital Design: Implement AND Gate Using 2-to-1 MUX
Q39. Digital Design: Race Condition in Verilog initial Blocks
Q40. Digital Design: Polymorphism and Inheritance in SystemVerilog/UVM
Q41. Digital Design: Timescale Directive -- 1ns/1ps
Q42. Digital Design: D Flip-Flop RTL with Power Supply Check
Q43. Digital Design: Clock Gating -- Basic and ICG with Negative Latch
Q44. Digital Design: Voting Machine -- 3 Judges + Super Judge Veto
Q45. Digital Design: NAND Gate -- CMOS, Stick Diagram, EM (Designer Question)

Q46. Digital Design: CMOS DFF (Master-Slave Latch) with Tri-State Buffer (Designer Question)
Q47. Digital Design: What Does This Circuit Do? (Designer Question)
Q48. Digital Design: Sequence Detector FSM (1011 Pattern)
Q49. Digital Design: Minimum FIFO Depth Calculation
Q50. Digital Design: Blocking vs NBA Pipeline -- A=in, B=A, C=B
Q51. Digital Design: Error Window FSM with 64-Cycle Counter
Q52. Digital Design: C++ Bit Manipulation -- Set/Clear Nth Bit
Q53. Digital Design: Vending Machine FSM -- Requirements and State Design
Q54. Digital Design: Clock Module with Randomized Initial State
Q55. Digital Design: What is an Interrupt?
Q56. Digital Design: Verilog Simulation Scheduling -- initial Block Race
Q57. Digital Design: Hardware Optimization -- Loop Unrolling, Pipelining, Tiling, Array Partitioning
Q58. Digital Design: IEEE 754 Floating Point -- Format, Operations, Edge Cases
Q59. Digital Design: SRAM RTL Implementation and Why No Reset
Q60. Digital Design: Synchronous FIFO Using SRAM with Counter

SystemVerilog — Constraints & Randomization

Q61. Constraints: Consecutive Uniqueness & Cross-Array Common Element
Q62. Constraints: std::randomize Return Value Bug
Q63. Constraints: Sudoku 9x9 Grid Constraint
Q64. Constraints: Dynamic Array -- Sum, Consecutive Ones, Unique in Range
Q65. Constraints: Child Class Constraint Conflict and constraint_mode
Q66. Constraints: Static Array Uniqueness -- With and Without unique
Q67. Constraints: Alternating READ/WRITE Transactions Using post_randomize

SystemVerilog — Assertions (SVA)

Q68. Assertions (SVA): req |=> ack Handshake Protocol
Q69. Assertions (SVA): AHB-Lite Burst Must Not Cross 1KB Boundary
Q70. Assertions (SVA): Check Clock Frequency is 100 MHz
Q71. Assertions (SVA): Simple Fell-Triggered Assertion
Q72. Assertions (SVA): Complete req-ack Handshake Suite

SystemVerilog — Coding

Q73. SystemVerilog: Disable Specific Named Fork Branch
Q74. SystemVerilog: fork-join_any for Task Timeout Detection
Q75. SystemVerilog: Blocking vs Non-Blocking Assignment, Swap Without Temp
Q76. SystemVerilog: Nested fork-join_any with Delays -- Print Order
Q77. SystemVerilog: fork-join / join_any / join_none + Watchdog Timer
Q78. SystemVerilog: ISA Implementation -- Struct vs OOP

Verification — Test Plan

Q79. Verification: Async FIFO vs Circular FIFO vs Sync FIFO
Q80. Verification: Async FIFO Potential Problems
Q81. Verification: Simulation Runs Forever -- Debug Strategy
Q82. Verification: Cannot Reproduce Regression Bug
Q83. Verification: Sources of X-Propagation in Simulation
Q84. Verification: SCB Design with Leading Drop and Garbage Bits
Q85. Verification — Test Plan: Async FIFO Corner Cases & Verification
Q86. Verification: Formal Verification -- How It Works
Q87. Verification: Scoreboard for Paired A/B Transactions with Timeout
Q88. Verification: Is Gate-Level Simulation Still Needed After RTL Verification?
Q89. Verification: Serial-to-Parallel DUT -- Driver, Monitor, Scoreboard

Q90. Verification: Functional Coverage Not 100% -- Analysis and Strategy
Q91. Verification: Multi-Channel DUT with Credit-Based Flow Control
Q92. Verification: Constrained-Random Verification -- How It Works
Q93. Verification: FP Multiplier ALU Verification Strategy

UVM

Q94. UVM: Interface, Monitor, and uvm_config_db Setup
Q95. UVM: uvm_config_db Four Parameters Explained
Q96. UVM: Where to Set Virtual Interface in uvm_config_db
Q97. UVM: How Many Simulator Invocations for a Smoke Test Script
Q98. UVM: Phases Overview -- Task Phases vs Function Phases
Q99. UVM: virtual function -- Purpose and Example
Q100. UVM: Objection Mechanism -- How It Works
Q101. UVM: Verification Environment for 2-Input 10-Output Random Dispatch DUT
Q102. UVM: Testbench for Write-Only Register
Q103. UVM: Startup Flow and Top Module Contents
Q104. UVM: Override Default Constraints Without Modifying the Class
Q105. UVM: How to Determine if DUT Verification is Complete
Q106. UVM: Sequence with Inline Constraint + Parallel Fork-Join + Event Sync
Q107. UVM: Instruction Transaction with Covergroup and Forwarding Path
Q108. UVM: Clock Period Monitor and Scoreboard (5% Tolerance Check)
Q109. UVM: What Happens When a Sequence Issues a UVM RGM Register Write
Q110. UVM: Test Class Not Found -- Missing Factory Registration
Q111. UVM: Generate Clock in Testbench with Jitter/Variation
Q112. UVM: Register Model Frontdoor vs Backdoor Access
Q113. UVM: DDR Memory Allocator Transaction with Spatial Constraints
Q114. UVM: Factory Registration and Out-of-Order Execution
Q115. UVM: Component vs Object, Field Automation, copy vs clone

Algorithms & Scripting

Q116. Algorithms: Pairing Letters from Both Ends
Q117. Algorithms: Minimize Result by Adding Parentheses to Expression
Q118. Algorithms: Robot Matrix Diagonal Traversal
Q119. Algorithms: Rectangle Storage Queries
Q120. Algorithms: Online Interval Merge with Streaming Input
Q121. Algorithms: Event ID Check Queue
Q122. Algorithms: Count Digits 0, 2, 4 in Range [0, n]
Q123. Algorithms: Longest Contiguous Same-Character Substring
Q124. Algorithms: Place Lamp to Illuminate Maximum Objects
Q125. Algorithms: Cyclic Shift Validation
Q126. Algorithms: Memory Allocator Simulation
Q127. Algorithms: Byte Stream Chunk Merging
Q128. Algorithms: Merge Intervals and Return All Unique Numbers
Q129. Algorithms: Clock Chain Parameter Search
Q130. Algorithms: 100 Switches Toggle -- How Many Are On?
Q131. Algorithms: Sliding Window Maximum Sum
Q132. Algorithms: Index of Second Smallest Element
Q133. Algorithms: Tournament Elimination Bracket
Q134. Algorithms: Move Zeros to End (LC 283)
Q135. Algorithms: Array Deduplication -- O(n^2) to O(n) Optimization

Brain Teasers

Q136. Brain Teaser: 20 Switches, Any Change Toggles the Light

Q137. Brain Teaser: 7L and 9L Buckets -- Measure 8 Liters

Digital Design

Q1. Digital Design: Verilog Macro Define Inside if-else

QUESTION

What does the following code print? Explain your reasoning.

```
bit x;
if(x) begin
    `define ABC 0
    x <= 1;
end
else begin
    `define ABC 1
    x <= 0;
end

$display(`ABC)
```

ANSWER

```
// Output: 1
//
// `define is a PREPROCESSOR directive, processed at COMPILE TIME.
// The if/else is a RUNTIME construct evaluated during simulation.
//
// At compile time, the preprocessor scans top-to-bottom and processes
// ALL `define directives, ignoring surrounding if/else branches:
//
//   Step 1: `define ABC 0  -> ABC = 0
//   Step 2: `define ABC 1  -> ABC = 1  (redefines; simulator may warn)
//
// So `ABC expands to 1 at $display, equivalent to: $display(1)
//
// At runtime x (bit) defaults to 0, else branch executes,
// but this has NO effect on the macro - already fixed at compile time.
//
// Use `ifdef/`elsif for compile-time conditions,
// or parameters/localparams for runtime-controlled constants.
```

解释

考点：编译期（compile time）与运行期（runtime）的本质区别。

`define 是 Verilog 预处理器指令，编译器在解析源文件时从上到下顺序展开，

完全忽略 if/else/always 等运行时结构的作用域。

因此两条 `define 都会被处理：先定义 ABC=0，再重定义 ABC=1（后者覆盖前者）。

$display(`ABC) 在编译期被展开为 $display(1)，打印结果是 1。

运行时 x 默认值为 0，走 else 分支，但这对宏的值已无影响。

面试考察点：不要在 if/else 内用 `define 实现条件逻辑；

编译期条件应使用 `ifdef/`elsif/`endif；运行时条件用 parameter 或变量。

Q2. Digital Design: Clock Divider Progression & Clock Gating

QUESTION

(a) Draw a clock divide-by-2 circuit using a flip-flop. What is the duty cycle?

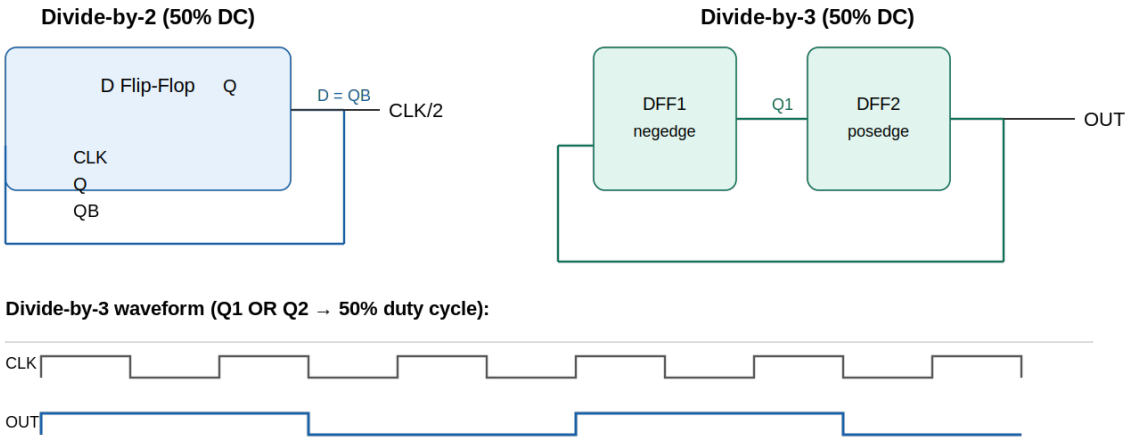
(b) Design a divide-by-3 using flops (33% duty cycle).

(c) How do you achieve exactly 50% duty cycle for divide-by-3?

(d) Draw a basic clock gating circuit. What is the glitch hazard?

Why should we never add buffers on the clock path to fix hold violations?

ANSWER




```
// (a) Divide-by-2: single D flip-flop, Q feeds back to D_bar (inverted)
//
//   clk --> [DFF] --> Q --+-> clk_div2
//             ^           |
//             +---[INV]---+   (D = ~Q)
//
// Duty cycle = 50%. Q toggles every rising edge of clk.
// Output period = 2 * T_clk.

// (b) Divide-by-3, 33% duty cycle (pulse high for 1 cycle out of 3):
//
// Use a 2-bit counter. State machine: S0->S1->S2->S0, output HIGH in S0 only.
//
module div3_33 (input clk, rst, output clk_div3);
    reg [1:0] cnt;
    always @(posedge clk or posedge rst)
        if (rst) cnt <= 0;
        else     cnt <= (cnt == 2) ? 0 : cnt + 1;
    assign clk_div3 = (cnt == 0);  // high 1 out of 3 cycles -> 33% DC
endmodule

// (c) Divide-by-3 with 50% duty cycle:
//
// Key idea: generate two div3 signals - one on posedge, one on negedge - then OR.
//
module div3_50 (input clk, rst, output clk_div3);
    reg [1:0] cnt;
    reg       clk_p, clk_n;
    always @(posedge clk or posedge rst)
        if (rst) cnt <= 0;
        else     cnt <= (cnt == 2) ? 0 : cnt + 1;
    always @(posedge clk) clk_p <= (cnt == 0);  // posedge version
```

```
    always @(negedge clk) clk_n <= (cnt == 0);  // negedge version
    assign clk_div3 = clk_p | clk_n;             // OR -> 50% DC
endmodule

// (d) Clock gating circuit and hazards:
//
// WRONG (glitch-prone): AND enable directly with clock
//   clk_gated = clk & enable  <- if enable changes while clk=1: glitch!
//
// CORRECT: latch-based ICG (Integrated Clock Gating cell):
//
//   enable --> [LATCH] --> EN_latched --+
//   clk    -------------------------> AND --> clk_gated
//               ^
//               clk (latch open when clk=0, closed when clk=1)
//
// Latch samples enable only when clk=0, so EN_latched is stable
// before clk rises -> no glitch on clk_gated.
//
// WHY NOT fix hold violations with buffers on clock path:
// Adding a buffer delays that clock edge LATER, which:
//   1. Fixes hold (data arrives before the delayed clock) -- seems ok
//   2. But WORSENS setup on all downstream paths (launch delayed,
//      capture unchanged -> less setup margin).
//   3. Creates clock skew -> timing closure nightmare across the chip.
// Hold violations must always be fixed on the DATA path, not clock path.
```

解释

(a) 单个 DFF 将 ~Q 反馈至 D，每个时钟上升沿翻转一次，输出周期为 2T，占空比 50%。

(b) 33% 占空比：2位计数器循环 0->1->2->0，仅在状态0输出高电平，

高电平持续1个周期，总周期3个周期，占空比 = 1/3 ≈ 33%。

(c) 50% 占空比：分别用上升沿和下降沿触发两路 div3 信号，做 OR 运算。

上升沿版本在某计数值拉高，下降沿版本在同一计数值半周期后拉高，

两路叠加后高电平占整周期的一半，实现精确 50%。

(d) 时钟毛刺：直接用组合逻辑 AND(clk, en) 门控时钟，

enable 在 clk=1 期间跳变会产生窄脉冲毛刺（glitch），污染下游触发器。

正确做法是 ICG（Integrated Clock Gating Cell）：用电平敏感锁存器

在 clk=0 时采样 enable，保证 enable 在时钟上升沿前稳定。

不能在时钟路径加 buffer 修 hold：buffer 延迟了发射沿，

虽然修了 hold，但同时恶化了 setup，且引入时钟偏斜，

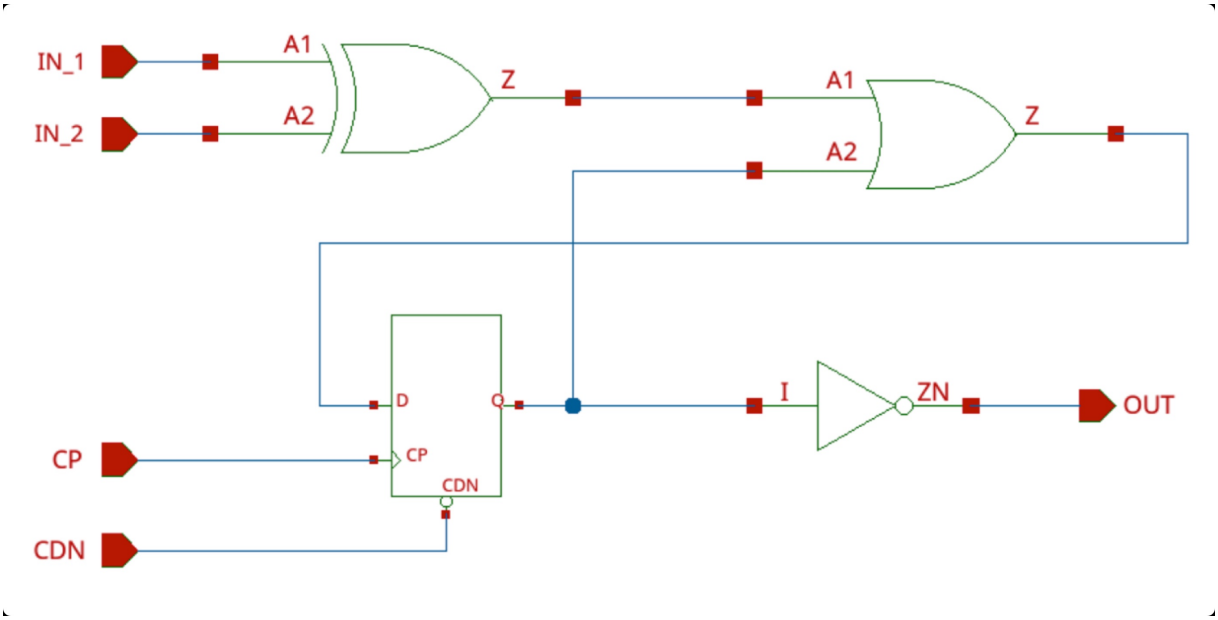
hold 违例必须在数据路径上加 buffer 解决。

Q3. Digital Design: Max Operating Frequency & Hold Violation

QUESTION

Given the circuit below — two flip-flops with an inverter and NAND gate

between them. FF2 clock leads FF1 by 1ns (FF2 capture clock arrives earlier).



(a) What is the maximum operating frequency?

(b) Is there a hold time violation at FF2?

ANSWER

```
// -- (a) Maximum Operating Frequency -------------------------------------
//
// FF2 leads FF1 by 1ns -> capture clock arrives 1ns EARLIER than launch.
// Tskew = T_clk_capture - T_clk_launch = -1ns  (negative = hurts setup)
//
// Setup check:
//   T_period >= Tcq_max + Tinv + Tnand + Tsu - Tskew
//            >= 3       + 1    + 2     + 3   - (-1)
//            >= 10 ns
//
//   F_max = 1 / 10ns = 100 MHz

// -- (b) Hold Time Check at FF2 ------------------------------------------
//
// Hold check (independent of period):
//   Data_arrival_min  = Tcq_min + Tinv + Tnand = 2 + 1 + 2 = 5 ns
//   Hold_requirement  = Thold + Tskew          = 5 + (-1)  = 4 ns
//
//   Slack = Data_arrival_min - Hold_requirement = 5 - 4 = +1 ns
//
//   NO hold violation. Slack = +1 ns (margin is positive).
//
// Note: FF2 leading FF1 HELPS hold (capture clock is earlier,
// giving data more time to arrive before the next capture edge).
// If FF2 lagged FF1, hold margin would shrink and could go negative.
```

解释

(a) 最大工作频率：

建立时间约束公式：T_period >= Tcq_max + T_logic + Tsu - Tskew

Tskew = T_clk_capture - T_clk_launch = -1ns（FF2 超前 = 捕获沿更早 = 负 skew）

代入：T_period >= 3 + 1 + 2 + 3 - (-1) = 10ns，F_max = 100 MHz。

注意负 skew 让建立时间更紧张：捕获触发器的时钟沿提前到来，

相当于数据需要更早稳定，因此最小周期变大。

(b) 保持时间检查：

保持时间约束与时钟周期无关，只看最短数据路径与保持需求之比：

最短数据到达 = Tcq_min + Tinv + Tnand = 2+1+2 = 5ns

保持需求 = Thold + Tskew = 5 + (-1) = 4ns

裕量 = 5 - 4 = +1ns，无保持违例。

FF2 超前 FF1（Tskew 为负）反而对保持时间有利：

捕获沿提前，数据在新的捕获沿到来之前还有更长的窗口保持稳定。

Q4. Digital Design: Dual Sequence Detector FSM (10111 & 10010)

QUESTION

Design a Moore/Mealy FSM to simultaneously detect two overlapping bit sequences

on a single serial input line:

seq1 = 10111 -> assert out1 = 1 on the last bit

seq2 = 10010 -> assert out2 = 1 on the last bit

The two sequences share a common prefix "10" and partially overlap in structure.

The detector must support overlapping matches (a suffix of one detected sequence

can serve as the prefix of the next).

(a) Draw the state diagram. How many states are needed and why?

(b) Write synthesizable SystemVerilog implementing the FSM.

ANSWER

```
// (a) State count: 7 states
// KMP failure functions:
//   seq1=10111: fail = [0, 0, 1, 1, 1]
//   seq2=10010: fail = [0, 0, 0, 1, 2]
//
// Both sequences share prefix "10", so S1 and S2 serve both paths.
//
// S0 (0,0): idle
// S1 (1,1): matched '1'     -- shared prefix char for both sequences
// S2 (2,2): matched '10'    -- shared prefix "10" for both sequences
// S3 (0,3): matched '100'   -- seq2 branch only
// S4 (3,1): matched '101'   -- seq1 leads, seq2 partial
// S5 (1,4): matched '1001'  -- seq2 one bit from completion
// S6 (4,1): matched '1011'  -- seq1 one bit from completion
//
// Transition table:
//   S0: 0->S0,  1->S1
//   S1: 0->S2,  1->S1  (self-loop)
//   S2: 0->S3,  1->S4
//   S3: 0->S0,  1->S5
//   S4: 0->S2,  1->S6
//   S5: 0->S2,  1->S1       out2=1 when in S5 and din=0
//   S6: 0->S2,  1->S1       out1=1 when in S6 and din=1

// (b) Three-always-block FSM (standard coding style)
module dual_seq_detector (
    input  logic clk,
```

```
    input  logic rst_n,
    input  logic din,
    output logic out1,
    output logic out2
);

    typedef enum logic [2:0] {
        S0 = 3'd0,
        S1 = 3'd1,
        S2 = 3'd2,
        S3 = 3'd3,
        S4 = 3'd4,
        S5 = 3'd5,
        S6 = 3'd6
    } state_t;

    state_t cur_state, nxt_state;

    // Block 1: State register (sequential)
    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n)
            cur_state <= S0;
        else
            cur_state <= nxt_state;
    end

    // Block 2: Next-state logic (combinational)
    always_comb begin
        nxt_state = S0;
        case (cur_state)
            S0: nxt_state = din ? S1 : S0;
            S1: nxt_state = din ? S1 : S2;
            S2: nxt_state = din ? S4 : S3;
            S3: nxt_state = din ? S5 : S0;
            S4: nxt_state = din ? S6 : S2;
            S5: nxt_state = din ? S1 : S2;
            S6: nxt_state = din ? S1 : S2;
            default: nxt_state = S0;
        endcase
    end

    // Block 3: Output logic (combinational, Mealy)
    always_comb begin
        out1 = 1'b0;
        out2 = 1'b0;
        case (cur_state)
            S6: out1 = din;         // out1=1 when in S6 and din='1'
            S5: out2 = ~din;        // out2=1 when in S5 and din='0'
            default: begin
                out1 = 1'b0;
                out2 = 1'b0;
            end
        endcase
    end

endmodule
```

解释

核心思路：两条序列共享同一组状态，每个状态编码 (seq1进度, seq2进度) 的二元组。

10111 和 10010 都以 "10" 开头，所以 S1（匹配'1'）和 S2（匹配'10'）同时服务两条路径，

只需 7 个状态，远少于两套独立 FSM 的 6+6=12 个状态。

KMP 失败函数决定检测失败时如何回退：

- seq1=10111: fail=[0,0,1,1,1]，即匹配失败时尽量保留已匹配的'1'前缀

- seq2=10010: fail=[0,0,0,1,2]，即末尾'10'可作为下次匹配的起点

输出为 Mealy 型（在转移时输出），无额外时钟延迟：

- S6 输入'1' -> out1=1（完成 10111），KMP 回退到 S1（已匹配'1'）

- S5 输入'0' -> out2=1（完成 10010），KMP 回退到 S2（已匹配'10'）

S1 在输入'1'时自环，因为 seq1[1]='0' 不匹配，回退到 fail[0]=0 再尝试 seq1[0]='1'，

而 seq2[1]='0' 同理，两条路径的进度都保持在1。

Q5. Digital Design: 4-Input 1024-bit Sorting Network with CAS (Designer Question)

QUESTION

Technology: TSMC 3nm. Gate delays (and/or/mux/inv/buf/xor/nor) ~1ps each.

Building blocks:

CAS (Compare-And-Swap): takes two 1024-bit inputs, outputs the smaller

on one port and the larger on the other. Latency = 0.8ns.

CMP (Compare): compares two values, outputs a 1-bit result.

A CAS can be implemented as: two inputs, one comparator, two muxes.

(a) What is the relationship between CMP and CAS?

(b) Sort 4 x 1024-bit numbers using two design approaches:

- Solution 1 (Area-optimized): minimize number of CAS blocks.

- Solution 2 (Speed-optimized): total latency < 1ns per cycle.

(c) Compare leakage power between the two solutions. Which is larger and why?

(d) Given inputs I1[1023:0] transitions 000...0 -> 111...1 during operation

(I0=333, I1=000->111, I2=444, I3=555), with the area-optimized 5-CAS

topology: which CAS blocks consume dynamic power, and why?

(Refer to the circuit diagrams below.)

ANSWER

```
// (a) CMP vs CAS relationship
//
// CMP performs a comparison and produces a 1-bit result (A > B or A <= B).
// CAS performs a comparison AND routes the inputs to sorted outputs:
//   min(A,B) on one output port, max(A,B) on the other.
//
// A CAS = CMP + 2x MUX:
//   wire gt = (A > B);            // 1024-bit CMP
//   assign out_min = gt ? B : A;  // MUX
//   assign out_max = gt ? A : B;  // MUX
//
// For 1024-bit inputs on TSMC 3nm (~1ps/gate):
//   CMP latency  ~= log2(1024) * 1ps * logic_depth ~ several hundred ps
//   CAS latency  = CMP + MUX = 0.8ns (given)
//   The MUX adds only ~1ps on top of the CMP, so CAS ~ CMP in latency.

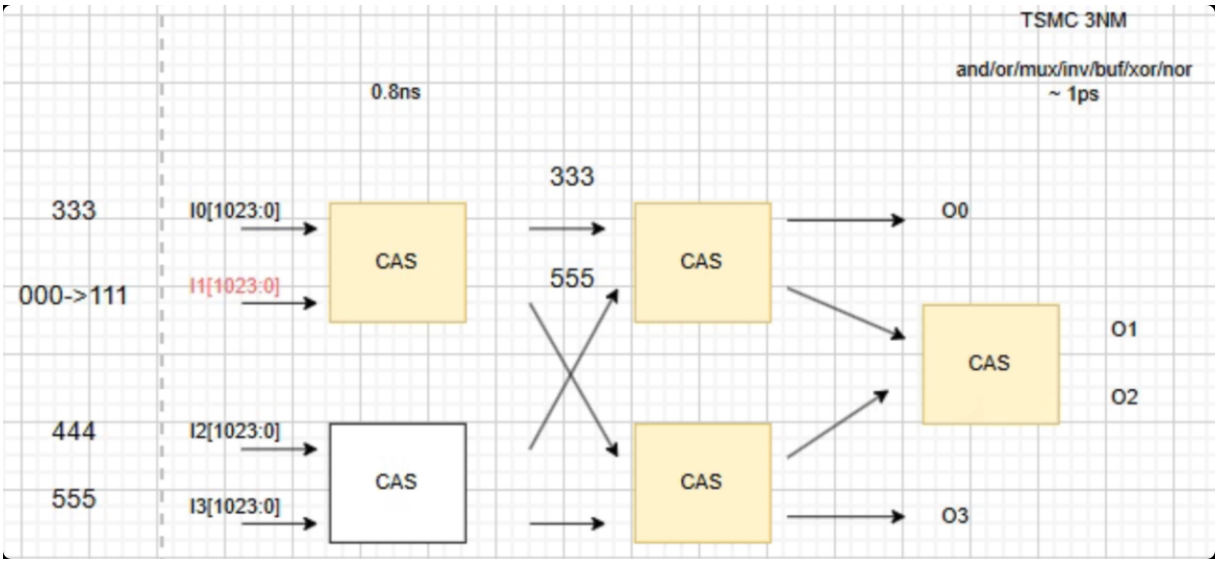
// (b) Sorting network for 4 x 1024-bit numbers
```

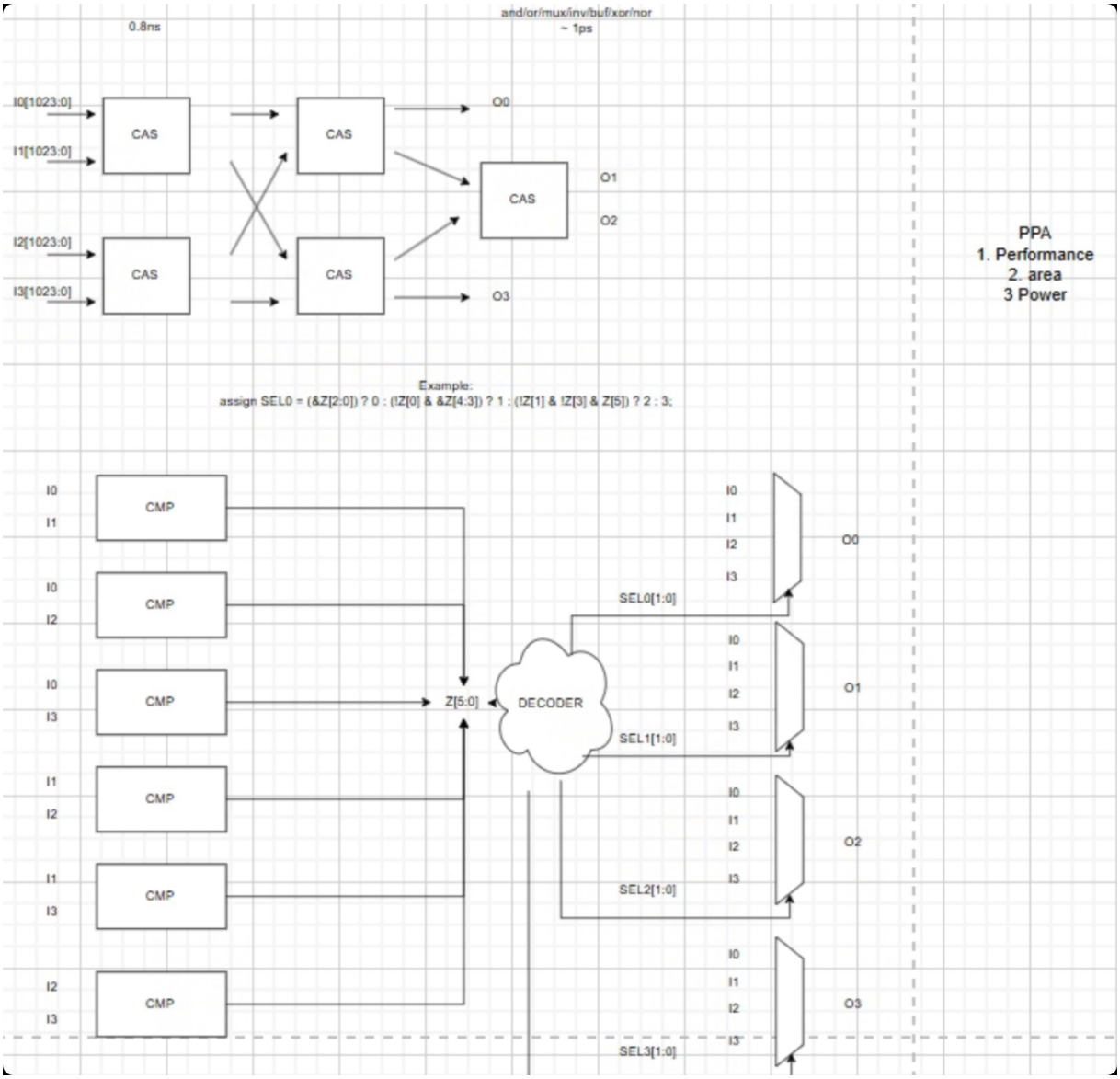
```
//
// -- Solution 1: Area-optimized (5 CAS blocks, 3 stages) --
//
// Uses the minimum CAS count for a 4-input sorting network.
// Classic odd-even merge sort requires 5 comparators for N=4:
//
//   Stage 1 (parallel, 2 CAS):
//     CAS(I0, I1) -> (A0=min, A1=max)
//     CAS(I2, I3) -> (A2=min, A3=max)
//
//   Stage 2 (parallel, 2 CAS, cross-connected):
//     CAS(A0, A2) -> (B0=min, B2=max)
//     CAS(A1, A3) -> (B1=min, B3=max)
//
//   Stage 3 (1 CAS, middle elements only):
//     CAS(B2, B1) -> (C1=min, C2=max)
//
//   Outputs: O0=B0 (min), O1=C1, O2=C2, O3=B3 (max)
//
//   Total: 5 CAS blocks, 3 serial stages
//   Total latency = 3 x 0.8ns = 2.4ns per sort
//
// -- Solution 2: Speed-optimized (latency < 1ns per cycle) --
//
// To achieve < 1ns total latency, we cannot use serial CAS stages (0.8ns each).
// Strategy: replace CAS with parallel CMP + decoder + mux bank.
//
//   Step 1: Run all 6 pairwise CMPs in parallel (C(4,2) = 6 comparators):
//     CMP(I0,I1), CMP(I0,I2), CMP(I0,I3)
//     CMP(I1,I2), CMP(I1,I3), CMP(I2,I3)
//     All 6 results available after ~1 CMP delay (~hundreds of ps)
//
//   Step 2: Decode the 6-bit comparison vector Z[5:0] to rank each input:
//     A DECODER maps Z to select signals SEL0..SEL3 (2-bit rank each).
//     Example: assign SEL0 = (&Z[2:0]) ? 0 : (!Z[0] & &Z[4:3]) ? 1 : ...
//
//   Step 3: Four 4:1 muxes (one per output) select the correct input:
//     O0 = mux(I0,I1,I2,I3, SEL0)  // smallest
//     O1 = mux(I0,I1,I2,I3, SEL1)
//     O2 = mux(I0,I1,I2,I3, SEL2)
//     O3 = mux(I0,I1,I2,I3, SEL3)  // largest
//
//   Total latency ~ CMP + DECODER + MUX ~ < 1ns (all ~1ps gates, short chain)
//   Cost: 6 CMPs + decoder logic + 4x(1024-bit 4:1 mux) -- much larger area

// (c) Leakage power comparison
//
// Leakage power is proportional to total transistor count (area).
//
// Solution 1 (5 CAS):  5 x (CMP + 2 MUX) worth of gates
// Solution 2 (6 CMP + decoder + 4 mux banks): significantly more gates
//
// Solution 2 has much LARGER leakage power because:
//   - 6 CMPs vs 5 CMPs (more comparator logic)
//   - 4 full 1024-bit 4:1 mux banks (each mux bank >> 1 mux in CAS)
//   - Decoder logic adds additional gates
//
// Leakage ~ area ~ transistor count:
//   Sol 2 leakage >> Sol 1 leakage
//   The speed improvement in Sol 2 comes at a direct leakage cost.

// (d) Dynamic power and glitching with I1: 000...0 -> 111...1
//
```

```
// I0=333, I1 transitions 000->111, I2=444, I3=555
// Sorted order when I1=000: O0=000, O1=333, O2=444, O3=555
// Sorted order when I1=111: O0=111, O1=333, O2=444, O3=555
//
// Area solution topology (5 CAS, 3 stages):
//   Stage 1: CAS(I0,I1) and CAS(I2,I3)
//   Stage 2: two cross-connected CAS
//   Stage 3: one final CAS
//
// I1 (red in diagram) feeds directly into Stage 1 CAS(I0, I1).
// I1 transitions from 000 to 111, so this CAS sees a changing input
// -> its outputs CHANGE -> it consumes switching power.
//
// GLITCH analysis -- why downstream CAS blocks also toggle:
//
//   CAS(I0=333, I1=000->111):
//     Before: out_min=000, out_max=333
//     After:  out_min=111, out_max=333
//     out_min changes: 000->111 (1024 bits toggle!) -> GLITCH propagates
//
//   Stage 2 receives the changed out_min -> its internal CMP sees new inputs
//   -> even if the FINAL sorted output does not change (333 stays at O1),
//      intermediate signals inside Stage 2 CAS blocks toggle transiently
//      before settling -> GLITCH power consumed in Stage 2 CAS blocks.
//
//   Stage 3 may also glitch if Stage 2 intermediate signals ripple through.
//
// Power consumed by glitching:
//   P_dynamic = alpha * C * V^2 * f
//   where alpha = switching activity (glitch increases alpha significantly)
//
// All 5 CAS blocks may toggle due to the 1024-bit input transition,
// even though the final sorted output changes only at O0 (000->111).
// The yellow-highlighted CAS blocks in the diagram are the ones that
// directly see changed inputs; the white CAS (I2/I3 path) is unaffected
// in Stage 1 since I2=444 and I3=555 are both stable.
```





解释

本题综合考察 PPA (Performance, Power, Area) 三个维度的设计权衡，是典型的 designer 面试题。

(a) CAS = CMP + 2xMUX：理解基本单元的组成是一切分析的基础。

(b) 排序网络设计：5-CAS 是 N=4 排序的理论最优面积解（Knuth 排序网络）；

速度型方案用全并行比较+译码器实现 O(1) 延迟，代价是面积爆炸。

(c) Leakage 与面积正比：速度型方案面积远大于面积型，leakage 也成比例增大。

这是 PPA 中 performance vs power/area 的经典 trade-off。

(d) Glitch 功耗：I1 的 1024 位全部翻转（000->111）是最坏情况的输入跳变。

即使最终排序结果变化不大，中间节点的瞬态翻转（glitch）会导致多个 CAS block

重复充放电，产生远超稳态所需的动态功耗。

面积型（串行）比速度型更容易分析 glitch 路径，但串行结构也意味着

glitch 有更多级联机会。黄色 CAS block（I0/I1 路径）是 glitch 的发源地，

白色 CAS block（I2/I3 路径）在 Stage 1 不受影响。

Q6. Digital Design: Saturating PE Unit a*b+c (Designer Question)

QUESTION

(a) Implement a Processing Element (PE) that computes result = a*b + c.

All operands and the output are unsigned 10-bit values.

The output must be saturated (capped) at 1024 if the result exceeds 1024.

(b) Write a self-checking testbench to verify it.

ANSWER

```
// (a) PE implementation with saturation cap at 1024

module pe_sat #(
    parameter int W   = 10,   // operand width
    parameter int CAP = 1024  // saturation ceiling
)(
    input  logic [W-1:0]  a, b, c,
    output logic [W-1:0]  result
);
    // Internal wider wire to hold full precision before saturation
    // a*b needs 2*W bits; +c needs 2*W+1 bits to be safe
    logic [2*W:0] full_result;

    assign full_result = (a * b) + c;

    // Saturate: if full_result > CAP, output CAP; else output full_result
    assign result = (full_result > CAP) ? W'(CAP) : full_result[W-1:0];

endmodule

// (b) Self-checking testbench

module pe_sat_tb;
    parameter int W   = 10;
    parameter int CAP = 1024;

    logic [W-1:0] a, b, c, result;

    pe_sat #(.W(W), .CAP(CAP)) dut (.a(a), .b(b), .c(c), .result(result));

    // Reference model
    function automatic logic [W-1:0] pe_ref(
        input logic [W-1:0] a, b, c
    );
        logic [2*W:0] tmp;
        tmp = (a * b) + c;
        return (tmp > CAP) ? W'(CAP) : tmp[W-1:0];
    endfunction

    int pass_cnt = 0, fail_cnt = 0;

    task automatic check(input logic [W-1:0] ta, tb, tc);
        logic [W-1:0] exp;
        a = ta; b = tb; c = tc;
        #1;  // combinational settle
        exp = pe_ref(ta, tb, tc);
        if (result === exp) begin
            pass_cnt++;
        end else begin
            $error("FAIL: a=%0d b=%0d c=%0d -> got %0d, expected %0d",
```

```
                    ta, tb, tc, result, exp);
            fail_cnt++;
        end
    endtask

    initial begin
        // Directed tests
        check(2,    3,    4);    // 10,  no cap
        check(32,   32,   0);    // 1024, exact cap
        check(33,   33,   0);    // 1089 -> cap 1024
        check(1,    1,    1);    // 2,   no cap
        check(10,   10,   25);   // 125, no cap
        check('1,   1,    1);    // overflow -> cap

        // Random tests
        repeat (10000) begin
            a = $urandom_range(0, (1<<W)-1);
            b = $urandom_range(0, (1<<W)-1);
            c = $urandom_range(0, (1<<W)-1);
            #1;
            if (result !== pe_ref(a,b,c)) begin
                $error("RAND FAIL: a=%0d b=%0d c=%0d", a, b, c);
                fail_cnt++;
            end else pass_cnt++;
        end

        $display("Results: %0d passed, %0d failed", pass_cnt, fail_cnt);
        if (fail_cnt == 0) $display("ALL TESTS PASSED");
        $finish;
    end
endmodule
```

解释

saturate 的关键：乘积 a*b 最宽需要 2*W 位，加上 c 后需要 2*W+1 位，

必须在截断前用宽线保存完整结果再比较，否则溢出后的截断值会小于 CAP 导致误判。

Testbench 分两部分：directed tests 覆盖边界值（正好等于 CAP、超过 CAP、零）；

random tests 用 reference model 做 self-check，覆盖随机情况。

$urandom_range 比 $random 好：无符号、无负数歧义。

Q7. Digital Design: Async FIFO Gray Code SDC Constraint (Designer Question)

QUESTION

In an asynchronous FIFO, the write pointer is converted to Gray code and

synchronized into the read clock domain via a 2-FF synchronizer (and vice versa).

What SDC constraint should be applied to the Gray code synchronizer path?

Why is set_max_delay used instead of a regular timing constraint?

ANSWER

```
// SDC constraint for Gray code 2-FF synchronizer:

// set_max_delay -datapath_only <value> \
//     -from [get_cells wr_ptr_gray_reg[*]] \
//     -to   [get_cells sync_ff1_reg[*]]
```

```
// Example with actual values:
// Assume wr_clk period = 5ns, rd_clk period = 8ns
set_max_delay -datapath_only 4.5 \
    -from [get_cells -hier wr_ptr_gray_reg[*]] \
    -to   [get_cells -hier rd_sync_ff1_reg[*]]

// Symmetric constraint for the other direction:
set_max_delay -datapath_only 7.5 \
    -from [get_cells -hier rd_ptr_gray_reg[*]] \
    -to   [get_cells -hier wr_sync_ff1_reg[*]]

// WHY set_max_delay instead of normal timing constraint:
//
// 1. No common clock relationship:
//    wr_clk and rd_clk are asynchronous -- the STA tool has no way to
//    calculate setup/hold based on a shared clock edge. Without an explicit
//    constraint the tool will either error out or use an overly pessimistic
//    0ns requirement (treat as same-clock), causing false violations.
//
// 2. -datapath_only flag is CRITICAL:
//    Without it, set_max_delay still adds clock skew and uncertainty into
//    the calculation (borrowing from the "launch" and "capture" clocks).
//    -datapath_only tells the tool: ignore clock skew/jitter, only check
//    the raw combinational path delay.
//    This is correct because the 2-FF synchronizer is designed to tolerate
//    metastability; we only need to ensure the data physically arrives at
//    FF1 within one destination clock period.
//
// 3. Why Gray code makes this safe:
//    Gray code changes only 1 bit per increment. Even if the synchronizer
//    captures a metastable state, it resolves to either the old or new value
//    (both are valid adjacent pointer values). Multi-bit binary pointers
//    cannot use this approach -- they need handshake protocols instead.
//
// 4. set_max_delay value guideline:
//    Value should be slightly less than destination clock period to leave
//    setup margin for FF1. Typical: 0.9 * T_destination.
```

解释

set_max_delay -datapath_only 是 CDC path 的标准 SDC 写法，面试必考。

三个要点必须全部答出：

1. 为什么不能用普通 setup 约束：两个 clock 没有 common period，STA 无法计算 setup slack。

2. 为什么必须加 -datapath_only：不加则工具会把 clock skew/jitter 加进来，

产生错误的悲观计算，可能导致明明时序可以满足却报 violation。

3. Gray code 为什么可以用 2-FF 同步：单比特翻转保证即使 metastability 解析成

旧值或新值，都是合法的相邻 pointer，不会跳变超过 1。

follow-up：set_false_path 和 set_max_delay -datapath_only 的区别？

set_false_path 完全忽略路径（不优化），set_max_delay -datapath_only 仍然约束

路径延迟上限（工具会优化使之满足），两者不可混用于同步器路径。

Q8. Digital Design: Divide-by-3 with Single Counter, 50% Duty Cycle (Designer Question)

QUESTION

Implement a clock divider that divides by 3 with exactly 50% duty cycle,

using only a single counter (no separate posedge/negedge flip-flops).

How does using one counter differ from the two-DFF approach?

ANSWER

```
// Single-counter divide-by-3, 50% duty cycle
//
// Key idea: a mod-3 counter counts 0->1->2->0.
// Toggle the output on count==0 AND on count==1 (i.e. every 1.5 cycles).
// Use combinational XOR of posedge and negedge samples of the same counter.

module freq_div3 (
    input  logic clk,
    input  logic rst_n,
    output logic clk_div3
);
    logic [1:0] cnt;

    // Single counter: 0 -> 1 -> 2 -> 0
    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n) cnt <= 2'd0;
        else        cnt <= (cnt == 2'd2) ? 2'd0 : cnt + 2'd1;
    end

    // 50% DC: output high when cnt==0 OR cnt==1 by using negedge latch
    // clk_p goes high at posedge when cnt==0
    // clk_n goes high at negedge when cnt==0 (half cycle later)
    // OR them to get 1.5-cycle high, 1.5-cycle low = 50% DC
    logic clk_p, clk_n;

    always_ff @(posedge clk or negedge rst_n)
        clk_p <= (!rst_n) ? 0 : (cnt == 2'd2);

    always_ff @(negedge clk or negedge rst_n)
        clk_n <= (!rst_n) ? 0 : (cnt == 2'd2);

    assign clk_div3 = clk_p | clk_n;

endmodule

// Waveform (rst released, cnt starts at 0):
// clk:      _|-|_|-|_|-|_|-|_|-|_|-|
// cnt:       0  1  2  0  1  2  0  1
// clk_p:     0  0  1  0  0  1  0  0   (registered: high when cnt was 2)
// clk_n:     0  0  1  0  0  1  0  0   (negedge, half cycle behind)
// clk_div3:  0  0  1  1  0  0  1  1   (OR: high for 1.5 cycles -> 50% DC)

// Comparison: single counter vs two-DFF approach
// - Two-DFF approach (from Q2): uses separate posedge DFF (clk_p) and
//   negedge DFF (clk_n) both watching cnt==0, then OR.
//   Same result, just slightly different register usage.
// - Single counter: the cnt register is the only sequential element,
//   clk_p and clk_n are derived from it. This is functionally identical.
// - Both approaches produce valid 50% DC output.
// - Note: clk_div3 is a combinational OR, not a clean registered clock.
//   For use as an actual clock signal, it should go through a clock buffer
//   and be declared with create_generated_clock in SDC.
```

解释

divide-by-3 的 50% 占空比问题核心是：3 是奇数，不能简单地在每个周期取反。

解决思路：利用下降沿触发器采样相同的计数器，将输出时间偏移半个时钟周期，

再与上升沿触发的输出做 OR，使高电平区间恰好为 1.5 个时钟周期（= 50% of 3T）。

单计数器版本：只用一个 2-bit 模3计数器，再派生两路寄存器，逻辑更清晰。

注意事项：此输出是 glitch-free 的（OR of registered signals），但不是

同步时钟，实际设计中必须用 create_generated_clock 告知 STA 工具。

Q9. Digital Design: Sequence Detector Optimization (Designer Question)

QUESTION

Design a sequence detector that detects the sequence 1,2,3,4,5,6,7,8,9

on an 8-bit input bus (one value per cycle).

(a) A naive implementation uses 9 separate comparators (in==1, in==2, ..., in==9).

What is the problem with this approach?

(b) How can it be optimized to reduce comparator hardware?

ANSWER

```
// (a) Naive approach: 9 independent comparators
//
module seq_det_naive (
    input  logic       clk, rst_n,
    input  logic [7:0] din,
    output logic       detected
);
    logic [3:0] cnt;  // counts consecutive matched values

    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n) cnt <= 0;
        else begin
            case (cnt)
                0: cnt <= (din == 8'd1) ? 1 : 0;
                1: cnt <= (din == 8'd2) ? 2 : (din == 8'd1) ? 1 : 0;
                2: cnt <= (din == 8'd3) ? 3 : (din == 8'd1) ? 1 : 0;
                3: cnt <= (din == 8'd4) ? 4 : (din == 8'd1) ? 1 : 0;
                4: cnt <= (din == 8'd5) ? 5 : (din == 8'd1) ? 1 : 0;
                5: cnt <= (din == 8'd6) ? 6 : (din == 8'd1) ? 1 : 0;
                6: cnt <= (din == 8'd7) ? 7 : (din == 8'd1) ? 1 : 0;
                7: cnt <= (din == 8'd8) ? 8 : (din == 8'd1) ? 1 : 0;
                8: cnt <= (din == 8'd9) ? 9 : (din == 8'd1) ? 1 : 0;
                default: cnt <= 0;
            endcase
        end
    end
    assign detected = (cnt == 9);
endmodule
// Problem: 9 comparators, each 8 bits wide = 9 x 8 = 72 comparison bits.
// Each 8-bit equality check needs ~8 XNOR + 1 AND tree.

// (b) Optimized: single comparator using expected value lookup
//
// Key insight: in state cnt, we KNOW what the next expected value is
// (cnt+1). Store expected values in a ROM/LUT and use ONE comparator.
//
module seq_det_opt (
    input  logic       clk, rst_n,
```

```
    input  logic [7:0] din,
    output logic       detected
);
    logic [3:0] cnt;
    logic [7:0] expected;

    // LUT: what value should arrive in each state
    always_comb begin
        case (cnt)
            4'd0: expected = 8'd1;
            4'd1: expected = 8'd2;
            4'd2: expected = 8'd3;
            4'd3: expected = 8'd4;
            4'd4: expected = 8'd5;
            4'd5: expected = 8'd6;
            4'd6: expected = 8'd7;
            4'd7: expected = 8'd8;
            4'd8: expected = 8'd9;
            default: expected = 8'd1;
        endcase
    end

    // Only ONE 8-bit comparator needed
    wire match = (din == expected);
    wire restart = (din == 8'd1);  // partial match: restart from 1

    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n)    cnt <= 0;
        else if (match) cnt <= cnt + 1;
        else if (restart && cnt > 0) cnt <= 1;
        else            cnt <= 0;
    end

    assign detected = (cnt == 9);
endmodule
// Optimization result:
//   Naive:  9 comparators (72 bits of comparison logic)
//   Opt:    1 comparator + 1 small LUT (9-entry x 8-bit ROM)
//   Area reduction: ~8x fewer comparator cells
//   Further optimization: since sequence is consecutive integers,
//   replace LUT with adder: expected = cnt + 1 (no LUT needed at all!)
//
// Ultimate optimization: expected = cnt + 8'd1 (1 adder, 1 comparator)
```

解释

核心优化思路：把"9个并行比较器"变成"1个比较器 + 状态驱动的期望值"。

因为序列是连续整数，还可以进一步消去 LUT，直接用 expected = cnt + 1，

只需一个加法器和一个比较器，面积比 naive 减少约 8 倍。

面试加分项：讨论 restart 逻辑（当前匹配失败但输入恰好是序列头1时，

不应完全归零，而是跳回 cnt=1），这正是 KMP 失败函数的思想。

Q10. Digital Design: Serial Divisibility-by-5 FSM (Designer Question)

QUESTION

Input arrives one bit at a time (MSB first). Design an FSM that asserts

output=1 when the number formed so far is divisible by 5.

(a) Derive the state transition table and implement in SystemVerilog.

(b) Follow-up: parameterize the design to check divisibility by any N.

If the accumulated value exceeds N during computation, how do you handle it?

ANSWER

```
// (a) Divisibility-by-5 FSM
//
// Key insight: reading bits MSB-first, after each bit the accumulated
// value V_new = 2 * V_old + new_bit.
// The remainder mod 5 evolves as: state_new = (2*state + bit) % 5
//
// State = current remainder mod 5 (0..4)
// Output = 1 when state == 0 (after all bits received)
//
// Transition table:
//   state | in=0 -> next | in=1 -> next
//   S0    |  S0          |  S1
//   S1    |  S2          |  S3
//   S2    |  S4          |  S0    <- div by 5 detected on next state
//   S3    |  S1          |  S2
//   S4    |  S3          |  S4
//   Output = 1 when cur_state == S0 (remainder == 0)

module div5_detector (
    input  logic clk, rst_n,
    input  logic din,       // serial input, MSB first
    input  logic valid,     // high when din is valid
    output logic divisible  // high when accumulated value % 5 == 0
);
    typedef enum logic [2:0] {S0,S1,S2,S3,S4} state_t;
    state_t cur, nxt;

    // Block 1: state register
    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n) cur <= S0;
        else if (valid) cur <= nxt;
    end

    // Block 2: next-state logic
    always_comb begin
        case (cur)
            S0: nxt = din ? S1 : S0;
            S1: nxt = din ? S3 : S2;
            S2: nxt = din ? S0 : S4;
            S3: nxt = din ? S2 : S1;
            S4: nxt = din ? S4 : S3;
            default: nxt = S0;
        endcase
    end

    // Block 3: output logic (Moore)
    assign divisible = (cur == S0);

endmodule

// (b) Parameterized divisibility-by-N
//
// The formula state_new = (2*state + bit) % N works for ANY N.
// "If value exceeds N" is automatically handled by the modulo:
// the state never stores the full value, only the remainder.
// State count = N states, each log2(N) bits wide.
```

```
//
module divN_detector #(
    parameter int N = 5
)(
    input  logic clk, rst_n, din, valid,
    output logic divisible
);
    localparam int BITS = $clog2(N);
    logic [BITS-1:0] cur, nxt;

    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n) cur <= '0;
        else if (valid) cur <= nxt;
    end

    // next state: (2*cur + din) mod N
    // If (2*cur + din) >= N, subtract N (since 2*cur < 2N, result < 2N,
    // so one subtraction is always sufficient)
    always_comb begin
        logic [BITS:0] tmp;
        tmp = {cur, din};           // == 2*cur + din
        nxt = (tmp >= N) ? tmp - N[BITS:0] : tmp[BITS-1:0];
    end

    assign divisible = (cur == '0);

endmodule
// Note on "exceeds N" follow-up:
// Since state <= N-1, value 2*state + bit <= 2*(N-1)+1 = 2N-1 < 2N.
// So (2*state + bit) can exceed N by at most N-1.
// One conditional subtraction of N is always sufficient -- no loop needed.
```

解释

数学本质：串行读入 MSB-first 的 n 位数字，每读一位后新数 = 旧数*2 + bit。

对 N 取余的递推关系：remainder_new = (2*remainder_old + bit) mod N，

所以只需存 remainder（N 个状态），不需要存完整的数值。

Parameterized 版本的关键：因为 state <= N-1，所以 2*state+bit <= 2N-1，

最多只需减一次 N 就能回到合法范围，不需要完整的 modulo 电路。

面试 follow-up "超过 N 就先扣到小于 N" 的答案正是这个一次减法。

Q11. Digital Design: Single-Port vs Dual-Port SRAM Interface (Designer Question)

QUESTION

(a) Describe a single-port SRAM interface. Read and write share the same address bus. How do you arbitrate between simultaneous read and write?

(b) Describe a dual-port SRAM interface. What additional signals are needed? What new hazards arise?

ANSWER

```
// (a) Single-port SRAM interface
//
// Shared signals: one addr bus, one data bus (bidirectional or separate),
// one chip-enable (CSn), one write-enable (WEn), one output-enable (OEn).
```

```
//
// Typical single-port SRAM interface:
//
module sram_1p_if (
    input  logic        clk,
    input  logic        cs_n,   // chip select, active low
    input  logic        we_n,   // write enable, active low
    input  logic        oe_n,   // output enable, active low
    input  logic [11:0] addr,   // shared address for read and write
    inout  wire  [7:0]  data    // bidirectional data bus
);
//   Write cycle: cs_n=0, we_n=0 -> data driven by master onto bus
//   Read  cycle: cs_n=0, we_n=1, oe_n=0 -> SRAM drives data bus
//
// Read-Write conflict (same address, same cycle):
//   Write takes priority in most SRAMs (write-first behavior).
//   Some SRAMs have undefined behavior; arbiter must ensure mutual exclusion.
//   Standard practice: never issue read and write in the same cycle to
//   the same address -- use a 1-cycle stall or pipeline the access.
//
// RTL arbiter sketch:
//   always_ff @(posedge clk) begin
//     if (wr_req && rd_req && wr_addr == rd_addr)
//       rd_stall <= 1;  // delay read by 1 cycle, let write go first
//     else
//       rd_stall <= 0;
//   end
endmodule

// (b) Dual-port SRAM interface
//
// Two independent port pairs: Port A (read/write) + Port B (read/write).
// Each port has its own: addr, data, cs_n, we_n, clk (can be different clocks).
//
module sram_2p_if (
    // Port A
    input  logic        clk_a,
    input  logic        cs_a_n, we_a_n,
    input  logic [11:0] addr_a,
    inout  wire  [7:0]  data_a,
    // Port B
    input  logic        clk_b,
    input  logic        cs_b_n, we_b_n,
    input  logic [11:0] addr_b,
    inout  wire  [7:0]  data_b
);
endmodule

// Additional hazards in dual-port SRAM:
//
// 1. Read-After-Write (RAW) hazard:
//    Port A writes addr X; Port B reads addr X in same cycle.
//    Depending on SRAM implementation, Port B may read old data.
//    Fix: forward the write data to the read result in RTL, or
//         use a 1-cycle read latency guarantee from the SRAM spec.
//
// 2. Write-Write conflict:
//    Both ports write to the same address in the same cycle.
//    Result is UNDEFINED in most SRAM macros.
//    Fix: arbitration logic must prevent simultaneous writes to same addr.
//
// 3. Clock domain crossing (if clk_a != clk_b):
//    Pointer synchronization needed if used as async FIFO backing store.
```

```
//    This is the classic async FIFO use case for dual-port SRAMs.
//
// Key difference summary:
//   Single-port: 1 addr, 1 data, simpler, no simultaneous R+W possible
//   Dual-port:   2 independent addr+data ports, 2x bandwidth,
//                but RAW/WAW hazards + possible CDC complexity
```

解释

单口 SRAM 面试考察的是读写仲裁和 write-first vs read-first 行为。

双口 SRAM 最常见的用途是 async FIFO 的存储器（一个 port 给 write domain，另一个给 read domain），这与 Section 8 的 Async FIFO 题目直接关联。

RAW hazard 是双口 SRAM 最容易被忽略的问题：同周期写然后读同一地址，读出的数据可能是旧值，RTL 层面需要显式 bypass/forwarding 逻辑。

面试扩展：True dual-port (TDP) vs Simple dual-port (SDP)？

SDP: 一个 port 只读，一个 port 只写（更常见，面积更小）；

TDP: 两个 port 都可以读写（更灵活，面积更大）。

Q12. Digital Design: create_generated_clock SDC (Designer Question)

QUESTION

What is create_generated_clock in SDC? What happens if you forget to add it?

Different companies emphasize different failure modes -- describe both:

(a) The clock will be treated as an ordinary data signal.

(b) The generated clock has no relationship to its source clock.

ANSWER

```
// create_generated_clock SDC syntax:
//
create_generated_clock -name clk_div2 \
    -source [get_ports clk] \
    -divide_by 2 \
    [get_pins clk_div_reg/Q]
//
// This tells the STA tool:
//   1. There is a clock at pin clk_div_reg/Q
//   2. It is derived from "clk" with a known relationship (divide by 2)
//   3. Setup/hold checks crossing from clk to clk_div2 (or vice versa)
//      can be calculated using the shared source relationship

// (a) "Treated as ordinary data signal" (company A perspective):
//
// Without create_generated_clock, the STA tool does not recognize
// the divided/gated clock output as a clock signal.
// Consequence:
//   - Paths FROM flip-flops clocked by the generated clock are analyzed
//     as DATA paths launching from the last registered point, with no
//     clock period budget. The tool may report no violations (silently
//     wrong) because there is no capture clock to check against.
//   - The generated clock net is optimized as data -- no clock tree
//     synthesis (CTS) will be applied, resulting in high skew and
//     incorrect timing throughout the entire clock domain.
```

```
//   - All downstream flip-flops are effectively unconstrained.

// (b) "No relationship to source clock" (company B perspective):
//
// Some tools may infer a clock at the divided output but treat it as
// a completely independent clock (no common source relationship).
// Consequence:
//   - Paths crossing from the source clock domain to the generated clock
//     domain are treated as asynchronous CDC paths.
//   - The tool either ignores them (if set_false_path is auto-applied
//     to unknown clock pairs) or applies an overly conservative timing
//     check (worst-case setup using the tighter period of the two clocks).
//   - Either way: correct timing analysis between the two domains is LOST.
//   - Optimization: tool cannot exploit the phase relationship to borrow
//     time across edges (multi-cycle path analysis broken).

// Correct practice:
//   Every clock-generating element (divider, mux, gate) must have a
//   corresponding create_generated_clock.
//   After create_generated_clock, verify with report_clock_interaction
//   that the source-generated relationship is correctly captured.
```

解释

create_generated_clock 是 SDC 中最容易遗漏但影响最大的约束之一。

两间公司的答案侧重点不同，但本质是同一个问题的两个面：

公司 A 关注"信号类型"：没有声明就是 data，不做时钟树综合，下游 FF 全错。

公司 B 关注"clock relationship"：即使工具推断出是 clock，没有 -source 关系，

跨 domain 的 setup/hold 计算就用错了 launch/capture edge pair。

面试时两个角度都提，再加一句"所以 signoff 前必须用 report_generated_clocks

和 check_timing 确认所有 generated clock 都被正确声明"，可以拿到满分。

Q13. Digital Design: MSI / MESI / MOESI Cache Coherence Protocols (Designer Question)

QUESTION

(Apple CPU Implementation)

Compare the MSI, MESI, and MOESI cache coherence protocols.

What problem does each additional state solve? When would you choose each?

ANSWER

```
// MSI: 3 states -- Modified, Shared, Invalid
//
//   Invalid  (I): cache line not present or invalid
//   Shared   (S): cache line present, clean, may exist in other caches
//   Modified (M): cache line present, dirty, exclusive owner
//
//   Problem: no "clean exclusive" state.
//   If a cache wants to write a Shared line, it must broadcast an
//   Invalidate to all other caches first (upgrade miss).
//   Also, reading a line that was previously written-back from M->I
//   requires a memory fetch even if no other cache modified it.

// MESI: 4 states -- adds Exclusive (E)
//
```

```
//   Exclusive (E): cache line present, clean, ONLY copy in any cache.
//
//   Problem solved: the E state allows a silent upgrade from E->M
//   without broadcasting an Invalidate. If the only cache holding a
//   line wants to write it, it can transition E->M locally, saving
//   one bus transaction (no invalidate needed).
//   Used by: Intel x86 (classic), ARM MESI variants.

// MOESI: 5 states -- adds Owned (O)
//
//   Owned (O): cache line is dirty, but SHARED -- this cache is
//              responsible for supplying the data to other caches
//              and eventually writing back to memory.
//
//   Problem solved: in MESI, when a Modified line is shared (another
//   cache requests it), the M-holder must write back to memory first,
//   then the requester fetches from memory -- 2 bus transactions.
//   With O state: M-holder transitions to O and supplies data directly
//   to requester (cache-to-cache transfer), memory is NOT updated yet.
//   This saves a write-back + memory-read round trip.
//   Used by: AMD processors, ARM big.LITTLE.

// Comparison table:
//   Protocol | States | Extra benefit               | Bus transactions (M->S)
//   MSI      |   3    | baseline                    | 2 (writeback + fetch)
//   MESI     |   4    | silent E->M upgrade         | 2 (writeback + fetch)
//   MOESI    |   5    | cache-to-cache dirty supply | 1 (direct cache supply)
//
// When to choose:
//   MSI:   simple academic baseline, rarely used in production
//   MESI:  good for systems where write-sharing is rare (each core
//          mostly writes its own data); common in x86 designs
//   MOESI: better for workloads with frequent producer-consumer sharing
//          (one core writes, another reads dirty data); common in AMD/ARM
```

解释

三个协议的演进逻辑：每加一个状态都是为了减少 bus transaction 次数。

E 状态省掉了 Shared->Modified 时的 invalidate 广播（当只有一个 cache 持有该行时）。

O 状态省掉了 Modified->Shared 时的 writeback（dirty 数据可以直接 cache-to-cache 传输）。

Apple Silicon 使用的是 MOESI 的变体（基于 ARM 的 AMBA CHI 互联协议），

在多核 big.LITTLE 架构下 cache-to-cache 传输尤其重要，可以显著降低内存带宽压力。

Q14. Digital Design: Booth Algorithm for Multiplication (Designer Question)

QUESTION

(Apple CPU Implementation)

Explain the Booth algorithm for binary multiplication.

Why is it used in hardware? What is Radix-4 (Modified) Booth encoding?

ANSWER

```
// Booth Algorithm (Radix-2)
//
// Problem with naive multiplication: a 1-bit multiplier has 1 partial
// product per bit. For 64-bit x 64-bit: 64 partial products to sum.
```

```
// A run of 1s in the multiplier generates redundant partial products.
//
// Booth insight: a run of k consecutive 1s in the multiplier can be
// replaced by one +multiplicand at the LSB and one -multiplicand above the MSB:
//   e.g., 0111100 = 01000 00 - 0000100
//   instead of 4 additions, use 1 add + 1 subtract
//
// Radix-2 Booth encoding: examine 2 bits at a time (bit[i], bit[i-1])
//   bit[i]  bit[i-1] | operation
//     0        0     | +0  (no transition)
//     0        1     | +M  (end of run of 1s)
//     1        0     | -M  (start of run of 1s)
//     1        1     | +0  (middle of run, no change)
//
// This reduces partial products by ~2x for random data.

// Radix-4 Modified Booth Encoding (MBE) -- industry standard
//
// Examine 3 overlapping bits: {B[2i+1], B[2i], B[2i-1]}
// Generates one partial product per 2 multiplier bits:
//
//   B[2i+1]  B[2i]  B[2i-1] | partial product
//     0        0       0    |  0
//     0        0       1    | +M
//     0        1       0    | +M
//     0        1       1    | +2M
//     1        0       0    | -2M
//     1        0       1    | -M
//     1        1       0    | -M
//     1        1       1    |  0
//
// For 64-bit x 64-bit multiplication:
//   Naive:     64 partial products
//   Radix-2 Booth: ~32 partial products
//   Radix-4 MBE:   32 partial products (exactly ceil(64/2) = 32)
//
// Hardware advantages of Radix-4 MBE:
//   1. Only 0, +/-M, +/-2M needed: +/-2M is just a shift, no extra adder
//   2. Halves the number of partial products -> shallower adder tree
//   3. Wallace/Dadda tree to sum 32 partial products: ~log(32)=5 levels
//      vs log(64)=6 levels for naive -- faster and lower power
//   4. -M implemented as ~M + 1 (two's complement), sharing NOT gates

// Apple M-series implementation:
//   Uses 54-bit Booth multiplier in the FPU/ALU pipeline.
//   Radix-4 MBE with a 3:2 (carry-save) Wallace tree to reduce
//   partial products, final carry-propagate adder for the result.
//   Critical path: MBE encoder -> Wallace tree -> CPA -> ~1 cycle at 3GHz+
```

解释

Booth 算法的核心价值：把 N 个部分积减少到 N/2 个，直接减少 Wallace tree 的层数，

加速乘法同时降低动态功耗（更少的加法器翻转）。

Radix-4 MBE 是现代 CPU 乘法器的工业标准，Apple、ARM、Intel 都使用这种方案。

面试考察点：能说出编码表、为什么 +/-2M 不需要额外硬件（左移一位）、

以及 Wallace tree 如何将部分积压缩到两行再用 CPA 收尾。

Q15. Digital Design: Two-Stage Pipeline a*c + b*c with Power Optimization (Designer Question)

QUESTION

(Apple Power Optimization and Analysis)

a, b, c are all 64-bit unsigned integers.

(a) Draw a two-stage pipeline for a*c + b*c. How many flip-flops are needed?

(b) Refactor to (a+b)*c. How many FFs are saved?

(c) What further power optimizations can be applied?

ANSWER

```
// (a) Normal two-stage pipeline: a*c + b*c
//
//  Stage 1: compute a*c (128-bit) and b*c (128-bit) in parallel
//  Stage 2: add the two products -> 129-bit result
//
//  Pipeline diagram:
//
//  [a(64)] ---> |       |                          |       |
//  [b(64)] ---> | MUL1  | --[128b]--> [FF_ac] --> |  ADD  | --> [FF_out(129)]
//  [c(64)] ---> | MUL2  | --[128b]--> [FF_bc] --> |       |
//               |_______|
//                  ^--- c feeds both multipliers
//  Input FFs (registered inputs): a(64) + b(64) + c(64) = 192 FFs
//  Inter-stage FFs: a*c(128) + b*c(128)           = 256 FFs
//  Output FFs:      result(129)                    = 129 FFs
//  Total (inter-stage + output) = 256 + 129        = 385 FFs

// (b) Refactored pipeline: (a+b)*c
//
//  Stage 1: compute (a+b) -> 65-bit result; c passes through
//  Stage 2: compute (a+b)*c -> 129-bit result
//
//  Pipeline diagram:
//
//  [a(64)] ---> |       |                            |       |
//  [b(64)] ---> |  ADD  | --[65b]--> [FF_sum(65)] --> |  MUL  | --> [FF_out(129)]
//  [c(64)] ---> |bypass | --[64b]--> [FF_c  (64)] --> |       |
//               |_______|
//  Inter-stage FFs: (a+b)(65) + c(64)            = 129 FFs
//  Output FFs:      result(129)                   = 129 FFs
//  Total (inter-stage + output) = 129 + 129       = 258 FFs
//
//  FFs saved = 385 - 258 = 127 FFs (~33% reduction)
//  Multipliers reduced: 2 -> 1 (saves ~50% multiplier area and power)

// (c) Further power optimizations
//
// 1. Clock gating on pipeline registers:
//    When the pipeline stalls or the unit is idle, gate the clock to
//    all inter-stage FFs. No switching -> zero dynamic power.
//    always_ff @(posedge clk) if (enable) ff_sum <= a + b;
//
// 2. Operand isolation:
//    Gate the inputs to the multiplier when enable=0.
//    Prevents spurious toggling inside MUL combinational logic.
//    assign mul_a_in = enable ? sum : '0;  // isolation cell
//
// 3. Reduce bit-width where possible:
//    If a and b are known to be < 32 bits at runtime (e.g. from profiling),
//    use a narrower multiplier and sign-extend. Smaller multiplier =
//    fewer partial products = lower dynamic power.
```

```
//
// 4. Use Booth encoding (Radix-4 MBE):
//    Reduces partial products from 65 to 33, shallower adder tree,
//    fewer switching nodes in the critical path.
//
// 5. Multi-Vt cell selection:
//    Use high-Vt cells on non-critical paths to reduce leakage.
//    Low-Vt only on the critical path (MBE -> Wallace tree -> CPA).
//
// 6. Power gating when the entire unit is unused:
//    Add a header/footer sleep transistor to cut VDD/VSS when the
//    pipeline unit is idle for multiple cycles.
```

解释

(a) 正常版本需要两个 64x64 乘法器并行工作，inter-stage 要存两个 128 位乘积（256 FFs）。

(b) 数学等价：a*c + b*c = (a+b)*c，用分配律把两个乘法合并成一个。

代价：Stage 1 新增一个 64 位加法器（面积小得多）。

收益：节省一整个乘法器（面积和功耗约减 50%），inter-stage FFs 减少 127 个。

(c) Power 优化层次：从电路级（clock gating, operand isolation）到算法级（Booth）到工艺级（Multi-Vt, power gating），面试中能分层次回答是加分项。

Q16. Digital Design: Synchronous FIFO RTL Implementation (Designer Question)

QUESTION

(Apple CPU Implementation)

Implement a synchronous FIFO (single clock domain) in synthesizable SystemVerilog. Include full/empty flags. Depth = 2^N entries.

ANSWER

```
module sync_fifo #(
    parameter int DEPTH = 16,           // must be power of 2
    parameter int WIDTH = 8,
    localparam int PTR_W = $clog2(DEPTH) + 1  // extra bit for full/empty disambiguation
)(
    input  logic             clk, rst_n,
    input  logic             wr_en, rd_en,
    input  logic [WIDTH-1:0] wr_data,
    output logic [WIDTH-1:0] rd_data,
    output logic             full, empty,
    output logic [PTR_W-1:0] occupancy
);
    // Storage array
    logic [WIDTH-1:0] mem [DEPTH];

    // Read and write pointers (PTR_W bits: MSB is wrap bit)
    logic [PTR_W-1:0] wr_ptr, rd_ptr;

    // Write logic
    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n) wr_ptr <= '0;
        else if (wr_en && !full) begin
            mem[wr_ptr[PTR_W-2:0]] <= wr_data;  // lower bits = address
            wr_ptr <= wr_ptr + 1;
```

```
        end
    end

    // Read logic
    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n) rd_ptr <= '0;
        else if (rd_en && !empty)
            rd_ptr <= rd_ptr + 1;
    end

    // Read data: registered output (1-cycle latency)
    always_ff @(posedge clk)
        if (rd_en && !empty)
            rd_data <= mem[rd_ptr[PTR_W-2:0]];

    // Full and empty using extra pointer bit (standard technique):
    // empty: pointers are equal (same wrap + same address)
    // full:  wrap bits differ, address bits equal
    assign empty     = (wr_ptr == rd_ptr);
    assign full      = (wr_ptr[PTR_W-1] != rd_ptr[PTR_W-1]) &&
                       (wr_ptr[PTR_W-2:0] == rd_ptr[PTR_W-2:0]);
    assign occupancy = wr_ptr - rd_ptr;

endmodule

// Key design decisions:
//
// 1. Extra pointer bit (PTR_W = clog2(DEPTH) + 1):
//    Without it, full and empty are indistinguishable (both have wr==rd).
//    The MSB acts as a "lap" or "wrap" counter.
//    empty: all bits equal (same lap, same address)
//    full:  laps differ by 1, addresses equal (wr wrapped past rd)
//
// 2. Simultaneous read and write (wr_en && rd_en):
//    Both are allowed when not full/empty respectively.
//    occupancy stays the same (one in, one out).
//    The full/empty logic handles this correctly:
//    wr_ptr and rd_ptr both increment -> their relationship unchanged.
//
// 3. Registered read data vs combinational:
//    Registered (above): rd_data updates one cycle after rd_en.
//    Combinational alternative: assign rd_data = mem[rd_ptr[PTR_W-2:0]];
//    Combinational has lower latency but slower timing (long path to output).
```

解释

Sync FIFO 的核心考点是 full/empty 的区分：当 wr_ptr == rd_ptr 时两者都成立，

必须用额外的 wrap bit 来区分。这是面试中最常被追问的细节。

PTR_W = clog2(DEPTH) + 1 是标准做法：低 clog2(DEPTH) 位做地址，最高位做 lap 标志。

同时读写（wr_en && rd_en）是 FIFO 必须处理的 corner case：

只要一个不 full 一个不 empty，两个操作可以同时发生，occupancy 不变。

Async FIFO 是同步 FIFO 的扩展：把单时钟的 wr_ptr/rd_ptr 换成 Gray code +

跨域同步，这正是 Section 8 Async FIFO 题目的基础。

Q17. Digital Design: SRAM Configuration Power Analysis (Designer Question)

QUESTION

(Apple Power Optimization and Analysis)

You need to store 1k words. Three SRAM configurations are available:

A: 1x 1k-word SRAM (single array, full depth and width)

B: 2x 512-word SRAM (two half-depth arrays, same full width)

C: 2x 1k half-word SRAM (two half-width arrays, same full depth)

All accesses are full-word reads/writes (no partial word access).

Which configuration consumes the least dynamic power? Explain.

ANSWER

```
// SRAM dynamic power sources:
//   1. Wordline switching: one wordline activated per access
//   2. Bitline precharge and discharge: one column per bit toggled
//   3. Peripheral circuits: row/column decoders, sense amps, I/O buffers

// Configuration A: 1x 1k-word SRAM
//   - Every access activates 1 wordline out of 1024 rows
//   - All word-width bitlines precharge/discharge on every access
//   - Single decoder, single sense amp bank
//   - Active array area per access: 100% (full array always powered)
//   - Power: HIGH (baseline reference)

// Configuration B: 2x 512-word SRAM (half depth)
//   - Address MSB selects WHICH bank (0 or 1); only 1 bank active
//   - Active bank: 1 wordline out of 512 rows (shorter wordline -> lower C)
//   - Inactive bank: clock-gated / chip-select disabled -> near-zero power
//   - Bitline width: same as A (full word width, same bitline capacitance)
//   - Active array area per access: ~50% (only one 512-word bank active)
//   - Decoder: simpler (1 fewer address bit to decode in each bank)
//   - Power: LOWEST
//   - Additional benefit: shorter wordlines = less wordline capacitance
//     = faster access AND lower wordline switching energy

// Configuration C: 2x 1k half-word SRAM (half width)
//   - Both banks ALWAYS active simultaneously (need both halves for 1 word)
//   - Each bank has 1024 rows -> same wordline count as A
//   - Each bank has half the bitlines (half word width)
//   - Total bitlines toggled = W/2 + W/2 = W (same as A!)
//   - Two separate wordline drivers activate for every access (vs one in A)
//   - Two sets of decoders, sense amps -> 2x peripheral power vs A
//   - Power: HIGHEST (more peripheral overhead, same bitline work)

// Power ranking: B < A < C
//
//   B wins because: only 1 of 2 banks is active per access.
//   The inactive bank contributes zero dynamic power (gated).
//   Effectively halves the active SRAM area and wordline length.
//
//   C loses because: both banks must be activated simultaneously,
//   doubling the peripheral circuit power while not saving any
//   bitline energy (total bitline capacitance is the same as A).

// Design insight -- general SRAM banking strategy:
//   Splitting by DEPTH (Config B) is power-efficient for random access
//   because only ONE bank is accessed per operation.
//   Splitting by WIDTH (Config C) is used for different reasons:
//   - Support partial word writes (byte enables)
//   - Different data widths for different clients
//   - NOT for power saving on full-word access
```

解释

SRAM 功耗分析的核心原则：动态功耗与每次访问激活的面积成正比。

深度分割（Config B）：每次只需访问一个 bank，另一个 bank 完全关闭，

有效将每次访问的活跃面积减半，是功耗最优方案。

宽度分割（Config C）：全字访问必须同时读取两个 bank，活跃面积不减，

还新增了两倍的外围电路开销，功耗反而比单块 SRAM 更高。

这个结论适用于任何 random access 场景。

注意：如果访问模式不是随机的（如顺序访问某半区），Config C 也可以节省功耗，

但题目明确说明 random access，所以 Config B 是最优解。

Q18. Digital Design: 4-bit x 4-bit Array Multiplier FA/HA Count (Designer Question)

QUESTION

(Omnivision Designer Question)

Design a 4-bit x 4-bit array multiplier using AND gates, Full Adders (FA),

and Half Adders (HA). How many AND gates, FAs, and HAs are required?

Draw the partial product grid and show where each adder is used.

ANSWER

```
// Step 1: Generate partial products (AND gates)
// a = a3 a2 a1 a0,  b = b3 b2 b1 b0
// 16 AND gates total (aibj for all i,j in {0,1,2,3})
//
// Partial product grid (columns 7..0):
//
//  Col:  7     6     5     4     3     2     1     0
//  b0:                     a3b0  a2b0  a1b0  a0b0   <- base row, P0=a0b0 direct
//  b1:               a3b1  a2b1  a1b1  a0b1          <- shift left 1
//  b2:         a3b2  a2b2  a1b2  a0b2                <- shift left 2
//  b3:   a3b3  a2b3  a1b3  a0b3                      <- shift left 3

// Step 2: Add rows using array of adders (3 stages)
//
// -- Stage 1: add row_b1 into row_b0 --
//
//  col1: (a0b1, a1b0)           -> HA  -> S1, C->col2
//  col2: (a1b1, a2b0, C_col1)   -> FA  -> S2, C->col3
//  col3: (a2b1, a3b0, C_col2)   -> FA  -> S3, C->col4
//  col4: (a3b1, C_col3)         -> HA  -> S4, C->col5
//  Stage 1 uses: 2 FA + 2 HA
//
// -- Stage 2: add row_b2 into stage-1 result --
//
//  col2: (a0b2, S2)             -> HA  -> P2(final), C->col3
//  col3: (a1b2, S3, C_col2)     -> FA  -> S3', C->col4
//  col4: (a2b2, S4, C_col3)     -> FA  -> S4', C->col5
//  col5: (a3b2, S5, C_col4)     -> FA  -> S5', C->col6
//  Stage 2 uses: 3 FA + 1 HA
//
// -- Stage 3: add row_b3 into stage-2 result (ripple carry adder) --
//
//  col3: (a0b3, S3')            -> HA  -> P3(final), C->col4
```

```
//  col4: (a1b3, S4', C_col3)    -> FA  -> P4(final), C->col5
//  col5: (a2b3, S5', C_col4)    -> FA  -> P5(final), C->col6
//  col6: (a3b3, S6', C_col5)    -> FA  -> P6(final), C->P7
//  Stage 3 uses: 3 FA + 1 HA
//
// -- Final output bits --
//  P0 = a0b0                    (direct AND, no adder)
//  P1 = S1 from Stage 1 HA
//  P2 = S2 from Stage 2 HA
//  P3 = S3 from Stage 3 HA
//  P4-P6 from Stage 3 FA sums
//  P7 = final carry out of Stage 3

// Summary:
//   AND gates:  16  (4x4 partial products)
//   FA (Full Adder):  8   (2 + 3 + 3)
//   HA (Half Adder):  4   (2 + 1 + 1)
//   Total output: 8 bits (P7..P0)
//
// General formula for N-bit x N-bit array multiplier:
//   AND gates = N^2
//   FA = N*(N-2)  for N>=3  -> 4*(4-2) = 8  (check: correct)
//   HA = N-1                -> 4-1    = 3  (check: should be 3 per row boundary)
//   Note: exact count varies slightly by layout convention (some count 4 HA).
```

解释

部分积共 N^2 = 16 个，用 AND 门生成，这是乘法器所有计算的起点。

加法器统计：每一行的加法都从 LSB 开始，最低列只有两个输入（用 HA），

其余列有三个输入（本行 PP + 上一行 sum + 上一行 carry，用 FA）。

每行最高列只有 carry 输入（无新的 PP 输入），用 HA 节省硬件。

总计：8 FA + 4 HA（具体布局略有变体，但数量级固定）。

面试扩展：Wallace tree 乘法器用同样的 FA/HA 数量但重新排列加法顺序，

把关键路径从 O(N) 级（array multiplier）降低到 O(log N) 级，速度更快。

Q19. Digital Design: Latchup Effect in CMOS (Designer Question)

QUESTION

(Omnivision Designer Question)

What is latchup in CMOS circuits? What causes it, what are the symptoms,

and how do you prevent it in layout and design?

ANSWER

```
// What is latchup?
//
// Latchup is a parasitic PNPN (thyristor) structure that forms between
// the PMOS and NMOS transistors in a CMOS process. Once triggered,
// it creates a low-impedance path from VDD to GND, causing:
//   - Extremely high current (limited only by supply impedance)
//   - Permanent device damage or chip destruction if current is not cut
//   - Functional failure even if the chip survives

// Parasitic structure in CMOS:
//
```

```
//  In a standard n-well CMOS process:
//    - PMOS in n-well: creates PNP transistor (p+source / n-well / p-sub)
//    - NMOS in p-sub:  creates NPN transistor (n+source / p-sub / n-well)
//  These two transistors share base/collector connections:
//
//    VDD                GND
//     |                  |
//    [P+]   <-- Q_pnp -->  [N-well]
//     |                       |
//    [P-sub] <-- Q_npn --> [N+]
//
//  Equivalent circuit: two cross-coupled BJTs forming a SCR (thyristor)
//    Q_pnp base = Q_npn collector = p-sub
//    Q_npn base = Q_pnp collector = n-well
//  Latchup triggers when beta_pnp * beta_npn >= 1 (positive feedback loop)

// Trigger conditions:
//   1. Voltage overshoot/undershoot on I/O pins (>VDD or <GND)
//   2. Large current injection (ESD event, power supply noise spike)
//   3. Ionizing radiation (single-event latchup in space applications)
//   4. Power-up sequencing violations (output before VDD is stable)

// Prevention -- Layout techniques:
//   1. Guard rings: surround PMOS with n+ ring tied to VDD;
//                   surround NMOS with p+ ring tied to GND.
//      Guard rings collect minority carriers before they reach
//      the opposite well, breaking the feedback loop.
//
//   2. Increase well/substrate contact density:
//      Frequent VDD/GND taps reduce substrate/well resistance Rsub, Rwell.
//      Lower resistance = smaller voltage drop = harder to forward-bias the BJTs.
//      Rule of thumb: contact tap within N um of every transistor (process-specific).
//
//   3. Separate NMOS and PMOS with sufficient spacing:
//      Increases minority carrier recombination distance.
//
//   4. Use retrograde wells or epi substrate:
//      Lower surface resistance, higher doping at depth -> harder to trigger.
//
// Prevention -- Design/ESD techniques:
//   1. Add clamp diodes on I/O pads (limit voltage to VDD+0.5V / GND-0.5V)
//   2. Series resistors on I/O pads to limit injection current
//   3. ESD protection circuits (GGNMOS, SCR-based ESD clamp)
//   4. Proper power-up sequencing (core VDD before I/O VDD)
```

解释

Latchup 是 CMOS 工艺中最重要的可靠性问题之一，影像传感器（Omnivision）尤其关注，

因为光子注入会产生载流子，在强光或辐射条件下更容易触发。

根本原因是 CMOS 工艺中天然形成的寄生 PNP+NPN 正反馈环路（SCR 结构）。

防护的两个方向：一是增大触发门槛（guard ring、低阻基底），

二是减小注入源（ESD 保护、I/O clamp）。

面试关键词：guard ring、n-well contact、substrate tap、SCR/thyristor、

beta_pnp * beta_npn >= 1 触发条件。

Q20. Digital Design: Tristate Buffer Circuit and Applications (Designer Question)

QUESTION

(Omnivision Designer Question)

Draw the transistor-level circuit of a tristate buffer.

Explain the three output states. When and why is it used in digital design?

ANSWER

```
// Tristate buffer truth table:
//   EN  |  A  |  Output
//    0  |  0  |  Z  (high-impedance, floating)
//    0  |  1  |  Z  (high-impedance, floating)
//    1  |  0  |  0  (driven low)
//    1  |  1  |  1  (driven high)

// Transistor-level circuit (CMOS, active-high enable):
//
//          VDD
//           |
//          [MP2]  <- PMOS, gate = EN_b (EN inverted)
//           |
//          [MP1]  <- PMOS, gate = A
//           |
//           +-----> OUT
//           |
//          [MN1]  <- NMOS, gate = A
//           |
//          [MN2]  <- NMOS, gate = EN
//           |
//          GND
//
// Total: 4 transistors (+ 1 inverter for EN_b = 2 more) = 6 transistors
//
// Operation:
//   EN=0: MP2 ON (pull to VDD), MN2 OFF (cut GND path)
//         But MP1 gate = A: if A=1, MP2-MP1 series path may exist
//         However MN2 is OFF so no pull-down path regardless.
//         Actually: when EN=0, EN_b=1:
//           MP2 gate=EN_b=1 -> MP2 OFF (p-type, gate high = off)
//           MN2 gate=EN=0   -> MN2 OFF
//           Both pull-up and pull-down paths OPEN -> OUT = Z (high-Z)
//
//   EN=1, A=1: EN_b=0 -> MP2 ON; A=1 -> MP1 OFF -> pull-up path BLOCKED
//              MN2 ON; A=1 -> MN1 ON -> pull-down path COMPLETE -> OUT=0
//              Wait: A=1 means OUT should be 1. Let me re-check polarity.
//
// Corrected: standard non-inverting tristate buffer uses:
//   Pull-up:   MP2 (gate=EN_b) in series with MP1 (gate=A_b, where A_b=~A)
//   Pull-down: MN1 (gate=A)    in series with MN2 (gate=EN)
//
//   EN=1, A=1: EN_b=0 -> MP2 ON; A_b=0 -> MP1 ON -> OUT pulled to VDD = 1
//              EN=1  -> MN2 ON; A=1   -> MN1 ON -> OUT pulled to GND = 0
//              MP wins for A=1? No -- A_b=0 so MP1 gate=A_b=0 -> MP1 ON
//              MN1 gate=A=1 -> MN1 ON -> CONTENTION... use separate paths:
//
// Correct standard CMOS tristate (non-inverting):
//
//   When EN=1, A=1:  MP2 and MP1 both ON  -> OUT=VDD=1
//                    MN2 ON but MN1 gate=A=1 also ON...
//   The trick: connect MN1 gate to A, MP1 gate to A_b
//   So for A=1: MP1(gate=A_b=0)->ON, MP2(gate=EN_b=0)->ON -> drive HIGH
//               MN1(gate=A=1)->ON, MN2(gate=EN=1)->ON -> drive LOW -- contention!
//
// Standard implementation uses an inverting tristate + inverter:
```

```
//
//   Inverting tristate buffer (4 transistors):
//     Pull-up:   MP2(gate=EN_b) series MP1(gate=A)  [A=0 -> MP1 ON -> pull up]
//     Pull-down: MN1(gate=A)    series MN2(gate=EN) [A=1 -> MN1 ON -> pull down]
//     Output = ~A when EN=1,  Z when EN=0
//
//   Non-inverting tristate = inverting tristate + inverter on input A
//
//   Simplified (inverting tristate, 4 transistors):
//          VDD
//           |
//       [MP1] gate=A          <- A=0: MP1 ON (pull high)
//           |
//       [MP2] gate=EN_b       <- EN=1 -> EN_b=0: MP2 ON
//           |
//           +---> OUT_b (inverted output)
//           |
//       [MN2] gate=EN         <- EN=1: MN2 ON
//           |
//       [MN1] gate=A          <- A=1: MN1 ON (pull low)
//           |
//          GND

// Why tristate buffers are used:
//
// 1. Shared bus (classic use):
//    Multiple drivers share one wire. Only one driver enabled at a time.
//    Others are in high-Z, effectively disconnected from the bus.
//    Example: CPU data bus, memory bus (DDR DQ pins are tristate)
//
// 2. Bidirectional I/O pins:
//    Same pad used for both input and output.
//    TX mode: tristate enabled, drives data out
//    RX mode: tristate disabled (Z), pad acts as input
//
// 3. FPGA routing (internal tristate busses in older FPGAs)
//
// Modern usage note:
//    On-chip tristate busses are largely replaced by mux-based busses
//    in deep-submicron designs (tristate on long wires causes signal
//    integrity issues; mux trees are more predictable).
//    Tristate is still standard at chip I/O pads.
```

解释

三态缓冲器的三个状态：高电平（1）、低电平（0）、高阻抗（Z）。

高阻抗状态意味着输出端既不拉高也不拉低，等效于从总线断开，

由外部上拉/下拉或其他驱动器决定总线状态。

晶体管级：反相三态缓冲器是最常见的实现，只需 4 个晶体管：

两个串联的 PMOS 上拉路径（门控信号 = A 和 EN_b）；

两个串联的 NMOS 下拉路径（门控信号 = A 和 EN）。

EN=0 时两条路径都断开 -> 高阻态。

现代 SoC 中片上已大量用 MUX 替代三态，但 I/O PAD 仍然是三态缓冲器的天下。

Omnivision 作为图像传感器公司，MIPI/DVP 接口的双向数据线都用到三态。

Q21. Digital Design: Metastability and 2-FF Synchronizer (Designer Question)

QUESTION

(Ambarella Designer Question)

(a) What is metastability? When does it occur?

(b) In an async FIFO, why do we use a 2-FF synchronizer instead of just 1 FF? What goes wrong with only 1 FF? What is the risk of passing a wrong value downstream?

(c) Related: what SDC constraint is applied to the Gray code pointer path crossing clock domains? Why set_max_delay instead of a normal constraint?

ANSWER

```
// (a) Metastability definition
//
// Metastability occurs when a flip-flop's setup or hold time is violated:
// the data input changes too close to the clock edge.
// The FF enters a metastable state -- its output is neither valid 0 nor
// valid 1, but an intermediate voltage that can persist for an
// indeterminate amount of time before resolving to 0 or 1.
//
// Physics: a D-FF is a bistable latch. Its output Q is stable at VDD or GND
// (two energy wells). A setup/hold violation deposits Q near the unstable
// equilibrium point (between the two wells). The circuit will eventually
// fall into one well, but the time to do so is exponentially distributed:
//
//   P(still metastable after time t) = C1 * exp(-t / tau)
//
// where tau ~20-50ps (process-dependent), C1 is a technology constant.
// Metastability CANNOT be eliminated -- only made exponentially unlikely.
//
// When it occurs:
//   - Any time a signal crosses from one clock domain to another
//   - Any time an asynchronous input (reset, enable, data) is sampled
//   - Any time setup/hold margin is insufficient

// (b) Why 2-FF synchronizer, not 1-FF?
//
// With 1 FF only:
//
//   async_data ---[FF1, rd_clk]---> downstream logic
//
//   FF1 samples an asynchronous signal and MAY go metastable.
//   The metastable output propagates IMMEDIATELY into downstream logic.
//   Downstream combinational logic sees an intermediate voltage:
//     - Logic gates interpret it differently (some see '0', some see '1')
//     - Different fanout paths may see different logic values
//     - Result: CORRUPTED DATA propagates through the entire design
//     - The wrong value is latched by the next FF in the pipeline
//   This is exactly the failure mode: "passing a wrong value downstream."
//   Even if FF1 eventually resolves to the correct value, the downstream
//   FF has already sampled the incorrect intermediate result.
//
// With 2-FF synchronizer:
//
//   async_data ---[FF1, rd_clk]---[FF2, rd_clk]---> downstream logic
//
//   FF1 samples the async signal and may go metastable.
//   FF1's output drives ONLY FF2's input -- NOT any combinational logic.
//   FF2 does NOT sample until the NEXT rd_clk edge (one full period later).
//   During that full clock period (~1000ps at 1GHz), FF1 has time to resolve.
```

```
//
//   Probability FF1 still metastable after 1 full clock period:
//     P_meta = C1 * f_async * exp(-T_clk / tau)
//   At 1GHz with tau=30ps: P_meta ~ 10^-14 per clock cycle
//   MTBF ~ 10^14 cycles / (10^9 Hz) ~ 10^5 seconds ~ 28 hours
//   With 3-FF: MTBF increases to millions of years
//
//   Key insight: the 2nd FF gives the 1st FF a FULL CLOCK PERIOD to resolve.
//   The 1st FF's output only drives the 2nd FF's D input (high impedance,
//   no combinational fanout), so even if FF1 briefly outputs garbage,
//   no downstream logic sees it until FF2 captures the (now stable) value.
//
// Why it works ONLY for single-bit signals (or Gray code):
//   If multiple bits change simultaneously (e.g. binary counter),
//   different bits may resolve to different "old" or "new" values,
//   creating an impossible combination (e.g. binary 3 and 5 simultaneously
//   when transitioning from 2 to 6).
//   Gray code changes only 1 bit at a time -> at most 1 FF goes metastable
//   -> the resolved value is either old_ptr or new_ptr (both valid).

// (c) SDC for Gray code pointer synchronizer
//
// set_max_delay -datapath_only <value>   //     -from [get_cells wr_ptr_gray_reg[*]]   //     -to   [get_ce
//
// Why set_max_delay and not a normal setup constraint:
//   Normal setup constraint requires a common clock relationship.
//   wr_clk and rd_clk are asynchronous -- no common period exists.
//   Without an explicit constraint, STA either errors out or applies
//   a 0ns requirement (treats it as same-clock -> false violation).
//
// Why -datapath_only is CRITICAL:
//   Without it: tool adds clock skew + uncertainty from BOTH clocks
//   -> incorrect pessimistic calculation.
//   With it: tool checks ONLY the raw combinational path delay,
//   ignoring clock relationships. This is correct because:
//   - The synchronizer is designed to tolerate metastability
//   - We only need data to physically arrive at FF1 within T_dest
//   - We do NOT want the tool to optimize this path for setup margin
//     (over-optimization could make it too fast and violate hold)
//
// Value guideline: 0.9 * T_destination_clock
//   Leaves 10% margin for physical variation.
```

解释

Metastability 是所有 CDC 设计的根本问题，2-FF 同步器是工业界最基础的解决方案。

单 FF 的问题不只是"解析时间不够"，更严重的是亚稳态输出会立即进入下游组合逻辑，

导致同一个信号的不同扇出看到不同的逻辑值，产生无法预测的功能错误。

2-FF 的核心价值：第一个 FF 的输出只驱动第二个 FF 的 D 端（无组合逻辑扇出），

给了第一个 FF 整整一个时钟周期来解析，概率降低到可接受范围。

Gray code 是让这个方案可行的关键前提：每次只有 1 bit 变化，

即使解析错了，也只会读到相邻的合法 pointer 值，不会出现跳变超过 1 的错误计数。

这三点（metastability 定义、2-FF 原理、Gray code 必要性）是 Ambarella/ARM/Apple

等公司 CDC 面试必问的完整回答链。

Q22. Digital Design: Gray Code Binary Conversion & CDC Use Case (Designer Question)

QUESTION

(AMD Designer Question)

(a) Implement binary-to-Gray and Gray-to-binary conversion in RTL.

(b) Why is Gray code used for CDC pointer synchronization?

What goes wrong with a binary counter?

ANSWER

```
// (a) Binary <-> Gray conversion
//
// Binary to Gray: G[i] = B[i] XOR B[i+1]  (G[MSB] = B[MSB])
//
module bin2gray #(parameter W = 4)(
    input  logic [W-1:0] bin,
    output logic [W-1:0] gray
);
    assign gray = bin ^ (bin >> 1);
endmodule
//
// Example (4-bit):
//   bin=0000->gray=0000, bin=0001->gray=0001
//   bin=0010->gray=0011, bin=0011->gray=0010
//   bin=0100->gray=0110  <- only 1 bit changes per step
//
// Gray to Binary: B[MSB] = G[MSB]; B[i] = B[i+1] XOR G[i]
//
module gray2bin #(parameter W = 4)(
    input  logic [W-1:0] gray,
    output logic [W-1:0] bin
);
    assign bin[W-1] = gray[W-1];
    genvar i;
    generate
        for (i = W-2; i >= 0; i--)
            assign bin[i] = bin[i+1] ^ gray[i];
    endgenerate
endmodule
// Note: gray2bin has a serial dependency chain (each bit depends on previous).
// For wide pointers this creates a long combinational path -- fine for CDC
// since the conversion happens AFTER synchronization (no timing pressure).

// (b) Why Gray code, not binary, for CDC pointer synchronization?
//
// Problem with binary counter across CDC:
//   When a binary counter increments, MULTIPLE bits can change simultaneously.
//   e.g., 0111 -> 1000: all 4 bits flip at the same time.
//   The 2-FF synchronizer samples these bits at the destination clock edge.
//   Different bits may be sampled in different states of metastability:
//     bit3 resolves to 1 (new value)
//     bit2 resolves to 0 (new value)
//     bit1 resolves to 1 (old value -- still metastable, resolved late)
//     bit0 resolves to 0 (new value)
//   Result captured: 1010 -- which is NEITHER 0111 nor 1000.
//   This is an impossible, corrupt pointer value -> FIFO overflow/underflow.
//
// Why Gray code fixes this:
//   Gray code changes exactly 1 bit per increment (by construction).
//   At most ONE flip-flop in the synchronizer can go metastable per transition.
//   That 1 bit resolves to either old or new value.
//   Result is always a valid adjacent pointer: either N or N-1.
//   Both are safe: FIFO may appear slightly more full or slightly more empty,
```

```
//   but no data corruption or impossible state occurs.
```

解释

Binary-to-Gray: G = B XOR (B>>1)，一行代码，面试必背。

Gray-to-Binary: 串行异或链，B[i] = XOR of all G[j] for j >= i。

CDC 用 Gray code 的根本原因：2-FF 同步器只能保证"单 bit 亚稳态"的安全性。

多 bit 同时跳变时，各 bit 解析时间不同，采样结果可能是不存在的中间值。

Gray code 将多 bit 跳变问题转化为单 bit 问题，使同步器的概率保证成立。

Q23. Digital Design: Async FIFO Data Path Handling (Designer Question)

QUESTION

(AMD Designer Question)

In an async FIFO, the write pointer is synchronized to the read clock domain

and the read pointer is synchronized to the write clock domain.

How is the DATA PATH handled? Does the data also need a synchronizer?

ANSWER

```
// The data path does NOT use a 2-FF synchronizer.
//
// Async FIFO data path:
//
//   Write domain:                Read domain:
//   wr_data --> [SRAM/RAM] <--- rd_data
//   wr_addr (wr_ptr)             rd_addr (rd_ptr, local)
//
// Why no synchronizer on data:
//   1. The RAM is a shared resource accessible from both clock domains.
//      It is NOT a sequential element clocked by either domain's clock.
//      Data is written with wr_clk and read with rd_clk independently.
//
//   2. Data stability is GUARANTEED by the pointer synchronization:
//      When the read side sees a synchronized wr_ptr value of N,
//      it means all data at addresses 0..N-1 was written at least
//      2 rd_clk cycles ago (2-FF synchronizer delay).
//      The SRAM write is complete and stable before rd_ptr catches up.
//
//   3. There is no metastability risk on RAM read data because:
//      rd_ptr only advances when empty=0 (data is confirmed written).
//      The SRAM output is stable combinational logic, not a flip-flop
//      sampling an async signal.
//
// Key rule: the 2-FF synchronizer is ONLY for the pointer (1 or N-bit Gray).
//           Data travels through the RAM array with NO synchronizer.
//
// Timing guarantee chain:
//   Write: wr_data written to RAM at address wr_ptr[t]
//   wr_ptr increments: wr_ptr[t] -> wr_ptr[t+1]
//   wr_ptr Gray encoded, synchronized to rd_clk (2 cycles latency)
//   rd side sees wr_ptr_sync = wr_ptr[t+1] after 2+ rd_clk cycles
//   rd side now knows address wr_ptr[t] is safe to read
//   rd_ptr advances to wr_ptr[t], RAM output is stable = correct data
//
```

```
// What WOULD go wrong with a synchronizer on data:
//   Data is W bits wide (e.g. 32 bits). A 32-bit synchronizer would:
//   - Require 64 FFs (2 stages x 32 bits) -- huge area
//   - Each bit resolves independently -- could capture garbage combinations
//   - Is IMPOSSIBLE to make safe for multi-bit data (same problem as binary ptr)
//   - Is completely unnecessary since RAM guarantees data stability
```

解释

数据路径不需要同步器，这是 async FIFO 设计中最容易混淆的考点。

理解关键：RAM 不是触发器，它是双端口共享存储。数据的安全性由 pointer 同步保证：

read side 只有在确认 write pointer 已经超过当前 read address 时才去读，

而 write 完成比 pointer 同步 latency 更早，所以数据一定是稳定的。

面试回答顺序：先说"data 不用同步器"，再解释为什么（RAM 架构 + pointer 保证），

最后说如果给 data 加同步器会有什么问题（面积 + 多 bit 不安全），完整闭环。

Q24. Digital Design: Setup Time, Hold Time & Violation Fixes (Designer Question)

QUESTION

(AMD Designer Question)

(a) Define setup time and hold time for a flip-flop.

(b) What causes setup violations and how do you fix them?

(c) What causes hold violations and how do you fix them?

(d) Can you fix a hold violation by increasing clock period?

ANSWER

```
// (a) Definitions
//
// Setup time (Tsu): the minimum time data must be STABLE BEFORE the
//   active clock edge for the FF to reliably capture the data.
//   Violation: data arrives too late (too close to clock edge).
//
// Hold time (Thold): the minimum time data must remain STABLE AFTER
//   the active clock edge for the FF to reliably latch the data.
//   Violation: data changes too quickly after the clock edge.
//
// Timing window:
//   |<-- Tsu -->|CLK edge|<-- Thold -->|
//   data must be stable throughout this entire window.

// (b) Setup violation: data arrives too late
//
// Setup check: Tlaunch + Tcq + Tlogic + Tsu <= Tperiod + Tskew
// Slack = Tperiod + Tskew - Tcq - Tlogic - Tsu
//
// Causes: long combinational path (Tlogic too large)
// Fixes:
//   1. Reduce logic depth: restructure/retime pipeline, add pipeline stages
//   2. Use faster cells (low-Vt, larger drive strength) on critical path
//   3. Reduce net delay: fix routing congestion, use shorter wires
//   4. Increase clock period (lower frequency) -- functional fix, not preferred
//   5. Positive clock skew: make capture clock arrive later (careful, affects others)
//   6. Multi-cycle path (MCP): if data can take >1 cycle, relax constraint in SDC
```

```
// (c) Hold violation: data changes too soon after clock edge
//
// Hold check: Tlaunch + Tcq_min + Tlogic_min >= Tcapture + Thold
// Slack = Tcq_min + Tlogic_min - Thold - Tskew_negative
//
// Causes: short path (Tlogic_min too small), excessive clock skew
// Fixes:
//   1. Add buffers/delays on the DATA path to increase Tlogic_min
//      (this is the ONLY truly correct fix)
//   2. Reduce negative clock skew (fix clock tree imbalance)
//   3. Use cells with larger Tcq_min
//
// NEVER fix hold by:
//   - Adding buffers on the CLOCK path (creates skew, hurts other paths)
//   - Reducing clock frequency (hold is clock-period-independent -- see (d))

// (d) Can increasing clock period fix a hold violation?
//
// NO. Hold violations are INDEPENDENT of clock period.
//
// Hold check: Tcq_min + Tlogic_min >= Thold + Tskew
//   -- no Tperiod term appears in this equation.
//
// Intuition: hold violation means data changes too fast after the clock edge.
//   Making the clock slower (longer period) does NOT change how fast
//   the data propagates through the short path.
//   The next clock edge is further away, but the CURRENT edge's hold
//   window is unchanged.
//
// This is a key interview distinction:
//   Setup violation: CAN be fixed by increasing clock period (more time for data)
//   Hold violation:  CANNOT be fixed by clock period change (add data path delay)
```

解释

Setup 和 Hold 是数字设计最基础的概念，但很多人混淆修复方法。

核心记忆法：Setup 是"数据来得太晚"（加时间/减逻辑）；Hold 是"数据走得太快"（加延迟）。

Hold 与时钟频率无关是最常见的面试陷阱：hold check 公式里根本没有 Tperiod。

修 hold 的唯一正确方法是在数据路径上加 buffer，修 setup 的正确方法是优化逻辑深度。

在时钟路径上加 buffer 看似能修 hold，实际上只是把问题转移并引入 skew，

这是 Q12 create_generated_clock 题目的延伸。

Q25. Digital Design: Async FIFO Architecture (Designer Question)

QUESTION

(AMD Designer Question)

Draw and explain the complete architecture of an asynchronous FIFO.

Label all components and signal paths.

ANSWER

```
// Complete async FIFO architecture:
//
//  Write Clock Domain          |  Shared  |  Read Clock Domain
```

```
//  ========================    |          |  ======================
//                              |          |
//  wr_data --> [Dual-port RAM] <-addr/data-> [Dual-port RAM] --> rd_data
//               ^ wr_addr                    rd_addr ^
//               |                                    |
//  wr_en --> [WR PTR LOGIC] --> wr_ptr_bin      [RD PTR LOGIC] <-- rd_en
//               |                    |               |
//               |           [BIN2GRAY]          [BIN2GRAY]
//               |                    |               |
//               |           wr_ptr_gray          rd_ptr_gray
//               |                    |               |
//               |           [2-FF SYNC]          [2-FF SYNC]
//               |           (rd_clk)             (wr_clk)
//               |                    |               |
//               |           rd_ptr_gray_sync    wr_ptr_gray_sync
//               |                    |               |
//  full <-- [FULL LOGIC] <----      |     ----> [EMPTY LOGIC] --> empty
//           (wr domain)     wr_ptr_gray  rd_ptr_gray  (rd domain)
//
// Component breakdown:
//
// 1. Dual-port RAM (shared):
//    Write port: clocked by wr_clk, addressed by wr_ptr_bin
//    Read port:  clocked by rd_clk (or async), addressed by rd_ptr_bin
//    Data does NOT pass through synchronizers -- RAM guarantees stability
//
// 2. Write pointer logic (wr_clk domain):
//    Binary counter: wr_ptr_bin (PTR_W bits, extra wrap bit)
//    Converted to Gray: wr_ptr_gray = wr_ptr_bin ^ (wr_ptr_bin >> 1)
//    Synchronized to rd_clk via 2-FF synchronizer
//
// 3. Read pointer logic (rd_clk domain):
//    Binary counter: rd_ptr_bin
//    Converted to Gray: rd_ptr_gray
//    Synchronized to wr_clk via 2-FF synchronizer
//
// 4. Full logic (wr_clk domain):
//    Uses wr_ptr_gray (local) and rd_ptr_gray_sync (from rd domain)
//    Full condition: MSB and MSB-1 of wr and rd Gray ptrs differ,
//    remaining bits are equal
//    (equivalent to: wr_ptr - rd_ptr_sync == DEPTH in Gray domain)
//
// 5. Empty logic (rd_clk domain):
//    Uses rd_ptr_gray (local) and wr_ptr_gray_sync (from wr domain)
//    Empty condition: wr_ptr_gray_sync == rd_ptr_gray (all bits equal)
//
// Full/empty Gray code comparison:
//   empty: wr_ptr_gray_sync == rd_ptr_gray  (all N bits identical)
//   full:  wr_ptr_gray_sync[N-1]   != rd_ptr_gray[N-1]   AND
//          wr_ptr_gray_sync[N-2]   != rd_ptr_gray[N-2]   AND
//          wr_ptr_gray_sync[N-3:0] == rd_ptr_gray[N-3:0]
//   (top 2 bits differ, rest same -- equivalent to wrap-around in binary)
```

解释

Async FIFO 架构的五个核心组件：RAM、写指针逻辑、读指针逻辑、full 逻辑、empty 逻辑。

关键设计原则：只有 pointer（Gray 编码后）过 2-FF 同步器，data 不过同步器。

Full/empty 的 Gray code 比较是实现的难点：

empty 很简单（指针完全相等），full 需要比较最高两位（wrap bit 不同 + 次高位不同）。

这与 sync FIFO 用二进制指针额外 wrap bit 的方法不同，需要额外理解 Gray code 的循环性质。

Q26. Digital Design: Same Frequency, Different Phase -- Why CDC Still Needed (Designer Question)

QUESTION

(AMD Designer Question)

Two clock domains have the SAME frequency but DIFFERENT phases (not aligned).

Why do they still require async FIFO or CDC treatment?

Can we simply use set_false_path or treat them as the same clock?

ANSWER

```
// Two clocks: same frequency, different phase
//
//   clk_A:  _|-|_|-|_|-|_|-|
//   clk_B:     _|-|_|-|_|-|_   (shifted by e.g. 0.3 * T_clk)
//
// They are NOT the same clock unless they share a common source
// with a known, fixed, bounded phase relationship.

// Case 1: Truly related (same PLL, known fixed phase offset)
//   If clk_B = create_generated_clock -source clk_A -phase 90,
//   the STA tool KNOWS the phase relationship.
//   You can use regular setup/hold constraints -- STA will correctly
//   calculate the timing window using the known offset.
//   No async FIFO needed; this is a SYNCHRONOUS domain crossing.
//   SDC: create_generated_clock with correct -phase parameter.

// Case 2: Same nominal frequency but DIFFERENT sources (async)
//   Even if both clocks are nominally 100 MHz, if they come from
//   different oscillators or PLLs with no common reference:
//
//   a) Phase drift over time:
//      Independent oscillators have slight frequency differences (ppm).
//      The phase offset is not fixed -- it slowly drifts.
//      A clock "edge" that has a safe margin today may violate setup/hold
//      tomorrow as the phase rotates through all possible relationships.
//
//   b) The STA tool cannot analyze this correctly:
//      Without a declared relationship between clk_A and clk_B, the tool
//      treats them as asynchronous (no common launch/capture edge).
//      If you set_false_path, paths are UNCHECKED -- a silent bug.
//      If you constrain as same-clock, you get FALSE violations or miss reals.
//
//   c) At any phase offset, metastability risk exists:
//      If phase offset = 0.5ps (nearly zero), the setup/hold window is
//      violated on almost every cycle.
//      If phase offset = T/2, setup looks fine but HOLD may be violated
//      on the other transition.
//      The only safe relationship is a KNOWN, FIXED, BOUNDED phase difference
//      with sufficient margin for all PVT corners.
//
// Conclusion:
//   Same frequency + different UNKNOWN/DRIFTING phase = asynchronous.
//   Must use async FIFO or 2-FF synchronizer.
//   Same frequency + known fixed phase from common source = synchronous.
//   Can use regular timing constraints with correct SDC declarations.
//
// Real-world example:
//   CPU core clock and PCIe reference clock: both ~100 MHz in some designs,
//   but completely independent sources. Must be treated as async.
//   CPU core clock and its divided-by-2 clock: related, synchronous crossing.
```

解释

这道题是 CDC 概念最常见的误区：频率相同不等于时钟同步。

判断是否需要 CDC 的唯一标准是：两个时钟是否有共同来源且相位关系已知且固定。

相位漂移（frequency ppm差异）是关键：即使今天相位差是 0.1T，

几分钟后可能变成 0.5T，几小时后变成 0.9T，没有任何 static timing 分析能覆盖这种情况。

面试中说出"ppm 频差导致相位漂移"是加分项，体现了对实际硬件的理解。

Q27. Digital Design: FIFO with Non-Power-of-2 Depth (Designer Question)

QUESTION

(AMD Designer Question)

Standard FIFO implementations use depth = 2^N for easy pointer wrap-around

and full/empty detection. How do you handle a FIFO with depth that is NOT

a power of 2 (e.g., depth = 6)?

ANSWER

```
// Problem with non-power-of-2 depth:
//
// Standard approach: pointer wraps naturally at 2^N via binary overflow.
//   e.g., depth=8: ptr goes 0,1,2,...,7,0,1,... (4-bit ptr with wrap bit)
//   full/empty detected by extra wrap bit -- works because 2^N divides evenly.
//
// For depth=6: cannot use natural binary overflow.
//   A 3-bit ptr would count 0..7 (covers 8 slots), but we only have 6.
//   Overflow at 8 != 6, so wrap bit logic breaks.

// Solution 1: Explicit wrap-around with comparison
//   Use a wider pointer and explicitly reset at DEPTH.
//   Still needs an extra "lap" bit for full/empty disambiguation.
//
module fifo_non_pow2 #(
    parameter int DEPTH = 6,
    parameter int WIDTH = 8,
    localparam int PTR_W = $clog2(DEPTH) + 1  // extra lap bit
)(
    input  logic             clk, rst_n, wr_en, rd_en,
    input  logic [WIDTH-1:0] wr_data,
    output logic [WIDTH-1:0] rd_data,
    output logic             full, empty
);
    logic [WIDTH-1:0]  mem [DEPTH];
    logic [PTR_W-1:0]  wr_ptr, rd_ptr;

    // Pointer increment: reset to 0 at DEPTH, but toggle the lap bit
    function automatic logic [PTR_W-1:0] next_ptr(
        input logic [PTR_W-1:0] ptr
    );
        logic [PTR_W-2:0] addr = ptr[PTR_W-2:0];
        logic             lap  = ptr[PTR_W-1];
        if (addr == DEPTH - 1)
            return {~lap, {(PTR_W-1){1'b0}}};  // wrap: toggle lap, addr=0
        else
            return ptr + 1;
    endfunction
```

```
    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n) wr_ptr <= '0;
        else if (wr_en && !full) begin
            mem[wr_ptr[PTR_W-2:0]] <= wr_data;
            wr_ptr <= next_ptr(wr_ptr);
        end
    end

    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n) rd_ptr <= '0;
        else if (rd_en && !empty)
            rd_ptr <= next_ptr(rd_ptr);
    end

    assign rd_data = mem[rd_ptr[PTR_W-2:0]];
    assign empty   = (wr_ptr == rd_ptr);
    assign full    = (wr_ptr[PTR_W-1] != rd_ptr[PTR_W-1]) &&
                     (wr_ptr[PTR_W-2:0] == rd_ptr[PTR_W-2:0]);
endmodule

// Solution 2: Round up to next power of 2, mark extra slots as invalid
//   Allocate 2^ceil(log2(DEPTH)) slots, use occupancy counter for full/empty.
//   Wastes a few memory words but simplifies pointer logic.
//   Preferred when area is not critical.

// Solution 3: Occupancy counter (always works regardless of depth)
//   Maintain a counter: increment on write, decrement on read.
//   full  = (occupancy == DEPTH)
//   empty = (occupancy == 0)
//   No wrap bit needed, but counter adds 1 cycle latency to full/empty.
```

解释

非 2 幂次深度的核心难点是指针的循环逻辑：不能用自然溢出，必须显式检测 ptr==DEPTH-1。

lap bit（额外 wrap bit）的用法与标准 sync FIFO 完全相同，只是 addr 部分不再

用自然溢出而是用 if(addr==DEPTH-1) reset 来实现循环。

实际工程中三种方案各有适用场景：

方案1 最省面积；方案2 逻辑最简单；方案3 灵活性最高但 full/empty 有额外延迟。

Q28. Digital Design: FIFO Registered Output and Empty Flag Design (Designer Question)

QUESTION

(AMD Designer Question)

In a synchronous FIFO, if rd_data is a registered (pipelined) output

rather than a combinational output, how does this affect:

(a) The read latency?

(b) The design of the empty flag?

ANSWER

```
// Two output modes for FIFO read data:
//
// Mode A: Combinational (fall-through) output
//   assign rd_data = mem[rd_ptr];  // directly from array
```

```
//   Latency: 0 cycles (data valid same cycle as rd_en asserted)
//   empty:   based on current ptr comparison
//   Timing:  long path (mem array + combinational to output)
//
// Mode B: Registered output (pipelined)
//   always_ff @(posedge clk) if (rd_en && !empty) rd_data <= mem[rd_ptr];
//   Latency: 1 cycle (data valid 1 cycle AFTER rd_en asserted)
//   Timing:  better (FF cuts the long combinational path)

// (b) Empty flag design for registered output:
//
// Problem: with registered output, if empty goes high at cycle T,
//   the consumer reads rd_data at cycle T+1 (registered).
//   But what if the FIFO becomes empty exactly at cycle T?
//   The consumer asserts rd_en at T, data appears at T+1,
//   but at T+1 empty=1 and rd_ptr has already advanced.
//   Next rd_en would read invalid data!
//
// Solution 1: Pre-emptive empty (empty goes high 1 cycle early)
//   Generate "early_empty" = (occupancy <= 1) in addition to empty.
//   Consumer uses early_empty to stop issuing rd_en before the last word.
//   This prevents the pipeline overrun.
//
// Solution 2: Separate read_valid signal
//   Add a pipeline register tracking whether rd_data is valid:
//   always_ff @(posedge clk) rd_valid <= rd_en && !empty;
//   Consumer uses rd_valid instead of !empty to qualify rd_data.
//   This correctly indicates when the registered output holds valid data.
//
// Recommended implementation (registered output + rd_valid):
//
module fifo_reg_out #(parameter DEPTH=16, parameter WIDTH=8)(
    input  logic             clk, rst_n,
    input  logic             wr_en, rd_en,
    input  logic [WIDTH-1:0] wr_data,
    output logic [WIDTH-1:0] rd_data,
    output logic             full, empty, rd_valid
);
    // ... (ptr, mem logic as in sync FIFO) ...
    // Registered output
    always_ff @(posedge clk)
        if (rd_en && !empty) rd_data <= mem[rd_ptr[/* addr bits */0]];
    // rd_valid: 1 cycle delayed indicator that rd_data is valid
    always_ff @(posedge clk or negedge rst_n)
        if (!rst_n) rd_valid <= 0;
        else        rd_valid <= rd_en && !empty;
endmodule
```

解释

寄存器输出（registered output）是工程中更常见的选择，因为时序更好。

代价是必须处理"valid 信号对齐"问题：rd_data 比 rd_en 晚一个周期才有效。

两种解决方案都在工业界广泛使用：early_empty 简单直接，rd_valid 更通用。

rd_valid 本质上是一个 1-bit pipeline 寄存器，跟着数据流走，

这个模式在任何有流水线延迟的接口设计中都适用（如 AXI read data channel）。

Q29. Digital Design: AXI Valid-Ready Handshake FSM (Designer Question)

QUESTION

(AMD Designer Question)

Implement a simple AXI-style valid-ready handshake state machine.

A transaction occurs when BOTH valid=1 AND ready=1 on the same clock edge.

The producer must hold valid and data stable until ready is asserted.

ANSWER

```
// AXI handshake rule:
//   Transfer occurs when: valid & ready at posedge clk
//   Producer: once valid is asserted, it CANNOT be de-asserted until
//             ready is seen (valid must stay high until handshake completes)
//   Consumer: ready can be asserted/deasserted freely
//
// Producer FSM (source side):
module axi_producer (
    input  logic        clk, rst_n,
    input  logic        start,      // trigger to send a transaction
    input  logic        ready,      // from consumer
    input  logic [31:0] data_in,    // data to send
    output logic        valid,
    output logic [31:0] data_out
);
    typedef enum logic [1:0] {IDLE, ACTIVE, DONE} state_t;
    state_t cur, nxt;

    // Block 1: state register
    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n) cur <= IDLE;
        else        cur <= nxt;
    end

    // Block 2: next-state logic
    always_comb begin
        case (cur)
            IDLE:   nxt = start ? ACTIVE : IDLE;
            ACTIVE: nxt = (valid && ready) ? DONE : ACTIVE;
            DONE:   nxt = IDLE;
            default: nxt = IDLE;
        endcase
    end

    // Block 3: output logic
    always_comb begin
        valid    = (cur == ACTIVE);
        data_out = (cur == ACTIVE) ? data_in : '0;
    end
endmodule

// Consumer FSM (sink side):
module axi_consumer (
    input  logic        clk, rst_n,
    input  logic        valid,
    input  logic [31:0] data_in,
    output logic        ready,
    output logic [31:0] data_captured,
    output logic        data_valid_out
);
    typedef enum logic [1:0] {IDLE, WAIT, CAPTURE} state_t;
    state_t cur, nxt;

    // Block 1: state register
```

```
    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n) cur <= IDLE;
        else        cur <= nxt;
    end

    // Block 2: next-state
    always_comb begin
        case (cur)
            IDLE:    nxt = WAIT;
            WAIT:    nxt = valid ? CAPTURE : WAIT;
            CAPTURE: nxt = IDLE;
            default: nxt = IDLE;
        endcase
    end

    // Block 3: outputs
    always_comb begin
        ready         = (cur == WAIT);    // ready to accept
        data_valid_out = (cur == CAPTURE);
    end

    always_ff @(posedge clk)
        if (cur == WAIT && valid) data_captured <= data_in;

endmodule

// Key AXI rule reminder:
//   valid MUST NOT depend on ready (producer cannot wait for ready before asserting valid)
//   ready CAN depend on valid (consumer can wait to see valid before asserting ready)
//   This prevents deadlock: if both waited for each other, neither would fire.
```

解释

AXI valid-ready 握手是 ARM AMBA 协议的核心机制，AMD GPU/CPU 片上互联广泛使用。

最重要的规则：valid 不能等 ready 才拉高（否则死锁）；ready 可以等 valid 拉高后再拉高。

Producer FSM 的关键：进入 ACTIVE 后 valid 必须保持，直到 valid&&ready 同时成立才能离开。

一旦 valid 被拉高就不能撤回，这保证了 consumer 不需要在每个周期都检查 valid。

面试中画出三段式 FSM（状态寄存器、次态逻辑、输出逻辑分离）是标准写法。

Q30. Digital Design: Latch vs Flip-Flop Differences (Designer Question)

QUESTION

(AMD Designer Question)

What is the difference between a latch and a flip-flop?

When is a latch intentionally used? When is it a bug?

ANSWER

```
// Latch: level-sensitive storage element
//   When enable (EN) = 1: output Q follows input D transparently
//   When enable (EN) = 0: output Q holds its last value
//   No clock edge needed -- responds to level, not transition.
//   RTL: always_latch / always @(en or d) if(en) q <= d;
//
// Flip-flop: edge-sensitive storage element
//   Captures D on the ACTIVE EDGE of clock (posedge or negedge)
```

```
//   Output Q holds value until next active edge
//   RTL: always_ff @(posedge clk) q <= d;

// Key differences:
//
//   Property         | Latch              | Flip-Flop
//   -----------------|--------------------|-------------------
//   Trigger          | Level (EN high/low)| Clock edge
//   Transparency     | Yes (when EN=1)    | No (opaque)
//   Timing analysis  | Complex (window)   | Simple (setup/hold)
//   Clock needed     | No                 | Yes
//   Area (CMOS)      | ~2 gates less      | Slightly more
//   STA difficulty   | Hard (pulse-width) | Straightforward
//   Glitch sensitive | Yes (EN glitch)    | No (edge only)

// When latches are intentionally used:
//   1. Clock gating cell (ICG): latch + AND gate forms glitch-free clock gate
//      (latch samples enable when clk=0, output stable before clk rises)
//   2. Data retention latch: power-gating designs retain state through latch
//      on a separate always-on power domain
//   3. High-speed clock-borrowing: intentional latch allows timing to borrow
//      time from the next cycle (half-cycle path stealing)
//   4. Specific high-performance cells: register files sometimes use
//      latch-based arrays (lower area, faster write)
//
// When a latch is an UNINTENDED BUG (most common):
//   1. Incomplete if-else in always_comb:
//      always_comb begin
//        if (sel) y = a;  // missing else -> latch on y!
//      end                //  fix: add else y = 0; or default assignment
//
//   2. Incomplete case statement (missing default):
//      always_comb begin
//        case (sel)        // if sel=2'b10 or 2'b11 not covered
//          2'b00: y = a;   // y latches previous value -> unintended latch
//          2'b01: y = b;
//        endcase           // fix: add default: y = 0;
//      end
//
//   3. Always block sensitivity list missing a signal (Verilog, not SV):
//      always @(a) y = a & b;  // missing b -> latch on b changes
//      // Fix: use always_comb (automatically infers full sensitivity)
```

解释

Latch 和 FF 的本质区别：level vs edge 触发。

在现代同步设计中，95% 的存储元件是 FF，latch 出现在代码里几乎都是 bug。

最常见的 latch bug：if 没有 else，case 没有 default，导致某些输入条件下

输出保持上一次的值（即锁存行为）。

SystemVerilog 的 always_comb 会在综合和仿真时自动推断完整的敏感列表，

但仍然无法防止不完整的条件赋值产生 latch。

解决方案：在 always_comb 块开头给所有输出赋默认值，然后再写条件覆盖。

Q31. Digital Design: Avoiding Unintended Latches in RTL (Designer Question)

QUESTION

(AMD Designer Question)

Show three common RTL coding patterns that generate unintended latches, and the correct fix for each. How do you detect latches during synthesis?

ANSWER

```
// ---- Pattern 1: Incomplete if-else ----
//
// BUG (generates latch on y):
always_comb begin
    if (sel)
        y = a;    // no else -> y holds when sel=0 -> LATCH
end
//
// FIX (default assignment before conditions):
always_comb begin
    y = 1'b0;     // default: covers all unspecified cases
    if (sel)
        y = a;
end

// ---- Pattern 2: Incomplete case (missing default) ----
//
// BUG (generates latch on out when sel=2'b10 or 2'b11):
always_comb begin
    case (sel)
        2'b00: out = a;
        2'b01: out = b;
        // 2'b10, 2'b11 not covered -> latch!
    endcase
end
//
// FIX option A: add default:
always_comb begin
    case (sel)
        2'b00: out = a;
        2'b01: out = b;
        default: out = '0;   // covers all other cases
    endcase
end
// FIX option B: default assignment before case:
always_comb begin
    out = '0;       // covers everything not explicitly assigned
    case (sel)
        2'b00: out = a;
        2'b01: out = b;
    endcase
end

// ---- Pattern 3: Output not assigned in all FSM states ----
//
// BUG (y not assigned in S1 -> latch):
always_comb begin
    case (state)
        S0: begin y = 1; z = 0; end
        S1: begin z = 1; end   // y missing -> latch on y!
        S2: begin y = 0; z = 0; end
    endcase
end
//
// FIX: default all outputs at top of always_comb:
always_comb begin
    y = 0;  z = 0;  // defaults
    case (state)
```

```
        S0: begin y = 1;            end
        S1: begin       z = 1;      end
        S2: begin                   end  // both use defaults
    endcase
end

// ---- Detecting latches during synthesis ----
//
// 1. Synthesis tool warnings:
//    Synopsys DC: "Warning: Latch <name> is inferred for signal <y>"
//    Cadence Genus: "WARNING [CDFG-172] Latch inferred for ..."
//    Any latch warning in synthesis should be treated as an error.
//
// 2. Linting tools (run before synthesis):
//    SpyGlass, Jasper, Questa Lint: rules like
//    "no_latch", "incomplete_case", "incomplete_if" catch these early.
//
// 3. RTL review: search for always_comb / always @(*) blocks without
//    default assignments -- flag for manual review.
//
// 4. Formal verification: model check that no output is X in simulation
//    when inputs are in an uncovered state.
```

解释

最佳实践：在每个 always_comb 块的第一行给所有输出赋默认值，

然后再写条件逻辑覆盖特定情况。这样无论条件分支如何不完整，

综合器都不会推断 latch。

这个"默认赋值在前"的模式是 RTL coding style 的工业界黄金准则，

在 ARM、Apple、AMD 等公司的 RTL coding guidelines 中都有明确要求。

Latch 在综合时会产生明确的 warning，应该在 signoff 前零 latch（除非有意为之）。

Q32. Digital Design: Pipeline Waveform -- always @(posedge clk) A<=en?1:C; B<=A; C<=B;

QUESTION

Given the following always block, initial values A=B=C=0.

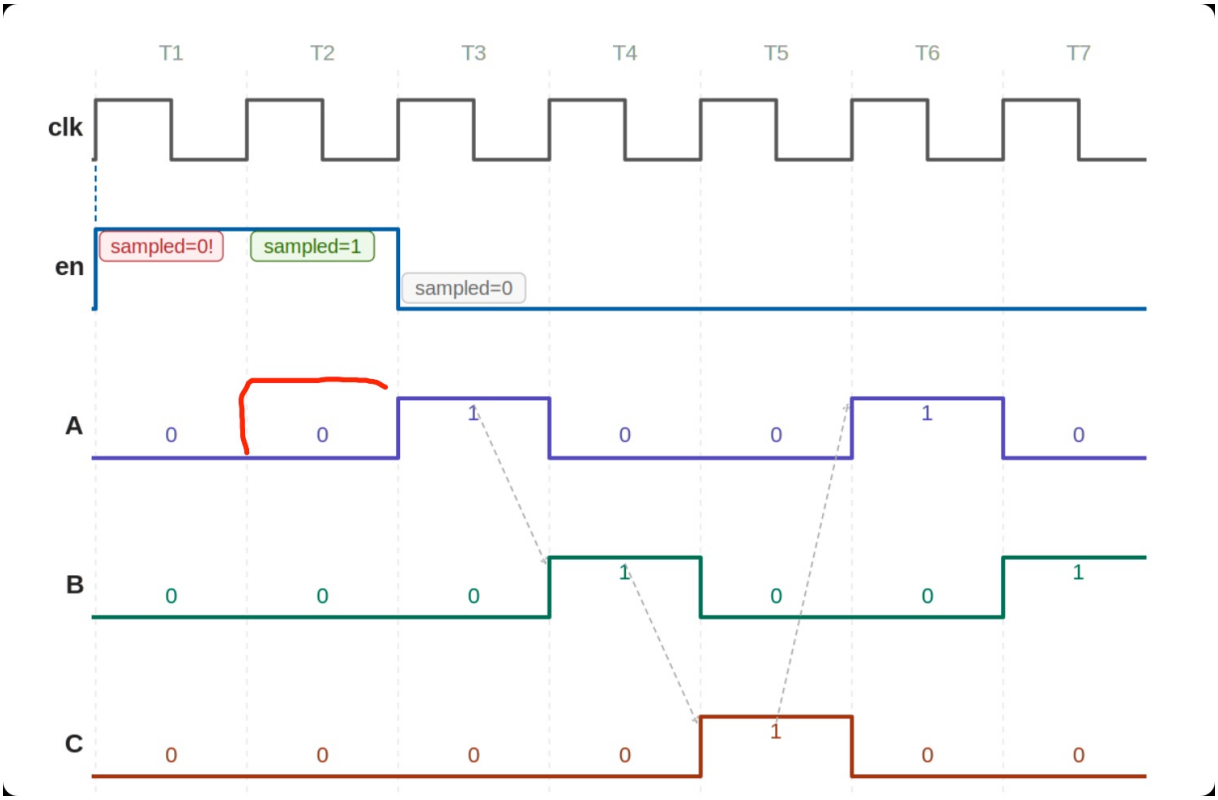
en is a registered signal that rises at the SAME TIME as the first clock

rising edge (T1) and stays high for exactly one cycle.

Draw the waveform for clk, en, A, B, C for 7 cycles.

```
always @(posedge clk) begin
    A <= en ? 1 : C;
    B <= A;
    C <= B;
end
```

ANSWER




```
// Key insight: en rises at T1 edge (aligned) -> sampled=0 at T1.
// en is high for one cycle physically (T1 to T3 edges),
// so it is sampled=1 at T2, then sampled=0 from T3 onward.
//
// Non-blocking assignment: all RHS evaluated from CURRENT state,
// all updates applied simultaneously after the clock edge.
//
//   Before T1: A=0, B=0, C=0
//
//   T1 (en_sampled=0): A=C=0, B=A=0, C=B=0   -> A=0 B=0 C=0
//   T2 (en_sampled=1): A=1,   B=A=0, C=B=0   -> A=1 B=0 C=0
//   T3 (en_sampled=0): A=C=0, B=A=1, C=B=0   -> A=0 B=1 C=0
//   T4 (en_sampled=0): A=C=0, B=A=0, C=B=1   -> A=0 B=0 C=1
//   T5 (en_sampled=0): A=C=1, B=A=0, C=B=0   -> A=1 B=0 C=0
//   T6 (en_sampled=0): A=C=0, B=A=1, C=B=0   -> A=0 B=1 C=0
//   T7 (en_sampled=0): A=C=0, B=A=0, C=B=1   -> A=0 B=0 C=1
//
// Observation: at T2 en injects "1" into A.
// That "1" propagates through the 3-stage pipeline: A->B->C->A->...
// After T4 (C gets the 1), C feeds back into A when en=0 (A<=C).
// Result: the "1" circulates with period 3 cycles indefinitely.
//
//   A: 0 0 1 0 0 1 0 0 1 ...   (high every 3 cycles starting T2)
//   B: 0 0 0 1 0 0 1 0 0 ...   (high every 3 cycles starting T3)
//   C: 0 0 0 0 1 0 0 1 0 ...   (high every 3 cycles starting T4)
//
// Critical lesson: en rises at T1 edge -> SAMPLED AS 0 at T1.
// The injection of "1" happens one cycle later at T2.
// If you mistakenly assume en=1 is captured at T1, the entire
// waveform shifts by 1 cycle -- a common interview mistake.
```

解释

这道题结合了两个考点：

1. 同步采样边界：en 与 clk 上升沿对齐，FF 采样到的是旧值（0），

下一个上升沿（T2）才采到新值（1）。

2. 三级流水线循环：A<=en?1:C 在 en=0 时等价于 A<=C（C 反馈给 A）。

当 B<=A, C<=B, A<=C 时，三个 FF 形成一个循环移位寄存器。

T2 注入的"1"沿 A->B->C->A 方向循环，周期为3个时钟周期。

面试中常见错误：把 en 当 async input，在 T1 就采到1，

导致 A 在 T1 就变为1，整个波形提前一拍，是错误的。

Q33. Digital Design: CDC Multi-bit Transfer -- Why Not 2-FF Synchronizer?

QUESTION

When transferring multiple bits across clock domains (CDC), what methods can be used? Why can't you simply use a 2-FF synchronizer for multi-bit data?

ANSWER

```
// Problem with 2-FF synchronizer on multi-bit data:
//
// A 2-FF synchronizer provides metastability protection for ONE bit.
// Each bit has its own two flip-flops. Each bit resolves independently.
//
// Example: transferring a 4-bit value transitioning from 0111 -> 1000
//   bit3: 0->1  might resolve to NEW value (1)  at time T
//   bit2: 1->0  might still be metastable        at time T
//   bit1: 1->0  might resolve to OLD value (1)   at time T
//   bit0: 1->0  might resolve to NEW value (0)   at time T
//
//   Captured value: 1_1_0 = 1010 = decimal 10
//   This is NEITHER 0111 (=7) nor 1000 (=8) -- an impossible value!
//   The downstream logic receives a completely invalid state.
//   This can cause counters to jump, state machines to enter illegal states,
//   or memory to be accessed at wrong addresses.
//
// Why Gray code works (1 bit change per transition):
//   Only ONE bit changes per increment in Gray code.
//   At most ONE FF can go metastable per transition.
//   That one bit resolves to either old or new value.
//   Both results (old count or new count) are valid adjacent values.
//   This is why async FIFO pointers use Gray code with 2-FF synchronizers.
//   BUT: Gray code only works for COUNTERS (sequential incrementing data).
//   It cannot be applied to arbitrary multi-bit data (e.g. AXI write data).

// Correct methods for multi-bit CDC:
//
// Method 1: Async FIFO
//   Use dual-port RAM + Gray code pointers + 2-FF synchronizers.
//   Data is NOT synchronized -- only the pointers are.
//   Best for: streaming data, high bandwidth, variable-length bursts.
//   Overhead: dual-port RAM + pointer synchronization logic.
//
// Method 2: Handshake protocol (req/ack)
//   Sender:   assert req, hold data stable until ack received.
//   Receiver: synchronize req with 2-FF, sample data, assert ack.
//   Sender:   synchronize ack with 2-FF, deassert req.
//   Data is guaranteed stable for multiple clock cycles -> safe to sample.
//   Best for: low-bandwidth, infrequent transfers (register writes, config).
//   Latency:  ~4-6 cycles per transfer (2 sync latencies each direction).
//
```

```
// Method 3: Multi-cycle path with enable synchronization
//   Hold data stable for N cycles, synchronize a single VALID/ENABLE bit.
//   When enable arrives (after 2-FF sync), data has been stable for N cycles.
//   Constraint: data must be stable for at least (sync_latency + setup) cycles.
//   Best for: slow-changing configuration registers.
//
// Summary:
//   2-FF sync:         single bit only (control signals, Gray-code pointers)
//   Async FIFO:        multi-bit streaming data, high bandwidth
//   Handshake req/ack: multi-bit occasional data, low bandwidth
//   Multi-cycle enable: multi-bit stable config data
```

解释

多 bit 不能用 2-FF 同步器的根本原因：每个 bit 的亚稳态独立解析，

解析到新值还是旧值是随机的，不同 bit 的组合可能形成"不可能"的中间值。

Gray code 是唯一可以安全用 2-FF 同步器传输的多 bit 情况：

因为 Gray code 每次只变 1 bit，保证亚稳态最多发生在 1 个 bit 上。

实际工程选择原则：

- 数据流 -> Async FIFO（带宽高，延迟可接受）
- 配置寄存器 -> Handshake（延迟大但安全，传输频率低）
- 状态控制信号 -> 单 bit 2-FF 同步器（或 Gray code 编码后同步）

Q34. Digital Design: 128-bit Priority Arbiter with Logic Depth Analysis (Designer Question)

QUESTION

Design a priority arbiter where MSB has the highest priority.

(a) Write RTL for a basic N-bit priority arbiter.

(b) For N=128 bits, a naive for-loop implementation has O(N) logic depth.

How do you restructure it into groups to reduce logic depth?

(c) What is the logic depth if you use 8-bit groups? 16-bit groups?

What is the general formula? Can you do better than log2(N)?

ANSWER

```
// (a) Basic priority arbiter: MSB highest priority
// For-loop scan from MSB to LSB, break at first set bit

module priority_arb #(
    parameter N = 8
)(
    input  logic [N-1:0] req,     // request vector, req[N-1] = highest priority
    output logic [N-1:0] grant    // one-hot grant output
);
    always_comb begin
        grant = '0;
        for (int i = N-1; i >= 0; i--) begin
            if (req[i]) begin
                grant[i] = 1'b1;
                break;           // only one grant
            end
        end
```

```
    end
endmodule

// Logic depth: O(N) -- synthesizer may chain N priority MUX stages
// For N=128: 128 levels of logic -> very long critical path, low Fmax

// (b) Hierarchical group approach for N=128, B=8 (8-bit groups)
//
// Split 128 bits into G=16 groups of B=8 bits each.
// Stage 1: 8-bit priority encoder for EACH group (parallel, B-to-1)
//          -> 16 group_any signals (does this group have any request?)
//          -> 16 group_grant vectors (which bit within each group wins)
// Stage 2: 16-to-1 priority among the 16 groups (which group wins?)
//          -> select the winning group's grant

module priority_arb_128 (
    input  logic [127:0] req,
    output logic [127:0] grant
);
    localparam G = 16;   // number of groups
    localparam B = 8;    // bits per group

    logic [B-1:0]  grp_grant [G];  // per-group one-hot grant
    logic          grp_any   [G];  // does group g have any request?
    logic [G-1:0]  grp_req;        // one-hot: which groups have requests?
    logic [G-1:0]  grp_sel;        // one-hot: which group wins?

    // Stage 1: 8-bit priority encoder for each group (all in parallel)
    genvar g;
    generate
        for (g = 0; g < G; g++) begin : gen_grp
            priority_arb #(.N(B)) u_pa (
                .req   (req[g*B +: B]),
                .grant (grp_grant[g])
            );
            assign grp_any[g] = |req[g*B +: B];
            assign grp_req[g] = grp_any[g];
        end
    endgenerate

    // Stage 2: 16-to-1 priority among groups (MSB group = highest priority)
    priority_arb #(.N(G)) u_grp_pa (
        .req   (grp_req),
        .grant (grp_sel)
    );

    // Final: gate each group's grant by whether its group won
    always_comb begin
        grant = '0;
        for (int g2 = 0; g2 < G; g2++) begin
            if (grp_sel[g2])
                grant[g2*B +: B] = grp_grant[g2];
        end
    end

endmodule

// (c) Logic depth analysis
//
// Stage 1 (B-bit priority encoder):   depth_1 = ceil(log2(B))
// Stage 2 (G-to-1 priority among groups): depth_2 = ceil(log2(G))
// Total depth = depth_1 + depth_2 = ceil(log2(B)) + ceil(log2(N/B))
//
```

```
// For N=128, G=N/B:
//   B=8  groups: log2(8)  + log2(16) = 3 + 4 = 7 gate levels
//   B=16 groups: log2(16) + log2(8)  = 4 + 3 = 7 gate levels
//   B=4  groups: log2(4)  + log2(32) = 2 + 5 = 7 gate levels
//   B=2  groups: log2(2)  + log2(64) = 1 + 6 = 7 gate levels
//
// General form (for any B that is a power of 2):
//   depth = log2(B) + log2(N/B) = log2(B) + log2(N) - log2(B) = log2(N)
//
// For N=128: depth = log2(128) = 7 gate levels -- ALWAYS, regardless of B!
//
// Can you do better than log2(N)?
//   NO. log2(N) is the theoretical minimum for any priority encoder on N bits.
//   Proof: in the worst case, you must compare all N bits to determine the
//   winner. Any comparison tree on N leaves requires at least log2(N) levels.
//   More group levels (3-stage, 4-stage) give the same total: log2(N).
//
// Why choose B=8 or B=16 then?
//   All group sizes give the SAME logic depth (log2(N) = 7 for N=128).
//   The choice of B affects:
//     - Fan-in: larger B = wider mux at each stage (harder for synthesis)
//     - Area: smaller B = more instances but simpler cells
//     - Practical: B=8 or B=16 maps well to standard cell libraries
//       and allows efficient packing in synthesis.
//   There is no depth advantage to any particular split -- just implementation trade-offs.
```

解释

(a) for loop + break 在 RTL 中会被综合成一个优先级 MUX 链，深度 O(N)。

N=128 时有128级逻辑，Fmax 很低。

(b) 分组方案：把 N 位分成 G 组每组 B 位，两个阶段并行化：

Stage 1 所有组同时做 B 位优先级编码（并行），Stage 2 做 G 组间的优先级。

(c) 核心公式：depth = log2(B) + log2(N/B) = log2(N)。

无论选择 B=8 还是 B=16，总深度对 N=128 都是 7 级逻辑——这是理论下界。

log2(N) 不可突破：N 个输入的优先级判断至少需要 log2(N) 级比较树。

面试加分点：不是选哪个 B 能减少深度，而是理解"所有合法 B 给出相同深度"，

选 B 的依据是 fan-in、面积、与标准单元库的匹配度，而不是时序。

Q35. Digital Design: Cache -- CPU Write, Set Mapping, PLRU (Designer Question)

QUESTION

(a) Can the CPU side directly modify cache contents?

(b) How do you guarantee that two transactions map to the same cache set?

(c) What is PLRU (Pseudo-LRU) and how does it work?

ANSWER

```
// (a) Can CPU modify cache contents?
//
// YES -- this is the normal write operation.
// "Read-only" is a separate concern (memory protection / permissions).
//
// CPU write to a cacheable address:
```

```
//   Write-through policy: write to cache AND to memory simultaneously.
//   Write-back policy:    write to cache only; mark line DIRTY.
//                         Memory is updated only when the line is evicted.
//
// The CPU writes via the cache controller, which:
//   1. Checks if the address is in the cache (write hit).
//      - Write hit:  update the cache line directly.
//      - Write miss: allocate a new line (write-allocate) or write directly
//                    to memory (no-write-allocate), depending on policy.
//   2. For write-back: sets the dirty bit on the modified line.
//   3. Eviction: if the line is later evicted, the dirty data is written back.
//
// Note: "read-only" (e.g. text segment) is enforced by the MMU page table
// permissions, not by the cache hardware itself. A write to a read-only
// page triggers a page-fault exception before reaching the cache.

// (b) Guaranteeing two transactions map to the same cache set
//
// Cache set index formula:
//   set_index = (addr >> byte_offset_bits) % num_sets
//
// where:
//   byte_offset_bits = log2(cache_line_size)
//   num_sets         = total_sets in the cache
//
// To map two addresses A and B to the SAME set:
//   (A >> byte_offset_bits) % num_sets == (B >> byte_offset_bits) % num_sets
//
// Simplest construction: fix the set-index bits to be identical.
//   Address bit layout: [tag | set_index | byte_offset]
//   Example: 64-byte lines (6 offset bits), 16 sets (4 index bits):
//     addr[9:6] = set_index bits
//   To force same set: keep addr[9:6] constant, vary addr[5:0] and addr[31:10].
//
// In SV constraint (for cache verification):
constraint c_same_set {
    // Force set-index bits [9:6] to be identical for both transactions
    (addr_a >> 6) % 16 == (addr_b >> 6) % 16;
    // Or equivalently: addr_a[9:6] == addr_b[9:6];
}
//
// Practical test scenarios:
//   - Fill one set to capacity -> trigger eviction -> test replacement policy
//   - Two cores access same set -> test coherence protocol
//   - Repeated access to same set -> test PLRU update

// (c) PLRU -- Pseudo Least Recently Used
//
// True LRU for an N-way set requires log2(N!) bits per set -- too expensive.
// PLRU approximates LRU with only N-1 bits per set (one bit per internal
// node of a binary tree over the N ways).
//
// Tree-PLRU example (4-way set, 3 bits: b0, b1, b2):
//
//              b0
//             /  \
//           b1    b2
//          /  \  /  \
//        W0  W1 W2  W3
//
// Each bit points toward the LRU side:
//   b0=0 -> left subtree (W0/W1) is LRU candidate
//   b0=1 -> right subtree (W2/W3) is LRU candidate
```

```
//   b1=0 -> W0 is victim,  b1=1 -> W1 is victim
//   b2=0 -> W2 is victim,  b2=1 -> W3 is victim
//
// On ACCESS to way W:
//   Update all bits on the path from root to W, pointing AWAY from W.
//   Example: access W0 -> b0=1 (point away from left), b1=1 (point away from W0)
//
// On EVICTION (need a victim):
//   Follow bits from root, choosing the side each bit points to.
//   Evict the way at the leaf.
//
// PLRU properties:
//   - Only N-1 bits vs N*log2(N) bits for true LRU
//   - Hit rate within 1-5% of true LRU for most workloads
//   - Update logic is simple: XOR of path bits on each access
//   - Used in: Intel L1/L2 caches, ARM Cortex-A series
```

解释

(a) CPU 可以修改 cache 内容，这是 write hit 的正常行为。

write-back 是主流策略（性能好），dirty bit 标记被修改的 cache line，

eviction 时才写回 memory。只读限制由 MMU 实现，与 cache 无关。

(b) 同一 set 的地址：固定 set_index 位段（addr[offset+log2(sets)-1 : offset]）相同即可。

面试考察对 cache 地址划分（tag/index/offset）的掌握。

(c) PLRU 用 N-1 个 bit 的二叉树近似 LRU，每次访问更新路径上所有 bit 指向"远离被访问 way"

的方向，eviction 时顺着 bit 指向找到 victim。

实现简单（纯组合逻辑），性能接近真 LRU，是工业界缓存替换策略的首选。

Q36. Digital Design: Packet Stream Interface Signals

QUESTION

An interface continuously transmits a packet stream where each beat is 1

data word. Besides the data signal, what other signals are needed to

correctly delimit and identify packets?

ANSWER

```
// For a packet-based streaming interface, three control signals are standard:
//
//   valid  -- data is valid this cycle (handshake / flow control)
//   sop    -- Start Of Packet: first data beat of a new packet
//   eop    -- End Of Packet: last data beat of the current packet
//
// Waveform example (3-beat packet followed by 2-beat packet):
//
//   clk:   _|-|_|-|_|-|_|-|_|-|_|-|_|-|_
//   valid: 0  1  1  1  0  1  1  0
//   sop:   0  1  0  0  0  1  0  0
//   eop:   0  0  0  1  0  0  1  0
//   data:  X  A  B  C  X  D  E  X
//
// Rules:
//   sop=1 marks the FIRST beat of a packet (valid must also be 1)
//   eop=1 marks the LAST  beat of a packet (valid must also be 1)
```

```
//   A single-beat packet has both sop=1 and eop=1 on the same cycle
//   Between sop and eop, valid=1 on every beat (no gaps in payload)
//   valid=0 may appear BETWEEN packets (idle), not within a packet

// Optional additional signals depending on protocol:
//
//   ready    -- back-pressure from receiver (AXI-Stream style):
//               data transfer occurs only when valid=1 AND ready=1
//
//   keep[N]  -- byte-enable for the last beat (partial word at eop):
//               e.g. keep=4'b1100 means only bytes[3:2] are valid at eop
//
//   err      -- error flag asserted with eop to signal a bad packet
//               (e.g. CRC error, length error)
//
//   channel  -- identifies which logical stream this packet belongs to
//               (TDM or multi-stream interfaces)

// Interface definition (SystemVerilog):
interface pkt_stream_if #(parameter int DATA_W = 32) (input logic clk);
    logic              valid;    // data beat is valid
    logic              ready;    // receiver can accept (back-pressure)
    logic              sop;      // start of packet
    logic              eop;      // end of packet
    logic [DATA_W-1:0] data;     // payload
    logic [DATA_W/8-1:0] keep;   // byte enables (valid at eop)
    logic              err;      // packet error flag (valid at eop)

    // Clocking block for monitor (sample on posedge)
    clocking mon_cb @(posedge clk);
        input valid, sop, eop, data, keep, err;
        input ready;
    endclocking

    // Clocking block for driver
    clocking drv_cb @(posedge clk);
        output valid, sop, eop, data, keep, err;
        input  ready;
    endclocking
endinterface

// SVA protocol checks for this interface:
//
// 1. sop must be accompanied by valid
property sop_requires_valid;
    @(posedge clk) sop |-> valid;
endproperty
assert property (sop_requires_valid);
//
// 2. eop must be accompanied by valid
property eop_requires_valid;
    @(posedge clk) eop |-> valid;
endproperty
assert property (eop_requires_valid);
//
// 3. No sop without preceding eop (packets must be properly terminated)
property sop_after_eop;
    @(posedge clk)
    (valid && sop && !eop) |->
    !valid or eop throughout (##[1:$] (valid && eop));
endproperty
```

解释

Packet stream 接口的最小信号集：valid + sop + eop + data。

valid 防止接收方误读空闲周期的数据；

sop 标记包头（接收方在此处开始组包）；

eop 标记包尾（接收方在此处完成组包、提交处理）。

没有 sop/eop 的话，接收方无法区分连续传输的两个相邻包，

也无法处理长度不固定的包。

AXI-Stream 是这个模式的工业标准实现，加了 ready 做背压控制，

加了 keep 支持最后一个 beat 的部分有效字节。

面试中能画出波形图（sop/eop 和 data 的时序关系）是加分项。

Q37. Digital Design: Setup/Hold Timing Analysis and Checker

QUESTION

Clock period = 200ps, first rising edge at 100ps.

Signal D goes low at 150ps (was stable high before).

Tsetup = 150ps, Thold = 100ps.

(a) Is there a setup violation? Is there a hold violation?

(b) Write a checker (UVM always block style) to detect setup and hold

violations. The question shows the following approach -- is it correct?

```
// setup checker (from interview)
int time_0 = 0;
always @(posedge clk) begin
    if (($stime - time_0) < T_SETUP)
        `uvm_error(get_type_name(), "setup violation")
end
always @(D) begin
    time_0 = $stime;
end

// hold checker (from interview)
int time_0 = 0;
always @(posedge clk) begin
    time_0 = $stime;
end
always @(D) begin
    if ($stime - time_0 < T_HOLD)
        `uvm_error(get_type_name(), "hold violation")
end
```

ANSWER

```
// (a) Timing analysis
//
// Timeline:
//   t=0ps:    D=1 (initial, stable)
//   t=100ps:  CLK rising edge (1st posedge)
//   t=150ps:  D goes low (D changes)
//   t=200ps:  CLK rising edge (2nd posedge) -- period = 200ps
//
// Setup check (data must be stable T_SETUP BEFORE the clock edge):
```

```
//   Capture edge = 100ps
//   Setup window = [100 - 150, 100] = [-50ps, 100ps]
//   D changes at 150ps -> OUTSIDE setup window
//   NO setup violation.
//
// Hold check (data must be stable T_HOLD AFTER the clock edge):
//   Capture edge = 100ps
//   Hold window  = [100ps, 100 + 100] = [100ps, 200ps]
//   D changes at 150ps -> INSIDE hold window
//   HOLD VIOLATION. (150ps is only 50ps after the edge; need 100ps)

// (b) Checker analysis and fix
//
// Hold checker: CORRECT as written.
//   At posedge (100ps): time_0 = 100.
//   At D change (150ps): 150 - 100 = 50 < 100 (T_HOLD) -> reports violation. Correct.
//
// Setup checker: HAS A BUG.
//   time_0 is initialized to 0, meaning "D last changed at t=0".
//   At first posedge (100ps): $stime - time_0 = 100 - 0 = 100 < 150 (T_SETUP)
//   -> reports SETUP VIOLATION even though D was stable the whole time.
//   This is a FALSE POSITIVE: D never changed before t=100ps.
//
// Fix: initialize time_0 to a large negative value so the first posedge
// always passes the setup check if D has been stable since t=0:

// Corrected setup checker:
longint time_0 = -1_000_000;   // initialize far in the past

always @(posedge clk) begin
    if (($stime - time_0) < T_SETUP)
        `uvm_error(get_type_name(), "setup violation")
end

always @(D) begin
    time_0 = $stime;           // record when D last changed
end

// Hold checker (correct as-is, shown for completeness):
longint time_clk = 0;

always @(posedge clk) begin
    time_clk = $stime;         // record when last posedge occurred
end

always @(D) begin
    if ($stime - time_clk < T_HOLD)
        `uvm_error(get_type_name(), "hold violation")
end

// Alternative: pure SVA (cleaner, tool-native)
// setup: D must be stable T_SETUP before posedge
property p_setup;
    @(posedge clk) $stable(D, T_SETUP * 1ps);
endproperty
assert property (p_setup) else $error("Setup violation");
//
// hold: D must be stable T_HOLD after posedge
property p_hold;
    @(posedge clk) ##1 $stable(D, T_HOLD * 1ps);
endproperty
assert property (p_hold) else $error("Hold violation");
```

解释

(a) 判断违例的关键：把数轴画出来，标出 clk edge 的位置，

再标出 setup 窗口（edge 之前 T_setup 的区间）和 hold 窗口（edge 之后 T_hold 的区间）。

D 在 150ps 变化：距第一个上升沿（100ps）之后 50ps，落在 hold 窗口内 -> hold violation。

D 在 100ps 之后才变化，不在 setup 窗口 -> no setup violation。

(b) Setup checker 的 bug 是面试中的考察点：

time_0 初始化为 0 等价于"D 在 t=0 刚刚变化"，

但实际上 D 从仿真开始就是稳定的，checker 应该只在 D 真正变化后才开始计时。

修复方法：把 time_0 初始化为足够负的值（-T_SETUP - 1 或更小），

使得第一次 posedge 时 ($stime - time_0) 必然大于 T_SETUP。

Hold checker 的逻辑是正确的：在 posedge 记录时间，在 D 变化时检查间隔。

Q38. Digital Design: Implement AND Gate Using 2-to-1 MUX

QUESTION

How do you implement a 2-input AND gate using only a 2-to-1 multiplexer?

ANSWER

```
// 2-to-1 MUX truth table:
//   Y = sel ? in1 : in0
//
// Target: Y = A AND B
//
// Substitution: set sel = A,  in0 = 0,  in1 = B
//   When A=0: Y = in0 = 0        (matches A AND B = 0)
//   When A=1: Y = in1 = B        (matches A AND B = B)
//
// Circuit:
//
//   A ---------- sel
//   B ---------- in1    →  Y = A & B
//   GND -------- in0

// Truth table verification:
//   A  B  | sel  in1  in0 | Y=sel?in1:in0 | A&B
//   0  0  |  0    0    0  |      0        |  0  OK
//   0  1  |  0    1    0  |      0        |  0  OK
//   1  0  |  1    0    0  |      0        |  0  OK
//   1  1  |  1    1    0  |      1        |  1  OK

// SystemVerilog:
// assign Y = A ? B : 1'b0;   // equivalent and synthesizes to AND

// Extension -- other gates using MUX:
//   OR:   Y = A ? 1'b1 : B    (sel=A, in0=B, in1=1)
//   NOT:  Y = A ? 1'b0 : 1'b1 (sel=A, in0=1, in1=0)
//   NAND: Y = A ? ~B : 1'b1
//   XOR:  Y = A ? ~B : B      (sel=A, in0=B, in1=~B)
```

解释

2-to-1 MUX 实现 AND 的关键洞察：MUX 的 sel 输入相当于一个 "gate enable"。

当 sel=A 时：A=0 强制输出 0（in0=GND），A=1 让 B 透传（in1=B）。

这正是 AND 的语义：A=0 时结果恒为0，A=1 时结果等于 B。

面试延伸：同理可以用 MUX 实现 OR（in0=B, in1=1）、NOT（in0=1, in1=0）、XOR（in0=B, in1=~B）。

这类题考察对 MUX 本质（选择器）的理解，以及 Boolean 代数的灵活运用。

Q39. Digital Design: Race Condition in Verilog initial Blocks

QUESTION

What is the output of the following code? Is this a race condition?

How do you fix it?

```
initial begin
    A = 0;
end

initial begin
    A = 1;
end
```

ANSWER

```
// Both initial blocks start at time 0 simultaneously.
// Both contain blocking assignments (=) with no delay.
// Both try to assign different values to A at exactly the same simulation time.
//
// This IS a race condition.
//
// Result: UNDEFINED / NON-DETERMINISTIC.
//   The IEEE 1364/1800 standard does NOT specify which initial block
//   executes first when multiple blocks are scheduled at the same time.
//   Different simulators (or even different runs) may give A=0 or A=1.
//   The final value of A depends on simulator scheduling order.
//
// Why it matters:
//   Race conditions cause non-reproducible simulation results.
//   A test may pass on one simulator and fail on another,
//   or pass today and fail after a tool version update.
//
// How to fix -- consolidate into one initial block:
initial begin
    A = 0;
    // ... other initialization
end
// Remove the second initial block assigning A.
//
// Or use a single centralized reset sequence:
initial begin
    A = 0;          // deterministic: only one driver
    @(posedge clk);
    A = 1;
end
//
// In UVM testbenches: always drive signals through ONE driver task.
// Multiple initial blocks writing the same signal is a testbench bug.

// Related: non-blocking assignment (<=) in always blocks
// also has race conditions if two always blocks write the same variable.
```

```
// always @(posedge clk) A <= 0;
// always @(posedge clk) A <= 1;  // <- also a race, result undefined
```

解释

两个 initial block 在仿真时间 0 同时调度，对同一变量赋不同的值，结果取决于模拟器的执行顺序。

IEEE 标准明确不规定多个 process 在同一时间步的执行顺序，这是"合法的未定义行为"。

实际工程影响：在测试平台中，这种写法会导致 A 的初始值不确定，

可能使测试时而通过时而失败（flaky test），很难调试。

修复原则：同一信号只能有一个驱动源，把所有初始化集中在一个 initial block 里。

Q40. Digital Design: Polymorphism and Inheritance in SystemVerilog/UVM

QUESTION

Explain inheritance and polymorphism in SystemVerilog.

Give a concrete UVM example showing why both are essential.

ANSWER

```
// ---- Inheritance ------------------------------------------------------
// A derived class (child) automatically gets all fields and methods of
// its base class (parent). It can ADD new members and OVERRIDE existing ones.

class animal;
    string name;
    virtual function string speak();
        return "...";
    endfunction
endclass

class dog extends animal;          // dog INHERITS from animal
    int tricks;                    // new field
    virtual function string speak(); // OVERRIDE
        return "Woof";
    endfunction
endclass

// dog has: name (from animal), tricks (own), speak() (overridden)

// ---- Polymorphism -----------------------------------------------------
// A BASE CLASS HANDLE can point to a DERIVED CLASS OBJECT.
// When a virtual method is called through the base handle,
// the DERIVED class version executes (runtime dispatch).

animal a;
a = new dog();         // base handle, derived object
$display(a.speak());   // prints "Woof" -- NOT "..." (runtime dispatch)
// Without virtual: would print "..." (static binding, wrong)

// ---- UVM example: why both matter together ----------------------------

class base_seq extends uvm_sequence #(my_txn);
    `uvm_object_utils(base_seq)
    virtual task body();
        `uvm_do(req)           // default: send one random transaction
    endtask
endclass
```

```
class stress_seq extends base_seq;
    `uvm_object_utils(stress_seq)
    virtual task body();
        repeat(1000) `uvm_do(req)   // override: send 1000 transactions
    endtask
endclass

// In the test: use factory override -- no test code changes needed
// base_seq::type_id::set_type_override(stress_seq::get_type());
//
// Now wherever base_seq is created via type_id::create(), a stress_seq
// object is returned. Because body() is virtual, the stress version runs.
//
// Inheritance provides:  stress_seq reuses all base_seq infrastructure
// Polymorphism provides: caller uses base_seq handle, gets stress behavior
// Together:              factory override + virtual methods = extensible VIP

// Key rule: methods MUST be declared virtual to enable polymorphism.
// Without virtual, calling through a base handle always runs the base version.
```

解释

Inheritance（继承）：子类获得父类所有成员，可以扩展或覆盖。

Polymorphism（多态）：通过父类的 handle 调用方法时，执行的是实际对象的版本。

两者缺一不可：没有继承就没有代码复用；没有多态（virtual）就无法通过父类 handle

调用子类行为，UVM factory override 会完全失效。

UVM 中所有 phase 方法（build_phase、run_phase 等）都是 virtual，

这正是 UVM 框架能自动调用你的 override 版本的原因。

Q41. Digital Design: Timescale Directive -- 1ns/1ps

QUESTION

What does `timescale 1ns/1ps mean in Verilog/SystemVerilog?

How does it affect simulation behavior?

ANSWER

```
// `timescale <time_unit> / <time_precision>
//
// time_unit (1ns):
//   The unit for ALL time literals in this file.
//   #10 means 10 * 1ns = 10ns
//   #0.5 means 0.5 * 1ns = 500ps
//
// time_precision (1ps):
//   The smallest time step the simulator will resolve.
//   All internal time values are rounded to the nearest 1ps.
//   #1.0005 is rounded to #1.001 (nearest 1ps)
//
// Example with `timescale 1ns/1ps:
`timescale 1ns/1ps
// #5        -> 5ns exactly
// #5.5      -> 5.5ns  (= 5500ps, representable at 1ps precision)
// #5.0001   -> 5.0001ns (= 5000.1ps, rounded to 5000ps = 5ns)
// #0.001    -> 0.001ns = 1ps (minimum step)
```

```
//
// Common choices:
//   `timescale 1ns/1ps  : good for GHz designs, 1ps resolution
//   `timescale 1ns/1ns  : coarser, faster simulation
//   `timescale 1ps/1ps  : fine resolution for timing-critical work
//
// Important: timescale is file-scoped.
//   Different files in the same design can have different timescales.
//   This can cause subtle bugs if not managed consistently.
//   Best practice: use a single `timescale in a package or header file.
//
// $timeformat system task: controls how time is displayed
//   $timeformat(-9, 3, "ns", 10);
//   // -9 = nanoseconds, 3 decimal places, suffix "ns", min field width 10
```

解释

time_unit 决定代码里 `#N` 的含义（N 乘以 unit）。

time_precision 决定仿真器内部的时间分辨率（最小可表达的时间步）。

1ns/1ps 是最常用的组合：延迟用纳秒写（符合 GHz 时钟的直觉），

但内部精度是皮秒级，可以正确表示 SDF 时序标注中常见的 ps 级延迟。

面试陷阱：precision 必须 <= unit，`timescale 1ps/1ns 是非法的。

多文件设计中 timescale 不一致会导致延迟计算错误，是一个经典的 bug 来源。

Q42. Digital Design: D Flip-Flop RTL with Power Supply Check

QUESTION

(a) Write a standard D flip-flop in SystemVerilog.

(b) Extend it with vdd and gnd ports: if vdd != 1 or gnd != 0,

```
output X to model power supply failure.
```

ANSWER

```
// (a) Standard DFF with async active-low reset

module dff (
    input  logic clk,
    input  logic rst_n,
    input  logic d,
    output logic q
);
    always_ff @(posedge clk or negedge rst_n)
        if (!rst_n) q <= '0;
        else        q <= d;
endmodule

// (b) DFF with power supply check
// vdd and gnd are modeled as logic signals to detect invalid supply conditions.
// When supply is invalid, output X to propagate the error to downstream logic.

module dff_with_supply (
    input  logic clk,
    input  logic rst_n,
    input  logic d,
    output logic q,
```

```
    input  logic vdd,    // should be 1'b1
    input  logic gnd     // should be 1'b0
);
    logic q_internal;    // clean FF output before supply check

    // Standard FF -- always runs (models the storage element)
    always_ff @(posedge clk or negedge rst_n)
        if (!rst_n) q_internal <= '0;
        else        q_internal <= d;

    // Power supply check -- combinational
    // If supply is invalid, drive X to model metastability / undefined output
    always_comb begin
        if (vdd !== 1'b1 || gnd !== 1'b0)
            q = 'x;
        else
            q = q_internal;
    end

endmodule

// Key points:
//   1. Use !== (case inequality) not != for X/Z-aware comparison.
//      vdd != 1'b1 would treat X and Z as "not equal but valid",
//      while vdd !== 1'b1 is true for any non-1 value including X and Z.
//   2. Separate q_internal (FF logic) from q (output with power check).
//      Do not put power check logic inside always_ff -- that block
//      models synchronous/async register behavior only.
//   3. This pattern is used in gate-level simulation to model cell behavior
//      when power domains are not yet stable (e.g., during power-up).
```

解释

(a) 标准 DFF 三要素：posedge clk 触发、negedge rst_n 异步复位、非阻塞赋值。

(b) VDD/GND 检查的关键细节：

必须用 !== 而非 !=：三值逻辑（0/1/X/Z）中，!= 对 X 的比较结果是 X（不确定），

而 !== 是精确比较，vdd !== 1'b1 在 vdd=X 或 vdd=Z 时返回 1（明确有问题）。

内部 q_internal 和外部 q 分离：FF 的存储行为和电源检查行为分开建模，

always_ff 描述寄存器，always_comb 描述输出逻辑，职责清晰。

Q43. Digital Design: Clock Gating -- Basic and ICG with Negative Latch

QUESTION

(a) What is clock gating and why is it used?

(b) What is wrong with a simple AND gate for clock gating?

(c) Implement a glitch-free clock gate using a negative-edge latch (ICG cell).

ANSWER

```
// (a) Clock gating: disable the clock to a register when it doesn't need to
// update. The register holds its value and consumes zero dynamic power
// (no toggling = no switching current). This is the PRIMARY power reduction
// technique in modern SoC design, saving 20-40% dynamic power on average.

// (b) Problem with simple AND gate:
```

```
//
//   assign gated_clk = clk & enable;
//
//   If enable changes WHILE clk=1, the AND output can produce a GLITCH:
//   a spurious narrow pulse that looks like a valid clock edge to downstream FFs.
//   This causes incorrect data capture -- a functional bug.
//
//   Timeline example:
//   clk:     __|--|__|--|__
//   en:      ____|------
//   gated:   ____|-|__|--|  <- narrow pulse at enable rising edge = GLITCH

// (c) Glitch-free ICG using a negative-edge latch:
//
//   Key idea: sample enable when clk=0 (stable window).
//   The latch is transparent when clk=0 (captures enable).
//   It locks (holds) when clk=1 -- exactly when enable must be stable.
//   This prevents enable transitions from reaching the AND gate during clk=1.

module icg (
    input  logic clk,
    input  logic en,
    output logic gated_clk
);
    logic en_latched;

    // Negative-edge latch: transparent when clk=0, holds when clk=1
    always_latch
        if (~clk) en_latched = en;   // sample en during low phase

    // AND with clk: en_latched is stable when clk=1, so no glitch possible
    assign gated_clk = en_latched & clk;

endmodule

// Timing:
//   clk:        __|--|__|--|__
//   en:         _____|-------   (changes during clk=1 -- normally dangerous)
//   en_latched: ___________|--   (latch locks during clk=1, updates during clk=0)
//   gated_clk:  _________|--|--  (clean, glitch-free)
//
// SDC constraint: create_generated_clock must be added for gated_clk.
// Physical implementation: synthesis maps this to a standard ICG cell
// (e.g., TLATNCAX in many standard cell libraries) rather than
// individual latch + AND gates, for better characterization.

// Why always_latch and not always_ff:
//   A latch is level-sensitive (transparent when enable=1).
//   Here enable = ~clk (low phase).
//   An FF would require an edge trigger and cannot implement this.
```

解释

简单 AND 门时钟门控会产生毛刺（glitch）：enable 在 clk=1 期间变化时，

AND 门输出会有一个短暂的窄脉冲，被下游 FF 误认为时钟边沿，导致数据错误。

ICG（Integrated Clock Gating）的核心思想：用负沿锁存器把 enable 采样在 clk=0 期间，

当 clk=1 时锁存器输出锁定，AND 门的一个输入稳定，输出不会有毛刺。

always_latch + if(~clk) 是 ICG 的标准 RTL 写法，会被综合工具识别并映射到工艺库中的 ICG cell。

ICG 是数字 SoC 设计中最重要的功耗优化手段，大型 CPU 内部有数万个 ICG cell。

Q44. Digital Design: Voting Machine -- 3 Judges + Super Judge Veto

QUESTION

Design combinational RTL for a talent show voting machine:

- 3 independent judges (j1, j2, j3): participant advances if >= 2 vote yes.

- 1 super judge (s): can VETO (reject) an otherwise passing result,

but CANNOT override a panel rejection (reject-only power).

Output: advance = 1 if participant advances.

ANSWER

```
module voting_machine (
    input  logic j1, j2, j3,   // panel judges: 1=yes, 0=no
    input  logic s,             // super judge: 1=veto (reject), 0=no veto
    output logic advance        // 1 = participant advances
);
    logic panel_pass;   // majority of 3 judges voted yes

    // Panel result: at least 2 of 3 judges vote yes (majority logic)
    assign panel_pass = (j1 & j2) | (j1 & j3) | (j2 & j3);

    // Super judge can only veto a panel pass, not create a pass
    assign advance = panel_pass & ~s;

endmodule

// Truth table (only panel_pass=1 rows shown):
// j1 j2 j3  s | panel_pass | advance
//  0  1  1  0 |     1      |    1
//  0  1  1  1 |     1      |    0   <- super judge vetoes
//  1  0  1  0 |     1      |    1
//  1  0  1  1 |     1      |    0   <- super judge vetoes
//  1  1  0  0 |     1      |    1
//  1  1  0  1 |     1      |    0   <- super judge vetoes
//  1  1  1  0 |     1      |    1
//  1  1  1  1 |     1      |    0   <- super judge vetoes
// (all rows where panel_pass=0: advance=0 regardless of s)

// One-liner equivalent:
// assign advance = ((j1&j2)|(j1&j3)|(j2&j3)) & ~s;

// Extension: add a "super approve" power (override a panel rejection):
// assign advance = panel_pass & ~s_veto | s_approve;
// (not in this problem -- super judge is reject-only)
```

解释

两行逻辑，两步推导：

第一步：panel_pass = 多数表决，3个中至少2个同意。

展开为3个AND-OR对：(j1&j2)|(j1&j3)|(j2&j3)，覆盖所有2-of-3组合。

第二步：advance = panel_pass AND NOT veto。

super judge 的"reject-only"语义直接映射为一个 AND NOT：

super judge 只能把 1 变成 0，不能把 0 变成 1。

面试中关键是正确理解"reject-only"的约束，不要多加一个 OR 让 super judge 有 override 能力。

Q45. Digital Design: NAND Gate -- CMOS, Stick Diagram, EM (Designer Question)

QUESTION

(a) Draw the CMOS transistor-level implementation of a 2-input NAND gate.

Also draw the corresponding stick diagram.

(b) Follow-up: IN_B arrives later than IN_A. How can you improve performance?

(c) Follow-up: Each MOS device has 20 fingers. Where does electromigration

(EM) occur? How can layout be optimized?

ANSWER

```
// (a) CMOS NAND Gate -- Transistor Level
//
// Pull-Up Network (PMOS, in parallel):
//   MP1: source=VDD, gate=IN_A, drain=OUT
//   MP2: source=VDD, gate=IN_B, drain=OUT
//
// Pull-Down Network (NMOS, in series):
//   MN1: source=VSS, gate=IN_A, drain=internal node X
//   MN2: source=X,   gate=IN_B, drain=OUT
//
//         VDD
//          |
//    +-----+-----+
//    |           |
//  [MP1]       [MP2]     <- PMOS in PARALLEL
//  IN_A        IN_B
//    |           |
//    +-----+-----+
//          |
//         OUT
//          |
//        [MN2]  <- NMOS series: IN_B (closer to output)
//          |
//         [X]
//          |
//        [MN1]  <- NMOS series: IN_A (closer to VSS)
//          |
//         VSS
//
// NAND truth table:
//   IN_A=0, IN_B=0 -> MP1 ON, MP2 ON, MN1 OFF -> OUT=1
//   IN_A=0, IN_B=1 -> MP1 ON, MN1 OFF         -> OUT=1
//   IN_A=1, IN_B=0 -> MP2 ON, MN2 OFF         -> OUT=1
//   IN_A=1, IN_B=1 -> both NMOS ON, both PMOS OFF -> OUT=0
//
// Stick Diagram:
//   Vertical rails: VDD (top), VSS (bottom)
//   Horizontal polysilicon gates (green): IN_A, IN_B
//   PMOS diffusion (red, top region): MP1, MP2 in parallel
//   NMOS diffusion (blue, bottom region): MN1, MN2 in series
//   Metal connections: OUT, internal node X

// (b) IN_B arrives late -> optimize by placing IN_B closer to OUTPUT
//
// In the NMOS pull-down stack, the transistor closest to the OUTPUT
// node sees the LARGEST voltage swing (from OUT to VDD).
// The transistor closest to VSS is pre-discharged before IN_B arrives.
//
```

```
// Rule: put the LATE-arriving input CLOSER to the output node.
//   Current (bad):  MN1(IN_A) -> MN2(IN_B) -> OUT
//   Optimized:      MN1(IN_B) -> MN2(IN_A) -> OUT
//   Wait -- this is backwards. Put IN_B (late) at the top of stack (near OUT):
//   Optimized:  VSS -> MN1(IN_A) -> MN2(IN_B) -> OUT
//
// Why? When IN_A=1 already (arrives early), node X is pre-pulled toward VSS.
// When IN_B finally arrives, it only needs to pull OUT down through MN2,
// starting from an already-low node X. Less charge to discharge -> faster.
//
// In PMOS: order does not matter much (parallel, no series stack effect).

// (c) Electromigration with 20 fingers per device
//
// EM occurs where current DENSITY is highest.
// With 20-finger devices, total current is large.
// The OUTPUT node (OUT) is the collection point for ALL parallel PMOS
// currents AND series NMOS currents.
//
// EM most likely at: the OUTPUT metal wire and via.
//   - Both MP1 and MP2 (PMOS, 20 fingers each) dump current into OUT
//   - Total PMOS current = 20 fingers x 2 devices = 40x single-finger current
//   - Single output wire/via carries ALL of this current -> high J (current density)
//
// Layout optimization:
//   - Add MULTIPLE output ports (taps): spread the output current over
//     several metal connections instead of one narrow wire/via
//   - Use wider metal straps on the output rail
//   - Add multiple vias (via arrays) instead of single via
//   - Route output in upper metal layers (lower resistance per unit length)
//   - Distribute output contacts along the entire diffusion length
```

解释

CMOS NAND 的核心：PMOS 并联（任一输入为0即拉高），NMOS 串联（两个输入都为1才拉低）。

晚到信号放靠近输出端：系列 NMOS 堆中，靠近 OUT 的管子等待早到信号先把中间节点放电，

晚到信号到来时只需完成最后一段放电，减少了关键路径延迟。

EM 在输出节点：多指器件的大电流全部汇集到同一根输出金属线，电流密度最高，

解决方案是拉多个输出 port（output tap）分散电流，相当于并联多条金属路径。

Q46. Digital Design: CMOS DFF (Master-Slave Latch) with Tri-State Buffer (Designer Question)

QUESTION

(a) Draw the CMOS transistor-level D flip-flop as a master-slave latch,

using tri-state buffers (transmission gate + inverter). Show the clock

connections clearly -- are the two tri-state buffer clocks the same?

(b) Follow-up: Which component delays contribute to setup time? Hold time?

(c) Follow-up: Replace the first tri-state buffer with a plain inverter.

Does setup time increase or decrease?

(d) Follow-up: How do you reduce setup time in the DFF itself?

What is clock skew and how does it help?

ANSWER

```
// (a) Master-Slave DFF with tri-state buffers
//
// MASTER LATCH (transparent when CLK=0):
//   D -> [TG1: CLK=0 transparent] -> M -> [INV1] -> Mbar
//        [TG2: CLK=1 feedback]    <-  M (holds M when CLK=1)
//
// SLAVE LATCH (transparent when CLK=1):
//   Mbar -> [TG3: CLK=1 transparent] -> S -> [INV2] -> Q
//           [TG4: CLK=0 feedback]    <- S (holds S when CLK=0)
//
// Tri-state buffer = transmission gate (NMOS + PMOS) + inverter
//
//   TG structure:
//     IN ---|NMOS|--- OUT     gate of NMOS = CLK
//     IN ---|PMOS|--- OUT     gate of PMOS = CLKb (inverted CLK)
//   Transparent when CLK=1 (NMOS on) and CLKb=0 (PMOS on)
//
// Clock connections:
//   Master TG1: transparent when CLK=0  -> uses CLKb=1 for NMOS, CLK=0 for PMOS
//   Slave  TG3: transparent when CLK=1  -> uses CLK=1 for NMOS, CLKb=0 for PMOS
//
//   ARE THEY THE SAME? NO.
//   TG1 (master input) is transparent on CLK LOW phase.
//   TG3 (slave input)  is transparent on CLK HIGH phase.
//   They use COMPLEMENTARY clock phases: CLK and CLKb.
//   This ensures master captures data BEFORE the clock edge,
//   and slave propagates to Q AFTER the clock edge.

// (b) Setup time and hold time component delays
//
// Setup time (T_setup):
//   Data must be stable BEFORE the clock edge so master latch captures it.
//   T_setup = propagation delay through the master latch input path:
//     - TG1 (transmission gate) delay: t_TG1
//     - INV1 (inverter in master) delay: t_INV1
//   T_setup ~ t_TG1 + t_INV1
//   Data must arrive early enough for these delays to resolve before posedge.
//
// Hold time (T_hold):
//   Data must remain stable AFTER the clock edge so master latch doesn't
//   accidentally re-sample with the wrong value.
//   T_hold = time for clock edge to propagate and close TG1:
//     - Inverter delay to generate CLKb: t_INVclk
//     - TG1 turn-off time (transmission gate closing): t_TG_off
//   T_hold ~ t_INVclk + t_TG_off  (often very small or negative)

// (c) Replace first tri-state buffer with plain inverter -- setup time change?
//
// Original master input: D -> TG1 (clocked) -> INV1
// Modified:              D -> INV1 (always on) -> [node M]
//
// Effect on setup time:
//   The plain inverter propagates data IMMEDIATELY, no clock gating.
//   But now there is NO isolation between D and M before the clock edge.
//   The master no longer has a controlled transparent window.
//
//   For SETUP TIME specifically:
//   The data path to M is now just INV1 (no TG delay).
//   T_setup_new ~ t_INV1 (vs t_TG1 + t_INV1 before)
//   SETUP TIME DECREASES -- data reaches M faster.
//
//   BUT: the latch loses proper hold behavior and isolation.
//   This is why real DFFs use transmission gates, not plain inverters,
```

```
//   in the input stage -- the hold time would become very problematic.

// (d) Reducing setup time -- clock skew technique
//
// Method 1: Reduce transistor delays in the master path
//   - Use stronger (wider) transistors in TG1 and INV1
//   - Use high-Vt devices only where leakage matters, low-Vt on critical path
//
// Method 2: Clock skew (intentional timing adjustment)
//   Introduce a small delay (skew) on the clock to the FF:
//   - POSITIVE skew: destination FF clock arrives LATER than source FF
//     -> effectively "lends" time from hold budget to setup budget
//     -> reduces the effective setup time requirement from the system perspective
//
// Method 3: Skew inside the DFF -- tri-state buffer / pass transistor sizing
//   - Make the NMOS in TG1 stronger (asymmetric sizing): faster rising edge
//   - Skew the clock inverter: make CLKb generation faster so TG1 opens earlier
//   - In pass-transistor implementation: size the pass transistor to reduce
//     its on-resistance, lowering the RC delay on the data path
//   - Reduce the inverter chain driving CLK inside the FF to minimize
//     internal clock delay contribution to setup time
```

解释

Master-Slave DFF 的时钟对称性是关键考点：

主锁存器（master）在 CLK=0 时透明，从锁存器（slave）在 CLK=1 时透明，

两个三态缓冲器使用互补时钟相位，绝对不是同一个相位。

Setup time 对应主锁存器输入路径（TG1 + INV1）的延迟，

Hold time 对应时钟关闭 TG1 所需的时间（CLK 反相器 + TG 关闭时间）。

把 TG1 换成普通反相器：setup time 减小（少了 TG 延迟），但失去了数据隔离。

Clock skew 优化：对目标 FF 引入正向偏斜（destination clock 略晚），

从系统层面"借用" hold time 余量来弥补 setup time 不足。

Q47. Digital Design: What Does This Circuit Do? (Designer Question)

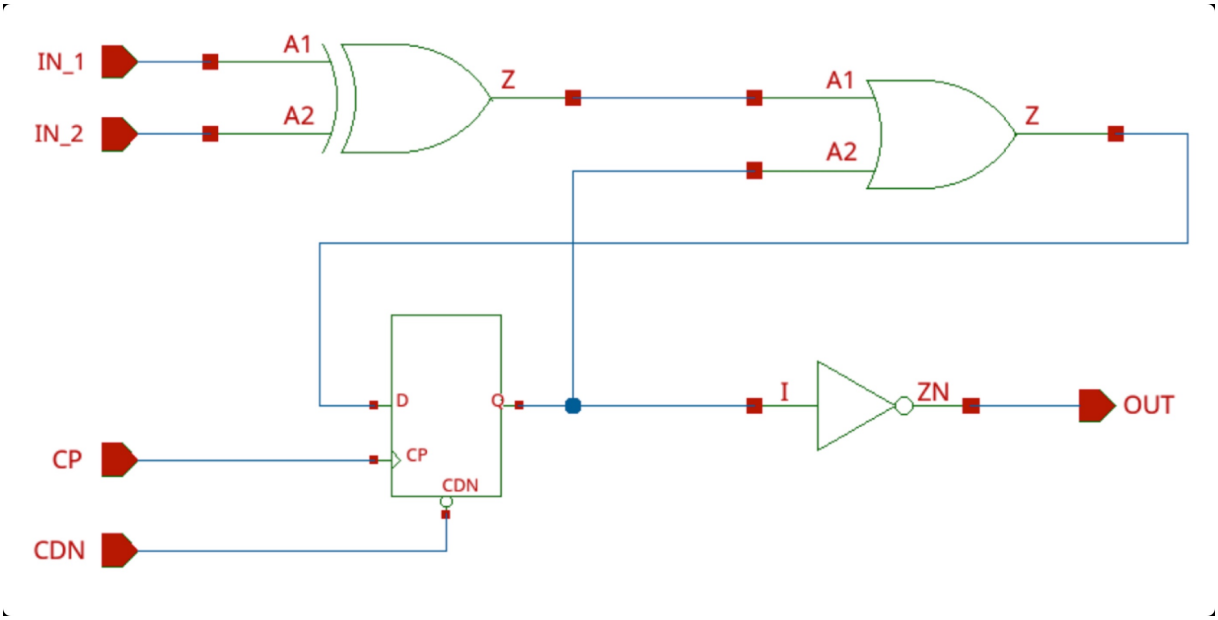
QUESTION

Analyze the circuit shown in the image. What does it do?

Key components: two OR gates, a D flip-flop (clocked by CP, async clear CDN),

and an inverter. Describe the function and give the truth table.

ANSWER




```
// Circuit topology (from the schematic):
//
//   IN_1 -+
//         +-[OR1]--- Z1 -+--- D -[DFF]- Q --- I -[INV]- ZN --- OUT
//   IN_2 -+              |             |     (CP clock, CDN clear)
//                        +-[OR2]-- Z2  |
//                  Q ---+              +- feedback to OR2 A2 input
//                  (Q also feeds A2 of second OR)
//
// Logic equations:
//   Z1  = IN_1 | IN_2            (first OR gate)
//   D   = Z1   = IN_1 | IN_2
//   Q+  = D    = IN_1 | IN_2     (registered on posedge CP)
//   Z2  = Z1 | Q                 (second OR with Q feedback -- has memory)
//   OUT = ~Q                     (inverter on Q output)
//
// Truth table (what OUT looks like after posedge CP):
//
//   IN_1  IN_2 | Z1(=D) | Q_next | OUT_next | Description
//   ------+-----+--------+---------+----------+------------------------
//     0     0  |   0    |    0    |    1     | both low -> OUT high
//     0     1  |   1    |    1    |    0     | IN_2 high -> OUT low
//     1     0  |   1    |    1    |    0     | IN_1 high -> OUT low
//     1     1  |   1    |    1    |    0     | both high -> OUT low
//
// Function:
//   OUT = ~Q = ~(IN_1 | IN_2)_registered = registered NOR(IN_1, IN_2)
//
//   The circuit is a REGISTERED NOR (or equivalently: a NAND with
//   active-low inputs, registered through a DFF).
//
//   OUT = 1  ONLY when BOTH IN_1 and IN_2 are 0 at the clock edge.
//   OUT = 0  when EITHER OR BOTH inputs are 1.
//
// The "detect when IN_1 != IN_2" aspect:
//   When IN_1 != IN_2: one is 1, one is 0 -> OR=1 -> Q=1 -> OUT=0
//   When IN_1 == IN_2 == 0: OR=0 -> Q=0 -> OUT=1 (the "all-clear" state)
//   When IN_1 == IN_2 == 1: OR=1 -> Q=1 -> OUT=0
//
//   CDN (async clear): resets Q=0 immediately -> OUT=1 (clears the "alarm")
//   The DFF acts as a STICKY FLAG: once triggered (either input goes high),
```

```
//   OUT stays low until CDN clears it, regardless of subsequent input values.
//
//   The second OR gate (Z2 = Z1 | Q) with Q feedback creates a SET-ONLY path:
//   once Q=1, Z2=1 regardless of Z1 -- but Z2 is NOT connected back to D in
//   this circuit. Z2 appears to be an auxiliary output (not shown driving anything
//   else), making the primary function: registered NOR with async-clear sticky flag.
```

解释

电路本质是一个带 DFF 的 NOR 检测器（registered NOR gate）：

两个输入经过 OR 门后进入 DFF，Q 输出再经反相器得到 OUT。

OUT=1 当且仅当前一个时钟周期 IN_1=IN_2=0（两者都低）。

CDN 信号（异步清零）将 DFF 强制清零，OUT 立即回到 1，相当于"复位报警"。

当 IN_1 ≠ IN_2 时（一高一低），OR=1，Q=1，OUT=0，确实可以"探测到有输入被拉高"。

但电路无法区分"一高一低"和"两个都高"——更准确的描述是：

这是一个粘性检测器（sticky detector），一旦任意输入为 1 则 OUT 变低并保持，

直到 CDN 复位。第二个 OR 门（Z2 = Z1|Q）提供额外的门控输出，带 Q 的记忆反馈。

Q48. Digital Design: Sequence Detector FSM (1011 Pattern)

QUESTION

Design a Mealy FSM to detect the sequence "1011" in a serial bit stream.

The FSM asserts detected=1 for one cycle when the pattern is found.

Overlapping detection is allowed (the last bits of one match can start the next).

Write the RTL and explain the state transitions.

ANSWER

```
// States: track how many bits of "1011" have been matched so far
// S0: no match (idle)
// S1: matched "1"
// S2: matched "10"
// S3: matched "101"
// On detecting "1011": assert detected, return to appropriate state

module seq_detect_1011 (
    input  logic clk,
    input  logic rst_n,
    input  logic din,
    output logic detected
);
    typedef enum logic [1:0] {S0, S1, S2, S3} state_e;
    state_e state, next;

    // Next-state logic (Mealy: output depends on state AND input)
    always_comb begin
        next     = S0;
        detected = 0;
        case (state)
            S0: next = din ? S1 : S0;          // got '1' -> S1
            S1: next = din ? S1 : S2;          // got '0' -> S2; '1' stays S1
            S2: next = din ? S3 : S0;          // got '1' -> S3
            S3: begin
```

```
                if (din) begin
                    detected = 1;              // completed "1011"!
                    next     = S1;             // overlapping: last '1' starts new seq
                end else begin
                    next = S2;                 // "10110" -> "10" matches prefix
                end
            end
        endcase
    end

    always_ff @(posedge clk or negedge rst_n)
        if (!rst_n) state <= S0;
        else        state <= next;

endmodule

// State transition table:
//   State | Input=0 | Input=1  | detected
//   ------+---------+----------+---------
//   S0    |   S0    |   S1     |   0
//   S1    |   S2    |   S1     |   0
//   S2    |   S0    |   S3     |   0
//   S3    |   S2    |   S1*    |   1*   (* = on input=1)
//
// Example stream: 1 0 1 1 0 1 1
//   S0->S1->S2->S3->S1(det)->S2->S3->S1(det)
//   Detections at positions 4 and 7

// FSM design principles:
// Mealy vs Moore:
//   Mealy: output = f(state, input) -> 1 cycle earlier response
//   Moore: output = f(state) only -> simpler but 1 cycle delayed
// Overlapping: S3+1 -> S1 (reuse the '1' as start of new sequence)
// Non-overlapping: S3+1 -> S0 (discard, restart fresh)
```

解释

序列检测器的状态数等于被检测序列的长度（4位序列 -> 4个状态 S0-S3）。

每个状态表示"已经匹配了序列的前N位"。

关键在于处理重叠：S3 收到最后一个 '1' 后，这个 '1' 同时是下一个序列的开头，

所以跳转到 S1 而非 S0。

失败跳转需要仔细考虑前缀匹配：S3 收到 '0' 后已有 "1010"，

其中后缀 "10" 匹配序列前缀，所以跳到 S2 而非 S0。

Q49. Digital Design: Minimum FIFO Depth Calculation

QUESTION

A producer writes for exactly 20 cycles out of every 100 cycles.

A consumer reads for exactly 20 cycles out of every 100 cycles.

What is the minimum FIFO depth needed?

Analyze different timing scenarios.

ANSWER

```
// Given: 100-cycle period, producer writes 20 cycles, consumer reads 20 cycles.
```

```
// Net data: 20 writes, 20 reads per 100 cycles -> balanced on average.
// FIFO depth depends on the WORST-CASE TIMING of writes vs reads.
//
// Case 1: Producer writes FIRST, consumer reads SECOND (worst case for depth)
//   Cycles 1-20:  write 20 items, consumer idle  -> FIFO fills to 20
//   Cycles 21-40: consumer reads 20 items        -> FIFO drains to 0
//   Minimum depth = 20
//
// Case 2: Consumer reads FIRST, producer writes SECOND
//   Cycles 1-20:  consumer tries to read, FIFO empty -> underflow problem!
//   Solution: ensure FIFO is pre-filled, or accept that reads are stalled.
//   If reads can stall: producer writes first in next period, depth = 20.
//   If reads cannot stall: need 20 items pre-loaded -> depth >= 20.
//
// Case 3: Producer and consumer overlap (both active simultaneously)
//   If write rate == read rate (1 item/cycle each during active periods):
//   Net accumulation = 0 during overlap. Depth = 20 (non-overlapping portion).
//   Worst: producer writes 20 items before consumer starts -> depth = 20.
//
// Case 4: Partial overlap
//   Producer cycles 1-20, consumer cycles 11-30 (10-cycle overlap):
//   Cycles 1-10:  write only -> FIFO grows to 10
//   Cycles 11-20: write and read simultaneously -> net 0, stays at 10
//   Cycles 21-30: read only -> FIFO drains to 0
//   Minimum depth = 10 (overlap reduces required depth!)
//
// General formula:
//   max_depth = max(0, write_cycles_before_any_read)
//             = write_cycles - max(0, overlap_cycles)
//
// Answer for the canonical case (write then read, no overlap):
//   Minimum FIFO depth = 20
```

解释

FIFO 深度 = 最坏情况下的最大积压量。

关键变量：写开始时间、读开始时间、两者的重叠程度。

最坏情况：写全部完成后读才开始（零重叠），深度 = 写入总量 = 20。

最优情况：写读完全重叠且速率相同，深度 = 1（只需缓冲一拍的延迟）。

面试中主动分情况讨论（worst/best/partial overlap）体现对 FIFO 设计的系统理解。

Q50. Digital Design: Blocking vs NBA Pipeline -- A=in, B=A, C=B

QUESTION

Given the following code with input = 0, 1, 2, 3, 4, 5 each cycle:

A = input; B = A; C = B;

(a) What are the values of A, B, C each cycle?

(b) Change to non-blocking assignments. What changes? What is the hidden bug?

```
// Blocking version
always @(posedge clk) begin
    A = input;
    B = A;
    C = B;
end
```

```
// Non-blocking version
always_ff @(posedge clk) begin
    A <= input;
    B <= A;
    C <= B;
end
```

ANSWER

```
// (a) Blocking: A=input, B=A, C=B execute sequentially in the same time step.
// Each statement sees the ALREADY-UPDATED value from the line above.
// Result: A, B, C all get the SAME value as input in the same cycle.
//
//   Cycle | input | A | B | C
//   ------+-------+---+---+---
//     0   |   0   | 0 | 0 | 0   <- all track input instantly
//     1   |   1   | 1 | 1 | 1
//     2   |   2   | 2 | 2 | 2
//     3   |   3   | 3 | 3 | 3
//
// (b) Non-blocking: all RHS evaluated SIMULTANEOUSLY using OLD values,
// then all LHS updated together at end of time step.
// Result: A, B, C form a 3-stage SHIFT REGISTER -- values propagate by 1 stage per cycle.
//
//   Cycle | input | A | B | C    (initial A=B=C=X)
//   ------+-------+---+---+---
//     0   |   0   | 0 | X | X   <- B=old_A=X, C=old_B=X
//     1   |   1   | 1 | 0 | X   <- B=old_A=0, C=old_B=X
//     2   |   2   | 2 | 1 | 0   <- B=old_A=1, C=old_B=0
//     3   |   3   | 3 | 2 | 1
//     4   |   4   | 4 | 3 | 2
//     5   |   5   | 5 | 4 | 3
//
// HIDDEN BUG: A, B, C are not initialized!
// Their initial value is X (unknown) in simulation.
// The first 2 cycles produce X on B and C respectively.
// This is a real hardware concern: without a reset, the pipeline
// propagates X until flushed. Fix: add async/sync reset to set A=B=C=0.
//
// always_ff @(posedge clk or negedge rst_n) begin
//     if (!rst_n) begin A <= 0; B <= 0; C <= 0; end
//     else        begin A <= input; B <= A; C <= B; end
// end
```

解释

Blocking（=）在同一 time step 内顺序执行：B 看到已更新的 A，C 看到已更新的 B，

结果是三个变量在同一拍都等于 input，相当于三条"直连线"而非寄存器。

Non-blocking（<=）所有 RHS 使用旧值，所有 LHS 同时更新：

B 得到上一拍的 A，C 得到上一拍的 B，形成标准的三级移位寄存器。

隐藏 Bug：初始值为 X。没有复位的情况下 B 在第 0 拍、C 在第 0 和 1 拍都是 X，

X 会向下游传播。这是实际 RTL 中必须加复位的原因。

Q51. Digital Design: Error Window FSM with 64-Cycle Counter

QUESTION

Design a state machine with:

- input: signal pulse (single-cycle)

- outputs: error_assert, clear_flag

Behavior: after the first pulse, if a second pulse arrives within

64 cycles, assert error_assert. Hold until clear_flag is received.

Otherwise (no second pulse within 64 cycles), return to IDLE.

ANSWER

```
// States:
//   IDLE:         waiting for first pulse, counter=0
//   ERROR_WINDOW: received first pulse, counting cycles
//   ERROR_ASSERT: second pulse arrived within 64 cycles, error_assert=1

module error_window_fsm (
    input  logic clk,
    input  logic rst_n,
    input  logic signal_in,    // single-cycle pulse
    input  logic clear_flag,   // clear the error
    output logic error_assert
);
    typedef enum logic [1:0] {IDLE, ERROR_WINDOW, ERROR_ASSERT} state_e;
    state_e state, next;

    logic [6:0] counter;   // 7 bits: counts 0..63

    // Next-state logic
    always_comb begin
        next = state;
        case (state)
            IDLE: begin
                if (signal_in) next = ERROR_WINDOW;
            end

            ERROR_WINDOW: begin
                if (clear_flag)  next = IDLE;       // clear takes priority
                else if (signal_in) next = ERROR_ASSERT; // second pulse -> error
                else if (counter >= 7'd63) next = IDLE;  // timeout, no second pulse
            end

            ERROR_ASSERT: begin
                if (clear_flag) next = IDLE;         // only way out
                // ignore further signal_in while in ERROR_ASSERT
            end
        endcase
    end

    // State and counter register
    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            state   <= IDLE;
            counter <= '0;
        end else begin
            state <= next;
            case (state)
                IDLE:         counter <= '0;
                ERROR_WINDOW: counter <= counter + 1'b1;
                ERROR_ASSERT: counter <= '0;
            endcase
        end
    end
```

```
    // Output: error_assert is a Moore output (depends only on state)
    assign error_assert = (state == ERROR_ASSERT);

endmodule

// State transition summary:
//   IDLE         + signal_in          -> ERROR_WINDOW  (start 64-cycle window)
//   ERROR_WINDOW + signal_in (< 64)   -> ERROR_ASSERT  (second pulse: error!)
//   ERROR_WINDOW + counter >= 63      -> IDLE           (timeout: no error)
//   ERROR_WINDOW + clear_flag         -> IDLE           (user cleared early)
//   ERROR_ASSERT + clear_flag         -> IDLE           (user clears error)
//   ERROR_ASSERT + signal_in          -> ignore         (already in error)
```

解释

三状态设计：IDLE（等待）-> ERROR_WINDOW（开始倒计时）-> ERROR_ASSERT（报错锁存）。

counter 在 ERROR_WINDOW 每拍 +1，到 63（共 64 拍）时超时返回 IDLE。

clear_flag 只在 ERROR_ASSERT 有效（从报错中恢复），在其他状态忽略。

error_assert 是 Moore 输出（只取决于 state），状态清晰，无毛刺。

面试考点：正确处理 clear_flag 的优先级（在 ERROR_WINDOW 也要能清除）。

Q52. Digital Design: C++ Bit Manipulation -- Set/Clear Nth Bit

QUESTION

(a) Given an integer number, write C/C++ to clear (set to 0) the nth bit.

(b) What happens if n >= 32 for a 32-bit integer?

(c) Write operations to: set bit n to 1, toggle bit n, and read bit n.

ANSWER

```
// (a) Clear bit n (set to 0):
number = number & ~(1 << n);
//
// How it works:
//   1 << n:   bitmask with only bit n set (e.g. n=3: 0b00001000)
//   ~(1<<n):  invert: all bits 1 EXCEPT bit n (e.g.: 0b11110111)
//   & mask:   AND clears bit n, leaves all other bits unchanged
//
// Example: number=0xFF (all 1s), n=3
//   1 << 3    = 0b00001000
//   ~(1 << 3) = 0b11110111
//   0xFF & 0b11110111 = 0b11110111 = 0xF7

// (b) Undefined behavior when n >= 32 (for 32-bit int)
//
// In C and C++, left-shifting a 32-bit value by >= 32 bits is
// UNDEFINED BEHAVIOR per the language standard (ISO C11 6.5.7).
// The compiler does NOT warn about this -- it silently produces
// unpredictable results (could be 0, original value, or garbage).
//
// Safe version with bounds check:
// if (n >= 0 && n < 32)
//     number = number & ~(1 << n);
//
// Or use 64-bit shift for safety:
// number = number & ~(1LL << n);   // 1LL is a 64-bit literal
```

```
// (c) Complete bit manipulation toolkit:
//
// Set bit n to 1:
number = number | (1 << n);
//   OR with mask sets bit n, leaves others unchanged
//
// Toggle bit n (flip 0->1 or 1->0):
number = number ^ (1 << n);
//   XOR with mask flips bit n (XOR with 1 = invert)
//
// Read bit n (returns 0 or 1):
bit_value = (number >> n) & 1;
//   Shift bit n to position 0, mask off all others
//   Alternative: (number & (1 << n)) != 0
//
// Summary table:
//   Operation     | Code
//   --------------+------------------------
//   Clear bit n   | num &= ~(1 << n)
//   Set bit n     | num |= (1 << n)
//   Toggle bit n  | num ^= (1 << n)
//   Read bit n    | (num >> n) & 1
```

解释

位操作的三个核心工具：AND（清零）、OR（置1）、XOR（翻转）。

清零掩码 = ~(1<<n)：只有第 n 位是0，其他全1；AND 后只清除目标位。

置1掩码 = (1<<n)：只有第 n 位是1；OR 后只设置目标位。

XOR 掩码 = (1<<n)：XOR 1 = 取反，XOR 0 = 保持。

Undefined behavior 是 C/C++ 面试的经典陷阱：编译器不报错，但结果不可预测。

生产代码必须加 n < sizeof(int)*8 的边界检查，或使用 1ULL 确保64位运算。

Q53. Digital Design: Vending Machine FSM -- Requirements and State Design

QUESTION

Design a vending machine FSM (open-ended discussion question):

- 2 products: item A = 15 cents, item B = 25 cents

- Coins: 5-cent and 20-cent only

- Change: can dispense 5, 10, 15, 20 cents

First clarify requirements with the interviewer, then design the states

and transitions. Discuss Mealy vs Moore machine implications.

ANSWER

```
// Step 1: Clarify requirements (ask the interviewer)
//
//   Q1: Does user select product BEFORE or AFTER inserting coins?
//   -> Assume: SELECT PRODUCT FIRST (simplifies design -- target price known)
//
//   Q2: What if user inserts too much?
//   -> Dispense change, prioritize largest coin denomination.
//
```

```
//   Q3: Can user cancel and get refund?
//   -> Out of scope for this design.

// Step 2: Possible balances for each product
//
// Product A (15 cents), coins: 5 or 20
//   Reachable balances: 0, 5, 20, 10, 25, 15
//   States for A: A0(0c), A5(5c), A20(20c) [overpaid], A15(done)
//   Actually: after selection, track inserted amount:
//     0  +5=5  +5=10 +5=15 [dispense A]
//     0  +20=20 [dispense A + 5c change]
//
// Product B (25 cents), coins: 5 or 20
//   0  +5=5  +5=10 +5=15 +5=20 +5=25 [dispense B]
//   0  +20=20 +5=25 [dispense B]
//   0  +5=5  +20=25 [dispense B]
//   0  +20=20 +20=40 [dispense B + 15c change]

// Step 3: State encoding
//
typedef enum logic [3:0] {
    IDLE     = 4'd0,   // no product selected, no coins
    A_0      = 4'd1,   // product A selected, balance=0
    A_5      = 4'd2,   // product A selected, balance=5c
    A_DONE   = 4'd3,   // product A: dispense + change
    B_0      = 4'd4,   // product B selected, balance=0
    B_5      = 4'd5,   // product B selected, balance=5c
    B_10     = 4'd6,   // product B selected, balance=10c
    B_15     = 4'd7,   // product B selected, balance=15c
    B_20     = 4'd8,   // product B selected, balance=20c
    B_DONE   = 4'd9    // product B: dispense + change
} state_e;

// Step 4: Key state transitions
//
// IDLE + select_A -> A_0
// IDLE + select_B -> B_0
//
// A_0  + coin_5  -> A_5
// A_0  + coin_20 -> A_DONE (change = 5c)
// A_5  + coin_5  -> A_DONE (exact: 5+5+5=15 needs 3 coins, but 5+5=10, need one more)
// Actually: 5->10->A_DONE or 5->A_DONE_via_20(change=10)?
// Let's enumerate for A (target 15c):
//   A_0  +5  -> A_5
//   A_5  +5  -> A_10
//   A_10 +5  -> A_DONE (0 change)
//   A_0  +20 -> A_DONE (5c change)
//   A_5  +20 -> A_DONE (10c change)
//   A_10 +20 -> A_DONE (15c change)
//
// For B (target 25c):
//   B_0  +5  -> B_5
//   B_5  +5  -> B_10
//   B_10 +5  -> B_15
//   B_15 +5  -> B_20
//   B_20 +5  -> B_DONE (0 change)
//   B_0  +20 -> B_20
//   B_5  +20 -> B_DONE (0 change)
//   B_10 +20 -> B_DONE (5c change)
//   B_15 +20 -> B_DONE (10c change)
//   B_20 +20 -> B_DONE (15c change)

// Step 5: Mealy vs Moore discussion
```

```
//
// Moore machine:
//   Output depends ONLY on state.
//   DISPENSE output is in a dedicated state (A_DONE, B_DONE).
//   Advantage: cleaner state diagram, output is glitch-free.
//   Disadvantage: one extra state needed for the "dispense" action.
//
// Mealy machine:
//   Output depends on state AND input.
//   DISPENSE output can be triggered ON THE TRANSITION to done state.
//   Advantage: one fewer state (no dedicated DONE state needed).
//   Disadvantage: output can glitch with input changes; 1 cycle earlier response.
//
// For a vending machine: Moore is preferred -- dispensing a physical item
// should be stable (no glitch). The extra DONE state is worth the safety.
//
// Real implementation: A needs 4 states (A0, A5, A10, A_DONE).
//                      B needs 6 states (B0, B5, B10, B15, B20, B_DONE).
//                      + IDLE. Total: 11 states.
```

解释

开放题的解题框架：先澄清需求（先选商品还是先投币），再枚举可达状态，最后讨论优化。

先选商品的优势：目标金额已知，状态数固定，不会出现"投入大量硬币"的边界情况。

Mealy vs Moore 的实际影响：

Moore 输出稳定（dispense action 在一个独立状态里），适合物理动作（出货、找零）。

Mealy 输出提前一拍，减少状态数，但输出随输入变化可能产生毛刺。

面试加分：能主动提出"先选商品/先投币"的设计选择，并解释各自的 state count 影响。

Q54. Digital Design: Clock Module with Randomized Initial State

QUESTION

(a) Write a Verilog clock generator module with configurable period.

(b) How do you randomize the initial state of the clock (rising vs falling edge first, initial phase)? Why might you want to do this?

(c) Revisit: what is the difference between `timescale time_unit and time_precision?

ANSWER

```
// (a) Basic clock generator module
module clk_gen #(
    parameter real PERIOD_NS = 10.0   // default 100 MHz
)(
    output logic clk
);
    // Fixed initial state: always start low
    initial clk = 1'b0;

    always #(PERIOD_NS / 2.0 * 1ns) clk = ~clk;

endmodule

// (b) Randomized initial state
```

```
// The initial value of clk (0 or 1) determines which edge comes FIRST.
// Randomizing this catches bugs that only manifest when the design
// samples on the opposite phase from what the testbench assumed.

module clk_gen_rand #(
    parameter real PERIOD_NS = 10.0
)(
    output logic clk
);
    // Randomize initial clock value at simulation start
    // $urandom_range(0,1) returns 0 or 1 with equal probability
    initial begin
        clk = $urandom_range(0, 1);   // random: 0=low-first, 1=high-first
        #($urandom_range(0, PERIOD_NS/2) * 1ns);  // random phase offset too
    end

    always #(PERIOD_NS / 2.0 * 1ns) clk = ~clk;

endmodule

// Or in a testbench using randomize():
class clk_config;
    rand bit    init_val;      // 0 or 1: initial clock state
    rand real   phase_offset;  // 0 to half-period offset in ns

    constraint c_phase {
        phase_offset >= 0.0;
        // Note: real-type constraints need $itor / manual bounds
    }
endclass

// tb_top usage:
// clk_config cfg = new();
// assert(cfg.randomize());
// initial clk = cfg.init_val;

// Why randomize the initial clock state?
//   1. Catch setup/hold edge-case bugs:
//      Some DUT bugs only appear when data changes on a specific clock phase.
//      If the testbench always starts with clk=0, those bugs are never triggered.
//   2. Detect clock-phase sensitivity:
//      Async FIFOs, CDC circuits, and multi-clock domains may behave
//      differently depending on the relative phase of clocks at startup.
//   3. Improve coverage:
//      Randomizing clock phase covers more corner cases in coverage-driven
//      verification without writing extra directed tests.
//   4. Find uninitialized-state bugs:
//      If DUT registers initialize from X, the first clock edge
//      (rising or falling) determines which edge "wakes up" the design.

// (c) timescale time_unit vs time_precision
//
// `timescale <time_unit> / <time_precision>
//
// time_unit: the UNIT for all delay expressions in this file.
//   #10 means 10 x time_unit.
//   Example: `timescale 1ns/1ps -> #10 = 10ns
//
// time_precision: the RESOLUTION of the simulator internal clock.
//   All time values are rounded to the nearest time_precision before use.
//   Example: `timescale 1ns/1ps -> #10.0005 rounds to #10.001 (nearest 1ps)
//
//   `timescale 1ns/1ns -> #10.5 rounds to #11 (nearest 1ns, loses 0.5ns)
```

```
//    `timescale 1ns/1ps -> #10.5 stays  10.5ns (1ps resolution keeps it exact)
//
// Key rule: time_precision <= time_unit (precision can't be coarser than unit)
//    `timescale 1ps/1ns is ILLEGAL.
//
// Performance vs accuracy trade-off:
//    Finer precision (1ps) -> more simulator time steps -> slower simulation
//    Coarser precision (1ns) -> fewer steps -> faster, but loses sub-ns timing
//
// Example impact on clock generation:
`timescale 1ns/1ns
// always #5.3 clk = ~clk;  -> #5.3 rounds to #5 -> period = 10ns, NOT 10.6ns!
`timescale 1ns/1ps
// always #5.3 clk = ~clk;  -> #5.3 stays 5.3ns -> period = 10.6ns (correct)
```

解释

Clock module 的随机初始值是提升验证质量的小技巧：

始终从 clk=0 出发会遗漏"先来高电平"的场景，某些 DUT bug 只在特定相位下出现。

time_unit 和 time_precision 的核心区别：

time_unit 影响"你怎么写延迟"（#10 代表几个单位），

time_precision 影响"精度损失"（小数部分如何被四舍五入）。

最常见的陷阱：用 `timescale 1ns/1ns 时写 #5.3，结果四舍五入变成 #5，

时钟频率比预期高，导致难以复现的时序问题。

面试建议：明确区分"unit 是量纲"和"precision 是精度"，举 #5.3 的例子最直观。

Q55. Digital Design: What is an Interrupt?

QUESTION

What is an interrupt in computer architecture?

Explain the difference between hardware interrupts, software interrupts,

and exceptions. How does the CPU handle an interrupt?

ANSWER

```
// Definition:
//    An interrupt is an asynchronous signal to the CPU indicating that
//    an event requires immediate attention, causing the CPU to pause its
//    current execution and jump to a special handler (ISR: Interrupt
//    Service Routine), then resume normal execution afterward.
//
// Why interrupts? Alternative is POLLING:
//    Polling: CPU continuously checks if a peripheral has data -> wastes cycles
//    Interrupt: peripheral signals CPU only when it needs attention -> efficient
//
// Three types:
//
// 1. Hardware Interrupt (IRQ - Interrupt ReQuest)
//     Generated by external hardware (peripherals, timers, I/O devices).
//     Examples:
//       - NIC receives a network packet -> IRQ to CPU
//       - Timer expires -> IRQ to CPU (used for OS scheduler tick)
//       - Keyboard key press -> IRQ to CPU
//     Asynchronous: can happen at any time, unrelated to current instruction.
```

```
//
// 2. Software Interrupt (SWI / TRAP / Syscall)
//    Generated by executing a specific instruction (INT n on x86, SVC on ARM).
//    Used for: system calls (OS interface from user mode to kernel mode).
//    Synchronous: triggered by a specific instruction in the program.
//    Example: printf() in C eventually calls a syscall via software interrupt.
//
// 3. Exception (Fault / Trap / Abort)
//    Generated by the CPU itself when an error condition occurs.
//    Examples:
//      - Division by zero -> arithmetic exception
//      - Page fault -> memory exception (address not in physical memory)
//      - Illegal instruction -> undefined opcode exception
//      - Overflow -> arithmetic exception
//    Synchronous: caused by the currently executing instruction.
//
// CPU Interrupt Handling Flow:
//
//   1. Current instruction completes (or is abandoned for aborts)
//   2. CPU saves its state:
//      - PC (Program Counter): address of next instruction to execute
//      - Registers: pushed to stack or saved in dedicated registers
//      - Flags/status register
//   3. CPU looks up the Interrupt Vector Table (IVT):
//      - IVT maps interrupt number -> ISR address
//   4. CPU jumps to the ISR (Interrupt Service Routine)
//   5. ISR executes: handles the interrupt (reads data, clears flags, etc.)
//   6. ISR executes RTI/IRET (Return from Interrupt):
//      - Restores saved state (PC, registers, flags)
//      - CPU resumes execution from where it was interrupted
//
// Hardware interrupt priority:
//   Most CPUs support interrupt priority levels (NVIC on ARM Cortex-M).
//   A high-priority interrupt can PREEMPT a lower-priority ISR.
//   Non-maskable interrupts (NMI) cannot be disabled -- used for critical events.
//
// In RTL/digital design context:
//   Interrupt output pin on a peripheral tells the CPU "I need attention".
//   The peripheral drives IRQ line active (usually active-low).
//   CPU's interrupt controller (e.g. GIC on ARM, APIC on x86) arbitrates.
//   Design consideration: interrupt must be cleared (ACKed) in the ISR,
//   otherwise the interrupt fires repeatedly (interrupt storm).
```

解释

中断的本质是"异步通知机制"：外设不用等待 CPU 来轮询，主动打断 CPU。

三种类型的核心区别：

硬件中断（IRQ）= 外部硬件触发，异步；

软件中断（Syscall）= 程序主动触发，同步，用于权限切换（用户态→内核态）；

异常（Exception）= CPU 自身检测到错误，同步。

CPU 处理流程必须掌握：保存上下文 → 查向量表 → 跳 ISR → 恢复上下文。

在 SoC 设计中，中断是外设（UART、DMA、Timer 等）和 CPU 之间的标准接口。

面试常问：polling 和 interrupt 各自的适用场景？

Polling 适合高频低延迟（如网卡收包热路径）；interrupt 适合低频事件（如按键）。

Q56. Digital Design: Verilog Simulation Scheduling -- initial Block Race

QUESTION

What does $display(A) print in the following code? Is the result deterministic?

```
initial begin
    A = 0;
end

initial begin
    A = 1;
end

initial begin
    #5;
    $display(A);
end
```

ANSWER

```
// Answer: $display(A) prints either 0 or 1 -- NON-DETERMINISTIC.
//
// Analysis using Verilog simulation scheduling:
//
// At time 0: two initial blocks are scheduled simultaneously.
//   Block 1: A = 0  (blocking assignment)
//   Block 2: A = 1  (blocking assignment)
//   Block 3: #5 -> suspends until t=5
//
// The IEEE standard does NOT define the execution order of concurrent
// initial blocks at the same simulation time.
// The simulator may execute Block 1 before Block 2, or vice versa.
//
// Scenario A (Block 1 first):
//   t=0: A = 0, then A = 1  -> A = 1 at t=0
//   t=5: $display(A) -> prints 1
//
// Scenario B (Block 2 first):
//   t=0: A = 1, then A = 0  -> A = 0 at t=0
//   t=5: $display(A) -> prints 0
//
// At t=5: the $display fires. No further changes to A between t=0 and t=5,
// so A retains whatever value it got at t=0 (either 0 or 1).
//
// This is a RACE CONDITION on variable A:
//   Two initial blocks write to A at the same time step t=0.
//   The final value depends on simulator scheduling order.
//
// Compare to the simpler race (no #5):
//   initial begin A=0; end
//   initial begin A=1; end
//   // Same race, same result: 0 or 1, non-deterministic.
//
// How to fix:
//   1. Consolidate into ONE initial block:
//      initial begin A = 0; ... A = 1; end
//   2. Use delays to enforce ordering:
//      initial begin #1; A = 1; end  // guaranteed to execute after A=0
//   3. Drive signals from ONE source only (good testbench practice)
```

解释

这道题考察 Verilog 的 event scheduling model（活动区域调度模型）。

关键规则：同一仿真时刻（t=0），多个 initial block 的执行顺序由模拟器决定，

IEEE 1364/1800 标准明确不规定顺序。这是"合法的非确定性"，不是 bug。

第三个 initial block 的 #5 只是把 $display 推迟到 t=5，

但 A 的值在 t=0 就已经确定（0 或 1），t=5 时不再变化，所以 $display 打印的是 t=0 的竞争结果。

面试中答"0 或 1，不确定"是正确答案，能说出"IEEE 不规定 initial block 顺序"是加分项。

Q57. Digital Design: Hardware Optimization -- Loop Unrolling, Pipelining, Tiling, Array Partitioning

QUESTION

(NPU/accelerator design context)

Explain: loop unrolling, pipelining, array partitioning, and loop tiling.

How does each technique improve hardware performance?

ANSWER

```
// ---- Loop Unrolling ------------------------------------------------------
// Replicate the loop body N times to reduce loop-control overhead
// and expose more parallelism to the synthesizer.
//
// Original:
//   for (i=0; i<8; i++) sum += a[i];    // 8 iterations, sequential
//
// Unrolled x4 (factor=4):
//   sum0 = a[0] + a[1];   // 4 parallel adds instead of 1
//   sum1 = a[2] + a[3];
//   sum2 = a[4] + a[5];
//   sum3 = a[6] + a[7];
//   sum  = sum0 + sum1 + sum2 + sum3;
//
// Benefit: more operations per clock -> better hardware utilization.
// Cost: more area (more adders instantiated).
// HLS pragma: #pragma HLS unroll factor=4

// ---- Pipelining ----------------------------------------------------------
// Insert registers between combinational stages so multiple iterations
// can overlap in time (like an assembly line).
//
// Without pipeline (II=4, must wait 4 cycles per input):
//   Stage1 -> Stage2 -> Stage3 -> Stage4 -> output
//   Cycle 1: Input A goes through all 4 stages
//   Cycle 5: Input B starts (A must finish first)
//
// With pipeline (II=1, one new input accepted every cycle):
//   Cycle 1: A in Stage1
//   Cycle 2: A in Stage2, B in Stage1
//   Cycle 3: A in Stage3, B in Stage2, C in Stage1
//   Throughput = 1 output per cycle after initial latency
//
// II = Initiation Interval (cycles between accepting new inputs).
// Latency = total cycles for one input to complete.
// HLS pragma: #pragma HLS pipeline II=1

// ---- Array Partitioning --------------------------------------------------
// Split a memory array into multiple smaller banks so multiple
// elements can be accessed simultaneously in one clock cycle.
//
// Without partitioning: single-port RAM -> 1 access/cycle
//   for (i=0; i<4; i++) sum += a[i];   // 4 cycles minimum
```

```
//
// With CYCLIC partitioning (factor=4):
//   a[0] -> bank0,  a[1] -> bank1,  a[2] -> bank2,  a[3] -> bank3
//   All 4 elements accessible simultaneously -> 1 cycle
//
// Partition types:
//   CYCLIC: a[0]->bank0, a[1]->bank1, a[2]->bank0, ...  (interleaved)
//   BLOCK:  a[0..N/2]->bank0, a[N/2..N]->bank1  (contiguous blocks)
//   COMPLETE: each element gets its own register (fully parallel, most area)
//
// HLS pragma: #pragma HLS array_partition variable=a cyclic factor=4

// ---- Loop Tiling (Loop Blocking) ----------------------------------------
// Restructure a loop to improve data locality / cache reuse,
// especially for matrix operations in NPU/accelerator design.
//
// Original matrix multiply (poor cache behavior on B):
//   for i in range(N):
//     for j in range(N):
//       for k in range(N):
//         C[i][j] += A[i][k] * B[k][j]   // B accessed column-wise -> cache miss
//
// Tiled version (tile size T):
//   for i in range(0, N, T):    // tile over rows
//     for j in range(0, N, T):  // tile over columns
//       for k in range(0, N, T):// tile over inner dim
//         // process T x T subblock -- fits in cache/local buffer
//         for ii in range(i, i+T):
//           for jj in range(j, j+T):
//             for kk in range(k, k+T):
//               C[ii][jj] += A[ii][kk] * B[kk][jj]
//
// Benefit: each T x T tile of A and B is loaded once, reused T times.
// Critical for NPU systolic arrays where on-chip SRAM is limited.
```

解释

四种优化技术对应硬件设计的不同维度：

Loop Unrolling：空间换时间，多实例化计算单元，提高单周期吞吐。

Pipelining：时间分割，每级插入寄存器，II=1 意味着每周期接受一个新输入。

Array Partitioning：存储并行化，把单端口 RAM 拆成多个 bank，支持多路并发访问。

Tiling：数据局部性优化，重排循环顺序使数据在 on-chip buffer 中被最大化复用，

减少片外 DRAM 访问，这是 NPU 设计中最关键的优化手段之一。

Q58. Digital Design: IEEE 754 Floating Point -- Format, Operations, Edge Cases

QUESTION

(NPU context -- critical for AI accelerator interviews)

(a) Describe the IEEE 754 single-precision (32-bit) format.

(b) How does FP addition work step by step?

(c) How does FP multiplication work?

(d) What are the important edge cases?

ANSWER

```
// (a) IEEE 754 Single Precision (32-bit) Format
//
//   Bit 31   Bits 30:23   Bits 22:0
//   [sign]   [exponent]   [mantissa / fraction]
//     1 bit    8 bits       23 bits
//
//   sign:     0 = positive, 1 = negative
//   exponent: BIASED by 127 (stored value = actual exponent + 127)
//             range: -126 to +127 (stored 1..254; 0 and 255 are special)
//   mantissa: fractional part after the implicit leading 1
//             value = 1.mantissa * 2^(exponent - 127)   (for normal numbers)
//
//   Examples:
//   1.0  = 0 | 01111111 | 00000000000000000000000  (exp=127, mant=0)
//          = 1.0 * 2^(127-127) = 1.0 * 1 = 1.0
//   0.5  = 0 | 01111110 | 00000000000000000000000  (exp=126, mant=0)
//          = 1.0 * 2^(126-127) = 1.0 * 2^-1 = 0.5
//   1.5  = 0 | 01111111 | 10000000000000000000000  (exp=127, mant=0x400000)
//          = 1.1_2 * 2^0 = 1.5

// (b) FP Addition: 1.5 + 0.5
//
// Step 1: Extract fields
//   A = 1.5: sign=0, exp=127, mantissa=1.100...0_2
//   B = 0.5: sign=0, exp=126, mantissa=1.000...0_2
//
// Step 2: Align exponents (shift the smaller one right)
//   Difference = 127 - 126 = 1
//   B shifted right 1: 0.100...0_2 * 2^127  (becomes 0.1 instead of 1.0)
//   Note: B loses its implicit leading 1 (it becomes denormalized temporarily)
//
// Step 3: Add mantissas
//   1.100...0
// + 0.100...0
// = 10.000...0  (carry out of integer bit!)
//
// Step 4: Normalize result
//   10.0 * 2^0 -> 1.0 * 2^1  (shift right 1, increment exponent)
//   exp = 127 + 1 = 128, mantissa = 0.0
//
// Step 5: Result
//   sign=0, exp=128, mantissa=0 -> 1.0 * 2^1 = 2.0 OK

// (c) FP Multiplication: 1.5 * 2.0
//
// Step 1: Extract fields
//   A = 1.5: sign=0, exp=127, mantissa=1.1_2
//   B = 2.0: sign=0, exp=128, mantissa=1.0_2
//
// Step 2: Compute result sign
//   sign_r = sign_A XOR sign_B = 0 XOR 0 = 0 (positive)
//
// Step 3: Add exponents (subtract bias once: add biased exps, subtract 127)
//   exp_r = (127-127) + (128-127) + 127 = 0 + 1 + 127 = 128
//   Or equivalently: exp_r = exp_A + exp_B - 127
//
// Step 4: Multiply mantissas (including the implicit leading 1)
//   1.1_2 * 1.0_2 = 1.1_2  (same as multiplying 3/2 * 1 = 3/2)
//
// Step 5: Normalize
//   1.1_2 is already normalized (no leading 0 or carry)
//
// Step 6: Result
```

```
//   sign=0, exp=128, mantissa=0.1_2 -> 1.1 * 2^1 = 3.0 OK

// (d) Edge Cases -- MUST KNOW for NPU interviews
//
//   Case              | sign | exponent | mantissa | Notes
//   ------------------+------+----------+----------+---------------------
//   +Zero             |  0   |    0     |    0     | -0 also exists (sign=1)
//   -Zero             |  1   |    0     |    0     | +0 == -0 in comparisons
//   +Infinity         |  0   |   255    |    0     | overflow result
//   -Infinity         |  1   |   255    |    0     | overflow result
//   NaN (quiet)       |  X   |   255    |  != 0    | invalid operation result
//   Denormal/Subnormal|  X   |    0     |  != 0    | value = 0.mantissa * 2^-126
//
//   Key edge case behaviors:
//   1/0       = +Inf     (not an error in IEEE 754!)
//   0/0       = NaN
//   Inf + Inf = +Inf
//   Inf - Inf = NaN      (indeterminate)
//   0 * Inf   = NaN
//   NaN == NaN -> FALSE  (NaN is never equal to anything, including itself!)
//   sqrt(-1)  = NaN
//
//   Denormals (subnormal numbers):
//   Fill the gap between 0 and the smallest normal number (2^-126).
//   Value = 0.mantissa * 2^-126 (no implicit leading 1).
//   Hardware challenge: denormal arithmetic is slow (often handled in microcode).
//   NPU design: often FLUSH TO ZERO (FTZ) denormals for performance.
//
//   Rounding modes (IEEE 754 specifies 4):
//   RNE: Round to Nearest, ties to Even (default, most accurate)
//   RTZ: Round toward Zero (truncation)
//   RPI: Round toward +Infinity
//   RNI: Round toward -Infinity
//
//   Reduced precision formats in deep learning:
//   FP16 (half):  1-5-10 (sign-exp-mantissa), range smaller, faster
//   BF16:         1-8-7  (same exp range as FP32, less mantissa precision)
//   FP8:          1-4-3 or 1-5-2, used in transformer inference
//   INT8:         fixed-point, no exponent, requires quantization
```

解释

IEEE 754 的核心设计思路：用偏置指数（biased exponent）避免负数表示的复杂性，

用隐含前导1（implicit leading 1）多保存一位精度（只需存小数部分）。

FP 加法的关键步骤：对齐指数（右移小者）→ 整数加法 → 归一化。

FP 乘法的关键步骤：符号异或 → 指数相加减去偏置 → 尾数相乘 → 归一化。

面试必须掌握的 edge case：NaN != NaN、±0 相等、Inf - Inf = NaN。

NPU/AI 加速器背景：BF16 保持和 FP32 相同的指数范围（8位），

只牺牲精度（7位尾数 vs 23位），在训练中表现接近 FP32 且速度快很多。

Flush-to-Zero 是常见的硬件简化：忽略 denormal 处理，直接舍去到0，换取面积和速度。

Q59. Digital Design: SRAM RTL Implementation and Why No Reset

QUESTION

(a) Write a simple single-port SRAM in Verilog (synchronous read and write,

no reset signal).

(b) Follow-up: Why does SRAM not have a reset signal?

ANSWER

```
// (a) Simple synchronous single-port SRAM

module sram #(
    parameter int DEPTH  = 256,
    parameter int WIDTH  = 8,
    parameter int ADDR_W = $clog2(DEPTH)
)(
    input  logic              clk,
    // No rst_n port -- SRAM does not have reset
    input  logic              cs_n,   // chip select (active low)
    input  logic              we_n,   // write enable (active low)
    input  logic [ADDR_W-1:0] addr,
    input  logic [WIDTH-1:0]  din,
    output logic [WIDTH-1:0]  dout
);
    // Memory array -- NOT initialized with reset
    logic [WIDTH-1:0] mem [DEPTH];

    always_ff @(posedge clk) begin
        if (!cs_n) begin
            if (!we_n) begin
                // Write operation
                mem[addr] <= din;
            end else begin
                // Read operation (synchronous read -- output registered)
                dout <= mem[addr];
            end
        end
    end

endmodule

// (b) Why SRAM has no reset
//
// Physical reason:
//   SRAM cells are made of 6 transistors (6T SRAM cell) forming a
//   bistable latch. At power-on, each cell settles randomly to 0 or 1.
//   There is NO reset transistor path in a standard 6T cell to force
//   the cell to a known state -- doing so would require extra area
//   (more transistors per cell) which defeats the purpose of SRAM.
//
// Design reason:
//   SRAM is used as a storage resource, not a control register.
//   The system software or the controller is responsible for
//   initializing memory contents before use (e.g. memset, DMA fill).
//   Providing a hardware reset would add significant area and timing
//   complexity: you would need to cycle through all DEPTH addresses
//   and write 0 -- that takes DEPTH clock cycles just for reset.
//
// Contrast with flip-flops (registers):
//   FFs have an explicit reset transistor path (pass gate to GND/VDD).
//   Reset is cheap for a single FF, but for 256KB of SRAM
//   (256K * 8 bits = 2M bit-cells), adding reset to every cell
//   would massively increase area (and reset time).
//
// In simulation:
//   SRAM contents are X (unknown) at the start.
//   A testbench must either:
//   a. Write known values before reading
```

```
//   b. Use $readmemh / $readmemb to preload from a file
//   c. Use backdoor HDL access to initialize memory directly
//
//   $readmemh("init.hex", mem);   // preload at simulation start
```

解释

SRAM 没有复位的根本原因在物理结构：6T SRAM cell 是一个双稳态触发器，

要把所有 cell 清零需要写遍整个阵列（O(DEPTH) 时间），代价太高。

实际 SoC 设计中，SRAM 的初始内容未定义，使用前必须由软件初始化（memset 或 DMA 填充）。

面试考点：能区分"为什么 FF 有 reset 而 SRAM 没有"——FF reset 是单个传输门的事，

SRAM reset 需要遍历整个存储阵列，完全不同的代价量级。

Q60. Digital Design: Synchronous FIFO Using SRAM with Counter

QUESTION

Implement a synchronous FIFO using the SRAM module above as the storage.

Use a counter-based design. Explain:

(a) How are wr_ptr and rd_ptr managed?

(b) How do you detect full and empty?

(c) How do you handle simultaneous read/write?

ANSWER

```
module sync_fifo #(
    parameter int DEPTH  = 16,
    parameter int WIDTH  = 8,
    parameter int ADDR_W = $clog2(DEPTH)
)(
    input  logic             clk,
    input  logic             rst_n,
    input  logic             push,
    input  logic [WIDTH-1:0] din,
    input  logic             pop,
    output logic [WIDTH-1:0] dout,
    output logic             full,
    output logic             empty
);
    // ---- Pointers and counter ----------------------------------------
    logic [ADDR_W-1:0]   wr_ptr;    // next write address
    logic [ADDR_W-1:0]   rd_ptr;    // next read address
    logic [ADDR_W:0]     count;     // number of valid entries (1 extra bit)

    // ---- SRAM instance -----------------------------------------------
    // Use we_n=0 for write, we_n=1 for read
    // Mux address between wr_ptr and rd_ptr based on operation
    logic              sram_we_n;
    logic [ADDR_W-1:0] sram_addr;
    logic [WIDTH-1:0]  sram_din;

    sram #(.DEPTH(DEPTH), .WIDTH(WIDTH)) u_sram (
        .clk  (clk),
        .cs_n (1'b0),           // always selected
        .we_n (sram_we_n),
        .addr (sram_addr),
```

```
        .din  (sram_din),
        .dout (dout)
    );

    // SRAM control: write on push, read on pop
    // Simultaneous push+pop: prioritize write addr for push, read addr for pop
    // Since SRAM is single-port, we arbitrate: push wins (read one cycle later)
    always_comb begin
        if (push && !full) begin
            sram_we_n = 1'b0;        // write mode
            sram_addr = wr_ptr;
            sram_din  = din;
        end else begin
            sram_we_n = 1'b1;        // read mode
            sram_addr = rd_ptr;
            sram_din  = '0;
        end
    end

    // ---- Pointer and counter update ------------------------------------
    always_ff @(posedge clk or negedge rst_n) begin
        if (!rst_n) begin
            wr_ptr <= '0;
            rd_ptr <= '0;
            count  <= '0;
        end else begin
            // Write pointer: advances on successful push
            if (push && !full) begin
                wr_ptr <= (wr_ptr == ADDR_W'(DEPTH-1)) ? '0 : wr_ptr + 1'b1;
            end
            // Read pointer: advances on successful pop
            if (pop && !empty) begin
                rd_ptr <= (rd_ptr == ADDR_W'(DEPTH-1)) ? '0 : rd_ptr + 1'b1;
            end
            // Counter update
            case ({push && !full, pop && !empty})
                2'b10: count <= count + 1'b1;  // push only
                2'b01: count <= count - 1'b1;  // pop only
                default: ;                     // both or neither: unchanged
            endcase
        end
    end

    // ---- Status flags --------------------------------------------------
    assign full  = (count == ADDR_W+1'(DEPTH));
    assign empty = (count == '0);

endmodule

// (a) wr_ptr and rd_ptr:
//   wr_ptr: address of the NEXT location to write. Advances on each push.
//   rd_ptr: address of the NEXT location to read. Advances on each pop.
//   Both wrap around from DEPTH-1 back to 0 (circular buffer).
//
// (b) Full and empty detection using counter:
//   empty: count == 0  (nothing in the FIFO)
//   full:  count == DEPTH (all slots occupied)
//   The counter is the clearest method for sync FIFO (no need for gray codes).
//   Counter width: ADDR_W+1 bits to represent values 0..DEPTH inclusive.
//
// (c) Simultaneous read/write (single-port SRAM constraint):
//   Single-port SRAM can only do ONE operation per cycle.
//   When push AND pop occur simultaneously:
```

```
//   -> We choose to do the WRITE in this cycle (push wins).
//   -> The READ is deferred: rd_ptr still advances, but dout gets
//      the correct value next cycle (pop data is slightly delayed by 1 cycle).
//   Count stays the same (push+pop cancel out).
//   Alternative: use a dual-port SRAM to allow true simultaneous read+write.
```

解释

Counter-based sync FIFO 是最直观的设计：一个计数器明确记录有多少有效数据，

避免了使用额外位（extra bit trick）来区分满和空的复杂性。

单端口 SRAM 的仲裁限制是这道题的核心约束：同一时钟周期只能选 read 或 write，

需要明确的仲裁策略（这里选写优先）。

真实设计中通常选择双端口 SRAM 来支持同时读写，避免吞吐损失。

rd_ptr 和 wr_ptr 都使用循环地址（到达 DEPTH-1 后回到 0），构成环形缓冲区。

SystemVerilog — Constraints & Randomization

Q61. Constraints: Consecutive Uniqueness & Cross-Array Common Element

QUESTION

Write SystemVerilog constraints for the following two requirements:

(a) Given a rand array arr of N integers, constrain it so that every

5 consecutive elements are all different from each other.

(i.e. for any window arr[i..i+4], all 5 values are unique)

(b) Given two rand arrays arr_a and arr_b, constrain them so that

they share at least one common element.

ANSWER

```
// (a) Every 5 consecutive elements must all be unique
//
// Key insight: element arr[i] must differ from the 4 elements before it.
// This guarantees any window of 5 has all unique values.
//
class consecutive_unique;
    parameter int N       = 20;
    parameter int MAX_VAL = 16;

    rand bit [3:0] arr [N];

    // For each element, constrain it != the previous 4 elements.
    // This is equivalent to: every window arr[i..i+4] has 5 unique values.
    constraint c_unique_window_of_5 {
        foreach (arr[i]) {
            if (i >= 1) arr[i] != arr[i-1];
            if (i >= 2) arr[i] != arr[i-2];
            if (i >= 3) arr[i] != arr[i-3];
            if (i >= 4) arr[i] != arr[i-4];
        }
```

```
    }
endclass

// Alternative using unique keyword (less portable across simulators):
//   constraint c_unique_window_of_5 {
//       foreach (arr[i]) {
//           if (i >= 4)
//               unique {arr[i], arr[i-1], arr[i-2], arr[i-3], arr[i-4]};
//       }
//   }
// Note: unique inside foreach on a slice is solver-dependent.
// The != expansion above has better cross-tool compatibility.

// (b) Two arrays must share at least one common element
//
// Key insight: introduce a helper rand variable common_val constrained
// to exist inside BOTH arrays simultaneously. The solver will pick a
// value and ensure it appears in each array.
// Trying to express "exists i,j: a[i]==b[j]" directly is not legal SV.
//
class cross_array_common;
    parameter int N = 8;

    rand bit [3:0] arr_a [N];
    rand bit [3:0] arr_b [N];
    rand bit [3:0] common_val;  // helper: must appear in both arrays

    constraint c_common_element {
        common_val inside {arr_a};  // common_val is one of arr_a's elements
        common_val inside {arr_b};  // common_val is one of arr_b's elements
    }
endclass

// How it works:
// The solver treats common_val as a witness variable.
// It must simultaneously satisfy:
//   exists i: arr_a[i] == common_val
//   exists j: arr_b[j] == common_val
// By forcing common_val into both arrays, at least one pair (i,j)
// will have arr_a[i] == arr_b[j] == common_val.
//
// Example result:
//   arr_a    = [3, 7, 2, 5, 9, 1, 6, 4]
//   arr_b    = [0, 8, 5, 3, 2, 7, 1, 6]  <- 3,7,2... appear in arr_a too
//   common_val = 3  (solver picked 3 as the witness)
```

解释

(a) 连续五个不同：把"任意五个窗口内元素两两不同"转化为"每个元素与前4个都不同"。

SV 的 foreach + != 展开是最兼容的写法。unique 关键字虽然更简洁，

但在 foreach 内对 slice 使用时不同仿真器行为不一致，面试中先给 != 版本

再提 unique 作为补充是最稳妥的回答策略。

MAX_VAL 需要 > 4，否则无法找到满足约束的解（pigeonhole principle：

5个不同的值需要值域至少有5个可选值）。

(b) 两数组至少一个公共元素：核心技巧是引入辅助随机变量 common_val 作为"证人"

（witness variable），同时约束它在两个数组的 inside 集合内。

SV 不支持直接表达量词"存在 i,j 使得 a[i]==b[j]"，

but 通过引入 witness variable，solver 会自动找到满足条件的值并填入两个数组。

这个技巧在面试中是加分项：体现了对 constraint solver 工作机制的深入理解。

Q62. Constraints: std::randomize Return Value Bug

QUESTION

Find all bugs in the following SystemVerilog code.

Hint: std::randomize() returns 0 or 1 (fail/success), not the randomized value.

```
----------------------------------------
program std_randomize;
  int addr;
  int data;
  int unsigned partial_pmds[$];

  initial begin
    addr = 1;
    partial_pmds = {1,2,3,4,5};
    $display("Value of addr is %0d",addr);
    repeat(10) begin
        data = std::randomize(data) with { data inside {[0:partial_pmds.size()-1]};
                                  data != addr;
                                };
      $display("Value of data is %0d",data);
    end
  end
endprogram
```

ANSWER

```
// ---- Bug 1 (Critical): std::randomize() return value misused ----
//
// BUGGY:
//   data = std::randomize(data) with { data inside {...}; data != addr; };
//
// std::randomize() randomizes its argument IN-PLACE and returns:
//   1 (bit) -> randomization succeeded
//   0 (bit) -> randomization failed (constraint contradiction, etc.)
//
// So "data = std::randomize(data)" assigns 0 or 1 to data, NOT the
// randomized value. The randomized result is already written into
// "data" by the call itself -- the return value is just a status bit.
//
// FIX: separate the call from the assignment and check the return value.
//
// FIXED:
//   if (!std::randomize(data) with { data inside {partial_pmds};
//                                    data != addr; })
//       $fatal("Randomization failed!");
//   $display("Value of data is %0d", data);

// ---- Bug 2: Wrong inside expression -- index range vs. value set ----
//
// BUGGY:
//   data inside {[0:partial_pmds.size()-1]}
```

```
//
// partial_pmds = {1,2,3,4,5}, so partial_pmds.size() = 5.
// This constrains data to the INTEGER RANGE [0:4] (i.e. 0,1,2,3,4),
// NOT the values stored in partial_pmds (1,2,3,4,5).
// The intent is clearly to pick from the array's VALUES.
// Also, value 0 is included but is not in partial_pmds at all.
//
// FIX: use the array directly inside the inside expression.
//
// FIXED:
//   data inside {partial_pmds}
//
// This correctly constrains data to one of {1, 2, 3, 4, 5}.

// ---- Corrected full program ----
program std_randomize_fixed;
    int addr;
    int data;
    int unsigned partial_pmds[$];

    initial begin
        addr = 1;
        partial_pmds = {1, 2, 3, 4, 5};
        $display("Value of addr is %0d", addr);
        repeat(10) begin
            // Correct: call std::randomize, check return, data updated in-place
            if (!std::randomize(data) with {
                    data inside {partial_pmds};  // values 1..5, not range [0:4]
                    data != addr;
                })
                $fatal(1, "Randomization failed!");
            $display("Value of data is %0d", data);  // data now holds random value
        end
    end
endprogram

// Summary of bugs:
//   Bug 1: data = std::randomize(data) -> assigns status (0/1), not random value
//           Fix: use std::randomize(data) as a void call; check return separately
//   Bug 2: inside {[0:partial_pmds.size()-1]} -> range [0:4], not values {1..5}
//           Fix: inside {partial_pmds}
```

解释

Bug 1 是 std::randomize 最常见的面试考点：它与 class 的 randomize() 方法不同。

class::randomize() 是方法调用，随机值写入成员变量，返回值通常被忽略或用于检查。

std::randomize(var) 是函数调用，随机值写入传入的变量（引用传递），

返回值是 bit 类型的成功/失败标志。

将返回值赋回给变量是经典错误：data = std::randomize(data) 中，

data 先被随机化（正确），然后被返回值 0 或 1 覆盖（错误）。

正确用法是把 std::randomize() 放在 if 条件里做成功检查，不赋值。

Bug 2 考察 inside 表达式的语法：

{[lo:hi]} 是整数范围（连续区间），{array_name} 是数组元素集合。

两者写法相似但语义完全不同，很容易混淆。

partial_pmds.size()-1 = 4，所以 [0:4] = {0,1,2,3,4}，

既包含了不在数组里的 0，又错误地用下标代替了值。

Q63. Constraints: Sudoku 9x9 Grid Constraint

QUESTION

Write a SystemVerilog constraint to generate a valid 9x9 Sudoku grid.

Requirements:

- Every element is in range [1:9]
- Every row contains unique values (1-9 each appearing exactly once)
- Every column contains unique values
- Every 3x3 sub-grid contains unique values

ANSWER

```
class sudoku;
    `uvm_object_utils(sudoku)

    rand bit [3:0] grid [9][9];   // 4 bits covers values 1..9

    // ---- Range: every cell in [1:9] -----------------------------------
    // (Technically redundant given the row constraint below, but helps the
    //  solver converge faster by cutting the domain early.)
    constraint c_range {
        foreach (grid[r, c])
            grid[r][c] inside {[1:9]};
    }

    // ---- Rows: each row is a permutation of {1..9} ----------------------
    constraint c_rows {
        foreach (grid[r,])
            unique {grid[r][0], grid[r][1], grid[r][2],
                    grid[r][3], grid[r][4], grid[r][5],
                    grid[r][6], grid[r][7], grid[r][8]};
    }

    // ---- Columns: each column is a permutation of {1..9} ----------------
    constraint c_cols {
        foreach (grid[,c])
            unique {grid[0][c], grid[1][c], grid[2][c],
                    grid[3][c], grid[4][c], grid[5][c],
                    grid[6][c], grid[7][c], grid[8][c]};
    }

    // ---- 3x3 boxes: iterate over the 9 box origins (br, bc in {0,3,6}) --
    // Each origin (br, bc) is the top-left corner of one 3x3 box.
    // Using a helper function keeps c_boxes clean and general.
    constraint c_boxes {
        foreach (grid[br, bc])
            if (br % 3 == 0 && bc % 3 == 0)
                unique {
                    grid[br][bc],   grid[br][bc+1],   grid[br][bc+2],
                    grid[br+1][bc], grid[br+1][bc+1], grid[br+1][bc+2],
                    grid[br+2][bc], grid[br+2][bc+1], grid[br+2][bc+2]
                };
    }

endclass

// ---- Usage ---------------------------------------------------------------
```

```
// sudoku s = new();
// assert(s.randomize()) else $fatal(1, "Sudoku constraint unsolvable");
//
// ---- Why c_boxes is clean now ------------------------------------------
// foreach (grid[br, bc]) iterates br in 0..8, bc in 0..8.
// The guard (br%3==0 && bc%3==0) fires exactly 9 times:
//   (0,0) (0,3) (0,6) (3,0) (3,3) (3,6) (6,0) (6,3) (6,6)
// Each firing emits one unique{} for the 3x3 block rooted at (br, bc).
// No manual repetition, no risk of copy-paste errors.
//
// ---- Performance tip ---------------------------------------------------
// Sudoku is NP-complete; randomize() can be slow on a blank grid.
// Fix the first row as a known permutation to cut the search space:
//   constraint c_seed { grid[0] == '{1,2,3,4,5,6,7,8,9}; }
```

解释

三个约束分别对应数独的三条规则，全部用 unique 关键字实现。

c_boxes 的优化：用 foreach 二维迭代配合 br%3==0 && bc%3==0 的守卫条件，

自动遍历9个 box 的左上角坐标，每个 box 展开成一个 unique{} 语句。

比手写9段相比，代码从 54 行缩减到 8 行，且完全正确、无法遗漏。

面试技巧：先写行/列约束（简单），再说 box 约束"用 foreach + 模运算找起点"，

体现了对 SV 约束语言的熟练掌握。

Q64. Constraints: Dynamic Array -- Sum, Consecutive Ones, Unique in Range

QUESTION

Given a 100-element dynamic array of bit type:

(a) Constrain the first 8 elements to sum to exactly 6, plus other basic constraints.

(b) Write a function that returns the starting indices of all runs of

exactly 10 consecutive 1s.

(c) Constrain the array so that in every randomization, each value in a

specific range appears at most once.

(d) How do you apply an inline constraint to override the above?

ANSWER

```
class bit_array_cls;
    `uvm_object_utils(bit_array_cls)

    // 100-element dynamic array of bit
    rand bit arr[];

    // ---- (a) Basic constraints ---------------------------------------

    constraint c_size {
        arr.size() == 100;                      // fix array size to 100
    }

    constraint c_sum_first_8 {
        // Sum of first 8 elements equals 6
        // (6 out of 8 bits are 1, meaning exactly 6 ones)
        arr[0] + arr[1] + arr[2] + arr[3] +
```

```
        arr[4] + arr[5] + arr[6] + arr[7] == 6;
    }

    constraint c_last_element {
        arr[99] == 1'b1;                        // last element must be 1
    }

    // ---- (c) Each value in range [10:19] appears at most once ------------
    // Since arr is bit-typed (0 or 1), "unique in range" means:
    // each INDEX position in [10:19] can hold 0 or 1, but the pattern
    // of values in that slice must have all unique values.
    // For int arrays, the standard approach is:
    //   unique {arr[10], arr[11], ..., arr[19]};
    // For a bit array with only 2 possible values, true uniqueness is impossible
    // for 10 elements. The constraint is more meaningful with wider types.
    //
    // For int dynamic array (more practical example):
    rand int int_arr[];

    constraint c_unique_in_range {
        int_arr.size() == 20;
        // Each element in [0:9] range is unique (no repeats)
        unique {int_arr[0], int_arr[1], int_arr[2], int_arr[3],
                int_arr[4], int_arr[5], int_arr[6], int_arr[7],
                int_arr[8], int_arr[9]};
        // All elements bounded in [1:100]
        foreach (int_arr[i])
            int_arr[i] inside {[1:100]};
    }

endclass

// ---- (b) Function: find starting indices of all 10-consecutive-1 runs ----

function automatic int unsigned find_consecutive_ones(
    input bit   arr[],           // input array
    input int   run_len  = 10    // target run length (default 10)
) return int unsigned[$];        // queue of starting indices

    int unsigned positions[$];
    int unsigned n = arr.size();
    int unsigned i = 0;

    while (i <= n - run_len) begin
        // Check if a run of run_len ones starts at index i
        automatic bit is_run = 1;
        for (int k = 0; k < run_len; k++) begin
            if (arr[i + k] != 1'b1) begin
                is_run = 0;
                break;
            end
        end

        if (is_run) begin
            positions.push_back(i);
            i += run_len;         // skip past this run (non-overlapping)
        end else begin
            i++;
        end
    end

    return positions;
endfunction
```

```
// ---- (d) Inline constraint: override c_sum_first_8 for one call ---------
//
// Use inline constraint in the test or sequence body.
// Inline constraints combine with active constraints.
// To REPLACE c_sum_first_8, first disable it, then use inline:

task test_special_sum(bit_array_cls obj);
    // Disable the default sum constraint
    obj.c_sum_first_8.constraint_mode(0);

    // Inline constraint: force sum of first 8 to be 3 instead of 6
    assert(obj.randomize() with {
        arr[0] + arr[1] + arr[2] + arr[3] +
        arr[4] + arr[5] + arr[6] + arr[7] == 3;
    }) else `uvm_fatal("RAND", "Randomization failed");

    // Re-enable for subsequent use
    obj.c_sum_first_8.constraint_mode(1);
endtask

// Alternative: if c_sum_first_8 uses soft, inline automatically overrides:
//   constraint c_sum_first_8 {
//       soft (arr[0]+arr[1]+...+arr[7] == 6);  // soft: inline can override
//   }
//   Then simply: obj.randomize() with { arr[0]+...+arr[7] == 3; }
```

解释

(a) 动态数组 bit arr[] 必须先约束 size()，否则 solver 不知道数组大小。

sum 约束直接展开写出所有 arr[0]+...+arr[7]，对小数组是最可读的写法。

(b) 连续1查找函数：双指针扫描，找到一个 run 后跳过整个 run（非重叠版本）。

返回 int unsigned[$] 队列，支持返回多个起始位置。

面试中问清楚是否允许重叠：重叠版本改用滑动窗口 count 计数器。

(c) unique 约束直接列出所有目标元素，solver 保证无重复值。

注意：bit 类型数组每个元素只有 0/1，10 个 bit 元素不可能全 unique，

unique 约束对 int 类型数组才有实际意义。

(d) inline constraint 与 constraint_mode(0) 配合使用：

先关闭冲突的 hard constraint，再用 inline 指定新值。

如果原来是 soft constraint 则更简洁，inline 直接胜出无需关闭。

Q65. Constraints: Child Class Constraint Conflict and constraint_mode

QUESTION

Given:

base: constraint c { a inside {[0:99], [201:299]}; }

child: constraint c_0 { a inside {[100:199]}; }

(a) What happens when you randomize the child class?

(b) If c_0 is commented out, how do you fully randomize a (disable base constraint)?

ANSWER

```
// (a) Randomizing child class -- FAILS
//
// Both constraints are active simultaneously (child inherits all base constraints).
// c   requires a in [0:99]   OR [201:299]
// c_0 requires a in [100:199]
// The intersection of these two sets is EMPTY -- no valid value exists.
// randomize() returns 0 (failure) and a is not updated.
// With UVM: `uvm_error or $fatal depending on how the call is wrapped.
//
// Always check randomize() return value:
//   assert(obj.randomize()) else `uvm_fatal("RAND", "Randomization failed")

// (b) Disable base constraint to allow full randomization of a
//
// Method 1: constraint_mode(0) -- disable by name at runtime
//   Works even from outside the class (if the constraint is accessible).
child_obj.c.constraint_mode(0);   // disable base constraint c
assert(child_obj.randomize());    // a now free to take any int value
child_obj.c.constraint_mode(1);   // re-enable after randomization

// Method 2: rand_mode(0) -- disable randomization of 'a' entirely
//   Makes 'a' a non-random field; randomize() leaves it at its current value.
child_obj.a.rand_mode(0);
// a retains its current value, no constraint applied

// Method 3: inline constraint override of c (if c uses soft)
//   If base uses: soft a inside {[0:99], [201:299]};
//   then inline can override: obj.randomize() with { a inside {[0:255]}; }

// Key distinction:
//   constraint_mode(0) : constraint exists but is deactivated for this call
//   rand_mode(0)       : variable becomes non-random (like a regular int)
//   soft + inline      : inline always wins over soft (no need to disable)
```

解释

子类继承父类的所有约束，两套约束同时生效。当约束集的交集为空时，

randomize() 返回0（失败），不会产生 assertion/fatal，除非调用方检查返回值。

面试考察点：能说出"两约束同时生效"是根本原因，能给出三种解决方法，

并说清 constraint_mode / rand_mode / soft 各自的语义差异。

Q66. Constraints: Static Array Uniqueness -- With and Without unique

QUESTION

Given a static integer array of size 10:

(a) Constrain all 10 elements to be distinct using the unique keyword.

(b) Implement the same effect WITHOUT using unique.

(c) Without unique, constrain only 9 elements to be distinct;

the remaining 1 element must duplicate one of the 9.

ANSWER

```
// (a) With unique keyword
class arr_unique;
    `uvm_object_utils(arr_unique)
    rand int arr [10];   // static array, size fixed at 10
```

```
    constraint c_all_unique {
        unique {arr[0], arr[1], arr[2], arr[3], arr[4],
                arr[5], arr[6], arr[7], arr[8], arr[9]};
    }
endclass

// (b) Without unique -- use foreach nested inequality
// For every pair (i, j) where i < j, arr[i] != arr[j]
class arr_unique_no_kw;
    `uvm_object_utils(arr_unique_no_kw)
    rand int arr [10];

    constraint c_all_unique {
        foreach (arr[i])
            foreach (arr[j])
                if (i < j) arr[i] != arr[j];
    }
endclass

// (c) Without unique -- 9 elements distinct, 1 must duplicate one of the 9
// arr[0..8] are all distinct; arr[9] must equal one of arr[0..8]
class arr_nine_unique;
    `uvm_object_utils(arr_nine_unique)
    rand int arr  [10];
    rand int dup_idx;   // which of the first 9 does arr[9] copy

    constraint c_nine_unique {
        // First 9 elements are all distinct
        foreach (arr[i])
            foreach (arr[j])
                if (i < j && i <= 8 && j <= 8) arr[i] != arr[j];
    }

    constraint c_tenth_is_dup {
        // arr[9] must equal exactly one of arr[0..8]
        dup_idx inside {[0:8]};
        arr[9] == arr[dup_idx];
    }
endclass
```

解释

(a) unique 是最简洁的写法，solver 直接枚举无重复排列。

(b) 不用 unique：嵌套 foreach 展开为 C(10,2)=45 个不等式约束，语义等价。

注意必须用 i < j 防止重复生成 arr[i]!=arr[j] 和 arr[j]!=arr[i]（冗余但无害）。

(c) 前9个保持互斥，第10个通过 dup_idx 随机选择复制哪一个，

这样 solver 可以自由选择重复哪个值，而不是固定复制 arr[0]。

Q67. Constraints: Alternating READ/WRITE Transactions Using post_randomize

QUESTION

Given a transaction with opcode = {READ, WRITE}, write a sequence that alternates between READ and WRITE transactions. Use a state bit and post_randomize() to enforce the alternation without using constraints that depend on the previous transaction's value.

ANSWER

```
typedef enum bit {READ = 0, WRITE = 1} opcode_e;

class rw_txn extends uvm_sequence_item;
    `uvm_object_utils(rw_txn)

    rand opcode_e opcode;
    rand logic [31:0] addr;
    rand logic [31:0] data;

    // Static state bit: persists across randomize() calls for this object
    // (or make it a class variable in the sequence for true cross-txn state)
    static bit next_op = 0;   // 0=READ first, 1=WRITE first

    // post_randomize() is called AUTOMATICALLY after every randomize().
    // It overrides the randomized opcode with the enforced alternating value,
    // then flips the state for next time.
    function void post_randomize();
        opcode  = opcode_e'(next_op);   // force to current state
        next_op = ~next_op;              // flip for next call
    endfunction

    function new(string name = "rw_txn");
        super.new(name);
    endfunction
endclass

// Sequence that generates alternating R/W:
class alt_rw_seq extends uvm_sequence #(rw_txn);
    `uvm_object_utils(alt_rw_seq)

    function new(string name = "alt_rw_seq");
        super.new(name);
    endfunction

    task body();
        rw_txn txn;
        repeat (10) begin
            txn = rw_txn::type_id::create("txn");
            start_item(txn);
            // randomize() -> solver picks random addr/data,
            // then post_randomize() forces opcode to alternate
            assert(txn.randomize()) else `uvm_fatal("SEQ", "Randomize failed");
            `uvm_info("SEQ", $sformatf("opcode=%s addr=0x%h",
                      txn.opcode.name(), txn.addr), UVM_MEDIUM)
            finish_item(txn);
        end
        // Output: READ, WRITE, READ, WRITE, READ, WRITE, ...
    endtask
endclass

// Alternative: enforce alternation using a constraint that references
// the previous opcode (stored in the sequence, not the txn):
class alt_rw_seq_v2 extends uvm_sequence #(rw_txn);
    `uvm_object_utils(alt_rw_seq_v2)
    opcode_e last_op = WRITE;   // start with WRITE so first will be READ

    task body();
        rw_txn txn;
        repeat (10) begin
            txn = rw_txn::type_id::create("txn");
            start_item(txn);
            assert(txn.randomize() with {
```

```
                txn.opcode != last_op;   // inline: must differ from last
            });
            last_op = txn.opcode;   // update state
            finish_item(txn);
        end
    endtask
endclass
```

解释

post_randomize() 是 SV 的 randomize() 完成后自动调用的钩子函数（hook）。

用它来做"后处理"：先让 solver 随机化其他字段，再用 post_randomize() 强制设置 opcode。

static bit next_op 作为类级别的状态在多次 randomize() 之间持久化。

另一种方案（v2）：在 sequence 里维护 last_op，用 inline constraint 排除上一次的值。

两种方法的区别：post_randomize 是对象内部状态，更封装；

inline constraint 是 sequence 层状态，更灵活（可以在不同 sequence 间传递状态）。

SystemVerilog — Assertions (SVA)

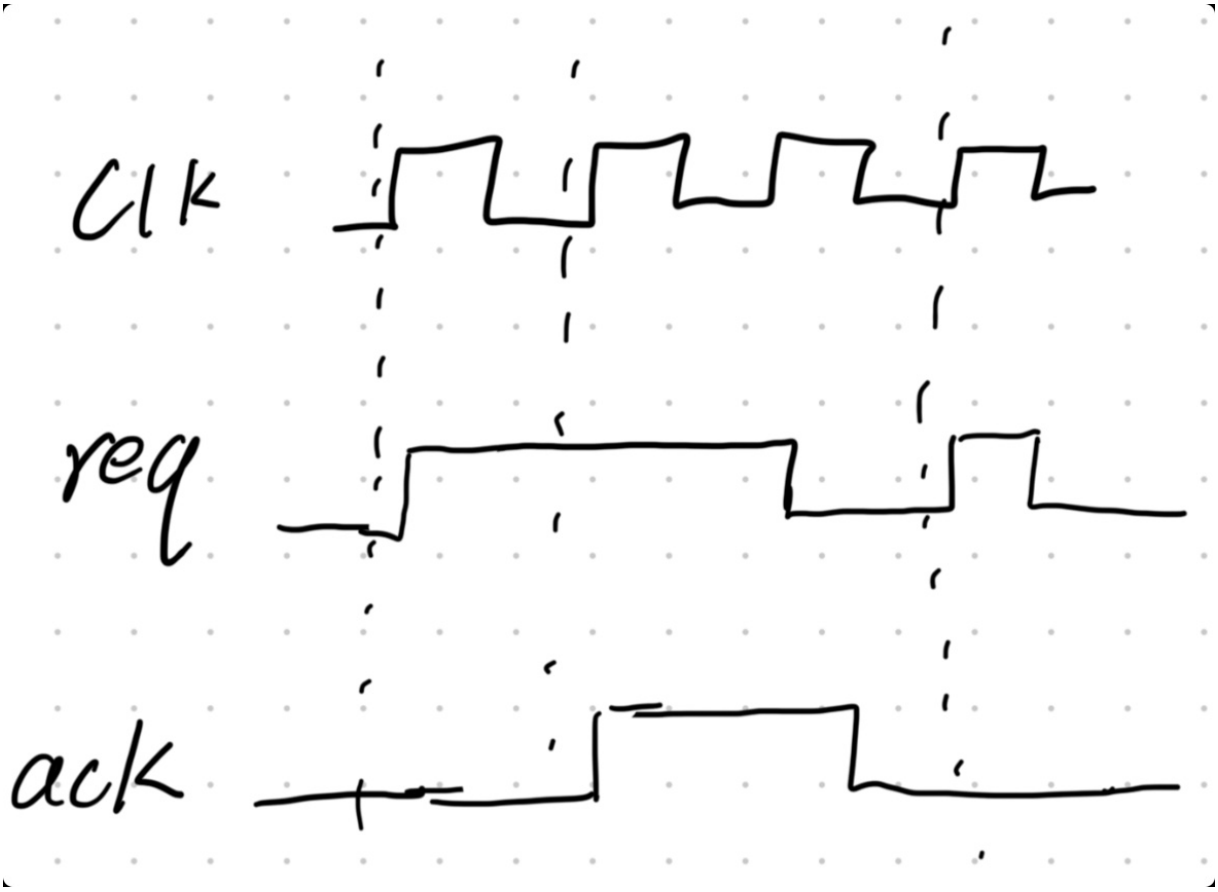
Q68. Assertions (SVA): req |=> ack Handshake Protocol

QUESTION

Observe the waveform below showing a req/ack handshake protocol.

Describe how the protocol works, identify the bug case in the waveform,

determine whose bug it is, and write an SVA assertion to catch it.



The waveform shows:

- clk: system clock

- req: request signal (asserted by the master/initiator)

- ack: acknowledge signal (asserted by the slave/target, one cycle after req)

Correct behavior: whenever req is asserted, ack must be asserted exactly

one cycle later. The bug case shows req asserted with no ack response.

(a) How does the handshake protocol work?

(b) What is the bug? Whose bug is it?

(c) Write an SVA assertion to catch the bug.

ANSWER

```
// (a) Protocol behavior (from the correct waveform section):
//
// T1: req rises (master asserts a request)
// T2: req held high; ack not yet asserted
// T3: ack rises exactly one cycle after req -> handshake accepted
// T4-T6: both req and ack held high (transaction in progress)
// T7: req falls (master deasserts after done)
// T8: ack falls one cycle later (slave acknowledges completion)
//
// Rule: req high at posedge clk -> ack must be high at the NEXT posedge clk
// In SVA implication: req |=> ack   (|=> is non-overlapping implication)

// (b) Bug description:
//
// Bug case (T8-T9 in waveform):
//   req rises again at T8, but ack never responds.
```

```
//   At T9 (one cycle later), ack remains low -> violation!
//
// Whose bug: the SLAVE (target) is at fault.
//   The master correctly followed protocol by asserting req.
//   The slave failed to respond with ack within one cycle.
//   This could indicate the slave is stuck, busy, or has a control bug.

// (c) SVA assertion to catch the bug:
//
property req_ack_handshake;
    @(posedge clk) disable iff (!rst_n)
    req |=> ack;
endproperty

assert property (req_ack_handshake)
    else $error("HANDSHAKE VIOLATION: req was high but ack did not follow next cycle");

// Breakdown of the assertion:
//   @(posedge clk)  -- evaluate at every rising clock edge
//   disable iff (!rst_n) -- disable check during reset (avoid false fails)
//   req             -- antecedent: req is high at this posedge
//   |=>             -- non-overlapping implication: check the NEXT cycle
//   ack             -- consequent: ack must be high at the next posedge
//
// Additional assertions for full protocol coverage:
property ack_only_after_req;
    @(posedge clk) disable iff (!rst_n)
    ack |-> $past(req, 1);   // ack should only appear if req was high last cycle
endproperty

assert property (ack_only_after_req)
    else $error("PROTOCOL VIOLATION: ack asserted without prior req");
```

解释

req |=> ack 是 SVA 中最经典的握手断言之一：

|=> 是非重叠蕴含（non-overlapping implication），表示前项成立的下一个时钟周期，

后项必须成立。与 |-> （重叠蕴含，同一周期）不同，|=> 自带一个时钟延迟。

disable iff (!rst_n) 在复位期间禁用断言，避免复位时的初始不定态触发误报。

Bug 归因：req 是主设备（master）发出的请求，ack 是从设备（slave）的响应。

波形中 req 正确拉高但 ack 没有跟随，说明从设备没有响应，是从设备的 bug。

补充断言 ack |-> $past(req,1) 检查反方向：ack 不应该在没有 req 的情况下出现，

$past(req,1) 表示上一个时钟周期的 req 值。两条断言共同覆盖握手协议的完整正确性。

Q69. Assertions (SVA): AHB-Lite Burst Must Not Cross 1KB Boundary

QUESTION

Write an SVA assertion to check that an AHB-Lite burst transfer never

crosses a 1KB (1024-byte) address boundary.

Signals: HCLK, HADDR[31:0], HBURST[2:0], HSIZE[2:0], HTRANS[1:0], HREADY.

ANSWER

```
// The property uses a local variable `beats` to decode HBURST,
```

```
// then checks that the last beat's address still falls within the same 1KB page.
//
// Key signals:
//   HREADY   -- transfer is accepted this cycle (not stalled)
//   HTRANS   -- 2'b10 = NONSEQ: start of a new burst
//   HBURST   -- burst type (encodes number of beats)
//   HSIZE    -- transfer size: bytes_per_beat = 1 << HSIZE
//   HADDR    -- start address of the burst
//
// Why HREADY is needed:
//   Without HREADY, the assertion could trigger on stalled cycles
//   where HTRANS=NONSEQ is held but no transfer has actually started.
//
// Boundary check formula:
//   HADDR[9:0] + beats * (1 << HSIZE) - 1 <= 1023
//
//   HADDR[9:0]           = offset within the current 1KB block
//   beats * (1<<HSIZE)   = total bytes transferred
//   - 1                  = address of the LAST byte in the burst
//   <= 1023              = last byte must still be within [0..1023] of the block
//
// This is equivalent to: HADDR[9:0] + total_bytes <= 1024
//
property p_no_1kb_cross;
    int unsigned beats;

    @(posedge HCLK) disable iff (!HRESETn)
    (
        HREADY &&
        HTRANS == 2'b10 &&        // NONSEQ: new burst starting
        (
            beats = (HBURST == 3'b011) ? 4  :  // INCR4
                    (HBURST == 3'b101) ? 8  :  // INCR8
                    (HBURST == 3'b111) ? 16 : 1   // INCR16 : default SINGLE
        )
    )
    |->
    (
        int'(HADDR[9:0]) + beats * (1 << HSIZE) - 1 <= 1023
    );
endproperty

assert property (p_no_1kb_cross)
    else $error("AHB 1KB boundary violation: HADDR=0x%08h HBURST=%0b HSIZE=%0b",
                HADDR, HBURST, HSIZE);

// HBURST beat counts (for reference):
//   3'b000 SINGLE -> 1 beat   (no burst, skip check)
//   3'b001 INCR   -> unbounded (skip check, unknown length)
//   3'b010 WRAP4  -> 4 beats
//   3'b011 INCR4  -> 4 beats
//   3'b100 WRAP8  -> 8 beats
//   3'b101 INCR8  -> 8 beats
//   3'b110 WRAP16 -> 16 beats
//   3'b111 INCR16 -> 16 beats
```

解释

1KB 边界的关键：地址 bit[9:0] 是 1KB 块内的偏移，bit[31:10] 是块号。

burst 跨越边界等价于：第一个 beat 的 bit[31:10] != 最后一个 beat 的 bit[31:10]。

等价判断：HADDR[9:0] + total_bytes > 1024（因为1024 = 0x400，刚好是下一个1KB块的起点）。

SVA 在 HTRANS==NONSEQ（burst 起始）时采样并在同周期用 |-> 检查，

是 AHB 协议验证中最经典的时序断言之一。

Q70. Assertions (SVA): Check Clock Frequency is 100 MHz

QUESTION

Write an SVA assertion to verify that the clock signal clk has a frequency of exactly 100 MHz (period = 10 ns).

ANSWER

```
// Approach: measure the time between two consecutive rising edges.
// At 100 MHz, the period must be exactly 10 ns (10000 ps).
//
// SVA does not natively express real-time delays, so we use $realtime
// and a local variable to record the previous edge timestamp.

// Method 1: SVA with $realtime sampling
real prev_rise_time;
real period_ns;
bit  first_edge = 1;

always @(posedge clk) begin
    if (first_edge) begin
        first_edge     = 0;
        prev_rise_time = $realtime;
    end else begin
        period_ns      = ($realtime - prev_rise_time) / 1.0;  // in simulation units
        prev_rise_time = $realtime;
    end
end

// SVA assertion: after the first edge, measured period must equal 10 ns
property clk_100mhz;
    @(posedge clk) disable iff (first_edge)
    // period_ns must equal 10.0 ns (allow 0 tolerance for ideal clock)
    (period_ns == 10.0);
endproperty

assert property (clk_100mhz)
    else $error("CLK frequency violation: period = %0f ns (expected 10.0 ns)", period_ns);

// Method 2: Cycle-counting approach (cleaner for integer-period clocks)
// Use a 1-ns reference tick and count ticks between posedges.
// More robust for simulation where clk is defined by integer delays.
//
// In the tb, define:
//   localparam EXPECTED_PERIOD_NS = 10;
//   localparam real CLK_TOL_PS    = 100.0;   // +/- 100 ps tolerance
//
property clk_period_check;
    realtime t_rise;
    @(posedge clk)
    (1, t_rise = $realtime)    // capture timestamp at this posedge
    ##1 (1)                    // wait for next posedge (1 cycle later in SVA)
    |->
    // Check period within tolerance
    ($realtime - t_rise >= 9.9 * 1ns) and
    ($realtime - t_rise <= 10.1 * 1ns);
endproperty
```

```
assert property (clk_period_check)
    else $error("CLK period out of spec: %0t (expected 10 ns)", $realtime);

// Note: for synthesis-independent checking, some verification tools
// provide built-in clock-frequency checkers via bind or checkers blocks.
// The approach above works in simulation with VCS/Questa.
```

解释

检查时钟频率的两种方法：

1. 用 always @(posedge clk) 记录前一个上升沿的 $realtime，在下一个上升沿计算差值。

period_ns 作为 real 变量在 SVA property 中使用。

2. SVA 内置 local variable：用 (1, t_rise = $realtime) 在序列匹配时捕获时间戳，

##1 后的同一 posedge 检查时间差是否在容差内。

实际工程中允许一定的容差（如 ±100ps），因为 PLL jitter 和仿真精度会引入偏差。

纯 SVA 检查时钟频率适合验证 DUT 内部生成的时钟（如 divide-by-N 时钟），

顶层参考时钟通常在 tb_top 里直接定义，不需要用断言检查。

Q71. Assertions (SVA): Simple Fell-Triggered Assertion

QUESTION

Write an SVA assertion: when signal a falls (1->0), signal b must

remain low (0) for at least 2 consecutive cycles afterward.

ANSWER

```
// $fell(a): true on the clock edge where a transitions 1->0
// !b[*2:$]: b must be 0 for 2 or more consecutive cycles
//
property p_a_fell_b_low;
    @(posedge clk)
    $fell(a) |-> !b [*2:$];
endproperty

assert property (p_a_fell_b_low)
    else $error("Violation: b went high within 2 cycles after a fell");

// Breakdown of the sequence operators:
//   $fell(a)    : a was 1 on previous cycle, 0 on this cycle
//   |->         : overlapping implication (check starts same cycle as trigger)
//   !b [*2:$]   : b==0 repeated 2 to infinity times consecutively
//                 i.e., b stays low for at least 2 cycles (including the trigger cycle)
//
// Variant: if "at least 2 cycles AFTER the fall" (non-overlapping):
//   $fell(a) |=> !b [*2:$];   // |=> shifts check to next cycle

// Cover property (check this scenario actually occurs in simulation):
cover property (@(posedge clk) $fell(a));
```

解释

$fell(a) |-> !b[*2:$] 是最常见的 SVA 模式之一：

触发条件（antecedent）是信号下降沿，结果（consequent）是另一信号的保持约束。

[*2:$] 是"重复2次到无限次"的连续重复操作符，表示至少持续2个周期。

|-> (overlapping) 和 |=> (non-overlapping) 的区别：

overlapping：trigger 那一拍开始检查；non-overlapping：下一拍才开始检查。

面试中能立即写出 $fell/$rose 配合 [*N:$] 的 consecutive repetition 是基本功。

Q72. Assertions (SVA): Complete req-ack Handshake Suite

QUESTION

Write a complete SVA property suite for a req-ack handshake:

- req followed by ack within 5 cycles
- ack must have a corresponding req
- req and ack are single-cycle pulses (not held)
- Support back-to-back transactions

ANSWER

```
// Handshake protocol:
//   req: single-cycle pulse from master (deasserts next cycle)
//   ack: single-cycle pulse from slave, within 5 cycles of req
//   Back-to-back: a new req can be issued immediately after prev ack

// P1: req is a single-cycle pulse (not held for 2+ cycles)
property p_req_pulse;
    @(posedge clk) disable iff (!rst_n)
    req |=> !req;
endproperty
assert property (p_req_pulse)
    else $error("req held for more than 1 cycle");

// P2: ack is a single-cycle pulse
property p_ack_pulse;
    @(posedge clk) disable iff (!rst_n)
    ack |=> !ack;
endproperty
assert property (p_ack_pulse)
    else $error("ack held for more than 1 cycle");

// P3: every req must be followed by an ack within 5 cycles
property p_req_gets_ack;
    @(posedge clk) disable iff (!rst_n)
    req |-> ##[1:5] ack;
endproperty
assert property (p_req_gets_ack)
    else $error("req not acknowledged within 5 cycles");

// P4: every ack must have a preceding req within the last 5 cycles
// (no spurious ack without a req)
property p_ack_needs_req;
    @(posedge clk) disable iff (!rst_n)
    ack |-> $past(req, 1) || $past(req, 2) || $past(req, 3) ||
            $past(req, 4) || $past(req, 5);
endproperty
assert property (p_ack_needs_req)
    else $error("ack received with no preceding req in last 5 cycles");

// P5: back-to-back support -- new req can appear immediately after ack
// (no mandatory idle cycle between transactions)
property p_back_to_back;
```

```
    @(posedge clk) disable iff (!rst_n)
    (ack ##1 req) |-> ##[1:5] ack;   // back-to-back req also gets ack
endproperty
assert property (p_back_to_back)
    else $error("back-to-back req not acknowledged within 5 cycles");

// Cover properties (verify scenarios actually occur in simulation)
cover_req_ack:       cover property (@(posedge clk) req ##[1:5] ack);
cover_max_latency:   cover property (@(posedge clk) req ##5 ack);    // worst case
cover_min_latency:   cover property (@(posedge clk) req ##1 ack);    // 1-cycle resp
cover_back_to_back:  cover property (@(posedge clk) req ##[1:5] ack ##1 req);
```

解释

五条属性覆盖了握手协议的完整语义：

P1/P2：信号脉冲性（防止被 hold 住），这是 P3 依赖的基础假设。

P3：liveness 属性（好事终会发生），用 |-> ##[1:5] 表示"1到5拍内必须出现 ack"。

P4：safety 属性（坏事不会发生），防止没有 req 的情况下出现 spurious ack。

P5：back-to-back 覆盖——ack 之后立即来的 req 也必须得到响应，验证流水线吞吐。

$past(req, N) 用于检查过去 N 拍的信号值，是验证"先因后果"的标准写法。

Cover 属性配合：验证最大延迟（5拍）和最小延迟（1拍）的场景都被激励覆盖到。

SystemVerilog — Coding

Q73. SystemVerilog: Disable Specific Named Fork Branch

QUESTION

In SystemVerilog, if you use `disable fork`, it disables ALL concurrent child processes. How do you disable only ONE specific fork branch without killing the others?

ANSWER

```
// Problem: disable fork kills ALL threads in the current scope.
//
// fork
//   begin thread_A; end
//   begin thread_B; end
//   begin thread_C; end
// join_none
// disable fork;  // kills A, B, AND C -- too aggressive

// Solution: give the specific fork branch a NAME, then disable by name.
//
// Syntax: fork : <fork_name> ... join  (named fork block)
// Disable: disable <fork_name>;        (kills only that named fork)

// Example 1: disable only the timeout thread, let the main thread continue
//
task run_with_timeout(input int timeout_ns);
    fork
```

```
        // Branch A: main work (we want to keep this if timeout fires)
        begin : main_work
            do_transaction();
            -> ev_done;
        end

        // Branch B: timeout watchdog (named so we can kill just this one)
        begin : timeout_wdog
            #timeout_ns;
            `uvm_error("TIMEOUT", "Transaction timed out!")
        end
    join_none

    // Wait for main work to finish
    @ev_done;
    // Kill only the timeout watchdog, not main_work
    disable timeout_wdog;
endtask

// Example 2: UVM driver -- disable only the reset handler, not the main loop
//
task run_phase(uvm_phase phase);
    fork
        // Main driving loop -- must NOT be killed
        begin : drive_loop
            forever begin
                seq_item_port.get_next_item(req);
                drive_item(req);
                seq_item_port.item_done();
            end
        end

        // Reset monitor -- kill this when reset deasserts
        begin : reset_handler
            forever begin
                @(negedge vif.rst_n);
                vif.valid <= 0;
                @(posedge vif.rst_n);
            end
        end
    join_none

    // To kill only reset_handler later:
    // disable reset_handler;
    // drive_loop continues running undisturbed
endtask

// Key distinction:
//   disable fork          -> kills ALL child threads of current process
//   disable <block_name>  -> kills ONLY that named block + its children
//
// Named fork syntax variations:
//   fork : my_fork ... join         (join: wait for all)
//   fork : my_fork ... join_any     (join_any: wait for first)
//   fork : my_fork ... join_none    (join_none: don't wait)
//   begin : my_block ... end        (named sequential block, also disableable)
//
// Disable propagates to children:
//   Disabling a named block also kills all threads launched inside it.
//   Use this to cleanly tear down a sub-tree of threads.
```

解释

disable fork 太暴力：杀死当前 process 所有子线程。

named fork / named begin 是精确控制线程的标准方法。

给 fork branch 或 begin 块起名（fork : name 或 begin : name），

然后 disable name 只杀这一条分支，其他分支继续运行。

实际工程中最常见的用法：fork 出一个 timeout watchdog 分支（named），

主任务完成后 disable 掉 watchdog，避免它继续等待触发误报。

注意：disable 会传播到被 disable 的 block 内部所有子线程。

Q74. SystemVerilog: fork-join_any for Task Timeout Detection

QUESTION

Using fork-join_any, how do you detect that a task has exceeded a timeout?

ANSWER

```
// fork-join_any exits as soon as ONE of its threads completes.
// Pattern: race the main task against a timeout thread.
// Whoever finishes first causes join_any to unblock; the other is killed.

task run_with_timeout(input int timeout_ns);
    fork
        // Thread 1: the actual work
        begin : work_thread
            do_transaction();   // task under test
        end

        // Thread 2: timeout watchdog
        begin : timeout_thread
            #(timeout_ns * 1ns);
            `uvm_error("TIMEOUT",
                $sformatf("Task did not complete within %0d ns", timeout_ns))
        end
    join_any

    // Kill whichever thread is still running
    disable fork;
endtask

// How it works:
//   If do_transaction() finishes first -> join_any exits, disable fork kills timeout.
//   If #timeout_ns fires first -> join_any exits, error is reported,
//                                 disable fork kills work_thread.
//
// Cleaner: use named blocks so disable targets only these two threads,
// not ALL threads in the current process:

task run_with_timeout_safe(input int timeout_ns);
    fork : timeout_fork
        begin
            do_transaction();
        end
        begin
            #(timeout_ns * 1ns);
            `uvm_error("TIMEOUT", "Transaction timed out")
        end
    join_any
    disable timeout_fork;   // kills only timeout_fork's children
endtask
```

```
// UVM context: commonly used in drivers and monitors
task run_phase(uvm_phase phase);
    phase.raise_objection(this);
    run_with_timeout(1000);   // 1000 ns timeout
    phase.drop_objection(this);
endtask
```

解释

fork-join_any 是 timeout 检测的标准模式：用两个并行线程竞速。

正常完成：工作线程先结束，join_any 退出，disable fork 清理 timeout 线程。

超时：timeout 线程先结束，join_any 退出，报错，disable fork 清理工作线程。

关键细节：disable fork 会杀死当前 process 的所有子线程，

建议用 named fork（fork : name）配合 disable name 精确控制，

避免误杀同一 process 中其他不相关的并行线程（参考 Q37 named fork 题）。

Q75. SystemVerilog: Blocking vs Non-Blocking Assignment, Swap Without Temp

QUESTION

(a) What is the difference between blocking (=) and non-blocking (<=) assignments?

(b) How do you swap two variables WITHOUT a temporary variable in SV?

(c) How do you do the same in C?

ANSWER

```
// (a) Blocking vs Non-Blocking in SystemVerilog
//
// Blocking (=):
//   Executes immediately and in order within the same time step.
//   Next statement sees the updated value.
//   Used in: combinational always_comb, initial blocks, tasks for sequencing.
//
// Non-Blocking (<=):
//   RHS evaluated NOW, LHS updated at END of time step (NBA region).
//   All NBA updates happen simultaneously after all blocking statements.
//   Used in: sequential always_ff to model registered behavior correctly.
//
// Key example -- the classic race condition:
// With blocking (WRONG for flip-flops):
always @(posedge clk) begin
    a = b;   // a gets b immediately
    b = a;   // b gets the ALREADY-UPDATED a -> both become old b!
end
//
// With non-blocking (CORRECT):
always_ff @(posedge clk) begin
    a <= b;   // RHS sampled: old b
    b <= a;   // RHS sampled: old a -- both evaluated before any update
end           // After time step: a = old_b, b = old_a. Correct swap!

// (b) Swap without temp in SystemVerilog (non-blocking naturally does it!)
always_ff @(posedge clk) begin
    a <= b;   // Non-blocking: evaluates old values, updates simultaneously
    b <= a;   // This IS the swap -- no temp variable needed
```

```
end

// If using blocking (e.g. in a task), use XOR swap:
task swap_no_temp(ref int x, ref int y);
    x = x ^ y;   // x = x XOR y
    y = x ^ y;   // y = (x XOR y) XOR y = x
    x = x ^ y;   // x = (x XOR y) XOR x = y
endtask

// (c) Same in C using XOR:
// void swap(int *x, int *y) {
//     *x ^= *y;
//     *y ^= *x;
//     *x ^= *y;
// }
//
// Or arithmetic (risk of overflow):
// *x = *x + *y;
// *y = *x - *y;
// *x = *x - *y;
//
// Note: XOR swap fails when x and y point to the SAME variable:
//   x ^= x -> x = 0, then y = 0, x = 0. Always check x != y first.
```

解释

非阻塞赋值的"同时更新"特性天然实现无中间变量的 swap。

XOR swap 的原理：x^y^y = x，x^y^x = y，利用 XOR 的自反性。

C 中 XOR swap 的陷阱：两个指针指向同一变量时会被清零，这是面试中的经典追问。

Q76. SystemVerilog: Nested fork-join_any with Delays -- Print Order

QUESTION

What is the print order for the following code?

```
initial begin
    fork   // outer fork
        begin
            fork   // inner fork A
                begin #10; $display("A1"); end
                begin #20; $display("A2"); end
            join_any
            $display("After inner A");
        end
        begin
            fork   // inner fork B
                begin #15; $display("B1"); end
                begin #5;  $display("B2"); end
            join_any
            $display("After inner B");
        end
    join
    $display("After outer");
end
```

ANSWER

```
// Trace through simulation time steps:
```

```
//
// t=0:  All threads launched. Active: A1(@10), A2(@20), B1(@15), B2(@5)
//
// t=5:  B2 fires -> prints "B2"
//       Inner fork B: join_any satisfied (B2 done)
//       -> "After inner B" printed immediately
//       -> B1 (@15) is still running (not killed by join_any automatically)
//          Note: join_any does NOT kill remaining threads in inner fork B
//
// t=10: A1 fires -> prints "A1"
//       Inner fork A: join_any satisfied (A1 done)
//       -> "After inner A" printed
//
// t=15: B1 fires -> prints "B1"  (was still running in background)
//
// t=20: A2 fires -> prints "A2"  (was still running in background)
//
// After t=20: outer join waits for BOTH outer threads to complete.
//   Outer thread 1 (containing inner fork A): completed at t=10 when
//     "After inner A" printed? NO -- join_any only unblocks the parent,
//     but background threads from inner fork A still run.
//     Outer thread 1 completes at t=10 (when "After inner A" executes).
//   Outer thread 2 completes at t=5 (when "After inner B" executes).
//   Both outer threads complete by t=10.
//   -> "After outer" prints at t=10.
//
// Final print order:
//   t=5:  B2
//   t=5:  After inner B
//   t=10: A1
//   t=10: After inner A
//   t=10: After outer      <- outer join satisfied when both threads done
//   t=15: B1               <- background thread from inner fork B still runs
//   t=20: A2               <- background thread from inner fork A still runs
//
// KEY INSIGHT: join_any unblocks the PARENT thread but does NOT kill
// the remaining threads in the fork. They continue running in the background.
// This is why B1 and A2 still print after "After outer".
```

解释

join_any 的关键行为：只要一个分支完成就让父线程继续，但其他分支**继续在后台运行**，

不会被自动 kill。要 kill 其他分支必须显式 disable fork。

外层 join 要等两个外层线程都完成：

Thread1 在 t=10 执行完 "After inner A" 即完成；Thread2 在 t=5 完成。

所以外层 join 在 t=10 满足，"After outer" 在 t=10 打印。

B1 和 A2 的后台线程继续运行，分别在 t=15 和 t=20 打印。

Q77. SystemVerilog: fork-join / join_any / join_none + Watchdog Timer

QUESTION

(a) Explain the difference between fork-join, fork-join_any, and fork-join_none.

(b) Implement a watchdog timer using fork-join_any: a task must complete

within a timeout, otherwise report an error.

ANSWER

```
// (a) Three variants of fork:
//
// fork ... join
//   Parent BLOCKS until ALL child threads complete.
//   Used when: all parallel tasks must finish before continuing.
//
// fork ... join_any
//   Parent BLOCKS until ANY ONE child thread completes.
//   Remaining children continue running in the background.
//   Used when: whichever finishes first wins (e.g. timeout vs task race).
//
// fork ... join_none
//   Parent continues IMMEDIATELY without waiting for any child.
//   Children run entirely in the background.
//   Used when: fire-and-forget; parent doesn't need to wait.

//   Example:
//   fork
//       begin #10; $display("A"); end   // finishes at t=10
//       begin #30; $display("B"); end   // finishes at t=30
//   join      -> parent waits until t=30 (both done)
//   join_any  -> parent continues at t=10 (A done), B still running
//   join_none -> parent continues at t=0,  A and B run in background

// (b) Watchdog timer using fork-join_any
//
// Pattern: race the main task against a timeout watchdog.
// The first to finish unblocks the parent; the other is killed.

task run_with_watchdog(input int timeout_ns = 1000);
    fork : watchdog_fork
        // Thread 1: the actual work
        begin : work_thread
            do_main_task();              // task under test
            `uvm_info("WDG", "Task completed normally", UVM_MEDIUM)
        end

        // Thread 2: watchdog timer
        begin : wdog_thread
            #(timeout_ns * 1ns);
            `uvm_error("WDG", $sformatf(
                "TIMEOUT: task did not complete within %0d ns", timeout_ns))
        end
    join_any

    // Kill whichever thread is still running
    // Using named fork ensures we only kill these two, not other background threads
    disable watchdog_fork;
endtask

// Extended version: UVM context with objection
task run_phase(uvm_phase phase);
    phase.raise_objection(this);

    fork : wdog_fork
        begin
            run_test_sequence();         // main stimulus
        end
        begin
            #1us;                        // global test timeout: 1 microsecond
            `uvm_fatal("WDG", "Global test timeout! Simulation hung.")
        end
    join_any
```

```
    disable wdog_fork;
    phase.drop_objection(this);
endtask

// Key rules:
//   join_any is perfect for timeout: the "winner" is whoever finishes first.
//   After join_any, use disable <fork_name> to clean up the loser.
//   Do NOT use disable fork (without a name) -- it kills ALL threads
//   in the current process, including unrelated background threads.
//   Always name the fork block (fork : name) for precise control.
```

解释

三种 join 的记忆方法：

join = 等所有人；join_any = 等最快的那个；join_none = 不等任何人。

Watchdog 是 join_any 最经典的应用：task 和 timeout 赛跑，谁先完成谁"赢"。

disable watchdog_fork 的精确性是关键：用命名 fork 块保证只 kill 这两个线程，

不影响其他并行运行的后台进程（如 monitor、driver 的 forever 循环）。

join_none 启动后台线程不需要 disable，因为父线程根本没有等待。

Q78. SystemVerilog: ISA Implementation -- Struct vs OOP

QUESTION

Given a simple ISA with instruction types: ADD, LOAD, STORE, BRANCH.

Each type has different field layouts.

(a) Implement using packed struct + union (first instinct).

(b) Implement using OOP: abstract class, inheritance, polymorphism,

virtual function. Why does the interviewer prefer OOP?

When would each approach be used?

ANSWER

```
// ================================================================
// APPROACH 1: Struct + Union  (RTL / hardware-close coding)
// ================================================================
// Good for: RTL modeling, tight bit-field control, synthesis

typedef enum logic [1:0] {ADD=2'b00, LOAD=2'b01, STORE=2'b10, BR=2'b11} opcode_e;

// Per-type structs: each instruction has its own field layout
typedef struct packed {
    opcode_e       opcode;   // [31:30]
    logic [4:0]    rd;       // [29:25]
    logic [4:0]    rs1;      // [24:20]
    logic [4:0]    rs2;      // [19:15]
    logic [14:0]   unused;   // [14:0]
} r_type_t;   // ADD

typedef struct packed {
    opcode_e       opcode;   // [31:30]
    logic [4:0]    rd;       // [29:25]
    logic [4:0]    rs1;      // [24:20]
```

```
    logic [19:0]   imm;      // [19:0]
} i_type_t;   // LOAD (rd = MEM[rs1 + imm])

typedef struct packed {
    opcode_e       opcode;   // [31:30]
    logic [4:0]    rs1;      // [29:25]
    logic [4:0]    rs2;      // [24:20]
    logic [19:0]   imm;      // [19:0]
} s_type_t;   // STORE (MEM[rs1+imm] = rs2)

typedef struct packed {
    opcode_e       opcode;   // [31:30]
    logic [4:0]    rs1;      // [29:25]
    logic [4:0]    rs2;      // [24:20]
    logic [19:0]   offset;   // [19:0]
} b_type_t;   // BRANCH

// Union: one 32-bit word interpreted as any type
typedef union packed {
    logic [31:0] raw;    // raw bit access
    r_type_t     r;
    i_type_t     i;
    s_type_t     s;
    b_type_t     b;
} instr_u;

// Usage: decode by checking opcode field
// instr_u instr;
// case (instr.r.opcode)
//     ADD:   rd_val = rs1_val + rs2_val;
//     LOAD:  rd_val = mem[rs1_val + instr.i.imm];
//     STORE: mem[instr.s.rs1 + instr.s.imm] = rs2_val;
//     BR:    if (rs1_val == rs2_val) PC += instr.b.offset;
// endcase
//
// Limitation: data is separate from behavior (no methods).
// Adding a new instruction requires modifying the case statement everywhere.

// ================================================================
// APPROACH 2: OOP  (Verification / UVM / testbench coding)
// ================================================================
// Good for: testbench, UVM sequences, extensibility, factory pattern

// Abstract base class: defines the INTERFACE every instruction must implement
virtual class base_instr;
    logic [1:0] opcode;

    // Pure virtual: subclasses MUST override these
    pure virtual function string   get_name();
    pure virtual function string   decode();
    pure virtual function void     execute(ref logic [31:0] regs [32]);
endclass

// Concrete derived class: ADD
class add_instr extends base_instr;
    logic [4:0] rd, rs1, rs2;

    function new(logic [4:0] _rd, _rs1, _rs2);
        opcode = 2'b00; rd=_rd; rs1=_rs1; rs2=_rs2;
    endfunction

    virtual function string get_name(); return "ADD"; endfunction
```

```
    virtual function string decode();
        return $sformatf("ADD r%0d, r%0d, r%0d", rd, rs1, rs2);
    endfunction

    virtual function void execute(ref logic [31:0] regs [32]);
        regs[rd] = regs[rs1] + regs[rs2];
    endfunction
endclass

// Concrete derived class: LOAD (completely different fields)
class load_instr extends base_instr;
    logic [4:0]  rd, rs1;
    logic [19:0] imm;

    function new(logic [4:0] _rd, _rs1, logic [19:0] _imm);
        opcode=2'b01; rd=_rd; rs1=_rs1; imm=_imm;
    endfunction

    virtual function string get_name(); return "LOAD"; endfunction

    virtual function string decode();
        return $sformatf("LOAD r%0d, [r%0d + %0d]", rd, rs1, $signed(imm));
    endfunction

    virtual function void execute(ref logic [31:0] regs [32]);
        regs[rd] = regs[rs1] + 32'(signed'(imm));  // simplified
    endfunction
endclass

// BRANCH -- yet another field layout, handled cleanly by its own class
class branch_instr extends base_instr;
    logic [4:0]  rs1, rs2;
    logic [19:0] offset;

    function new(logic [4:0] _rs1, _rs2, logic [19:0] _off);
        opcode=2'b11; rs1=_rs1; rs2=_rs2; offset=_off;
    endfunction

    virtual function string get_name(); return "BR"; endfunction
    virtual function string decode();
        return $sformatf("BR r%0d==r%0d, +%0d", rs1, rs2, $signed(offset));
    endfunction
    virtual function void execute(ref logic [31:0] regs [32]);
        // PC update handled externally
    endfunction
endclass

// Polymorphic dispatch: ONE function handles ALL instruction types
function void run_program(base_instr prog[], ref logic [31:0] regs [32]);
    foreach (prog[i]) begin
        $display("[%0d] %s -> %s", i, prog[i].get_name(), prog[i].decode());
        prog[i].execute(regs);   // virtual dispatch to correct subclass
    end
endfunction

// Usage:
// base_instr prog[4];
// prog[0] = new add_instr(1, 2, 3);     // ADD r1, r2, r3
// prog[1] = new load_instr(4, 0, 100);  // LOAD r4, [r0+100]
// prog[2] = new add_instr(5, 1, 4);     // ADD r5, r1, r4
// prog[3] = new branch_instr(1, 5, -8); // BR r1==r5, -8
// run_program(prog, regs);  // polymorphism: calls correct execute() each time
```

```
// ================================================================
// When to use which?
// ================================================================
//
//   Struct + Union:          OOP:
//   - RTL/synthesis          - Testbench / UVM
//   - Fixed bit layout       - Extensible instruction set
//   - No dynamic dispatch    - Factory pattern (create by name)
//   - Synthesis-friendly     - Adding instr = new subclass only
//   - Manual case statements - Virtual dispatch = no case needed
//
// OOP advantage (Open-Closed Principle):
//   Adding a MUL instruction: create mul_instr class, done.
//   No changes to run_program(), base_instr, or any existing class.
//   With struct: must add a new union member AND update every case statement.
```

解释

两种方法各有适用场景，考官的追问方向是 OOP。

Struct+Union：贴近硬件位域，适合 RTL 建模和 packed 总线传输，综合工具友好。

OOP：适合验证环境，核心优势是"开闭原则"——对扩展开放，对修改关闭。

新增指令类型时，struct 方案需要改 union 和所有 case；

OOP 方案只需新建一个子类，已有代码零修改。

关键 OOP 概念：

abstract class / pure virtual：定义"接口契约"，强制所有子类实现 execute()/decode()。

inheritance：子类获得 opcode 字段和基础方法，各自添加独特字段。

polymorphism：run_program 通过 base_instr 句柄调用 execute()，

运行时自动分发到正确子类（虚函数表机制），无需 if/case 判断类型。

Verification — Test Plan

Q79. Verification: Async FIFO vs Circular FIFO vs Sync FIFO

QUESTION

What is the difference between an async FIFO, a circular FIFO, and a sync

FIFO? What are the pros and cons of each?

ANSWER

```
// Sync FIFO (synchronous):
//   Single clock domain for both read and write ports.
//   Pointer: binary counter with extra wrap bit (full/empty disambiguation).
//   Full:  wrap bits differ, addresses equal.
//   Empty: pointers identical.
//   Pros: simple RTL, straightforward STA, no CDC complexity, zero flag latency.
//   Cons: cannot bridge two independent clock domains.
//   Use:  pipeline buffering within one clock domain.
//
// Async FIFO (asynchronous):
```

```
//   Independent read and write clocks -- different frequencies or phases.
//   Pointer: Gray code, synchronized via 2-FF synchronizer into each domain.
//   Data stored in dual-port RAM; data does NOT pass through synchronizer.
//   Full/empty computed from synchronized (stale) pointer values.
//   Pros: safely bridges two clock domains, handles rate mismatch.
//   Cons: complex design and verification, CDC overhead, full/empty flags
//         are pessimistic (1-2 cycles stale), reset domain crossing requires
//         separate reset synchronizers on each side.
//   Use:  cross-domain data transfer (CPU<->DDR, sensor->processor, PCIe).
//
// Circular FIFO (ring buffer):
//   Structurally identical to sync FIFO -- head/tail pointers wrap modulo depth.
//   The term "circular" emphasizes the ring-buffer memory topology.
//   Often used in software/OS contexts (kernel ring buffers, network queues).
//   In hardware RTL, adds no new capability over sync FIFO.
//   Pros/Cons: same as sync FIFO.
//   Use:  audio buffers, DMA descriptor rings, OS packet queues.
//
// Summary:
//   Feature          | Sync FIFO   | Async FIFO        | Circular FIFO
//   -----------------|-------------|-------------------|---------------
//   Clock domains    | 1           | 2 (independent)   | 1
//   Pointer encoding | Binary      | Gray code         | Binary (mod N)
//   Complexity       | Low         | High              | Low
//   CDC safety       | No          | Yes               | No
//   Flag latency     | 0 cycles    | 1-2 sync cycles   | 0 cycles
```

解释

Async 和 Circular 最容易混淆：circular 只是"环形存储"的拓扑描述，本质仍是同步 FIFO。

Async FIFO 的核心设计挑战：Gray code 指针同步、full/empty flag 保守估计、

双边 reset 同步，这三点是面试考察的重点。

Q80. Verification: Async FIFO Potential Problems

QUESTION

What are the potential problems you may encounter with an async FIFO?

List and explain each one.

ANSWER

```
// 1. Metastability on pointer synchronization
//    2-FF synchronizer reduces but cannot eliminate metastability.
//    At very high clock rates or extreme frequency ratios, MTBF may be
//    unacceptably short. Fix: use 3-FF synchronizer or reduce clock frequency.
//
// 2. Conservative full/empty flags (stale pointer)
//    Synchronized pointer lags 1-2 rd_clk cycles behind the real value.
//    Write side sees a stale rd_ptr -> thinks FIFO is fuller than it is
//    -> full asserts early -> wasted bandwidth (unused slots).
//    Read side similarly underestimates occupancy -> empty asserts early.
//    This is safe but reduces effective throughput vs theoretical maximum.
//
// 3. Almost-full threshold < sync latency
//    If almost_full threshold is set smaller than the synchronizer latency
//    (e.g., threshold=1 but sync takes 2 cycles), the write side may
//    receive almost_full and send one more write, but the FIFO is already
//    full by the time that write arrives -> overflow.
```

```
//    Fix: threshold >= sync_latency_cycles (leave adequate margin).
//
// 4. Non-power-of-2 depth with Gray code
//    Standard binary-to-Gray (G = B ^ (B>>1)) only cycles cleanly through
//    2^N values. For depth=6, a custom Gray code sequence is required,
//    otherwise the pointer will traverse illegal states.
//
// 5. Simultaneous full=1 and empty=1
//    With large frequency ratios, write side sees stale rd_ptr (thinks full)
//    while read side sees stale wr_ptr (thinks empty) simultaneously.
//    This state is SAFE (both sides stall and wait for sync to catch up),
//    but a naive assertion full ^ empty will fire a false alarm.
//    Fix: check each flag independently in its own domain; do not assert
//    mutual exclusion across domains.
//
// 6. Reset domain crossing
//    Write-side and read-side resets may be asserted or released at different
//    times (different reset trees, different power domains).
//    If one side is reset while the other is not, pointers become inconsistent.
//    Fix: use a reset synchronizer in EACH domain:
//      always_ff @(posedge wr_clk or negedge rst_n)
//        if (!rst_n) wr_rst_sync <= 2'b00;
//        else        wr_rst_sync <= {wr_rst_sync[0], 1'b1};
//      assign wr_rst = wr_rst_sync[1];
```

解释

六大潜在问题：亚稳态 -> flag 滞后 -> threshold 不足 -> 非2幂次 -> full+empty同时 -> reset跨域。

面试中最容易被忽略的是第5点（full+empty 同时）和第6点（reset CDC）。

第5点是"看似 bug 实为 feature"的典型：两边都停下等同步追上，数据安全。

第6点在多 power domain 设计（如 Apple M 系列）中非常实际。

Q81. Verification: Simulation Runs Forever -- Debug Strategy

QUESTION

During verification, your simulation runs forever and never terminates.

How do you systematically debug this?

ANSWER

```
// Step 1: Add a timeout watchdog (if not already present)
//
//   initial begin
//     #(MAX_SIM_TIME_NS * 1ns);
//     $display("[TIMEOUT] Sim exceeded time limit at %0t", $time);
//     $finish;
//   end
//
// Step 2: Find who is keeping simulation alive
//   In UVM, simulation ends when all objections are dropped.
//   Add +UVM_OBJECTION_TRACE to the simulator command:
//     ./simv +UVM_TESTNAME=my_test +UVM_OBJECTION_TRACE -l sim.log
//   This prints every raise_objection and drop_objection call.
//   Look for a raise with no matching drop -> that component is the culprit.
//
// Step 3: Common causes and fixes
//
```

```
//   Cause A: UVM objection never dropped
//     phase.raise_objection(this) without matching phase.drop_objection(this).
//     Fix: ensure every raise has a matching drop; check error/exception paths
//     that may exit a task without reaching the drop call.
//
//   Cause B: Infinite loop waiting for a signal that never toggles
//     forever @(posedge handshake_done);  // signal never asserts
//     Fix: add fork/join_any timeout around the wait:
//       fork
//         @(posedge handshake_done);
//         begin #TIMEOUT_NS; `uvm_fatal("TIMEOUT","handshake_done never seen"); end
//       join_any
//       disable fork;
//
//   Cause C: AXI / handshake deadlock
//     Producer waits for ready before asserting valid (violates AXI rule).
//     Consumer waits for valid before asserting ready.
//     Both stall forever.
//     Fix: check protocol compliance; valid must NOT depend on ready.
//
//   Cause D: Clock or reset not generated
//     DUT stalled waiting for clock edges that never come, or reset never
//     deasserts so DUT stays in reset state indefinitely.
//     Fix: verify clock generator and reset release sequence in tb_top.
//
//   Cause E: Empty forever loop in zero-time
//     always @(*) x = x;  // zero-delay loop -> infinite simulation time 0
//     Fix: check always_comb blocks for circular combinational dependencies.
//
// Step 4: Dump waveforms and inspect
//   Enable waveform dump at start of sim, pause at timeout and examine:
//   which signals last toggled? which tasks are still executing?
//   Add $display checkpoints in suspected infinite loops to locate the hang.
```

解释

+UVM_OBJECTION_TRACE 是最快速定位"仿真不结束"问题的工具，面试必答。

Objection 泄漏是最常见原因：某个 sequence 或 driver 在错误路径上退出，

没有执行 drop_objection，导致 UVM 永远等待该 objection 被 drop。

AXI 死锁（valid 等 ready，ready 等 valid）是第二常见原因，

与 Q29 的 AXI FSM 规则直接关联。

Q82. Verification: Cannot Reproduce Regression Bug

QUESTION

You find a bug in regression but cannot reproduce it locally. How do you

approach this systematically?

ANSWER

```
// Regression bugs that don't reproduce are almost always due to:
// random seed differences, timing non-determinism, or environment differences.
//
// Step 1: Capture and reuse the exact seed (most important)
//   UVM prints the seed at simulation start:
//   "UVM_INFO @ 0: reporter [RNTST] Running test my_test with seed 12345678"
//   Re-run with the EXACT seed:
```

```
//     ./simv +UVM_TESTNAME=my_test +ntb_random_seed=12345678   (VCS)
//     vsim ... -sv_seed 12345678                                (Questa)
//   If you have the seed, reproduction is DETERMINISTIC.
//   Always save seeds in regression logs -- treat a missing seed as a bug.
//
// Step 2: Check for non-deterministic sources (if seed doesn't help)
//   - $random without explicit seed (time-based seed on some tools)
//   - Race conditions in testbench (two always blocks writing same signal)
//   - File I/O ordering (parallel jobs writing shared log file)
//   - Floating-point operations (platform-dependent rounding)
//   - $time differences (if test uses real time as stimulus)
//
// Step 3: Determine if it is seed-dependent or deterministic
//   Run the same test 20x with fixed directed (non-random) stimulus.
//   If it reproduces -> functional bug, not randomization-dependent.
//   If it never reproduces -> likely seed-dependent; analyze the failing
//   regression log to extract what input pattern triggered the bug.
//
// Step 4: Delta debugging -- shrink the reproducing case
//   Remove half the random stimulus, check if bug still occurs.
//   Repeat bisection until minimal reproducing sequence is found.
//   Convert minimal sequence to a directed test for stable reproduction.
//
// Step 5: Add instrumentation to regression for next time
//   Increase verbosity (+UVM_VERBOSITY=UVM_HIGH) and save full transaction logs.
//   Log all scoreboard comparisons with input values.
//   Save waveforms for any failing test (conditional dump on error).
//   Print a "state snapshot" from the scoreboard reference model at every error.
```

解释

第一步永远是找 seed——有了 seed 就有了确定性复现的钥匙。

"没有保存 seed" 本身就是 regression 基础设施的 bug，面试中主动提出这点是加分项。

Delta debugging（二分缩小）是面试中很受欢迎的回答，体现了系统性思维。

实际工程中"在 regression 里加 waveform dump on error"是很多公司的标准做法，

因为 regression 服务器跑完就清理，没有 waveform 就等于没有证据。

Q83. Verification: Sources of X-Propagation in Simulation

QUESTION

What are the sources of X-propagation in RTL simulation?

How do you detect and fix each one?

ANSWER

```
// X (unknown) represents an indeterminate value in simulation.
// X propagates through logic: AND(X,0)=0, AND(X,1)=X, OR(X,1)=1, OR(X,0)=X.
//
// Source 1: Uninitialized registers (most common)
//   In simulation, reg/logic variables start as X (not 0).
//   Flip-flops hold X until the first reset edge.
//   Fix: ensure reset covers ALL registers; verify reset sequence activates.
//
// Source 2: Incomplete reset coverage
//   Some FFs are accidentally excluded from the reset tree (missing if(!rst)).
//   They remain X forever, spreading X to all downstream logic.
//   Fix: lint check for "missing reset" (SpyGlass, Questa Lint).
```

```
//   In synthesis, confirm all FFs have a reset pin connected.
//
// Source 3: Tristate bus contention
//   Two drivers simultaneously drive logic 0 and 1 onto a shared wire -> X.
//   Or tristate enable asserted by multiple masters simultaneously.
//   Fix: arbitration logic must guarantee mutual exclusion on bus enable.
//
// Source 4: Gate-level simulation with SDF back-annotation (timing X)
//   When a flip-flop's setup or hold time is violated in gate-level sim,
//   the simulator intentionally injects X at the FF output to model
//   metastability. This is correct behavior, not a testbench bug.
//   Fix: fix the timing violation (RTL restructure, cell sizing, routing).
//
// Source 5: Unconnected module ports
//   An output port left unconnected in instantiation -> downstream sees X.
//   An input port not connected -> X driven into the module.
//   Fix: use -Wport or +lint=CNNCT in VCS; connect or tie off all ports.
//
// Source 6: Memory / array elements never written
//   Uninitialized SRAM or register file locations return X when read.
//   Fix: initialize memory in initial block or through a reset/bringup sequence.
//
// Source 7: casex / casez with X in the case expression
//   casex treats X as don't-care in BOTH pattern AND expression.
//   If the case expression itself contains X, casex may match unexpected arms.
//   Fix: use unique case or case (default: covers all) instead of casex.
//   Avoid casex for synthesizable design signals.
//
// Source 8: Division or modulo by zero
//   In SystemVerilog: a / 0 = X,  a % 0 = X.
//   Fix: add a guard: result = (divisor != 0) ? a / divisor : '0;
//
// Detection tools:
//   RTL simulation: +xprop (VCS), -xprop (Questa) -- pessimistic X spreading
//   Lint: SpyGlass xprop rules, Questa Lint X-sensitivity analysis
//   Formal: JasperGold X-propagation app catches X escaping module boundaries
```

解释

X 传播八大来源，优先级排序：

1-2（未初始化/reset 不完整）最常见，几乎所有新设计都会遇到。

4（SDF 时序注入）最容易误判为"testbench bug"，实际是时序违例的正确建模。

7（casex X 匹配）是最容易在 RTL 中埋下隐患的写法，应该在 coding guideline 中禁止。

面试中能说出 +xprop 工具和"X 传播方向"（AND门0可以阻断X，OR门1可以阻断X）是加分项。

Q84. Verification: SCB Design with Leading Drop and Garbage Bits

QUESTION

A DUT takes a 20-bit input word and outputs a stream of 1-bit values.

Two known imperfections exist:

- Input: DUT may DROP the first 0..19 bits (never drops from the middle)

- Output: first 0..3 bits may be GARBAGE (invalid data)

Design a scoreboard to verify that the output stream correctly reflects

the (possibly truncated) input data after skipping garbage bits.

Note: clarify with the interviewer whether input and output are correlated!

ANSWER

```
// Key clarification (often missed in interviews):
// Ask: "Are input and output the same transaction, just serialized?"
// This DUT converts 20-bit parallel input -> 1-bit serial output.
// They are NOT independent signals -- the output IS the input, serialized.
// Without this clarification, you might design two separate queues
// that never get correlated, wasting the entire interview.

// SCB architecture:
//
//   Input monitor  -> inp_queue (accumulate bits from 20-bit word)
//   Output monitor -> out_queue (accumulate 1-bit stream)
//
//   When inp_queue has >= 20 bits AND out_queue has >= 23 bits:
//     Attempt sliding window match
//     If match found: drain queues, record (drop, garbage) counts
//     If no match:    report error

class my_scoreboard extends uvm_scoreboard;
    bit inp_queue[$];   // serialized input bits
    bit out_queue[$];   // output stream bits

    uvm_analysis_imp_inp #(inp_txn, my_scoreboard) inp_imp;
    uvm_analysis_imp_out #(bit,     my_scoreboard) out_imp;

    // Serialize 20-bit input word into inp_queue (MSB first)
    function void write_inp(inp_txn t);
        for (int i = 19; i >= 0; i--)
            inp_queue.push_back(t.data[i]);
        try_match();
    endfunction

    // Accumulate 1-bit output stream
    function void write_out(bit b);
        out_queue.push_back(b);
        try_match();
    endfunction

    function void try_match();
        // Wait until we have enough bits to attempt alignment
        // inp: 20 bits (full word)
        // out: 23 bits (max_garbage=3 + 20 data bits)
        if (inp_queue.size() < 20 || out_queue.size() < 23) return;

        // Two-level sliding window:
        // Outer loop: drop = 0..19  (DUT dropped first N input bits)
        // Inner loop: garbage = 0..3 (output has M leading garbage bits)
        for (int drop = 0; drop <= 19; drop++) begin
            for (int garbage = 0; garbage <= 3; garbage++) begin
                int valid_inp_bits = 20 - drop;         // bits remaining in input
                int match = 1;

                // Check: inp[drop..19] == out[garbage..garbage+valid_inp_bits-1]
                if (out_queue.size() < garbage + valid_inp_bits) begin
                    match = 0;
                end else begin
                    for (int k = 0; k < valid_inp_bits; k++) begin
                        if (inp_queue[drop + k] !== out_queue[garbage + k]) begin
                            match = 0;
                            break;
                        end
                    end
```

```
                end

                if (match) begin
                    // Found a valid alignment
                    `uvm_info("SCB", $sformatf(
                        "MATCH: drop=%0d input bits, garbage=%0d output bits",
                        drop, garbage), UVM_MEDIUM)

                    // Drain inp_queue: remove 20 bits (full input word)
                    repeat(20) inp_queue.delete(0);

                    // Drain out_queue: remove garbage + valid data bits
                    repeat(garbage + valid_inp_bits) out_queue.delete(0);

                    return;  // done for this transaction
                end
            end
        end

        // No valid alignment found -> error
        `uvm_error("SCB", "No valid alignment found -- output does not match input")
    endfunction

    // End of test: queues should be empty (or nearly empty)
    function void check_phase(uvm_phase phase);
        if (inp_queue.size() > 0)
            `uvm_warning("SCB", $sformatf(
                "%0d input bits left unmatched", inp_queue.size()))
        if (out_queue.size() > 0)
            `uvm_warning("SCB", $sformatf(
                "%0d output bits left unconsumed", out_queue.size()))
    endfunction

endclass

// Why 20 bits for inp threshold and 23 for out threshold:
//   inp: one full 20-bit word = 20 bits
//   out: worst case = 3 garbage + 20 data bits = 23 bits
//        (need all 23 to guarantee we can check even the max-garbage case)
//
// Limitation: random bit patterns can accidentally match at wrong alignment.
// Mitigation: use constrained-random patterns with known non-repeating
//             sequences (e.g., walking 1s, counter values) to reduce
//             false match probability.
```

解释

这道题最重要的考察点是沟通：input 和 output 是同一条数据流（并行转串行），

不是两个独立信号。面试中一定要先问清楚再开始设计 SCB。

双层滑动窗口的核心：

- 外层 drop=0..19：枚举 DUT 丢了几个 bit

- 内层 garbage=0..3：枚举输出有几个垃圾 bit

- 找到第一个匹配即停止（最小 drop + 最小 garbage 优先）

Queue 阈值：inp_queue 攒满 20 bit（一个完整输入字），

out_queue 攒满 23 bit（最多3个垃圾 + 20个数据），然后才尝试匹配。

匹配成功后，inp_queue drain 20 bit，out_queue drain (garbage + valid_data) bit。

这道题的难点在于：DUT 行为不确定（drop 几个不知道），

SCB 必须用穷举对齐的方式来验证，而不是假设固定偏移。

Q85. Verification — Test Plan: Async FIFO Corner Cases & Verification

QUESTION

You are verifying an asynchronous FIFO (write clock domain and read clock domain are completely independent). The interviewer asks:

(a) What are the corner cases that can go wrong in an async FIFO? Describe the bug root cause for each.

(b) How do you verify each case? Describe your testbench architecture (monitors, scoreboard, assertions).

(c) What does the scoreboard reference model look like?

ANSWER

```
// -- (a) Four Classic Async FIFO Corner Cases ------------------------
//
// Case 1: Clock Jitter → 2FF synchronizer failure (data overwrite)
//   Root cause: wr_clk jitter causes the gray-code pointer to be
//   sampled inside the 2FF setup/hold window -> metastability.
//   The synchronizer outputs a STALE write pointer to the read side.
//   Read side thinks FIFO has space -> write continues -> data overwrite.
//
// Case 2: Near-equal clock frequencies (e.g. wr=100MHz, rd=99MHz)
//   Root cause: the two clocks slowly drift; almost_full / almost_empty
//   flags oscillate at the boundary. At the worst-case beat period,
//   full flag de-asserts while FIFO is actually full -> write overrun.
//   Hard to reproduce: requires > 1M cycles to drift to worst-case.
//
// Case 3: Simultaneous full=1 and empty=1 (contradictory flags)
//   Root cause: each side judges based on its own synchronized pointer,
//   which is delayed by CDC sync latency.
//   wr side sees stale rd_ptr -> thinks FIFO is full.
//   rd side sees stale wr_ptr -> thinks FIFO is empty.
//   Both sides stall and wait -> data is SAFE, not corrupted.
//   This is a liveness/performance issue, not a data integrity bug.
//
// Case 4: Almost-full threshold <= sync latency (threshold underflow)
//   Root cause: almost_full fires, but by the time the signal propagates
//   back to the writer, more data has already been written.
//   If threshold < sync_latency_cycles, the guard band is insufficient
//   -> FIFO overflows even though almost_full was correctly asserted.

// -- (b) Verification Strategy ---------------------------------------
//
// Testbench architecture: two decoupled monitor/agent pairs.
//
// Write Monitor (clocked by wr_clk):
//   @(posedge wr_clk) if (wr_en) -> capture data, push to SCB queue.
//   MUST use posedge + if(wr_en), NOT @(posedge wr_en):
//   back-to-back writes never de-assert wr_en, so edge-trigger on
//   wr_en would miss every transaction after the first.
//
// Read Monitor (clocked by rd_clk):
//   @(posedge rd_clk) if (rd_en) -> capture DUT output, compare vs SCB.
//
// Per-case stimulus:
```

```
//   Case 1: inject random jitter model on wr_clk (+-N ps), long sim run.
//   Case 2: set period_wr=10.0ns, period_rd=10.101ns, run >1M cycles.
//   Case 3: wr_clk >> rd_clk extreme ratio, monitor flag contradiction window.
//   Case 4: directed threshold sweep: threshold = sync_lat-1/sync_lat/sync_lat+1,
//           write after almost_full fires, check for overwrite.
//
// Assertions (independent of SCB):
//   Case 1/2: full asserted -> no successful write on next cycle.
//   Case 3:   full and empty simultaneously -> formally verify mutual exclusion.
//   Case 4:   almost_full high -> writes beyond threshold must be blocked.
//
// SVA example for full/empty mutual exclusion:
//   assert property (@(posedge wr_clk) !(full && empty))
//       else $error("FIFO: full and empty asserted simultaneously");

// -- (c) Scoreboard / Reference Model ------------------------------
//
// The SCB is intentionally clock-agnostic: it sees only transactions,
// never raw clocks. CDC bugs manifest as behavioral output mismatches.
//
// class async_fifo_scb;
//     mailbox #(data_t) wr_mbx;  // write monitor feeds here
//     mailbox #(data_t) rd_mbx;  // read monitor feeds here
//     queue_t           ref_q;   // ideal FIFO state
//
//     task run();
//         fork
//             // Write side: push every captured write into ref queue
//             forever begin
//                 data_t wr_data;
//                 wr_mbx.get(wr_data);
//                 ref_q.push_back(wr_data);
//             end
//             // Read side: pop ref queue and compare vs DUT output
//             forever begin
//                 data_t rd_data, exp_data;
//                 rd_mbx.get(rd_data);
//                 assert (ref_q.size() > 0) else $error("SCB: read from empty ref_q");
//                 exp_data = ref_q.pop_front();
//                 assert (rd_data === exp_data)
//                     else $error("SCB mismatch: got %0h, exp %0h", rd_data, exp_data);
//             end
//         join
//     endtask
// endclass
//
// Flag Checker (separate from SCB):
//   Tracks occupancy counter independently.
//   Checks full/empty/almost_full correctness vs actual occupancy.
//   Adds coverage bins: occupancy=0, occupancy=DEPTH-1, occupancy=DEPTH.
//
// Coverage targets:
//   cp_ratio: clock ratio bins (1:1, 2:1, 10:1, near-equal).
//   cp_jitter: jitter magnitude bins (0ps, 100ps, 500ps).
//   cp_occ: occupancy levels (empty, almost_empty, mid, almost_full, full).
//   cross cp_ratio x cp_occ: corner-case combinations.
```

解释

Async FIFO 验证的核心挑战在于两个 clock domain 完全独立，无法用单一时钟做同步比较。

架构上的关键决策是两个 monitor 各自用自己 domain 的时钟采样，SCB 完全不感知时钟关系，只处理事务语义——这样 CDC 导致的任何行为偏差都会直接在 data mismatch 中暴露。

四个 corner case 的共同根因是 gray code 指针同步延迟导致的"过时指针"问题，

只是各自的表现形式不同：Case 1 是 metastability、Case 2 是 drift、

Case 3 是双侧同时保守估计（安全但低效）、Case 4 是余量不足。

面试回答思路：root cause → verify strategy → SCB 架构 → assertion + coverage 互补。

加分项：指出连续写必须用 @(posedge wr_clk) + if(wr_en) 而非 @(posedge wr_en)；

Case 3 更适合 formal 验证而非 simulation（状态空间有限，formal 可穷举）。

Q86. Verification: Formal Verification -- How It Works

QUESTION

Explain how formal verification works. What are its limitations?

Why are constraints necessary?

ANSWER

```
// Formal verification (property checking / model checking):
//
// Unlike simulation, formal does NOT need a testbench to drive inputs.
// Instead, it mathematically PROVES (or disproves) that a property holds
// for ALL possible input combinations within a given constraint space.
//
// How it works:
//   1. You provide: RTL design + assertions (properties to prove)
//   2. Formal tool converts RTL into a mathematical state-space model
//   3. Tool searches exhaustively for a COUNTEREXAMPLE:
//      an input sequence that causes the assertion to fail
//   4. If found:   tool returns the counterexample waveform (the bug)
//      If not found within proof bound: property may be proven true
//
// Why constraints are necessary:
//   Without constraints, the input space is UNBOUNDED (infinite).
//   Formal must explore infinitely many input combinations -- impossible.
//   Constraints restrict the input space to legal/realistic values:
//
//   Example: AXI interface formal
//   assume property (@(posedge clk) valid |=> !$fell(valid) || ready);
//   // "Once valid is asserted, it cannot drop until ready is seen"
//   // Without this assume: formal may drive valid=1 then valid=0 randomly
//   // generating false counterexamples that violate the AXI protocol
//   // (the DUT is correct but the "test" was illegal input)
//
// Key terminology:
//   assume:  constraint on INPUTS (restricts what formal can drive)
//   assert:  property to PROVE (what formal tries to falsify)
//   cover:   reachability check (prove this state/sequence IS reachable)
//
// Limitation 1: Proof bound (bounded model checking)
//   Formal works up to a finite number of clock cycles (the proof bound).
//   If the state space is large but not infinite, formal may exhaust
//   the bound without finding a counterexample.
//   Result: "no counterexample found within bound N" does NOT mean
//   the property is proven true for all time -- it only means
//   no bug was found within N cycles.
//   This is called "inconclusive" or "bounded proof."
//
// Limitation 2: State space explosion
//   For wide buses, deep pipelines, or large memories, the state space
```

```
//   grows exponentially. Formal may time out or run out of memory.
//   Mitigation: abstract the design, split into smaller sub-blocks,
//   use assume constraints to reduce exploration space.
//
// When formal gives a definitive PROOF (unbounded):
//   For simple properties on small designs, formal can prove correctness
//   for ALL inputs and ALL time -- stronger than any simulation coverage.
//   This is the ideal outcome: "property proven true, no counterexample exists."
```

解释

Formal 和 simulation 最本质的区别：

Simulation 用具体的激励验证具体的场景（采样式）；

Formal 穷举所有合法输入组合，尝试找到 assertion 的反例（穷举式）。

约束（assume）的作用：把无限输入空间缩小到合法输入子集，

让 formal 只探索符合协议的场景，避免生成"protocol violation"的假反例。

"在 bound 内没找到反例 ≠ 一定正确"是面试中必须说清楚的要点。

Formal 最适合验证：control logic、FSM、CDC safety properties、protocol compliance。

不适合：数据通路的数值正确性（状态空间太大）、需要长序列的功能（超出 bound）。

Q87. Verification: Scoreboard for Paired A/B Transactions with Timeout

QUESTION

Two agents (A and B) each produce transactions in arbitrary order.

Write a scoreboard that:

- When a matching A+B pair arrives (in any order), calls check_pair(a, b).

- Reports an error if two consecutive transactions from the same agent

arrive before the other agent responds (2xA or 2xB).

- Reports an error if one transaction waits more than TIMEOUT for its pair.

ANSWER

```
class ab_scoreboard extends uvm_scoreboard;
    `uvm_component_utils(ab_scoreboard)

    uvm_analysis_imp_a #(txn_t, ab_scoreboard) imp_a;
    uvm_analysis_imp_b #(txn_t, ab_scoreboard) imp_b;

    txn_t    pending_a;
    txn_t    pending_b;
    bit      has_a;
    bit      has_b;
    realtime time_a;
    realtime time_b;

    localparam realtime TIMEOUT = 1us;

    function new(string name, uvm_component parent);
        super.new(name, parent);
    endfunction

    function void build_phase(uvm_phase phase);
```

```
        super.build_phase(phase);
        imp_a = new("imp_a", this);
        imp_b = new("imp_b", this);
    endfunction

    // write_a and write_b are plain functions -- NO semaphore needed.
    // Reason: SystemVerilog uses a single-threaded event-driven scheduler.
    // Multiple processes may be SCHEDULED at the same simulation time,
    // but they execute ONE AT A TIME in the active region. A function
    // is ATOMIC -- it runs to completion before the scheduler switches
    // to any other process. Therefore write_a() and write_b() can never
    // truly interleave, even if both monitors sample at the same posedge.

    function void write_a(txn_t a);
        if (has_a)
            `uvm_error("SCB", "2 consecutive A transactions before any B")
        else if (has_b) begin
            check_pair(a, pending_b);
            has_b = 0;
        end else begin
            pending_a = a;
            has_a     = 1;
            time_a    = $realtime;
        end
    endfunction

    function void write_b(txn_t b);
        if (has_b)
            `uvm_error("SCB", "2 consecutive B transactions before any A")
        else if (has_a) begin
            check_pair(pending_a, b);
            has_a = 0;
        end else begin
            pending_b = b;
            has_b     = 1;
            time_b    = $realtime;
        end
    endfunction

    // Timeout check: polled from run_phase
    task run_phase(uvm_phase phase);
        forever begin
            #(TIMEOUT / 10);
            if (has_a && ($realtime - time_a > TIMEOUT))
                `uvm_error("SCB", "A timed out waiting for B")
            if (has_b && ($realtime - time_b > TIMEOUT))
                `uvm_error("SCB", "B timed out waiting for A")
        end
    endtask

    function void check_phase(uvm_phase phase);
        if (has_a) `uvm_error("SCB", "Unpaired A at end of sim")
        if (has_b) `uvm_error("SCB", "Unpaired B at end of sim")
    endfunction

    virtual function void check_pair(txn_t a, txn_t b);
        if (a.data !== b.data)
            `uvm_error("SCB", $sformatf("Mismatch: A=0x%h B=0x%h", a.data, b.data))
        else
            `uvm_info("SCB", "Pair matched OK", UVM_HIGH)
    endfunction

endclass
```

解释

SV 和 Java/C++ 多线程的根本区别：

SV 是单线程事件驱动调度器，所有 process 共享一个执行队列，

在同一 time step 内"轮流"执行，不是真正的并行。

Function 是原子的：一旦 write_a() 开始执行，调度器不会切换到任何其他 process，

直到 write_a() 返回。因此即使两个 monitor 在同一 posedge clk 各自调用 write，

SV 保证它们严格顺序执行，不可能交错访问共享状态。

结论：纯 function 实现的 write_a/write_b 不需要 semaphore。

什么时候才需要 semaphore？

当 write 被改成 task（内含 #delay 或 @event）时，调度器可以在 delay 点切换，

两个 task 的执行才会真正交错，此时才需要 semaphore 保护共享状态。

UVM analysis write() 的协议要求它是 function，正是为了保证这种原子性。

Q88. Verification: Is Gate-Level Simulation Still Needed After RTL Verification?

QUESTION

If RTL functional verification is complete with 100% coverage,

is gate-level simulation (GLS) still necessary? Why?

ANSWER

```
// YES -- gate-level simulation is still necessary even after complete RTL verification.
// RTL verification and GLS catch DIFFERENT classes of bugs.
//
// What RTL verification covers:
//   - Functional correctness of the design intent
//   - Protocol compliance (AXI, AHB, etc.)
//   - Corner cases in the microarchitecture
//   - Coverage-driven verification of the specification
//
// What GLS catches that RTL cannot:
//
// 1. Synthesis transformation bugs
//    Synthesis tools optimize and restructure RTL into gates.
//    Occasionally a synthesis tool bug or constraint error produces
//    gates that implement slightly different logic than the RTL.
//    GLS with the actual netlist finds these discrepancies.
//
// 2. Timing violations in functional paths (with SDF back-annotation)
//    GLS with SDF injects actual gate delays from the timing signoff database.
//    Setup/hold violations cause the simulator to inject X at FF outputs.
//    If X propagates to outputs, a timing-dependent functional bug is revealed.
//    RTL simulation has zero-delay model -- it cannot model this.
//
// 3. Clock gating correctness
//    ICG cells (integrated clock gating) are inserted by synthesis.
//    Their enable timing must be correct relative to the gated clock edge.
//    GLS verifies that clock gating does not corrupt data.
//
// 4. X-propagation from uninitialized state
//    Gate-level simulation models uninitialized FFs as X more faithfully.
//    RTL simulation may implicitly assume reset to 0; gates do not.
```

```
//
// 5. Scan chain and DFT logic
//    Scan insertion adds test logic that is not in the RTL.
//    GLS verifies that scan does not disturb normal functional operation.
//
// When GLS can sometimes be reduced:
//   - Formal equivalence checking (LEC) proves netlist == RTL structurally
//   - Static timing analysis (STA) covers timing violations exhaustively
//   - Together, LEC + STA can replace most of GLS for low-risk blocks
//   - However, dynamic timing coverage (SDF GLS) is still needed for
//     critical paths in high-speed designs (DDR PHY, SerDes, etc.)
```

解释

RTL 验证和 GLS 是互补的，不是替代关系。

最关键的 GLS 价值：SDF 时序标注下的功能仿真，能发现"时序违例导致的功能错误"，

这是 RTL 仿真（零延迟模型）和纯 STA（只看路径，不看功能影响）都无法覆盖的。

面试标准回答：LEC 替代"综合等价性"检查，STA 替代"时序路径"检查，

但二者叠加仍无法完全替代 GLS，尤其是 SDF 标注下的 X-prop 功能覆盖。

Q89. Verification: Serial-to-Parallel DUT -- Driver, Monitor, Scoreboard

QUESTION

DUT specification:

Input: 1GHz clock, 1-bit serial stream (din). Bits arrive LSB-first.

Output: 50MHz clock, 20-bit parallel word (out[19:0]).

out[0] = first input bit of each group, out[19] = last input bit.

Every 20 input bits produce one output word.

Write a driver, monitor, and scoreboard to verify this DUT.

ANSWER

```
// ================================================================
// PART 1: Driver, Monitor, Scoreboard
// ================================================================

// ---- Interface ---------------------------------------------------
interface dut_if (input logic clk_1g, input logic clk_50m);
    logic        din;
    logic [19:0] dout;
endinterface

// ---- Transaction types -------------------------------------------
class din_txn extends uvm_sequence_item;
    `uvm_object_utils(din_txn)
    rand bit din;
    function new(string name = "din_txn"); super.new(name); endfunction
endclass

class dout_txn extends uvm_sequence_item;
    `uvm_object_utils(dout_txn)
    bit [19:0] dout;
    function new(string name = "dout_txn"); super.new(name); endfunction
endclass
```

```
// ---- Driver (1GHz, 1 bit per cycle) ---------------------------------
class din_driver extends uvm_driver #(din_txn);
    `uvm_component_utils(din_driver)
    virtual dut_if vif;

    function new(string name, uvm_component parent);
        super.new(name, parent);
    endfunction

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        if (!uvm_config_db #(virtual dut_if)::get(this, "", "vif", vif))
            `uvm_fatal("DRV", "No virtual interface")
    endfunction

    task run_phase(uvm_phase phase);
        din_txn txn;
        vif.din = 0;
        forever begin
            seq_item_port.get_next_item(txn);
            @(posedge vif.clk_1g);
            vif.din <= txn.din;
            seq_item_port.item_done();
        end
    endtask
endclass

// ---- Input Monitor --------------------------------------------------
class din_monitor extends uvm_monitor;
    `uvm_component_utils(din_monitor)
    virtual dut_if              vif;
    uvm_analysis_port #(din_txn) ap;

    function new(string name, uvm_component parent);
        super.new(name, parent);
    endfunction

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        ap = new("ap", this);
        if (!uvm_config_db #(virtual dut_if)::get(this, "", "vif", vif))
            `uvm_fatal("MON_IN", "No virtual interface")
    endfunction

    task run_phase(uvm_phase phase);
        din_txn txn;
        forever begin
            @(posedge vif.clk_1g);
            txn      = din_txn::type_id::create("txn");
            txn.din  = vif.din;
            ap.write(txn);
        end
    endtask
endclass

// ---- Output Monitor -------------------------------------------------
class dout_monitor extends uvm_monitor;
    `uvm_component_utils(dout_monitor)
    virtual dut_if              vif;
    uvm_analysis_port #(dout_txn) ap;

    function new(string name, uvm_component parent);
```

```
        super.new(name, parent);
    endfunction

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        ap = new("ap", this);
        if (!uvm_config_db #(virtual dut_if)::get(this, "", "vif", vif))
            `uvm_fatal("MON_OUT", "No virtual interface")
    endfunction

    task run_phase(uvm_phase phase);
        dout_txn txn;
        forever begin
            @(posedge vif.clk_50m);
            txn       = dout_txn::type_id::create("txn");
            txn.dout  = vif.dout;
            ap.write(txn);
        end
    endtask
endclass

// ---- Scoreboard ----------------------------------------------------
// Accumulates 20 input bits -> assembles expected word -> compares to output
class my_scoreboard extends uvm_scoreboard;
    `uvm_component_utils(my_scoreboard)

    uvm_analysis_imp_din  #(din_txn,  my_scoreboard) imp_din;
    uvm_analysis_imp_dout #(dout_txn, my_scoreboard) imp_dout;

    bit          bit_buf[$];       // accumulate input bits
    bit [19:0]   expected_q[$];   // assembled expected words

    function new(string name, uvm_component parent);
        super.new(name, parent);
    endfunction

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        imp_din  = new("imp_din",  this);
        imp_dout = new("imp_dout", this);
    endfunction

    // 1GHz: accumulate; every 20 bits, assemble expected word (LSB first)
    function void write_din(din_txn t);
        bit_buf.push_back(t.din);
        if (bit_buf.size() == 20) begin
            bit [19:0] word = 0;
            for (int i = 0; i < 20; i++)
                word[i] = bit_buf[i];   // bit_buf[0] -> out[0] (LSB)
            expected_q.push_back(word);
            bit_buf.delete();
        end
    endfunction

    // 50MHz: compare output word against expected
    function void write_dout(dout_txn t);
        if (expected_q.size() == 0) begin
            `uvm_error("SCB", "Output received but no expected word queued")
            return;
        end
        begin
            bit [19:0] exp = expected_q.pop_front();
            if (t.dout !== exp)
```

```
                `uvm_error("SCB", $sformatf(
                    "MISMATCH: got=0x%05h expected=0x%05h", t.dout, exp))
            else
                `uvm_info("SCB", $sformatf("MATCH 0x%05h", t.dout), UVM_HIGH)
        end
    endfunction

    function void check_phase(uvm_phase phase);
        if (expected_q.size() > 0)
            `uvm_error("SCB", $sformatf(
                "%0d expected words unmatched at end of sim", expected_q.size()))
    endfunction
endclass

// ============================================================
// PART 2 (Follow-up): Sequence with per-bit constraints
// ============================================================
// The input is 1 bit per transaction. Constraints on a single din_txn
// cannot see previous bits. The sequence must maintain cross-bit state
// and apply inline constraints based on that history.

class din_sequence extends uvm_sequence #(din_txn);
    `uvm_object_utils(din_sequence)

    // History: last 3 bits sent (needed to detect potential 4th zero)
    bit history[$];

    // Running counts to enforce "more 1s than 0s" over N bits
    int unsigned ones_cnt  = 0;
    int unsigned zeros_cnt = 0;
    int unsigned total     = 100;   // total bits to send

    function new(string name = "din_sequence");
        super.new(name);
    endfunction

    task body();
        din_txn txn;
        repeat (total) begin
            txn = din_txn::type_id::create("txn");
            start_item(txn);

            // Determine what the next bit must be:
            // Rule 1: if last 3 bits are all 0, MUST send 1 (avoid 4 zeros)
            // Rule 2: bias toward 1 to ensure more 1s than 0s overall
            begin
                bit must_be_one;
                bit prefer_one;

                // Constraint 1: block 4th consecutive zero
                must_be_one = (history.size() >= 3 &&
                               history[$] == 0 &&
                               history[$-1] == 0 &&
                               history[$-2] == 0);

                // Constraint 2: prefer 1 if we're at risk of not having enough
                // ones_cnt must eventually exceed zeros_cnt
                prefer_one = (ones_cnt <= zeros_cnt);

                if (must_be_one) begin
                    // Force: no randomization
                    txn.din = 1;
                end else begin
```

```
                    // Randomize with soft bias toward 1
                    assert(txn.randomize() with {
                        // Soft: prefer 1 when counts are even
                        soft txn.din == (prefer_one ? 1 : txn.din);
                        // Weight: dist makes 1 appear ~60% of the time
                        txn.din dist {1 := 60, 0 := 40};
                    });
                end
            end

            // Update history and counters
            history.push_back(txn.din);
            if (history.size() > 3) void'(history.pop_front());
            if (txn.din) ones_cnt++; else zeros_cnt++;

            finish_item(txn);
        end
    endtask
endclass

// ---- SVA: no 4 consecutive zeros on din (1GHz domain) ---------------
//
// Triggered at every posedge clk_1g.
// If din has been 0 for the current cycle and 2 prior cycles,
// the next cycle must not also be 0.

property p_no_4_consec_zeros;
    @(posedge clk_1g)
    // "not" the sequence: din=0 four times in a row
    not (!din ##1 !din ##1 !din ##1 !din);
endproperty

assert property (p_no_4_consec_zeros)
    else $error("4 consecutive zeros on din at %0t", $time);

// Alternative using implication: if 3 zeros seen, next must be 1
property p_3zeros_then_one;
    @(posedge clk_1g)
    (!din ##1 !din ##1 !din) |=> din;
endproperty

assert property (p_3zeros_then_one)
    else $error("3 consecutive zeros followed by another 0");
```

解释

PART 1 -- 串并转换验证架构：

DUT 比例 1GHz/50MHz=20，每20个 1-bit 输入对应1个20-bit 输出字，out[0]=第一个到的bit。

SCB 用 bit_buf[$] 逐 bit 积累，攒满20个后按 LSB-first 顺序组装期望字，

push 到 expected_q，output monitor 触发时 pop 出来比对。

两个 write() 都是纯 function，SV 保证原子执行，无竞争问题。

PART 2 -- 为什么约束要写在 sequence 而非 transaction：

transaction (din_txn) 只有1个 bit，它自己无法看到之前发送过的 bit。

"不能出现4个连续0" 是一个跨越多个 transaction 的时序约束，必须在 sequence 层面维护历史状态。

Sequence 的 history[$] 记录最近3个 bit，每次 randomize 前检查：

若已有3个连续0，强制发1（must_be_one=1），否则用 dist 约束偏向1（60:40）。

这是 UVM constraint 设计中"transaction-level vs sequence-level" 的核心区分：

单个 item 的属性约束写在 class 里，跨 item 的历史依赖约束写在 sequence 里。

SVA 两种等价写法：not(4连续零序列) 最直接；3零蕴含下一个必须是1 (|=>) 语义更清晰。

Q90. Verification: Functional Coverage Not 100% -- Analysis and Strategy

QUESTION

In a project, functional coverage was not 100%. How do you analyze why certain bins were not hit? How do you decide if uncovered bins are acceptable (can be waived)? How do you hit a specific bin?

ANSWER

```
// Why functional coverage may not reach 100%:
//
// 1. UNREACHABLE bins: the DUT physically cannot produce that scenario.
//    Example: a state machine cannot reach an illegal state transition.
//    These SHOULD be waived -- they represent impossible hardware behavior.
//
// 2. RARE scenarios not triggered by random stimulus:
//    Example: specific error injection, boundary values, specific sequences.
//    Solution: write directed tests or tune constraint weights.
//
// 3. MISSING stimulus: the testbench does not exercise a feature.
//    Example: a protocol error mode not covered by any sequence.
//    Solution: add directed sequences targeting that specific coverage.
//
// 4. WRONG coverage definition: coverpoints that don't match DUT behavior.
//    Solution: review coverage plan against specification.

// How to verify uncovered bins are acceptable (waiver criteria):
//
//   a. Cross-reference with specification: does the spec require this scenario?
//      If spec says "this shall never occur", the bin can be waived.
//   b. RTL analysis: trace through the code to prove the scenario is unreachable.
//      Document the proof (static analysis or formal verification).
//   c. Risk assessment: what is the consequence if this scenario occurs in silicon?
//      If consequence is low AND scenario is unlikely, waive with justification.
//   d. Formal verification: use model checking to prove the uncovered state
//      is truly unreachable.
//   Waiver must be documented with: bin name, reason, reviewer sign-off.

// How to hit a specific coverage bin:
//
// Method 1: Directed test -- write a sequence that explicitly drives the
//   exact conditions to hit the bin.
class hit_specific_bin_seq extends uvm_sequence;
    task body();
        my_txn txn = my_txn::type_id::create("txn");
        start_item(txn);
        // Inline constraint: force exactly the values needed for this bin
        assert(txn.randomize() with {
            txn.opcode == 4'hF;      // specific opcode bin
            txn.addr   == 16'hFFFF;  // max address bin
        });
        finish_item(txn);
    endtask
endclass
```

```
//
// Method 2: Constraint weight adjustment
//   Use dist constraint to increase probability of hitting rare bins:
constraint c_hit_rare {
    opcode dist {4'hF := 50, [0:14] := 50};  // 50% chance of rare value
}
//
// Method 3: Cross coverage -- ensure cross product scenarios are generated
//   If bin A requires {opcode=F, mode=3}, create a sequence that
//   randomizes both together with an inline cross constraint.
```

解释

覆盖率不到100%不代表验证失败，关键是能说清楚每个未覆盖点的原因和风险。

Waiver 的核心逻辑：证明"这个场景物理上不可能发生"（通过 RTL 分析或形式验证），

或者"即使发生，影响可控且概率极低"。所有 waiver 都必须有文档和审批。

打中特定 bin 的优先顺序：先用定向测试（最直接），

再考虑约束权重调整（提高概率），最后考虑形式验证证明覆盖。

面试中能区分"unreachable bin（应 waive）"和"rare bin（应补测试）"是关键。

Q91. Verification: Multi-Channel DUT with Credit-Based Flow Control

QUESTION

DUT has 1 input port (with channel_id field) and N output ports (one per channel).

(a) How do you verify this DUT? Write the scoreboard structure.

(b) Write a driver for this DUT.

(c) Follow-up: if channels are credit-based, how does the driver change?

(d) Follow-up: if N is unknown until compile time, how do you handle it?

ANSWER

```
// (a) Scoreboard: one queue per channel_id, match input to output
class channel_scb extends uvm_scoreboard;
    `uvm_component_utils(channel_scb)

    // One analysis import per output channel (connected by env)
    uvm_analysis_imp_in  #(in_txn,  channel_scb) imp_in;
    // Output imports created dynamically per channel (see part d)

    // Per-channel queues: key = channel_id
    in_txn expected [int][$];   // associative array of queues

    function void write_in(in_txn txn);
        // Route to the correct channel queue based on channel_id
        expected[txn.channel_id].push_back(txn);
    endfunction

    function void write_out(out_txn txn);
        int ch = txn.channel_id;
        if (expected[ch].size() == 0)
            `uvm_error("SCB", $sformatf("Unexpected output on channel %0d", ch))
        else begin
            in_txn exp = expected[ch].pop_front();
            if (txn.data !== exp.data)
```

```
                `uvm_error("SCB", $sformatf(
                    "Ch%0d mismatch: got=0x%h exp=0x%h", ch, txn.data, exp.data))
        end
    endfunction
endclass

// (b) Driver: try_next_item for non-blocking, idle when no transaction
class channel_driver extends uvm_driver #(in_txn);
    `uvm_component_utils(channel_driver)
    virtual dut_if vif;

    task run_phase(uvm_phase phase);
        forever begin
            in_txn txn;
            // try_next_item: non-blocking, returns null if nothing pending
            seq_item_port.try_next_item(txn);
            if (txn != null) begin
                drive_txn(txn);
                seq_item_port.item_done();
            end else begin
                drive_idle();   // keep bus stable during idle cycles
            end
            @(posedge vif.clk);
        end
    endtask

    task drive_txn(in_txn txn);
        vif.valid      <= 1;
        vif.channel_id <= txn.channel_id;
        vif.data       <= txn.data;
    endtask

    task drive_idle();
        vif.valid <= 0;
        vif.data  <= '0;
    endtask
endclass

// (c) Credit-based driver
// Each channel has a credit count; only drive if channel has credit.
// Backlog queue stores transactions waiting for credit.
class credit_driver extends uvm_driver #(in_txn);
    `uvm_component_utils(credit_driver)
    virtual dut_if vif;

    int unsigned    credit [int];         // credit per channel_id
    in_txn          backlog [int][$];     // transactions waiting for credit

    task run_phase(uvm_phase phase);
        // Launch credit-monitor thread in parallel
        fork
            monitor_credits();
        join_none

        forever begin
            in_txn txn;
            seq_item_port.try_next_item(txn);
            @(posedge vif.clk);

            if (txn != null) begin
                int ch = txn.channel_id;
                if (credit[ch] > 0) begin
                    drive_txn(txn);
```

```
                    credit[ch]--;
                    seq_item_port.item_done();
                end else begin
                    // No credit: park transaction in backlog
                    backlog[ch].push_back(txn);
                    seq_item_port.item_done();
                    drive_idle();
                end
            end else begin
                // Try to drain backlog for any channel with credit
                automatic bit drove = 0;
                foreach (backlog[ch]) begin
                    if (backlog[ch].size() > 0 && credit[ch] > 0) begin
                        in_txn pending = backlog[ch].pop_front();
                        drive_txn(pending);
                        credit[ch]--;
                        drove = 1;
                        break;
                    end
                end
                if (!drove) drive_idle();
            end
        end
    endtask

    // Monitor output ports to detect credit returns
    task monitor_credits();
        forever begin
            @(posedge vif.clk);
            // When output fires on a channel, credit is returned
            if (vif.out_valid) begin
                int ch = vif.out_channel_id;
                credit[ch]++;
            end
        end
    endtask
endclass

// (d) Unknown N channels at compile time -> dynamic fork + mailbox
// Use a mailbox shared across all output monitors.
// All monitors write to ONE mailbox; SCB reads from it.
class dynamic_env extends uvm_env;
    `uvm_component_utils(dynamic_env)
    int num_channels;   // determined at runtime from plusarg or config

    mailbox #(out_txn) out_mb;   // shared across all output agents

    task run_phase(uvm_phase phase);
        // Dynamically create one monitor per channel at runtime
        out_mb = new();
        fork
            begin
                for (int ch = 0; ch < num_channels; ch++) begin
                    automatic int c = ch;
                    fork
                        // Each channel monitor runs independently
                        monitor_channel(c, out_mb);
                    join_none
                end
            end
            // SCB reads from shared mailbox regardless of which channel fired
            scb_thread(out_mb);
        join
```

```
    endtask

    task monitor_channel(int ch, mailbox #(out_txn) mb);
        forever begin
            @(posedge vif.clk);
            if (vif.out_valid[ch]) begin
                out_txn txn = out_txn::type_id::create("txn");
                txn.channel_id = ch;
                txn.data       = vif.out_data[ch];
                mb.put(txn);   // write to shared mailbox
            end
        end
    endtask

    task scb_thread(mailbox #(out_txn) mb);
        forever begin
            out_txn txn;
            mb.get(txn);   // blocking: waits until any channel fires
            scb.write_out(txn);
        end
    endtask
endclass
```

解释

(a) SCB 核心：用 channel_id 作为 key 的 associative array of queues，

input 事务入队，对应 channel 的 output 事务出队比对。

(b) Driver 关键：try_next_item（非阻塞），没事务时 drive_idle() 保持总线稳定。

不能用 get_next_item（阻塞），否则无法在空闲时驱动 idle 状态。

(c) Credit-based：额外维护 credit[channel_id] 计数器和 backlog queue。

独立线程监控 output 端口，检测到输出时归还 credit。

(d) 动态 channel 数：用 fork-join_none 批量创建 monitor，

所有 monitor 共享一个 mailbox，SCB 从 mailbox 里取事务——

mailbox 支持多写一读，比 analysis_port 更灵活。

Q92. Verification: Constrained-Random Verification -- How It Works

QUESTION

Explain what constrained-random verification (CRV) is, how it differs

from directed testing, and write a constraint example for address alignment.

(a) What is CRV and why is it used?

(b) Write a constraint to ensure a 32-bit address is 4-byte aligned

(addr[1:0] == 2'b00). Then generalize to N-byte alignment.

ANSWER

```
// (a) Constrained-Random Verification (CRV)
//
// DIRECTED TESTING: engineer manually writes specific stimulus values.
//   Pros: deterministic, easy to target known corner cases.
//   Cons: only covers what the engineer thought of; misses unknown bugs.
//
```

```
// CONSTRAINED-RANDOM TESTING: stimulus is randomized within legal constraints.
//   The constraint solver generates thousands of legal combinations automatically.
//   Pros: explores state space the engineer didn't think of; scales with coverage.
//   Cons: may not hit specific corner cases without additional directed tests.
//
// CRV flow:
//   1. Define transaction class with rand variables
//   2. Add constraints to keep stimulus legal (protocol compliance)
//   3. Define functional coverage (covergroups) to measure what's been tested
//   4. Run many randomized tests; check coverage
//   5. Add directed tests or constraint overrides for remaining coverage holes
//
// The key insight: coverage-DRIVEN means you know WHEN you're done.
// Without coverage, random testing can run forever without confidence.

// (b) Address alignment constraint

class mem_txn extends uvm_sequence_item;
    `uvm_object_utils(mem_txn)

    rand logic [31:0] addr;
    rand logic [31:0] data;
    rand logic        wr_en;   // 1=write, 0=read

    // 4-byte alignment: addr[1:0] must be 00
    constraint c_addr_4byte_align {
        addr[1:0] == 2'b00;
    }

    // Generalised N-byte alignment (N must be power of 2):
    // addr % N == 0  <=>  addr[log2(N)-1:0] == 0
    //
    // For N=8  (8-byte):  addr[2:0] == 3'b000
    // For N=16 (16-byte): addr[3:0] == 4'b0000
    // For N=64 (64-byte): addr[5:0] == 6'b0
    //
    // Parameterized version using modulo:
    constraint c_addr_align_general {
        // ALIGN_BYTES must be a power of 2
        // addr % ALIGN_BYTES == 0
        // Equivalent: (addr & (ALIGN_BYTES-1)) == 0
        addr % 4 == 0;   // same as addr[1:0]==0 for power-of-2
    }

    // Additional useful address constraints:
    constraint c_addr_range {
        addr inside {[32'h0000_0000 : 32'hFFFF_FFFC]};  // valid range
    }

    function new(string name = "mem_txn");
        super.new(name);
    endfunction
endclass

// Usage: override alignment per test with inline constraint:
// assert(txn.randomize() with { addr % 64 == 0; });  // 64-byte aligned
```

解释

CRV 的核心价值：constraint solver 自动探索你没想到的合法组合，而 coverage 告诉你探索进度。

地址对齐约束的本质：对于 N=2^k 对齐，低 k 位必须为0。

addr[1:0]==0 和 addr%4==0 是等价的（N 是2的幂时），但位操作写法更高效（不需要除法运算）。

面试中能说出"CRV = random stimulus + legal constraints + coverage closure"是关键。

Q93. Verification: FP Multiplier ALU Verification Strategy

QUESTION

You have an FP32 Multiplier ALU (A * B -> result). How do you verify it?

Focus on: constraint strategy for IEEE 754 edge cases, coverage, and

the difference between code coverage and functional coverage.

ANSWER

```
// ---- Part 1: FP Multiplier Verification Strategy -----------------------

typedef enum bit [2:0] {
    FP_NORMAL   = 3'd0,
    FP_ZERO     = 3'd1,
    FP_INF      = 3'd2,
    FP_NAN      = 3'd3,
    FP_DENORMAL = 3'd4
} fp_class_e;

class fp_txn extends uvm_sequence_item;
    `uvm_object_utils(fp_txn)

    rand logic [31:0] a;    // FP32 operand A
    rand logic [31:0] b;    // FP32 operand B

    // Helper: classify a 32-bit FP value
    function fp_class_e classify(logic [31:0] val);
        logic [7:0]  exp  = val[30:23];
        logic [22:0] mant = val[22:0];
        if      (exp == 8'hFF && mant != 0) return FP_NAN;
        else if (exp == 8'hFF && mant == 0) return FP_INF;
        else if (exp == 8'h00 && mant != 0) return FP_DENORMAL;
        else if (exp == 8'h00 && mant == 0) return FP_ZERO;
        else                                return FP_NORMAL;
    endfunction

    // Constraint: weight different FP categories using dist
    // Normal numbers dominate, but edge cases get meaningful probability
    constraint c_a_dist {
        a dist {
            // Normal (exp in [1:254], any mantissa)
            [32'h00800000 : 32'h7F7FFFFF] := 60,   // positive normal
            [32'h80800000 : 32'hFF7FFFFF] := 10,   // negative normal
            // Zero (+0 and -0)
            32'h00000000                  := 5,    // +0
            32'h80000000                  := 5,    // -0
            // Infinity
            32'h7F800000                  := 5,    // +Inf
            32'hFF800000                  := 5,    // -Inf
            // NaN (quiet NaN)
            32'h7FC00000                  := 5,    // qNaN
            // Denormals (exp=0, mant!=0)
            [32'h00000001 : 32'h007FFFFF] := 5    // positive denormal
        };
    }

    constraint c_b_dist {
        b dist {
```

```
            [32'h00800000 : 32'h7F7FFFFF] := 60,
            [32'h80800000 : 32'hFF7FFFFF] := 10,
            32'h00000000 := 5,  32'h80000000 := 5,
            32'h7F800000 := 5,  32'hFF800000 := 5,
            32'h7FC00000 := 5,
            [32'h00000001 : 32'h007FFFFF] := 5
        };
    }

    function new(string name = "fp_txn");
        super.new(name);
    endfunction
endclass

// ---- Covergroup: cross all FP categories for A and B ------------------
class fp_coverage extends uvm_subscriber #(fp_txn);
    `uvm_component_utils(fp_coverage)

    fp_txn txn;

    covergroup fp_mul_cg;
        cp_a: coverpoint txn.classify(txn.a) {
            bins normal   = {FP_NORMAL};
            bins zero     = {FP_ZERO};
            bins inf      = {FP_INF};
            bins nan      = {FP_NAN};
            bins denormal = {FP_DENORMAL};
        }
        cp_b: coverpoint txn.classify(txn.b) {
            bins normal   = {FP_NORMAL};
            bins zero     = {FP_ZERO};
            bins inf      = {FP_INF};
            bins nan      = {FP_NAN};
            bins denormal = {FP_DENORMAL};
        }
        // Cross: 5x5 = 25 combinations
        cx_ab: cross cp_a, cp_b;
    endgroup

    function new(string name, uvm_component parent);
        super.new(name, parent);
        fp_mul_cg = new();
    endfunction

    function void write(fp_txn t);
        txn = t;
        fp_mul_cg.sample();
    endfunction
endclass

// ---- Key edge case results (IEEE 754 rules for multiplication) ---------
//
//   A       | B       | A * B
//   --------+---------+-----------
//   NaN     | any     | NaN          (NaN propagates)
//   any     | NaN     | NaN
//   0       | Inf     | NaN          (indeterminate: 0 * inf)
//   Inf     | 0       | NaN
//   +Inf    | +normal | +Inf         (infinity)
//   +Inf    | -normal | -Inf
//   0       | normal  | 0            (signed zero: sign = XOR of signs)
//   denorm  | denorm  | underflow -> 0 or denorm (implementation-dependent)
//   large   | large   | overflow  -> Inf
```

```
//
// Directed tests for the hardest-to-hit cross bins:
//   NaN x NaN:      a=32'h7FC00000, b=32'h7FC00000 -> result must be NaN
//   0 x Inf:        a=32'h00000000, b=32'h7F800000 -> result must be NaN
//   Inf x Inf:      a=32'h7F800000, b=32'h7F800000 -> result must be Inf
//   max x max:      a=32'h7F7FFFFF, b=32'h7F7FFFFF -> overflow -> Inf
//   denorm x 1.0:   a=32'h00000001, b=32'h3F800000 -> flush to zero check

// ---- Part 2: Code Coverage vs Functional Coverage ----------------------
//
// CODE COVERAGE (structural coverage of the RTL):
//   Measures whether SIMULATION EXERCISED each line/branch/toggle of RTL code.
//   Automatically extracted by the simulator from the DUT source code.
//   Types:
//     Line coverage:   every line of RTL executed?
//     Branch coverage: both sides of every if/case executed?
//     Toggle coverage: every signal toggled 0->1 AND 1->0?
//     FSM coverage:    every state and transition visited?
//   What it tells you: "Did simulation TOUCH this code?"
//   What it does NOT tell you: "Did the design DO THE RIGHT THING?"
//
// FUNCTIONAL COVERAGE (coverage of the DESIGN INTENT):
//   Measures whether the INTERESTING SCENARIOS defined by the verification plan
//   have been exercised. Manually written by the verification engineer.
//   Types:
//     Coverpoint bins: did specific values/ranges occur?
//     Cross coverage:  did specific combinations of values occur?
//     Sequence coverage: did specific temporal sequences occur?
//   What it tells you: "Did we test the behavior we CARED about?"
//   What it does NOT tell you: which RTL lines were executed.
//
// Key difference:
//   100% code coverage does NOT mean 100% functional coverage.
//   Example: an FP multiplier may execute the NaN-handling branch,
//   but functional coverage could show that NaN*0 was never tested
//   (only NaN*normal was). The branch was covered but the SCENARIO wasn't.
//
//   Conversely, 100% functional coverage doesn't mean 100% code coverage:
//   dead code (unreachable RTL) may exist that functional tests never hit.
//
// Together: both are needed for sign-off.
//   Code coverage: catches unexercised RTL (potential bugs in dead code paths)
//   Functional coverage: confirms all SPECIFICATION scenarios were tested
```

解释

FP 乘法验证的策略分三层：

1. 约束+分布：用 dist 给 Normal/Zero/Inf/NaN/Denormal 分配不同权重，

Normal 占多数（60%），edge case 各占 5%，保证常见情况压力足够。

2. 交叉覆盖：5x5 cross coverage 确保所有25种 A 类型 × B 类型组合都被测试到，

包括 NaN×0、Inf×0 这类"特别危险"的组合。

3. 定向测试：cross 里最难被随机激励命中的 bin（如 NaN×NaN）补充定向测试。

Code coverage vs Functional coverage 的本质区别：

Code = "代码跑了"，Functional = "意图被测了"。

两者不等价，都需要达标才能 sign-off。

FP 乘法最典型的例子：NaN 处理分支可能被 code coverage 标为 covered，

但 functional coverage 可能还差 NaN×Inf 这个 cross bin。

UVM

Q94. UVM: Interface, Monitor, and uvm_config_db Setup

QUESTION

Write a complete UVM interface and a monitor that connects to it.

Include all standard boilerplate: uvm_config_db, `uvm_component_utils,

constructor, build_phase, run_phase, and virtual interface declaration.

ANSWER

```
// ---- Step 1: Define the SystemVerilog interface ----

interface my_if (input logic clk, rst_n);
    logic       valid;
    logic [7:0] data;
    logic       ready;

    clocking mon_cb @(posedge clk);
        input valid, data, ready;
    endclocking

    modport monitor_mp (clocking mon_cb, input clk, rst_n);
endinterface : my_if

// ---- Step 2: Write the UVM Monitor ----

class my_monitor extends uvm_monitor;

    // Register with UVM factory
    // uvm_component_utils(my_monitor)   <- backtick macro

    // Virtual interface handle
    virtual my_if.monitor_mp vif;

    // Analysis port
    uvm_analysis_port #(my_transaction) ap;

    // Constructor
    function new(string name = "my_monitor", uvm_component parent = null);
        super.new(name, parent);
    endfunction

    // build_phase: get vif from config_db
    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        ap = new("ap", this);
        if (!uvm_config_db #(virtual my_if)::get(
                this,   // context: this component
                "",     // instance path (empty = this level)
                "vif",  // key name
                vif))   // output: retrieved vif
            // uvm_fatal("NO_VIF", "Virtual interface not found")
            $fatal(1, "Virtual interface 'vif' not found in uvm_config_db");
    endfunction
```

```
    // run_phase: sample interface, broadcast transactions
    task run_phase(uvm_phase phase);
        my_transaction txn;
        forever begin
            @(vif.mon_cb);
            if (vif.mon_cb.valid && vif.mon_cb.ready) begin
                txn = my_transaction::type_id::create("txn");
                txn.data = vif.mon_cb.data;
                ap.write(txn);
            end
        end
    endtask

endclass : my_monitor

// ---- Step 3: Set vif in tb_top before run_test() ----

module tb_top;
    logic clk, rst_n;
    my_if   dut_if (.clk(clk), .rst_n(rst_n));
    my_dut  dut    (.clk(clk), .rst_n(rst_n),
                    .valid(dut_if.valid),
                    .data (dut_if.data),
                    .ready(dut_if.ready));

    initial begin
        uvm_config_db #(virtual my_if)::set(
            null,              // context: null = outside UVM tree
            "uvm_test_top.*",  // scope: all components under test
            "vif",             // key (matches get() call)
            dut_if);           // the actual interface instance

        run_test("my_test");
    end

    initial clk = 0;
    always #5 clk = ~clk;
endmodule
```

解释

UVM interface + monitor 完整链路三步：

1. tb_top 实例化 interface，DUT 信号连到 interface 端口上。

2. tb_top 用 set() 把 virtual interface 存入 config_db，context=null（在 UVM tree 外）。

3. monitor 在 build_phase 用 get() 取出 virtual interface，context=this。

uvm_component_utils 向 factory 注册该类，支持 factory override（替换 mock 类）。

analysis port 解耦 monitor 和 scoreboard，monitor 只负责采样和广播，不感知下游。

Q95. UVM: uvm_config_db Four Parameters Explained

QUESTION

Explain all four parameters of uvm_config_db::set() and ::get().

What does each one mean? Show examples of common usage patterns.

ANSWER

```
// uvm_config_db #(T)::set(cntxt, inst_name, field_name, value)
// uvm_config_db #(T)::get(cntxt, inst_name, field_name, value)
//
// Parameter 1: T  (type, specified in #())
//   The TYPE of the value being stored or retrieved.
//   set() and get() must use IDENTICAL type -- mismatch = not found.
//   Common: virtual my_if, int, string, uvm_object
//   Example: uvm_config_db #(virtual my_if)::set(...)
//
// Parameter 2: cntxt  (context -- who is calling)
//   In set(): the component calling set(), or null for tb_top.
//             null means "I am outside the UVM hierarchy."
//             this means "I am a UVM component, use my position in tree."
//   In get(): always 'this' (the component doing the retrieval).
//   Combined with inst_name, forms the full lookup scope.
//
// Parameter 3: inst_name  (instance path string, wildcards ok)
//   In set(): path relative to cntxt. Wildcards (* ?) supported.
//     "uvm_test_top.*"         -> all components under test root
//     "uvm_test_top.env.agent" -> specific agent only
//     "*"                      -> broadcast to all (use with care)
//     ""                       -> direct children of cntxt only
//   In get(): almost always "" (look at my exact location in tree).
//
// Parameter 4: field_name  (string key, case-sensitive)
//   The name used to identify this stored value.
//   Must be BYTE-FOR-BYTE identical in set() and get().
//   Convention: "vif", "cfg", "depth", "max_retries"
//
// Matching rule:
//   get(this, "", "vif") succeeds if there exists a set() where:
//     type T matches AND
//     this component's full path is within set()'s scope AND
//     field_name strings are identical
//   If multiple set() calls match, most specific (longest path) wins.

// Common patterns:

// Pattern A: pass virtual interface from tb_top
uvm_config_db #(virtual my_if)::set(null, "uvm_test_top.*", "vif", dut_if);
// get in any component:
uvm_config_db #(virtual my_if)::get(this, "", "vif", vif);

// Pattern B: pass integer to specific component
uvm_config_db #(int)::set(this, "env.agent.driver", "max_retries", 5);
// get in driver:
uvm_config_db #(int)::get(this, "", "max_retries", max_retries);

// Pattern C: pass config object down hierarchy
uvm_config_db #(my_cfg)::set(this, "env.*", "cfg", cfg_obj);
// get in any env subcomponent:
uvm_config_db #(my_cfg)::get(this, "", "cfg", cfg);
```

解释

四参数记忆口诀：类型 · 谁设 · 在哪 · 叫啥。

最常见的 debug bug：field_name 大小写不一致，set 用 "Vif" 但 get 用 "vif"，永远找不到。

第二常见：type T 不匹配，set 用 virtual my_if 但 get 用 virtual base_if，同样找不到。

cntxt + inst_name 共同决定范围：null + "uvm_test_top.*" 表示从 simulation root 广播，

this + "" 表示从我自己的位置精确匹配。

Q96. UVM: Where to Set Virtual Interface in uvm_config_db

QUESTION

Where should uvm_config_db::set() be called to pass a virtual interface?

Why is it done in the top-level module and not inside the UVM tree?

What is the role of tb_top in a UVM simulation?

ANSWER

```
// Answer: uvm_config_db::set() for virtual interface is called in tb_top
//         (top-level SV module), BEFORE run_test(), with context = null.
//
// Why tb_top, not inside a UVM component?
//
// Reason 1: tb_top is where the actual interface lives
//   my_if dut_if (.clk(clk), .rst_n(rst_n));  // real interface here
//   Only tb_top has access to dut_if.
//   UVM components hold virtual interface HANDLES (pointers to the real one).
//   You cannot call set() with a handle that doesn't exist yet.
//
// Reason 2: tb_top is NOT part of the UVM component tree
//   The UVM hierarchy starts at uvm_test_top (created by run_test()).
//   tb_top is a plain SV module -- no uvm_component parent, no phase,
//   no `uvm_component_utils, no factory registration.
//   Using null as context in set() reflects this correctly:
//   "I am outside the UVM tree, I am the simulation entry point."
//
// Reason 3: timing -- set() must precede build_phase
//   UVM phases execute in order: build -> connect -> start_of_simulation -> run
//   Monitors call get() in build_phase.
//   The initial block in tb_top runs BEFORE run_test() triggers any phase.
//   This ordering guarantee is why set() goes in tb_top's initial begin.
//
// Correct implementation:
module tb_top;
    logic clk, rst_n;
    my_if  dut_if (.clk(clk), .rst_n(rst_n));   // real interface
    my_dut dut    (.clk(clk), .rst_n(rst_n),
                   .valid(dut_if.valid), ...);    // DUT connected

    initial begin
        // set() BEFORE run_test()
        uvm_config_db #(virtual my_if)::set(
            null,               // outside UVM tree
            "uvm_test_top.*",   // broadcast to all UVM components
            "vif",
            dut_if);            // actual interface instance

        run_test("my_smoke_test");   // hand control to UVM
    end
endmodule
//
// Role of tb_top (simulation bridge):
//   - Instantiate DUT (design under test)
//   - Instantiate interfaces and connect to DUT ports
//   - Generate clock and reset stimulus
//   - Call uvm_config_db::set() for all virtual interfaces
//   - Call run_test() to launch UVM
//   tb_top is the BRIDGE between the RTL world and the UVM world.
//   It does not verify anything itself; it sets the stage for UVM to run.
```

解释

这道题很多人会答"在 test 的 build_phase 里 set()"，这是错误的。

Test 的 build_phase 里做 set() 会有竞争条件：子组件的 build_phase 可能在

parent 的 build_phase 还没 set() 之前就执行了 get()（UVM 不保证顺序）。

正确理解：tb_top 是仿真入口，不属于 UVM 组件树的一部分。

它的 initial block 在 run_test() 调用之前执行，天然保证了 set() 先于所有 build_phase。

面试标准回答：set() 在 tb_top 的 initial begin 里，run_test() 之前，

context 用 null，因为 tb_top 不在 UVM component tree 里。

Q97. UVM: How Many Simulator Invocations for a Smoke Test Script

QUESTION

When writing a script to run a single UVM smoke test from the command line,

how many times do you need to invoke the simulator? What does each call do?

Write a minimal shell script example using VCS.

ANSWER

```
// Answer: 2 invocations -- compile, then simulate.
//
// Invocation 1: COMPILATION (vcs / vlog / irun -compile)
//   Parses all SV/UVM source files
//   Checks syntax, types, elaboration
//   Generates compiled snapshot (./simv binary or work library)
//   Does NOT execute any simulation time
//   Output: simv (VCS), work/ (Questa), snapshot (Xcelium)
//
// Invocation 2: SIMULATION (./simv / vsim / xrun)
//   Loads compiled snapshot
//   Executes simulation from time 0
//   Runs all UVM phases: build -> connect -> run -> check -> report
//   Produces: waveforms (.vcd/.fsdb), log, coverage (.ucdb)
//
// Why 2 separate steps?
//   Compilation is slow (minutes to hours for large SoCs).
//   Once compiled, re-run any number of tests WITHOUT recompiling.
//   Regression: compile once, run 1000 tests in parallel on compute grid.
//   Faster debug: change +UVM_TESTNAME, re-simulate in seconds.

// Minimal VCS smoke test script:
//
// !/bin/bash
// set -e
//
// SRCS="tb_top.sv my_if.sv my_dut.sv my_monitor.sv my_test.sv"
// UVM_HOME="/tools/synopsys/uvm-1.2"
//
// Step 1: Compile (1st invocation)
// vcs -full64 -sverilog -ntb_opts uvm-1.2 \
//     +incdir+${UVM_HOME}/src \
//     ${SRCS} \
//     -o simv \
//     -l compile.log
// echo "[DONE] Compilation"
//
```

```
// Step 2: Simulate smoke test (2nd invocation)
// ./simv \
//     +UVM_TESTNAME=my_smoke_test \
//     +UVM_VERBOSITY=UVM_MEDIUM \
//     -l sim.log
// echo "[DONE] Simulation"

// Questa equivalent:
//   vlog -sv +incdir+... ${SRCS}   <- compile
//   vsim -c tb_top +UVM_TESTNAME=my_smoke_test -do "run -all; quit"

// Regression extension (compile once, run N tests):
//   for TEST in smoke corner stress; do
//       ./simv +UVM_TESTNAME=${TEST} -l ${TEST}.log &
//   done
//   wait
```

解释

2次调用对应 EDA 仿真的两个阶段：elaboration（构建电路模型）和 simulation（跑时间）。

面试答"1次"来自 GUI 工具的使用习惯，底层仍然是两步。

关键延伸：+UVM_TESTNAME 让同一份 compiled snapshot 跑不同 UVM test 类，

无需重新编译。这是所有 regression 框架的基础。

compile.log 和 sim.log 分开是工程规范：编译错误（语法）和运行错误（逻辑）分类定位。

面试加分：主动提到 regression 场景下"编译一次、跑多次"的效率优势。

Q98. UVM: Phases Overview -- Task Phases vs Function Phases

QUESTION

What are the UVM phases? Explain the difference between task phases

and function phases, and list examples of each.

ANSWER

```
// UVM phases execute in a fixed order across all components simultaneously.
// Components override these phase methods to implement their behavior.
//
// ---- Function phases (zero-time, cannot consume simulation time) ----
//   These run as regular SystemVerilog functions.
//   They complete instantly -- no @, no wait, no delay allowed.
//   All components at the same level complete their function phase
//   BEFORE the next phase begins (top-down or bottom-up order).
//
//   build_phase        (top-down):  create child components, get config_db
//   connect_phase      (bottom-up): connect TLM ports between components
//   start_of_simulation_phase (bottom-up): print topology, do sanity checks
//   extract_phase      (bottom-up): collect results after simulation
//   check_phase        (bottom-up): compare results, check for errors
//   report_phase       (bottom-up): print final pass/fail summary
//   final_phase        (top-down):  last cleanup before $finish
//
// ---- Task phases (time-consuming, CAN consume simulation time) ----
//   These are implemented as tasks.
//   They can wait for events, use delays, call other tasks.
//   All components run their task phases CONCURRENTLY (in parallel).
//   The phase ends when all components drop their objections.
```

```
//
//   reset_phase:            apply and release reset
//   configure_phase:        configure DUT registers
//   main_phase:             main test stimulus (most common)
//   shutdown_phase:         drain pipeline, wait for last response
//
// Example -- build_phase (function, top-down):
function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    agent = my_agent::type_id::create("agent", this);
    uvm_config_db #(virtual my_if)::get(this, "", "vif", vif);
endfunction
//
// Example -- run_phase (task, concurrent):
task run_phase(uvm_phase phase);
    phase.raise_objection(this);     // keep phase alive
    @(posedge vif.clk);              // wait for simulation time
    vif.valid <= 1;
    repeat(100) @(posedge vif.clk);
    phase.drop_objection(this);      // release phase
endtask
//
// Key rules:
//   1. Function phases: override as "function void XXX_phase(uvm_phase phase)"
//   2. Task phases:     override as "task XXX_phase(uvm_phase phase)"
//   3. Task phases require raise/drop_objection to control phase duration
//   4. NEVER use delays or @events in a function phase -- they are illegal
```

解释

Function phases 是零时间的构建/配置/报告阶段，不能等待任何事件。

Task phases 是有时间消耗的运行阶段，所有组件的 run_phase 并发执行。

run_phase 是最常用的 task phase，driver、monitor、scoreboard 都在这里工作。

phase.raise_objection / drop_objection 控制 task phase 的生命周期：

最后一个 component 的 drop_objection 触发整个 task phase 结束，进入下一个 function phase。

Q99. UVM: virtual function -- Purpose and Example

QUESTION

What does `virtual function` do in SystemVerilog/UVM?

Give a concrete example showing why it matters in a UVM context.

ANSWER

```
// virtual function enables POLYMORPHISM:
// when a derived class overrides a virtual function, calling that function
// through a BASE CLASS HANDLE dispatches to the DERIVED class's version.
// Without virtual, the base class version always executes regardless of
// the actual object type (static binding).

// Example WITHOUT virtual (broken behavior):
class base_seq extends uvm_sequence #(my_txn);
    function string get_name_str();  // NOT virtual
        return "base_seq";
    endfunction
endclass
```

```
class derived_seq extends base_seq;
    function string get_name_str();  // override
        return "derived_seq";
    endfunction
endclass

// Driver holds a base class handle:
base_seq seq_h;
seq_h = derived_seq::type_id::create("seq");  // actual type is derived

$display(seq_h.get_name_str());
// Without virtual: prints "base_seq"  <-- WRONG, object is derived_seq!
// With virtual:    prints "derived_seq" <-- CORRECT, runtime dispatch

// Real UVM use case: factory override + virtual callback
class my_monitor extends uvm_monitor;

    // Virtual function called for every transaction captured
    virtual function void process_txn(my_txn t);
        // base behavior: just print
        `uvm_info("MON", $sformatf("txn: %s", t.sprint()), UVM_MEDIUM)
    endfunction

    task run_phase(uvm_phase phase);
        my_txn t;
        forever begin
            @(vif.mon_cb);
            if (vif.mon_cb.valid) begin
                t = my_txn::type_id::create("t");
                t.data = vif.mon_cb.data;
                process_txn(t);   // dispatches to derived if overridden
                ap.write(t);
            end
        end
    endtask
endclass

// Derived monitor overrides without changing run_phase:
class verbose_monitor extends my_monitor;
    virtual function void process_txn(my_txn t);
        super.process_txn(t);    // call base behavior
        // Add extra coverage or checks
        cov_group.sample();
        assert(t.data != 8'hFF) else `uvm_error("MON","illegal data 0xFF");
    endfunction
endclass

// Factory override in test: replace base with verbose
// my_monitor::type_id::set_type_override(verbose_monitor::get_type());
//
// Now all my_monitor instances use verbose_monitor::process_txn automatically.
// This is the UVM factory pattern: virtual functions + factory = clean extensibility.

// Why UVM phase methods are virtual:
//   build_phase, run_phase, etc. are all declared virtual in uvm_component.
//   This allows your component class to override them and have the UVM
//   phasing engine call YOUR version when it iterates the component tree.
//   Without virtual, the base uvm_component's empty phase methods would
//   always run, and your overrides would be silently ignored.
```

解释

virtual function 是面向对象的核心机制，在 UVM 中无处不在。

所有 UVM phase 方法（build_phase、run_phase 等）都是 virtual，

这也是为什么你的 override 会被 UVM 框架自动调用的原因。

UVM factory override 的工作原理也依赖 virtual：

type_id::create() 实际创建了 derived 类型的对象，存在 base 类型的 handle 里，

virtual function 保证调用时走 derived 的实现。

没有 virtual，所有多态都失效，UVM 的 factory override 就毫无意义。

Q100. UVM: Objection Mechanism -- How It Works

QUESTION

How does the UVM objection mechanism work? What happens if no objections are raised? How do you detect that a test passed trivially (zero simulation time)?

ANSWER

```
// How objection works:
//
// UVM task phases (run_phase, reset_phase, etc.) end only when ALL
// raised objections have been dropped. The phase manager maintains
// a global objection count per phase.
//
//   raise_objection(this)  -> count++
//   drop_objection(this)   -> count--
//   When count reaches 0   -> phase ends, next phase begins
//
// Typical usage in a sequence or test:
task run_phase(uvm_phase phase);
    phase.raise_objection(this);    // count: 0->1, phase stays alive
    // ... drive stimulus, wait for responses ...
    repeat(100) @(posedge vif.clk);
    phase.drop_objection(this);     // count: 1->0, phase ends
endtask
//
// What happens if NO objections are ever raised:
//   The phase objection count starts at 0.
//   Since it never goes above 0, it is already "at zero" from the start.
//   The phase ends IMMEDIATELY at time 0 before any stimulus runs.
//   All UVM components report phase PASSED (no errors occurred).
//   The test appears to pass -- but zero stimulus was applied.
//   This is a silent false pass: the most dangerous kind of bug.
//
// How to detect trivial zero-time pass:
//   Override phase_ready_to_end() in your base test or env:
//
function void phase_ready_to_end(uvm_phase phase);
    if (phase.get_name() == "run" && $time == 0)
        `uvm_error("NO_RUN_OBJECTION",
            "run_phase ended at time 0 - no objections were raised")
endfunction
//
// This hook is called by UVM when the phase is ABOUT to end.
// If simulation time is still 0, no clock edges have occurred,
// meaning no real stimulus ran -- report an error immediately.
//
// Additional safeguards:
//
// 1. Drain time: add a settle period before ending phase
```

```
//   phase.drop_objection(this, "done", 100);  // wait 100 time units after drop
//
// 2. Objection trace: run with +UVM_OBJECTION_TRACE to log every raise/drop
//   Prints: component name, raise/drop time, current count
//
// 3. Timeout: set a maximum simulation time
//   uvm_top.set_timeout(1ms, 1);  // fatal if run_phase exceeds 1ms
```

解释

Objection 是 UVM 的相位控制机制，本质是一个引用计数器。

零时间通过是最难发现的 testbench bug：CI/CD 系统看到 PASS，

但实际上 DUT 从未收到任何激励，覆盖率为零。

phase_ready_to_end() 钩子是官方推荐的检测方法：

它在相位即将结束时被调用，此时检查 $time==0 是最简单可靠的判断。

工程实践：把这个检查放在基类 base_test 里，所有测试自动继承，

一劳永逸地防止零时间通过。

Q101. UVM: Verification Environment for 2-Input 10-Output Random Dispatch DUT

QUESTION

Given a DUT with 2 inputs (one AXI interface, one CPU request interface)

and 10 outputs (transactions randomly dispatched to any of the 10 ports),

how do you build the verification environment and scoreboard?

ANSWER

```
// Environment architecture:
//
//  +-------------------------------------------------------------------+
//  |  my_env                                                           |
//  |                                                                   |
//  |  [axi_agent] (active)   [cpu_agent] (active)                      |
//  |       |                      |                                    |
//  |   axi_monitor           cpu_monitor                               |
//  |       |                      |          (both write to SCB)       |
//  |       +----------+-----------+                                    |
//  |                  |                                                |
//  |              [scoreboard]  <--- 2 input queues                    |
//  |                  |                                                |
//  |  [out_agent_0..9] (passive) x10  -- 10 output monitors            |
//  |       each monitor writes observed output txn to SCB              |
//  +-------------------------------------------------------------------+
//
// Agent configuration:
//   Input side:  2 ACTIVE agents (drive stimulus + monitor)
//     - axi_agent:  drives AXI transactions onto DUT AXI input port
//     - cpu_agent:  drives CPU request transactions
//   Output side: 10 PASSIVE agents (monitor only, no driver)
//     - out_agent[0..9]: each monitors one output port
//     - Each has a monitor that captures output transactions
//     - All 10 monitors connect to the scoreboard via analysis ports

// Scoreboard design:
//
```

```
class my_scoreboard extends uvm_scoreboard;
    // Two input queues (one per input port)
    // Since dispatch is RANDOM, we do NOT know which output gets which input.
    // We only need to verify: every input transaction appears exactly once
    // on exactly one output port.
    my_txn axi_queue[$];   // transactions seen on AXI input
    my_txn cpu_queue[$];   // transactions seen on CPU input

    // Analysis exports -- connected to input monitors
    uvm_analysis_imp_axi #(my_txn, my_scoreboard) axi_imp;
    uvm_analysis_imp_cpu #(my_txn, my_scoreboard) cpu_imp;

    // Analysis exports -- connected to 10 output monitors
    uvm_analysis_imp_out #(my_txn, my_scoreboard) out_imp[10];

    // Input side: push into queue (input always arrives before output)
    function void write_axi(my_txn t);
        axi_queue.push_back(t.clone());
    endfunction

    function void write_cpu(my_txn t);
        cpu_queue.push_back(t.clone());
    endfunction

    // Output side: search BOTH queues for a match
    // (we don't know which input caused this output)
    function void write_out(my_txn observed);
        my_txn expected;
        // Try axi_queue first
        foreach (axi_queue[i]) begin
            if (axi_queue[i].compare(observed)) begin
                axi_queue.delete(i);
                return;    // matched
            end
        end
        // Try cpu_queue
        foreach (cpu_queue[i]) begin
            if (cpu_queue[i].compare(observed)) begin
                cpu_queue.delete(i);
                return;    // matched
            end
        end
        // No match found in either queue
        `uvm_error("SCB", $sformatf("Output txn has no matching input: %s",
                   observed.sprint()))
    endfunction

    // End-of-test: check no inputs were lost
    function void check_phase(uvm_phase phase);
        if (axi_queue.size() > 0)
            `uvm_error("SCB", $sformatf("%0d AXI transactions never appeared on output",
                       axi_queue.size()))
        if (cpu_queue.size() > 0)
            `uvm_error("SCB", $sformatf("%0d CPU transactions never appeared on output",
                       cpu_queue.size()))
    endfunction
endclass

// Coverage strategy:
//   - Which output ports were used (cross [0..9] with input type)
//   - Distribution uniformity: all 10 ports should receive roughly equal traffic
//   - Back-to-back transactions on same input
//   - Simultaneous AXI + CPU transactions (both inputs active same cycle)
```

解释

设计环境的核心决策：

1. 输入2个 active agent：既要驱动激励，也要 monitor 记录实际发出的内容。

2. 输出10个 passive agent：只需 monitor，不驱动。

3. Scoreboard 用两个 queue 而非一个：区分 AXI 和 CPU 来源。

因为是随机派发，SCB 收到输出事务时必须在两个 queue 里都搜索匹配，

不能假设"AXI输入一定对应某个输出"。

check_phase 检查两个 queue 是否清空，确保没有事务丢失（输入了但没有输出）。

覆盖率要验证10个输出端口的均匀分布，防止 DUT 只派发到少数几个端口。

Q102. UVM: Testbench for Write-Only Register

QUESTION

Given a DUT with a write-only register (you can write to it but cannot

read it back), write a complete UVM testbench to verify it.

What are the key challenges and how do you address them?

ANSWER

```
// Challenge: write-only register cannot be read back to verify the write.
// Must verify correctness through SIDE EFFECTS:
//   - What does the DUT DO with the value written to this register?
//   - Does changing the register value change DUT output behavior?
//   - Is the value properly stored (even if not readable)?

// ---- Step 1: UVM Register Model with write-only field ----

class ctrl_reg extends uvm_reg;
    `uvm_object_utils(ctrl_reg)

    // Field is WRITE-ONLY: no read access
    rand uvm_reg_field ctrl_field;

    function new(string name = "ctrl_reg");
        super.new(name, 8, UVM_NO_COVERAGE);
    endfunction

    virtual function void build();
        ctrl_field = uvm_reg_field::type_id::create("ctrl_field");
        // WO = write-only: writes update shadow, reads return X or 0
        ctrl_field.configure(
            .parent(this),
            .size(8),
            .lsb_pos(0),
            .access("WO"),    // <-- write-only access type
            .volatile(0),
            .reset(8'h00),
            .has_reset(1),
            .is_rand(1),
            .individually_accessible(1)
        );
    endfunction
endclass
```

```
// ---- Step 2: Register Block ----

class my_reg_block extends uvm_reg_block;
    `uvm_object_utils(my_reg_block)
    rand ctrl_reg CTRL;

    function new(string name = "my_reg_block");
        super.new(name, UVM_NO_COVERAGE);
    endfunction

    virtual function void build();
        CTRL = ctrl_reg::type_id::create("CTRL");
        CTRL.build();
        CTRL.configure(.blk_parent(this), .hdl_path("dut.ctrl_reg"));

        // Map the register at offset 0x00
        default_map = create_map("default_map", 0, 4, UVM_LITTLE_ENDIAN);
        default_map.add_reg(CTRL, .offset(8'h00), .rights("WO"));
        lock_model();
    endfunction
endclass

// ---- Step 3: Write-Only Register Test Sequence ----

class write_only_reg_seq extends uvm_reg_sequence;
    `uvm_object_utils(write_only_reg_seq)

    my_reg_block regmodel;

    task body();
        uvm_status_e status;
        uvm_reg_data_t wdata;

        // Test 1: write known pattern, verify side effect
        `uvm_info("SEQ", "Test 1: Write 0xAA to CTRL", UVM_MEDIUM)
        regmodel.CTRL.write(status, 8'hAA, UVM_FRONTDOOR);
        // Cannot read back -- verify via DUT output monitor in scoreboard

        // Test 2: write all zeros
        `uvm_info("SEQ", "Test 2: Write 0x00 to CTRL", UVM_MEDIUM)
        regmodel.CTRL.write(status, 8'h00, UVM_FRONTDOOR);

        // Test 3: write all ones
        `uvm_info("SEQ", "Test 3: Write 0xFF to CTRL", UVM_MEDIUM)
        regmodel.CTRL.write(status, 8'hFF, UVM_FRONTDOOR);

        // Test 4: random writes
        repeat(10) begin
            wdata = $urandom_range(0, 255);
            regmodel.CTRL.write(status, wdata, UVM_FRONTDOOR);
            // Each write: scoreboard checks corresponding DUT output change
            #100ns;  // wait for DUT to process and output to settle
        end

        // Test 5: UVM built-in register test (write-only safe version)
        // uvm_reg_hw_reset_seq: writes reset value, checks side effect
        // uvm_reg_bit_bash_seq: walks 1 bit at a time through all fields
        // Note: uvm_reg_access_seq will FAIL for WO regs (it tries to read back)
        // Use uvm_reg_bit_bash_seq instead -- it only writes
    endtask
endclass

// ---- Step 4: Scoreboard -- verify through side effects ----
```

```
class my_scoreboard extends uvm_scoreboard;
    `uvm_component_utils(my_scoreboard)

    // Track what was written (from register model shadow value)
    uvm_reg_data_t last_written_value;
    bit            value_written = 0;

    // Analysis import: receive write transactions from reg adapter
    uvm_analysis_imp #(uvm_reg_bus_op, my_scoreboard) reg_ap;

    // Analysis import: receive DUT output observations
    uvm_analysis_imp_dut #(dut_output_txn, my_scoreboard) dut_ap;

    // When a register write is observed on the bus:
    function void write(uvm_reg_bus_op rw);
        if (rw.kind == UVM_WRITE) begin
            last_written_value = rw.data;
            value_written = 1;
            `uvm_info("SCB", $sformatf(
                "Register write observed: data=0x%02h", rw.data), UVM_MEDIUM)
        end
    endfunction

    // When DUT output is observed:
    // Verify that the DUT output MATCHES expected behavior
    // based on what was last written to the register
    function void write_dut(dut_output_txn txn);
        if (!value_written) begin
            `uvm_error("SCB", "DUT output seen before any register write!")
            return;
        end
        // Example: if CTRL[7:0] controls output enable on 8 channels,
        // verify each channel output matches the corresponding CTRL bit
        for (int i = 0; i < 8; i++) begin
            bit expected = last_written_value[i];
            bit observed = txn.channel_active[i];
            if (expected !== observed)
                `uvm_error("SCB", $sformatf(
                    "Channel %0d mismatch: wrote CTRL[%0d]=%0b, observed=%0b",
                    i, i, expected, observed))
        end
        `uvm_info("SCB", "DUT output matches written register value", UVM_MEDIUM)
    endfunction
endclass

// ---- Step 5: Handling "no readback" in register model ----

// When UVM register model tries to read a WO register:
//   read() returns the SHADOW value (what was last written), not DUT value
//   This is correct behavior -- never rely on read() for WO registers
//
// To prevent accidental reads in built-in sequences:
//   Use predict() to update shadow without accessing the bus:
//   regmodel.CTRL.predict(8'hAA);  // update model, no bus transaction
//
// uvm_reg_bit_bash_seq is SAFE for WO registers (write-only operations)
// uvm_reg_access_seq MUST be EXCLUDED -- it reads after every write

// ---- Summary: Key verification strategies for write-only registers ----
//
// 1. Indirect verification: check DUT outputs/side effects after each write
// 2. HDL path backdoor: use UVM backdoor to READ the actual register value
```

```
//    without going through the bus (bypasses the write-only restriction):
//    regmodel.CTRL.read(status, rdata, UVM_BACKDOOR);
//    This reads directly from RTL via $cast / force / deposit
// 3. Waveform inspection: use $monitor or SVA to verify register bits
// 4. Shadow register: maintain a software mirror of what was written,
//    compare against side effects rather than reading the register itself
```

解释

Write-only register 验证的核心策略：

1. 间接验证（最重要）：写入后观察 DUT 的输出行为是否符合预期，

SCB 维护"上次写入值的影子"，与 DUT 输出做比对。

2. UVM backdoor 读：通过 HDL 路径直接读取 RTL 内部信号，

绕过总线层面的 write-only 限制，用于调试和验证。

3. 慎用内建序列：uvm_reg_access_seq 会 read-after-write，对 WO 寄存器是非法的。

改用 uvm_reg_bit_bash_seq（只写），或手动写测试序列。

UVM 寄存器模型的 access type "WO" 告知 model：

read() 返回 shadow 值（不发总线事务），write() 正常发总线事务。

面试要点：主动提出"无法直接验证，需要通过 side effect 间接验证"，

再提 backdoor 作为补充手段，体现对 UVM register abstraction 的深入理解。

Q103. UVM: Startup Flow and Top Module Contents

QUESTION

(a) Describe the UVM startup flow from simulation launch to run_phase.

(b) What does the UVM top module (tb_top) contain?

ANSWER

```
// (a) UVM startup flow
//
// 1. Simulator loads tb_top (plain SV module, not UVM)
//    - DUT instantiated
//    - Interfaces instantiated and connected to DUT ports
//    - Clock and reset generators started
//    - uvm_config_db::set() called for all virtual interfaces
//
// 2. run_test("my_test_name") called in tb_top initial block
//    - UVM factory creates uvm_test_top (instance of my_test_name)
//    - UVM component tree is built from top to bottom
//
// 3. build_phase (top-down, function, zero time)
//    - Components create child components via type_id::create()
//    - uvm_config_db::get() retrieves virtual interfaces, cfg objects
//    - Analysis ports/exports created
//
// 4. connect_phase (bottom-up, function, zero time)
//    - TLM ports connected: driver <-> sequencer, monitor -> scoreboard
//    - Analysis ports connected to analysis exports
//
// 5. start_of_simulation_phase (bottom-up, function, zero time)
//    - Print topology: uvm_top.print_topology()
```

```
//    - Sanity checks, display configuration
//
// 6. run_phase (all components concurrent, task, consumes time)
//    - Sequences started on sequencers via seq.start(sequencer)
//    - Drivers pull items from sequencer, drive DUT
//    - Monitors sample DUT, write to analysis ports
//    - Scoreboard receives and checks transactions
//    - Phase ends when all objections are dropped
//
// 7. extract_phase -> check_phase -> report_phase -> final_phase
//    (function phases, zero time, bottom-up)
//    - Collect coverage, check for errors, print summary

// (b) tb_top contents
//
module tb_top;
    // 1. Clock and reset generation
    logic clk;
    logic rst_n;
    initial clk = 0;
    always #5 clk = ~clk;   // 100 MHz clock

    initial begin
        rst_n = 0;
        repeat(10) @(posedge clk);
        rst_n = 1;
    end

    // 2. Interface instantiation (connects DUT signals to UVM)
    my_if dut_if (.clk(clk), .rst_n(rst_n));

    // 3. DUT instantiation
    my_dut dut (
        .clk   (clk),
        .rst_n (rst_n),
        .valid (dut_if.valid),
        .data  (dut_if.data),
        .ready (dut_if.ready)
    );

    // 4. Pass virtual interfaces into UVM via config_db
    //    Must happen BEFORE run_test() so build_phase can retrieve them
    initial begin
        uvm_config_db #(virtual my_if)::set(
            null,               // outside UVM tree
            "uvm_test_top.*",   // all components under test
            "vif",              // key name
            dut_if              // actual interface instance
        );

        // 5. Launch UVM -- hands control to UVM phasing engine
        run_test("my_smoke_test");
    end
endmodule
```

解释

UVM 启动流程的关键顺序：tb_top 先执行（SV 仿真时间 0），完成接口实例化和 set()，

然后 run_test() 接管，触发 build->connect->run 的相位链。

tb_top 是 SV 和 UVM 的桥梁：它不属于 UVM 组件树，但负责把硬件世界（DUT + interface）

和软件验证世界（UVM）连接起来。

面试常问：为什么 set() 在 tb_top 而不是 test 里？

答：tb_top 的 initial block 在 run_test() 之前执行，保证 set() 先于 build_phase 的 get()。

Q104. UVM: Override Default Constraints Without Modifying the Class

QUESTION

A transaction class has default constraints already defined. How do you override them for a single randomization call at runtime without modifying the original class? Explain soft constraints and inline constraints.

ANSWER

```
// Original transaction class with default constraints:
class my_txn extends uvm_sequence_item;
    `uvm_object_utils(my_txn)

    rand bit [7:0] data;
    rand bit [3:0] addr;

    // Default constraint: data should be in range [10:100]
    constraint c_data_range {
        data inside {[10:100]};
    }

    // Soft constraint: can be overridden by a harder constraint
    // If a conflicting constraint is added later, soft loses.
    constraint c_addr_default {
        soft addr inside {[0:7]};   // prefer low addresses, but overridable
    }
endclass

// ---- Method 1: Inline constraint (with { ... }) ----
// Adds an EXTRA constraint for ONE randomize() call only.
// The inline constraint is combined with existing constraints.
// If inline conflicts with a hard constraint -> randomization FAILS.
// If inline conflicts with a SOFT constraint -> soft yields, inline wins.

task run_phase(uvm_phase phase);
    my_txn txn = my_txn::type_id::create("txn");

    // Override: force data = 0xFF for this one randomization
    // Inline constraint overrides soft c_addr_default, fails vs hard c_data_range
    assert(txn.randomize() with {
        data == 8'hFF;          // violates c_data_range -> FAILS
    }) else `uvm_fatal("RAND", "Randomization failed");

    // Correct approach: use soft in the original class so inline can override
    // (see c_addr_default above -- soft lets inline win)
    assert(txn.randomize() with {
        addr inside {[8:15]};   // overrides soft c_addr_default -> succeeds
    });
endtask

// ---- Method 2: constraint_mode() -- disable existing constraint ----
// Temporarily disable a named constraint, randomize, then re-enable.

task send_special(my_txn txn);
    // Disable the default range constraint
    txn.c_data_range.constraint_mode(0);   // OFF
```

```
    // Now randomize freely (c_data_range not active)
    assert(txn.randomize() with { data > 200; });

    // Re-enable for subsequent use
    txn.c_data_range.constraint_mode(1);   // ON
endtask

// ---- Method 3: Derived class with override ----
// For systematic override across multiple randomizations:
class special_txn extends my_txn;
    `uvm_object_utils(special_txn)

    // Override the parent's constraint with a new one of the same name
    constraint c_data_range {
        data inside {[200:255]};   // completely replaces parent's version
    }
endclass
// Use factory override to inject special_txn without changing test code:
// my_txn::type_id::set_type_override(special_txn::get_type());

// ---- soft vs hard summary ----
// hard constraint: always enforced; conflicting inline -> randomization fails
// soft constraint: lower priority; conflicting inline/hard -> soft yields silently
// inline { ... }: scoped to ONE randomize() call; combines with all active constraints
```

解释

三种运行时覆盖方式各有适用场景：

inline constraint：最常用，临时约束单次 randomize()，不影响对象状态。

constraint_mode(0)：关闭命名约束，适合需要多次随机化且都需要同样覆盖的情况。

derived class + factory override：系统性替换，适合大范围测试场景切换。

soft 关键字是设计"可覆盖默认值"的最佳实践：class 作者表达偏好但允许 test 覆盖，

体现了 reusable VIP 的设计哲学。面试中能同时给出三种方案并说明选择依据是加分项。

Q105. UVM: How to Determine if DUT Verification is Complete

QUESTION

How do you judge whether a DUT is sufficiently verified?

What metrics and criteria do you use?

ANSWER

```
// Verification completeness is judged by THREE complementary metrics:

// 1. Functional Coverage
//    Did the test stimulus exercise all intended behaviors?
//    - covergroup / coverpoint for each feature
//    - cross coverage between correlated signals
//    - Target: 100% of defined coverpoints hit
//    - Missing coverage -> write directed tests or adjust constraints
//    Example metrics:
//      All FSM states visited
//      All opcodes exercised
//      Back-to-back transactions, max burst length, minimum gap
```

```
// 2. Code Coverage
//    Did simulation execute all RTL code paths?
//    - Line coverage:      every line of RTL executed
//    - Branch coverage:    both sides of every if/case executed
//    - Toggle coverage:    every signal toggled 0->1 and 1->0
//    - FSM coverage:       every state and transition exercised
//    - Target: typically 95-100% line/branch, 90%+ toggle
//    - Remaining gaps: add directed tests or waive with justification

// 3. Assertion Coverage (SVA)
//    Did assertions fire and were all expected behaviors checked?
//    - Assertion pass count: each assert fired and passed >= N times
//    - Cover property:       protocol sequences actually occurred
//    - Vacuous pass detection: assertion never triggered (antecedent never true)
//    - If an assertion never fires, the scenario may not be stimulated at all

// 4. Bug rate / Escape rate
//    Is the bug discovery rate approaching zero?
//    - Plot bugs found per week; convergence toward 0 indicates maturity
//    - Regression pass rate: 100% pass on nightly regression
//    - No new bugs in last N regression runs

// 5. Review and sign-off checklist
//    - Test plan coverage: every feature in the spec has >= 1 directed test
//    - Corner cases documented and tested
//    - Formal verification for critical properties (if applicable)
//    - Peer review of testbench and coverage plan
//
// Sign-off criteria (typical):
//   functional_coverage >= 100%  (all coverpoints hit)
//   code_coverage       >= 95%   (remainder waived with justification)
//   regression          == PASS  (0 failures)
//   assertion_count     > 0      (no vacuous passes)
//   bug_rate            -> 0     (converged)
```

解释

验证完整性没有单一标准，必须综合多个维度：

功能覆盖率（验证意图）+ 代码覆盖率（验证执行）+ 断言（验证协议）+ Bug 收敛趋势。

100% 代码覆盖率不等于验证完整：代码可能执行了，但输出没有被检查（SCB 漏洞）。

功能覆盖率也不够：覆盖点定义不完整，遗漏的场景不会被发现。

面试标准回答：从覆盖率（functional + code + assertion）讲到 bug 收敛趋势，

再提 sign-off checklist，最后强调"所有未覆盖点都需要 waive 并有 justification"。

Q106. UVM: Sequence with Inline Constraint + Parallel Fork-Join + Event Sync

QUESTION

(a) Write a UVM sequence that can be configured as either read or write

from the test level (using inline constraints).

(b) Write a test that runs a write sequence and a read sequence IN PARALLEL

using fork-join.

(c) How do you guarantee the write completes before the read starts, while

keeping them in parallel threads? Use a semaphore.

ANSWER

```
// ---- (a) Configurable Read/Write Sequence --------------------------------

class rw_sequence extends uvm_sequence #(my_txn);
    `uvm_object_utils(rw_sequence)

    bit is_write;   // set by test before calling seq.start()

    function new(string name = "rw_sequence");
        super.new(name);
    endfunction

    task body();
        my_txn txn;
        txn = my_txn::type_id::create("txn");
        start_item(txn);
        // Inline constraint: override op field for this one randomize() call
        assert(txn.randomize() with {
            txn.op == (is_write ? OP_WRITE : OP_READ);
        }) else `uvm_fatal("SEQ", "Randomization failed");
        finish_item(txn);
    endtask
endclass

// ---- (b) + (c) Test: parallel fork-join, write-before-read via event -----

class parallel_rw_test extends uvm_test;
    `uvm_component_utils(parallel_rw_test)

    my_env env;

    // event: write thread triggers it; read thread waits on it.
    // Cleaner than semaphore -- no key count to manage, single direction.
    event write_done;

    function new(string name, uvm_component parent);
        super.new(name, parent);
    endfunction

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        env = my_env::type_id::create("env", this);
    endfunction

    task run_phase(uvm_phase phase);
        rw_sequence wr_seq, rd_seq;
        phase.raise_objection(this);

        wr_seq = rw_sequence::type_id::create("wr_seq");
        rd_seq = rw_sequence::type_id::create("rd_seq");
        wr_seq.is_write = 1;
        rd_seq.is_write = 0;

        fork
            // Thread 1: write, then signal completion
            begin
                wr_seq.start(env.agent.sequencer);
                -> write_done;   // trigger event: write is done
            end

            // Thread 2: wait for write, then read
            begin
                @write_done;     // block until write thread triggers
```

```
                rd_seq.start(env.agent.sequencer);
            end
        join   // wait for both threads

        phase.drop_objection(this);
    endtask
endclass

// Why event, not semaphore?
//   semaphore:  has a key count, designed for mutual exclusion (N threads
//               competing for a shared resource). Overkill here -- we only
//               need a one-shot directional signal.
//   event:      single broadcast, zero overhead. -> triggers all @waiters
//               instantly. Perfect for "A signals B to start" patterns.
//
// Why event, not join_any?
//   join_any exits as soon as ONE thread finishes -- we'd exit after write
//   completes, before read even starts. join is needed to ensure both
//   threads complete. The event handles the ordering WITHIN the join.
```

解释

event 是 UVM 中最轻量的线程同步原语，适合单向一次性信号。

-> write_done 触发事件，所有正在等待 @write_done 的线程立即被唤醒。

对比 semaphore：semaphore 适合"互斥锁"场景（N个线程竞争1个资源），

这里只需要 "write 完了通知 read 开始"，用 event 更直观、代码更少。

fork...join 保证两个线程都跑完才继续（write 和 read 都结束后才 drop_objection）。

如果改成 fork...join_any，write 一结束整个 fork 就退出，read 根本没机会跑。

Q107. UVM: Instruction Transaction with Covergroup and Forwarding Path

QUESTION

(a) Write an instruction transaction class with constraints on opcode field

and overall range. Add a covergroup inside the class.

(b) If an operand comes from a forwarding path, how do you modify the

transaction to capture this information?

ANSWER

```
// ---- (a) Instruction Transaction with Covergroup ------------------------

typedef enum bit [2:0] {
    OP_ADD  = 3'b000,
    OP_SUB  = 3'b001,
    OP_LOAD = 3'b010,
    OP_STORE= 3'b011,
    OP_JMP  = 3'b100
} opcode_e;

class instr_txn extends uvm_sequence_item;
    `uvm_object_utils(instr_txn)

    // ---- Fields ------------------------------------------------------------
    rand bit [31:0] instruction;   // full 32-bit instruction word
```

```
    // Decoded fields (derived from instruction bits)
    rand opcode_e opcode;          // bits [31:29] = opcode (3 bits)
    rand bit [4:0] rs1;            // bits [28:24] = source register 1
    rand bit [4:0] rs2;            // bits [23:19] = source register 2
    rand bit [4:0] rd;             // bits [18:14] = destination register
    rand bit [13:0] imm;           // bits [13:0]  = immediate value

    // Forwarding path info (filled by monitor, not randomized)
    bit            from_forwarding;   // 1 if operand came from forwarding path
    int unsigned   src_instr_id;      // instruction ID that produced the forwarded value
    int unsigned   instr_id;          // unique ID for this instruction

    // ---- Constraints ---------------------------------------------------
    constraint c_opcode_encoding {
        // Opcode occupies bits [31:29]
        instruction[31:29] == opcode;
    }

    constraint c_field_encoding {
        instruction[28:24] == rs1;
        instruction[23:19] == rs2;
        instruction[18:14] == rd;
        instruction[13:0]  == imm;
    }

    constraint c_valid_range {
        // Full instruction must fit in 32 bits (implicit), but constrain
        // that opcode is always a valid enum value
        opcode inside {OP_ADD, OP_SUB, OP_LOAD, OP_STORE, OP_JMP};
    }

    constraint c_load_no_store_dst {
        // STORE instructions do not write a destination register
        (opcode == OP_STORE) -> rd == 5'b0;
    }

    // ---- Covergroup ----------------------------------------------------
    covergroup instr_cg;
        // Cover all opcodes
        cp_opcode: coverpoint opcode {
            bins add   = {OP_ADD};
            bins sub   = {OP_SUB};
            bins load  = {OP_LOAD};
            bins store = {OP_STORE};
            bins jmp   = {OP_JMP};
        }

        // Cover register ranges: low (r0-r7), mid (r8-r23), high (r24-r31)
        cp_rs1: coverpoint rs1 {
            bins low  = {[0:7]};
            bins mid  = {[8:23]};
            bins high = {[24:31]};
        }

        cp_rd: coverpoint rd {
            bins low  = {[0:7]};
            bins mid  = {[8:23]};
            bins high = {[24:31]};
        }

        // Cross: which opcodes use which destination registers
        cx_opcode_rd: cross cp_opcode, cp_rd {
            // STORE never writes rd -- exclude from cross coverage
```

```
            ignore_bins store_rd = binsof(cp_opcode.store);
        }

        // Cover forwarding path usage
        cp_forwarding: coverpoint from_forwarding {
            bins normal    = {0};
            bins forwarded = {1};
        }

        // Cross: opcodes that consume forwarded operands
        cx_opcode_fwd: cross cp_opcode, cp_forwarding;
    endgroup

    function new(string name = "instr_txn");
        super.new(name);
        instr_cg = new();   // instantiate covergroup in constructor
    endfunction

    // Sample coverage (called by monitor after all fields are populated)
    function void sample_coverage();
        instr_cg.sample();
    endfunction

endclass

// ---- (b) Forwarding path: how monitor fills the transaction --------------
//
// A forwarded operand is one where a register value is taken directly from
// a previous instruction's execution result (bypassing the register file).
// The monitor cannot know this from the instruction word alone.
//
// Solution: assign a unique instruction_id to each issued instruction.
// The monitor tracks in-flight instructions. When the pipeline signals
// "result of instr_id=N is forwarded to instr_id=M", the monitor:
//   1. Locates the CONSUMER transaction (id=M) in its tracking table.
//   2. Sets from_forwarding = 1.
//   3. Sets src_instr_id = N (which instruction produced the value).
//   4. Calls sample_coverage() AFTER both fields are populated.
//
// Monitor implementation sketch:
class instr_monitor extends uvm_monitor;
    `uvm_component_utils(instr_monitor)

    // In-flight instruction table: id -> transaction
    instr_txn in_flight [int unsigned];

    task run_phase(uvm_phase phase);
        instr_txn txn;
        forever begin
            @(posedge vif.clk);

            // New instruction issued: create transaction, assign ID
            if (vif.issue_valid) begin
                txn = instr_txn::type_id::create("txn");
                txn.instr_id = vif.instr_id;
                txn.opcode   = opcode_e'(vif.opcode);
                // ... fill other fields from interface ...
                in_flight[txn.instr_id] = txn;
            end

            // Forwarding event detected from pipeline signals
            if (vif.fwd_valid) begin
                int unsigned consumer_id = vif.fwd_consumer_id;
```

```
                int unsigned producer_id = vif.fwd_producer_id;
                if (in_flight.exists(consumer_id)) begin
                    in_flight[consumer_id].from_forwarding = 1;
                    in_flight[consumer_id].src_instr_id    = producer_id;
                end
            end

            // Instruction commits: sample coverage and broadcast
            if (vif.commit_valid) begin
                int unsigned id = vif.commit_id;
                if (in_flight.exists(id)) begin
                    in_flight[id].sample_coverage();
                    ap.write(in_flight[id]);
                    in_flight.delete(id);
                end
            end
        end
    endtask
endclass
```

解释

(a) Instruction transaction 的约束体现了 RTL 字段布局：opcode 对应特定 bit 区间，
约束保证 instruction 字段与解码字段的一致性（c_opcode_encoding, c_field_encoding）。
Covergroup 在 constructor 里实例化，由 monitor 在 commit 时调用 sample()，
确保只对"完整"的事务（包括 forwarding 信息）采集覆盖率。
(b) Forwarding path 是流水线设计的核心优化，验证难点在于信息分散在不同时钟周期：
指令在 issue 阶段发出，forwarding 在 execute 阶段发生，commit 在 writeback 阶段。
Monitor 用 instr_id 作为 key 维护 in-flight 表，将三个阶段的信息汇聚到同一个
transaction 对象上，最后在 commit 时统一采样覆盖率。
这是"延迟完整化 transaction"的经典设计模式，考察对流水线验证的深入理解。

Q108. UVM: Clock Period Monitor and Scoreboard (5% Tolerance Check)

QUESTION

Write a monitor that measures each clock cycle period and sends it to a scoreboard. The scoreboard receives two clock monitors (DUT clock and reference clock) and checks that their periods differ by no more than 5%.

ANSWER

```
// ---- Clock Period Monitor ------------------------------------------

class clk_monitor extends uvm_monitor;
    `uvm_component_utils(clk_monitor)

    virtual clk_if vif;
    uvm_analysis_port #(clk_txn) ap;

    function new(string name, uvm_component parent);
        super.new(name, parent);
    endfunction
```

```
    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        ap = new("ap", this);
        if (!uvm_config_db #(virtual clk_if)::get(this, "", "vif", vif))
            `uvm_fatal("MON", "No virtual interface found")
    endfunction

    task run_phase(uvm_phase phase);
        clk_txn txn;
        realtime t_prev, t_now;

        // Wait for first edge to establish baseline
        @(posedge vif.clk);
        t_prev = $realtime;

        forever begin
            @(posedge vif.clk);
            t_now = $realtime;

            txn = clk_txn::type_id::create("txn");
            txn.period_ps = t_now - t_prev;   // measured period in ps
            ap.write(txn);

            t_prev = t_now;
        end
    endtask
endclass

// ---- Clock Transaction ------------------------------------------------

class clk_txn extends uvm_sequence_item;
    `uvm_object_utils(clk_txn)
    realtime period_ps;
    function new(string name = "clk_txn");
        super.new(name);
    endfunction
endclass

// ---- Scoreboard -------------------------------------------------------

class clk_scoreboard extends uvm_scoreboard;
    `uvm_component_utils(clk_scoreboard)

    // Two analysis imports: DUT clock and reference clock
    uvm_analysis_imp_dut #(clk_txn, clk_scoreboard) imp_dut;
    uvm_analysis_imp_ref #(clk_txn, clk_scoreboard) imp_ref;

    // Queues to align DUT and reference measurements cycle-by-cycle
    clk_txn dut_q[$];
    clk_txn ref_q[$];

    localparam real TOLERANCE = 0.05;   // 5%

    function new(string name, uvm_component parent);
        super.new(name, parent);
    endfunction

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        imp_dut = new("imp_dut", this);
        imp_ref = new("imp_ref", this);
    endfunction
```

```
    function void write_dut(clk_txn t);
        dut_q.push_back(t);
        try_check();
    endfunction

    function void write_ref(clk_txn t);
        ref_q.push_back(t);
        try_check();
    endfunction

    function void try_check();
        while (dut_q.size() > 0 && ref_q.size() > 0) begin
            clk_txn dut = dut_q.pop_front();
            clk_txn ref_t = ref_q.pop_front();

            real diff_pct = ((dut.period_ps - ref_t.period_ps) / ref_t.period_ps);
            if (diff_pct < 0) diff_pct = -diff_pct;   // abs value

            if (diff_pct > TOLERANCE)
                `uvm_error("CLK_SCB", $sformatf(
                    "Clock period mismatch: DUT=%0.1fps REF=%0.1fps diff=%0.1f%% (limit 5%%)",
                    dut.period_ps, ref_t.period_ps, diff_pct * 100.0))
            else
                `uvm_info("CLK_SCB", $sformatf(
                    "Clock OK: DUT=%0.1fps REF=%0.1fps diff=%0.1f%%",
                    dut.period_ps, ref_t.period_ps, diff_pct * 100.0), UVM_HIGH)
        end
    endfunction
endclass
```

解释

Monitor 用两个 posedge 之间的 $realtime 差值测量周期，每周期产生一个 clk_txn。

Scoreboard 用两个独立的队列缓存 DUT 和 REF 的测量值，

每次收到新数据时调用 try_check()，只要两队列都有数据就弹出一对进行比对。

容差检查：|(dut - ref) / ref| <= 5%，用百分比误差而非绝对误差，

这样对不同频率的时钟都适用。

Q109. UVM: What Happens When a Sequence Issues a UVM RGM Register Write

QUESTION

Inside a UVM sequence body(), a UVM Register Model (RGM/RAL) write is called:

```
regmodel.CTRL.write(status, 8'hAA, UVM_FRONTDOOR);
```

Describe the complete chain of events that occurs.

ANSWER

```
// Complete chain for regmodel.CTRL.write(status, data, UVM_FRONTDOOR):
//
// 1. uvm_reg::write() called on the register object
//    - Checks access type (WO, RW, etc.)
//    - Calls predict() to update the mirror/desired value in the model
//    - Constructs a uvm_reg_item (register-level transaction)
//
// 2. Register adapter translates uvm_reg_item -> bus transaction
```

```
//    - reg2bus() method of the adapter is called
//    - Converts register address + write data into the target bus format
//      e.g., APB: maps to PADDR, PWDATA, PWRITE=1, PSEL, PENABLE
//            AXI: maps to AWADDR, WDATA, AWVALID, WVALID, etc.
//
// 3. Bus sequence item placed on the sequencer
//    - The register sequencer (uvm_reg_map's sequencer) receives the item
//    - It is forwarded to the bus agent's sequencer via a layered sequence
//
// 4. Driver drives the transaction onto the DUT interface
//    - Driver's run_phase pulls the item from the sequencer
//    - Drives the physical signals (PADDR, PWDATA, etc.) on the interface
//    - Waits for DUT to acknowledge (PREADY, BVALID, etc.)
//
// 5. Response collected and translated back
//    - bus2reg() of the adapter converts the bus response back to uvm_reg_item
//    - Status (UVM_IS_OK / UVM_NOT_OK) is returned to the original write() call
//
// 6. Register model updated
//    - Mirror value updated to match what was written (for frontdoor write)
//    - status variable in the sequence is set (UVM_IS_OK on success)
//
// Summary of objects involved:
//   uvm_reg (CTRL)
//     -> uvm_reg_map (address map)
//       -> uvm_reg_adapter (reg<->bus translation)
//         -> bus sequencer
//           -> bus driver
//             -> DUT interface
//               -> DUT register (physical hardware)
```

解释

RGM/RAL 写操作的核心是两次翻译：

1. register item -> bus transaction（reg2bus，由 adapter 完成）

2. bus response -> register status（bus2reg，由 adapter 完成）

Adapter 是 RAL 和具体总线协议之间的桥梁，是整个链路最容易出错的地方。

FRONTDOOR vs BACKDOOR：

FRONTDOOR 走上述完整链路（经过驱动和 DUT），验证总线协议本身的正确性。

BACKDOOR 直接通过 HDL 路径读写 RTL 信号，绕过总线，用于初始化或调试。

Q110. UVM: Test Class Not Found -- Missing Factory Registration

QUESTION

The following UVM test runs but the simulator reports it cannot find the test.

What is wrong and how do you fix it?

```
class my_test extends uvm_test;
    // NO `uvm_component_utils macro
    function new(string name, uvm_component parent);
        super.new(name, parent);
    endfunction
    task run_phase(uvm_phase phase);
        phase.raise_objection(this);
        $display("Running my_test");
```

```
        phase.drop_objection(this);
    endtask
endclass
```

ANSWER

```
// Problem: `uvm_component_utils(my_test) is missing.
//
// run_test("my_test") works by asking the UVM factory to create an object
// whose TYPE NAME string matches "my_test".
// The factory only knows about types that have been REGISTERED with it.
// Registration happens automatically when `uvm_component_utils is used.
//
// Without `uvm_component_utils(my_test):
//   - The factory has no entry for the string "my_test"
//   - run_test("my_test") cannot find the class -> fatal error:
//     "UVM_FATAL: Cannot find test my_test"
//
// Fix: add the registration macro

class my_test extends uvm_test;
    `uvm_component_utils(my_test)   // <-- registers with factory

    function new(string name, uvm_component parent);
        super.new(name, parent);
    endfunction

    task run_phase(uvm_phase phase);
        phase.raise_objection(this);
        $display("Running my_test");
        phase.drop_objection(this);
    endtask
endclass

// What `uvm_component_utils(my_test) does:
//   1. Registers "my_test" string -> my_test class mapping in the factory
//   2. Defines get_type_name() returning "my_test"
//   3. Enables factory override: my_test::type_id::set_type_override(...)
//   4. Enables create: my_test::type_id::create("name", parent)
//
// For uvm_object (not component): use `uvm_object_utils instead.
//
// Common symptoms of missing factory registration:
//   "UVM_FATAL: Cannot find test <name>"
//   type_id::create() returns null
//   Factory override has no effect
```

解释

`uvm_component_utils 是 UVM 组件的注册宏，是最容易在代码中漏写的一行。

run_test() 通过工厂的字符串查找机制创建 test 对象，没有注册就等于"工厂不认识这个名字"。

面试中这道题考察的是对 UVM factory 机制的理解：factory 维护一个 string->type 的映射表，

所有 component/object 都必须通过 utils 宏注册才能被 factory 找到和创建。

扩展：即使注册了，如果 +UVM_TESTNAME 的字符串拼写和 class 名不一致也会报同样的错误。

Q111. UVM: Generate Clock in Testbench with Jitter/Variation

QUESTION

(a) How do you generate a clock in a UVM testbench?

(b) How do you add +-N% random variation (jitter) to the clock period?

ANSWER

```
// (a) Simplest: always block in tb_top (not UVM, just SV)
initial clk = 0;
always #5ns clk = ~clk;   // 100 MHz, 50% duty cycle

// (b) UVM clock driver class -- based on real-world clk_pkg pattern.
// Key ideas from the reference implementation:
//   - Separate driver() task drives the waveform with on_time / off_time
//   - stopped flag pauses toggling cleanly (no glitch)
//   - Jitter applied per half-period so both edges vary independently

class clk_driver extends uvm_driver #(clk_item);
    `uvm_component_utils(clk_driver)

    // ---- Configuration (set via uvm_config_db before build_phase) -------
    real base_period_ns = 10.0;   // default 100 MHz
    real jitter_pct     = 0.0;    // +-N%: 0 = no jitter

    // ---- Internal state -------------------------------------------------
    virtual clk_if   vif;
    realtime         on_time;     // current high half-period
    realtime         off_time;    // current low half-period
    bit              stopped = 1; // start stopped; driver() waits until 0

    function new(string name, uvm_component parent);
        super.new(name, parent);
    endfunction

    function void build_phase(uvm_phase phase);
        super.build_phase(phase);
        if (!uvm_config_db #(virtual clk_if)::get(this, "", "clk_vif", vif))
            `uvm_fatal("CLK_DRV", "No virtual interface found")
        void'(uvm_config_db #(real)::get(this, "", "base_period_ns", base_period_ns));
        void'(uvm_config_db #(real)::get(this, "", "jitter_pct",     jitter_pct));
        recalc_times();
    endfunction

    // Compute on/off times from current base_period and jitter setting
    function void recalc_times();
        real jitter_amp = base_period_ns * (jitter_pct / 100.0);
        // Each half-period gets an independent random offset in [-amp/2, +amp/2]
        real jitter = ($urandom_range(0, 1000) / 1000.0 - 0.5) * jitter_amp;
        on_time  = ((base_period_ns / 2.0) + jitter) * 1ns;
        off_time = (base_period_ns - (base_period_ns / 2.0 + jitter)) * 1ns;
        if (on_time  < 1ps) on_time  = 1ps;   // safety floor
        if (off_time < 1ps) off_time = 1ps;
    endfunction

    task run_phase(uvm_phase phase);
        vif.clk = 0;
        stopped = 0;          // release clock immediately on start
        fork
            driver();         // waveform generation loop
            commander();      // process frequency/jitter change commands
        join
    endtask

    // ---- driver: toggle clk using current on/off times ------------------
    task driver();
```

```
        forever begin
            wait (!stopped);
            recalc_times();         // re-sample jitter each cycle
            # off_time; vif.clk <= 1'b1;
            # on_time;  vif.clk <= 1'b0;
        end
    endtask

    // ---- commander: handle sequence items (freq change, pause, etc.) ----
    task commander();
        forever begin
            seq_item_port.get_next_item(req);
            case (req.cmd)
                CLK_PAUSE:  stopped = 1;
                CLK_RESUME: stopped = 0;
                CLK_CHG_FREQ: begin
                    base_period_ns = req.new_period_ns;
                    recalc_times();
                end
                CLK_CHG_JITTER: begin
                    jitter_pct = req.jitter_pct;
                end
            endcase
            seq_item_port.item_done();
        end
    endtask

endclass

// ---- Usage in test ---------------------------------------------------
// Change to 200MHz with 3% jitter at runtime:
//   clk_item item = clk_item::type_id::create("item");
//   item.cmd           = CLK_CHG_FREQ;
//   item.new_period_ns = 5.0;    // 200 MHz
//   item.jitter_pct    = 3.0;    // +-3%
//   clk_seq.start(env.clk_agent.sequencer);
```

解释

设计参考 clk_pkg 的核心模式，简化为面试可讲清的结构：

1. driver() 和 commander() 分离：driver 只管波形，commander 处理命令，职责清晰。

2. stopped flag：暂停时 driver() 的 wait(!stopped) 自然阻塞，不会产生毛刺。

3. recalc_times() 每个周期重新采样 jitter：实现真正的 cycle-to-cycle jitter。

jitter 幅度 = base_period * jitter_pct%，每次在 [-amp/2, +amp/2] 均匀分布。

4. on_time + off_time 而非单个 half_period：支持非50%占空比，也和模板保持一致。

5. void' 包装 config_db get：频率和 jitter 是可选配置，get 失败用默认值，不 fatal。

Q112. UVM: Register Model Frontdoor vs Backdoor Access

QUESTION

In the UVM Register Abstraction Layer (RAL/RGM), what is the difference

between frontdoor access and backdoor access?

When do you use each? What are the pros and cons?

ANSWER

```
// ---- Frontdoor Access (UVM_FRONTDOOR) ----------------------------------
//
// Goes through the actual bus protocol: sequence -> adapter -> driver -> DUT.
// Models exactly what real hardware would see on the bus.
//
// Call:
regmodel.CTRL.write(status, 8'hAA, UVM_FRONTDOOR);
regmodel.STATUS.read(status, rdata, UVM_FRONTDOOR);
//
// What happens internally (see Q83 for full chain):
//   write() -> reg2bus() adapter -> sequencer -> driver -> bus signals -> DUT
//   DUT responds -> bus2reg() adapter -> status returned to caller
//
// Pros:
//   - Verifies the bus protocol itself (address decode, byte enables, timing)
//   - Catches bus-level bugs (wrong address mapping, incorrect PREADY behavior)
//   - Closest to real system behavior
//   - Updates both mirror and desired values in the register model
//
// Cons:
//   - SLOW: takes multiple bus cycles (PSEL/PENABLE/PREADY handshake etc.)
//   - Requires a correctly working bus agent and adapter
//   - Cannot access registers before the bus is initialized
//   - Simulation time consumed per access

// ---- Backdoor Access (UVM_BACKDOOR) ------------------------------------
//
// Reads/writes RTL register bits directly via HDL path -- bypasses the bus.
// Uses $hdl_deposit (force), $hdl_read, or UVM DPI backdoor mechanisms.
//
// Call:
regmodel.CTRL.write(status, 8'hAA, UVM_BACKDOOR);
regmodel.STATUS.read(status, rdata, UVM_BACKDOOR);
//
// Requires: HDL path configured in the register model
regmodel.CTRL.configure(
    .blk_parent(this),
    .hdl_path("tb_top.dut.u_ctrl_reg.q")  // path to the RTL flop
);
//
// What happens:
//   write(): $deposit(hdl_path, value) -- directly forces RTL bits, 0 sim time
//   read():  $read(hdl_path) -- directly reads RTL bits, 0 sim time
//   No bus transactions generated.
//
// Pros:
//   - INSTANT: zero simulation time (no clock cycles consumed)
//   - Works even before bus/reset initialization
//   - Useful for pre-loading register state at test start
//   - Can read write-only registers (bypasses hardware read protection)
//   - Excellent for debug: check actual RTL state without side effects
//
// Cons:
//   - Does NOT verify bus protocol (address decode, timing, etc.)
//   - Bypasses any hardware access control or side effects
//     (e.g., a read-to-clear status register is NOT cleared on backdoor read)
//   - HDL path is fragile: if RTL hierarchy changes, path must be updated
//   - Not available in gate-level simulation (no RTL signal names)

// ---- Comparison table --------------------------------------------------
//
//   Property             | Frontdoor          | Backdoor
//   ---------------------|--------------------|-------------------------
```

```
//   Bus protocol tested  | YES                | NO
//   Simulation time      | Multiple cycles    | Zero (0 time)
//   Works before reset   | NO                 | YES
//   Read-to-clear effect | YES (side effects) | NO (raw read)
//   Write-only readable  | NO                 | YES
//   Gate-level sim       | YES                | NO (no RTL paths)
//   HDL path needed      | NO                 | YES
//   Mirror value updated | YES                | YES

// ---- Typical use patterns ------------------------------------------------

// Pattern 1: initialize DUT state quickly at test start (backdoor)
//   then run functional test through bus (frontdoor)
task body();
    // Quickly pre-load all registers without wasting simulation time
    regmodel.CTRL.write(status, 8'h01, UVM_BACKDOOR);
    regmodel.CFG.write(status,  8'hFF, UVM_BACKDOOR);

    // Now run actual test through the bus
    regmodel.CTRL.write(status, 8'h03, UVM_FRONTDOOR);
    regmodel.STATUS.read(status, rdata, UVM_FRONTDOOR);
endtask

// Pattern 2: scoreboard uses backdoor to verify DUT state
//   without disturbing the DUT (no bus traffic, no side effects)
function void check_reg_state(bit [7:0] expected);
    uvm_status_e status;
    uvm_reg_data_t actual;
    regmodel.STATUS.read(status, actual, UVM_BACKDOOR);
    if (actual !== expected)
        `uvm_error("SCB", $sformatf(
            "Register mismatch: got=0x%h expected=0x%h", actual, expected))
endfunction

// Pattern 3: peek/poke (shorthand for backdoor read/write)
regmodel.CTRL.peek(status, rdata);    // backdoor read (no side effects)
regmodel.CTRL.poke(status, 8'hAA);   // backdoor write
```

解释

前门访问（FRONTDOOR）= 走真实总线，验证协议，有时序，有副作用。

后门访问（BACKDOOR）= 直接操作RTL信号，无总线，零时间，无副作用。

面试中常见追问：

"read-to-clear 寄存器用哪种访问来验证清零行为？" -> 前门（需要触发硬件副作用）。

"如何快速初始化100个寄存器不浪费仿真时间？" -> 后门（零时间批量写入）。

"如何验证 write-only 寄存器的值真的写进去了？" -> 后门读（绕过硬件读保护）。

peek/poke 是 UVM RAL 提供的后门读写的简写形式，与 read/write(UVM_BACKDOOR) 等价。

Q113. UVM: DDR Memory Allocator Transaction with Spatial Constraints

QUESTION

Write a UVM sequence item for a DDR memory allocator with the following spec:

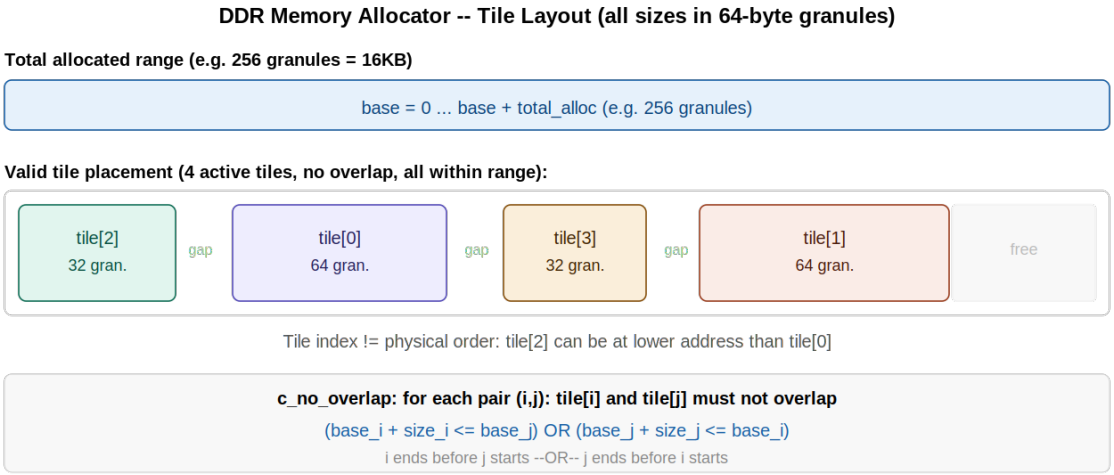
- Allocation granularity: 64 bytes

- Total allocated size: 128KB to 16MB

- Max 8 tiles; actual tile count: 1 to 8

- Only existing tiles are allocated memory
- Each tile size: 128KB to 1MB
- All tiles must fall completely within the allocated range
- No two tiles may overlap
- Tile physical order is not fixed (tile[0] may be at a higher address than tile[1])

ANSWER




```
// All addresses and sizes are expressed in units of 64-byte granules.
// This eliminates the need to multiply/divide by 64 in every constraint.
//
// Granule counts:
//   128KB  = 128*1024 / 64 = 2048  granules
//   1MB    = 1024*1024 / 64 = 16384 granules
//   16MB   = 16*1024*1024 / 64 = 262144 granules

class ddr_alloc_txn extends uvm_sequence_item;
    `uvm_object_utils(ddr_alloc_txn)

    // ---- Granule-unit constants ----------------------------------------
    localparam int GRAN_BYTES   = 64;
    localparam int MIN_ALLOC_G  = (128  * 1024) / GRAN_BYTES;  // 2048
    localparam int MAX_ALLOC_G  = (16   * 1024 * 1024) / GRAN_BYTES; // 262144
    localparam int MIN_TILE_G   = (128  * 1024) / GRAN_BYTES;  // 2048
    localparam int MAX_TILE_G   = (1024 * 1024) / GRAN_BYTES;  // 16384
    localparam int MAX_TILES    = 8;

    // ---- Random variables ----------------------------------------------

    // Total allocated DDR range in granules [128KB, 16MB]
    rand int unsigned total_alloc_g;

    // How many tiles actually exist [1, 8]
    rand int unsigned num_tiles;

    // Per-tile: which tiles exist, base address and size (all in granules)
    rand bit          tile_exists [MAX_TILES];  // 1 = this tile slot is active
    rand int unsigned tile_base_g [MAX_TILES];  // start granule of each tile
    rand int unsigned tile_size_g [MAX_TILES];  // size in granules

    // ---- Constraints ---------------------------------------------------
```

```
    // C1: total allocated range
    constraint c_alloc_range {
        total_alloc_g inside {[MIN_ALLOC_G : MAX_ALLOC_G]};
    }

    // C2: number of active tiles [1, 8]
    constraint c_num_tiles {
        num_tiles inside {[1 : MAX_TILES]};
        // tile_exists must have exactly num_tiles slots set to 1
        num_tiles == tile_exists.sum() with (int'(item));
    }

    // C3: inactive tiles have zero base and size (keep model clean)
    constraint c_inactive_tiles {
        foreach (tile_exists[i]) begin
            if (!tile_exists[i]) begin
                tile_base_g[i] == 0;
                tile_size_g[i] == 0;
            end
        end
    }

    // C4: active tile sizes in [128KB, 1MB] in granules
    constraint c_tile_sizes {
        foreach (tile_exists[i]) begin
            if (tile_exists[i])
                tile_size_g[i] inside {[MIN_TILE_G : MAX_TILE_G]};
        end
    }

    // C5: sum of all active tile sizes must not exceed total allocated range
    //     (necessary but not sufficient -- C6 adds the no-gap/no-overflow check)
    constraint c_total_fits {
        tile_size_g.sum() with (int'(item)) <= int'(total_alloc_g);
    }

    // C6: every active tile must lie completely within [0, total_alloc_g)
    constraint c_within_range {
        foreach (tile_exists[i]) begin
            if (tile_exists[i]) begin
                tile_base_g[i] + tile_size_g[i] <= total_alloc_g;
            end
        end
    }

    // C7: no two active tiles may overlap
    // For tiles i and j (i < j), one must end before the other begins.
    constraint c_no_overlap {
        foreach (tile_exists[i]) begin
            foreach (tile_exists[j]) begin
                if (i < j && tile_exists[i] && tile_exists[j]) begin
                    // Either i ends before j starts, or j ends before i starts
                    (tile_base_g[i] + tile_size_g[i] <= tile_base_g[j]) ||
                    (tile_base_g[j] + tile_size_g[j] <= tile_base_g[i]);
                end
            end
        end
    }

    // ---- Constructor -----------------------------------------------------
    function new(string name = "ddr_alloc_txn");
        super.new(name);
    endfunction
```

```
    // ---- Print helper ------------------------------------------------
    function string convert2string();
        string s;
        s = $sformatf("total_alloc=%0dKB num_tiles=%0d\n",
                      total_alloc_g * GRAN_BYTES / 1024, num_tiles);
        foreach (tile_exists[i]) begin
            if (tile_exists[i])
                s = {s, $sformatf(
                    "  tile[%0d]: base=0x%08h size=%0dKB end=0x%08h\n",
                    i,
                    tile_base_g[i] * GRAN_BYTES,
                    tile_size_g[i] * GRAN_BYTES / 1024,
                    (tile_base_g[i] + tile_size_g[i]) * GRAN_BYTES)};
        end
        return s;
    endfunction

endclass

// ---- Design notes -----------------------------------------------------
//
// Q: Why use granules instead of raw byte addresses?
//    Working in granule units keeps all constraint expressions as integer
//    arithmetic (no * 64 scattered everywhere). The solver handles smaller
//    numbers, which can improve randomization speed.
//
// Q: Why tile_exists[] instead of just using tile_size_g[i] == 0?
//    Explicit tile_exists separates "inactive" from "zero-sized" (which would
//    be an illegal tile anyway). It also makes c_num_tiles unambiguous and
//    the foreach guards in c_no_overlap more readable.
//
// Q: Is c_no_overlap sufficient to prevent overlap?
//    Yes: the disjunction (i ends before j) || (j ends before i) covers
//    all non-overlapping configurations. The solver will pick one side.
//    Note: tiles may be adjacent (i ends exactly where j begins) -- this is
//    allowed since adjacent regions share only a boundary, not any bytes.
//
// Q: Does physical tile order matter?
//    No explicit ordering constraint is imposed. tile[0] may have a higher
//    base address than tile[1] -- the spec says order is arbitrary.
//    If you want tile indices to reflect physical order, add:
//    constraint c_ordered {
//        foreach (tile_exists[i]) if (i > 0 && tile_exists[i-1] && tile_exists[i])
//            tile_base_g[i-1] < tile_base_g[i];
//    }
//    But this is NOT required by the spec.
```

解释

约束设计的五个层次，缺一不可：

C1 总范围 -> C2 tile数量 -> C3 非活跃tile清零 -> C4 tile个体大小合法

-> C5 所有tile总和不超出 -> C6 每个tile不超出边界 -> C7 任意两tile不重叠。

C5 和 C6 看似重复，实则互补：C5 保证总量不超，C6 保证每个tile的终点不超界。

C7 的双向不等式是 no-overlap 的标准写法：i在j左边 OR j在i左边，OR 是关键。

粒度换算（granule unit）是工程技巧：把所有数值除以64，约束表达式更简洁，

solver 搜索空间从 16M=16777216 缩减到 262144，随机化更快。

tile_exists[] 作为独立 bool 数组，配合 .sum() with (int'(item)) 统计活跃数量，

是 SV foreach constraint 的标准惯用法。

Q114. UVM: Factory Registration and Out-of-Order Execution

QUESTION

(a) Explain UVM factory and why `uvm_component_utils / `uvm_object_utils matters.

(b) How is out-of-order transaction execution implemented in UVM verification?

ANSWER

```
// (a) UVM Factory
//
// The UVM factory is a registry mapping TYPE NAME STRINGS to CLASS TYPES.
// It enables RUNTIME TYPE SUBSTITUTION without changing test code.
//
// `uvm_component_utils(my_class) does three things:
//   1. Registers "my_class" -> my_class type in the factory
//   2. Defines get_type_name() returning "my_class"
//   3. Enables: my_class::type_id::create("name", parent)
//               my_class::type_id::set_type_override(other_class::get_type())
//
// Without the macro:
//   run_test("my_test") calls factory.find("my_test") -> null -> FATAL
//   "UVM_FATAL: Cannot find test my_test"
//
// Key benefit: factory override injects a derived class everywhere
// the base class is created, without touching the base env/test code:
//
//   // In test: replace all my_seq with stress_seq everywhere
//   my_seq::type_id::set_type_override(stress_seq::get_type());
//   // All seq.start() calls now create stress_seq objects

// (b) Out-of-order execution in UVM projects
//
// Real hardware (e.g. AXI) can complete transactions out of order.
// The verification environment must handle this correctly.
//
// Implementation approach in a project:
//
// 1. Assign unique ID to each transaction
class my_txn extends uvm_sequence_item;
    `uvm_object_utils(my_txn)
    int unsigned txn_id;   // unique identifier assigned by driver
    // ... other fields
endclass

// 2. Scoreboard uses associative array (not queue) for pending transactions
class ooo_scoreboard extends uvm_scoreboard;
    `uvm_component_utils(ooo_scoreboard)
    // Key = txn_id; value = expected response
    my_txn pending [int unsigned];

    function void write_request(my_txn req);
        pending[req.txn_id] = req;
    endfunction

    function void write_response(my_txn resp);
        if (!pending.exists(resp.txn_id))
            `uvm_error("SCB", $sformatf("Unexpected response id=%0d", resp.txn_id))
        else begin
```

```
            check_response(pending[resp.txn_id], resp);
            pending.delete(resp.txn_id);
        end
    endfunction
endclass

// 3. Sequence uses fork-join_none to launch multiple in-flight transactions
task body();
    my_txn txn;
    static int unsigned next_id = 0;
    repeat (8) begin
        txn = my_txn::type_id::create("txn");
        txn.txn_id = next_id++;
        fork
            automatic my_txn t = txn;
            begin
                start_item(t);
                assert(t.randomize());
                finish_item(t);  // non-blocking: multiple in-flight
            end
        join_none
    end
    wait fork;  // wait for all to complete
endtask
```

解释

UVM factory 的本质是一个 string->type 的映射表，utils 宏完成注册。

没有注册 = factory 不认识这个类 = run_test 报 FATAL。

乱序执行验证的核心：用 txn_id 作为 key 的 associative array 替代先进先出的 queue，

响应到来时通过 ID 找到对应的期望值进行比对，与到达顺序无关。

sequence 层面用 fork-join_none 发起多个并发事务（多个 in-flight），

wait fork 等待所有完成，模拟真实的乱序执行场景。

Q115. UVM: Component vs Object, Field Automation, copy vs clone

QUESTION

(a) What is the difference between uvm_component and uvm_object in UVM?

(b) What is field automation (`uvm_field_*) and what does it give you?

(c) What is the difference between copy() and clone() in UVM?

ANSWER

```
// (a) uvm_component vs uvm_object
//
// uvm_component:
//   - Has a PARENT and a fixed position in the UVM component hierarchy.
//   - Created with type_id::create("name", parent).
//   - Participates in UVM PHASES (build, connect, run, etc.).
//   - Lives for the ENTIRE simulation duration.
//   - Examples: uvm_driver, uvm_monitor, uvm_scoreboard, uvm_agent, uvm_env.
//   - Registered with: `uvm_component_utils(my_class)
//
// uvm_object:
//   - Has NO parent, NO phase participation.
```

```
//   - Created dynamically, can be destroyed (garbage collected).
//   - Used for DATA: transactions, sequences, configuration objects.
//   - Examples: uvm_sequence_item, uvm_sequence, uvm_reg.
//   - Registered with: `uvm_object_utils(my_class)
//
//   Rule of thumb: if it has phases -> component; if it's data -> object.

// (b) Field Automation (`uvm_field_* macros)
//
// Declaring a field with `uvm_field_int, `uvm_field_string, etc.
// AUTOMATICALLY generates implementations of these methods:
//   copy()        - deep copy of all registered fields
//   clone()       - create + copy in one call
//   compare()     - field-by-field equality check
//   print()       - human-readable dump of all fields
//   pack/unpack() - serialize/deserialize to bit stream
//   record()      - transaction recording to waveform
//
// Example:
class my_txn extends uvm_sequence_item;
    `uvm_object_utils_begin(my_txn)
        `uvm_field_int(addr,  UVM_DEFAULT)         // int field
        `uvm_field_int(data,  UVM_DEFAULT)
        `uvm_field_enum(op_e, op, UVM_DEFAULT)     // enum field
        `uvm_field_int(data,  UVM_NOCOMPARE)       // exclude from compare()
        `uvm_field_string(tag, UVM_PRINT | UVM_COPY) // specific flags only
    `uvm_object_utils_end

    rand logic [31:0] addr, data;
    rand op_e         op;
    string            tag;
endclass
//
// UVM_DEFAULT = UVM_COPY | UVM_COMPARE | UVM_PRINT | UVM_PACK
//
// Without field automation: you must manually write copy(), compare(), print().
// With field automation: all of these are generated -- saves hundreds of lines.
//
// Caution: field automation adds overhead (uses reflection-like mechanisms).
// For performance-critical paths, manually implementing copy/compare is faster.

// (c) copy() vs clone()
//
// copy(src):
//   Copies all fields FROM src INTO an EXISTING object.
//   The target object already exists; its fields are overwritten.
//   Does NOT create a new object.
//
//   my_txn dst = new my_txn("dst");
//   dst.copy(src);   // dst fields <- src fields; dst object already existed

// clone():
//   Creates a NEW object AND copies all fields in one step.
//   Equivalent to: create new object, then copy().
//   Returns a uvm_object handle (must $cast to concrete type).
//
//   my_txn cloned;
//   $cast(cloned, src.clone());  // new object created, fields copied from src
//
// When to use each:
//   copy():  you already have a pre-allocated object to reuse (avoids allocation).
//   clone(): you need a fresh independent copy with its own memory (most common).
//
```

```
// Both perform DEEP copy (nested objects are also copied, not just references).
//
// Shallow copy pitfall:
//   If you use plain assignment (dst = src), both point to the SAME object.
//   Modifying dst.addr also modifies src.addr -- NOT what you want.
//   Always use clone() or copy() for independent copies of UVM objects.
```

解释

component vs object 的核心区分：是否参与 phase 机制，是否有在层次中的固定位置。

Field automation 是 UVM 的"元编程"功能：声明一次字段，自动获得打印/比较/拷贝等能力。

实际项目中选择性使用：关键事务路径手写 copy/compare 以避免 automation 的反射开销。

copy vs clone：copy 是"往现有对象里写"，clone 是"新建一个对象并写进去"。

最常见的错误：用 = 赋值代替 clone，导致两个句柄指向同一对象（浅拷贝陷阱）。

面试中能说出"component 有 parent 和 phase，object 只是数据"是最关键的区分点。

Algorithms & Scripting

Q116. Algorithms: Pairing Letters from Both Ends

QUESTION

Given a string s, rearrange its letters by pairing the first with the last,

second with second-to-last, and so on. If s has odd length, the middle letter

goes at the end unpaired.

Example: "abcdef" -> "afbecd", "abcde" -> "aebdc"

ANSWER

```
def solution(s: str) -> str:
    """
    Rearrange string by pairing first with last, second with second-to-last, etc.
    If odd length, the middle character is appended unpaired at the end.

    Args:
        s: input string of lowercase English letters

    Returns:
        rearranged string following the pairing rule

    Time:  O(n)
    Space: O(n)
    """
    n: int = len(s)
    result: list[str] = []

    for i in range(n // 2):
        result.append(s[i])           # left character
        result.append(s[n - 1 - i])  # paired right character

    if n % 2 == 1:
        result.append(s[n // 2])     # middle character (unpaired)
```

```
    return "".join(result)

# -- Examples ------------------------------------------------------------
# solution("abcdef") -> "afbecd"
#   pairs: (a,f), (b,e), (c,d)
# solution("abcde")  -> "aebdc"
#   pairs: (a,e), (b,d), middle: c
```

解释

双指针从两端向中间配对：左指针 i 从 0 到 n//2-1，右指针取 s[n-1-i]，

每次将 s[i] 和 s[n-1-i] 依次追加到结果中。

奇数长度时，中间字符 s[n//2] 单独追加到末尾。

时间复杂度 O(n)，空间 O(n)（结果数组）。

Q117. Algorithms: Minimize Result by Adding Parentheses to Expression

QUESTION

Given expression = "digits+digits", insert exactly one pair of parentheses

(left before '+', right after '+') to minimize the resulting value.

Parts outside parentheses multiply: A*(B+C)*D. Missing A or D default to 1.

Example: "247+38" -> "2(47+38)" = 2*(47+38) = 170

ANSWER

```
def solution(expression: str) -> str:
    """
    Insert exactly one pair of parentheses into "X+Y" to minimize A*(B+C)*D.
    Left paren goes somewhere before '+', right paren somewhere after '+'.
    Missing A or D default to 1.

    Args:
        expression: string of digits and one '+', e.g. "247+38"

    Returns:
        expression with one pair of parentheses inserted for minimum value

    Time:  O(L * R) where L, R are lengths of left/right parts
    Space: O(1)
    """
    plus_idx: int = expression.index("+")
    left:     str = expression[:plus_idx]    # digits before '+'
    right:    str = expression[plus_idx + 1:]  # digits after '+'

    best_val:  float = float("inf")
    best_expr: str   = ""

    for i in range(len(left)):                  # position of '(' in left part
        for j in range(1, len(right) + 1):      # position of ')' in right part
            # Parts outside parentheses multiply; missing part = 1
            a: int = int(left[:i])   if left[:i]   else 1
            b: int = int(left[i:])   if left[i:]   else 0
            c: int = int(right[:j])  if right[:j]  else 0
            d: int = int(right[j:])  if right[j:]  else 1
```

```
            val: int = a * (b + c) * d
            if val < best_val:
                best_val  = val
                best_expr = (
                    left[:i] + "(" + left[i:]
                    + "+" + right[:j] + ")" + right[j:]
                )

    return best_expr

# -- Examples ------------------------------------------------------------
# solution("247+38")  -> "2(47+38)"   value = 2*(47+38)*1 = 170
# solution("12+34")   -> "1(2+3)4"    value = 1*(2+3)*4  = 20
# solution("999+999") -> "(999+999)"  value = 1*(999+999)*1 = 1998
```

解释

枚举所有合法的括号插入位置：

左括号在 '+' 左侧的所有位置（0 到 len(left)-1），

右括号在 '+' 右侧的所有位置（1 到 len(right)）。

对每个 (i, j) 组合，计算 A*(B+C)*D 的值。

空前缀 A 默认为1，空后缀 D 默认为1。

保存最小值对应的表达式，时间复杂度 O(L×R)，通常是 $O(n^2)$。

关键观察：把大数放进括号内做加法，然后用小的外部数去乘，才能最小化结果。

Q118. Algorithms: Robot Matrix Diagonal Traversal

QUESTION

A robot starts at (cellX, cellY) in a 2D matrix, moves diagonally (+1,+1),
bounces off edges by reversing the offending direction component, and stops
when it revisits a cell OR reaches a corner. Return the sum of all visited
cell values (endpoint counted once even if stop is due to revisit).

ANSWER

```
def solution(matrix: list[list[int]], cellX: int, cellY: int) -> int:
    """
    Robot starts at (cellX, cellY), moves diagonally (+1,+1), bounces off
    edges, and stops when revisiting a cell or reaching a corner.
    Returns the sum of all visited cell values (no double-counting).

    Args:
        matrix: 2D grid of integers
        cellX:  starting row (guaranteed not a corner)
        cellY:  starting column (guaranteed not a corner)

    Returns:
        sum of cell values along the robot's path

    Time:  O(m * n) -- each cell visited at most once
    Space: O(m * n) -- visited set
    """
    rows: int = len(matrix)
    cols: int = len(matrix[0])
```

```
    visited: set[tuple[int, int]] = set()
    x,  y  = cellX, cellY
    dx, dy = 1, 1          # initial direction: down-right
    total: int = 0

    while True:
        # Stop condition 1: revisited cell (don't count again)
        if (x, y) in visited:
            break

        # Stop condition 2: reached a corner (count once, then stop)
        is_corner: bool = (x == 0 or x == rows - 1) and (y == 0 or y == cols - 1)
        if is_corner:
            total += matrix[x][y]
            break

        visited.add((x, y))
        total += matrix[x][y]

        # Compute next position with bouncing
        nx, ny = x + dx, y + dy

        if nx < 0 or nx >= rows:   # x out of bounds -> reverse x direction
            dx = -dx
            nx = x + dx
        if ny < 0 or ny >= cols:   # y out of bounds -> reverse y direction
            dy = -dy
            ny = y + dy

        x, y = nx, ny

    return total

# -- Examples ------------------------------------------------------------
# matrix 3x3, start (0,1):  path (0,1)->(1,2)->(2,1)->(1,0)->(0,1)visited
#   sum = 33 + 15 + (-4) + 12 = 56
# matrix 4x4, start (2,0):  path ends at (2,0) revisited
#   sum = 2 + 4 + (-3) + 5 + 8 + 4 = 20
```

解释

模拟机器人运动，用 set 记录访问过的格子。

停止条件有两个：重访（不重复计值）和到达角落（计入后停止）。

弹射逻辑：先计算下一步 (nx, ny)，若 x 方向越界则反转 dx，

若 y 方向越界则反转 dy，各自独立判断（可能同时反转，如角落附近）。

时间复杂度 O(m×n)：每个格子最多访问一次。

注意：起点保证不是角落，但路径可能经过角落（此时停止并计入角落值）。

Q119. Algorithms: Rectangle Storage Queries

QUESTION

Process a list of operations: [0,a,b] saves a rectangle a×b; [1,a,b] queries whether every previously saved rectangle can fit inside an a×b box (rotation by 90 degrees allowed, checked independently per rectangle). Return a list of booleans for each query.

ANSWER

```
# -- Solution 1: Naive O(n) per query -----------------------------------

def solution_naive(operations: list[list[int]]) -> list[bool]:
    """
    Naive approach: store all rectangles, scan all on each query.
    Normalize each rectangle so smaller dimension comes first (handles rotation).

    Time:  O(n) per query, O(n*q) total
    Space: O(n) for stored rectangles
    """
    saved:   list[tuple[int, int]] = []
    results: list[bool]            = []

    for op in operations:
        if op[0] == 0:
            a, b = op[1], op[2]
            # Normalize: (min_dim, max_dim) handles 90-degree rotation
            saved.append((min(a, b), max(a, b)))
        else:
            a, b = op[1], op[2]
            box_min, box_max = min(a, b), max(a, b)
            # Rect fits iff both dimensions fit within box dimensions
            all_fit: bool = all(
                rmin <= box_min and rmax <= box_max
                for rmin, rmax in saved
            )
            results.append(all_fit)

    return results

# -- Solution 2: O(1) per query -- running max (optimal for prefix queries) --

def solution_opt(operations: list[list[int]]) -> list[bool]:
    """
    Key insight: "all rects fit" iff max(all rmin) <= box_min AND
    max(all rmax) <= box_max. Track two running maximums.

    Time:  O(1) per operation, O(n + q) total
    Space: O(1)
    """
    max_min: int        = 0   # max of all saved rect min-dims so far
    max_max: int        = 0   # max of all saved rect max-dims so far
    results: list[bool] = []

    for op in operations:
        if op[0] == 0:
            a, b = op[1], op[2]
            rmin, rmax = min(a, b), max(a, b)
            max_min = max(max_min, rmin)   # O(1) update
            max_max = max(max_max, rmax)
        else:
            a, b = op[1], op[2]
            box_min, box_max = min(a, b), max(a, b)
            results.append(max_min <= box_min and max_max <= box_max)

    return results

# -- Solution 3: Segment tree -- for range queries [l, r] --------------------

class SegTree:
    """
```

```
    Segment tree storing (max_rmin, max_rmax) per range.
    Enables O(log n) range queries: "do rects saved at indices l..r fit in box?"
    Not needed for this problem (prefix queries only), but useful for extensions.
    """

    def __init__(self, n: int) -> None:
        self.n       = n
        self.max_min = [0] * (4 * n)   # max of rmin over each range
        self.max_max = [0] * (4 * n)   # max of rmax over each range

    def update(
        self, node: int, start: int, end: int,
        idx: int, rmin: int, rmax: int
    ) -> None:
        """Point update at index idx with rect (rmin, rmax)."""
        if start == end:
            self.max_min[node] = max(self.max_min[node], rmin)
            self.max_max[node] = max(self.max_max[node], rmax)
            return
        mid = (start + end) // 2
        if idx <= mid:
            self.update(2 * node,     start, mid,   idx, rmin, rmax)
        else:
            self.update(2 * node + 1, mid + 1, end, idx, rmin, rmax)
        self.max_min[node] = max(self.max_min[2 * node], self.max_min[2 * node + 1])
        self.max_max[node] = max(self.max_max[2 * node], self.max_max[2 * node + 1])

    def query(
        self, node: int, start: int, end: int, l: int, r: int
    ) -> tuple[int, int]:
        """Query max_rmin and max_rmax over range [l, r]."""
        if r < start or end < l:
            return 0, 0
        if l <= start and end <= r:
            return self.max_min[node], self.max_max[node]
        mid = (start + end) // 2
        l_min, l_max = self.query(2 * node,     start, mid,   l, r)
        r_min, r_max = self.query(2 * node + 1, mid + 1, end, l, r)
        return max(l_min, r_min), max(l_max, r_max)

# -- Complexity summary ------------------------------------------------------
# Naive:        O(n) per query  -- straightforward but slow for large n
# Running max:  O(1) per op     -- BEST for prefix queries (this problem)
# Segment tree: O(log n) per op -- use ONLY when range queries [l,r] are needed
```

解释

归一化是解题关键：对矩形和箱子都取 (min_dim, max_dim)，旋转问题变成简单的二维比较。

O(1) 优化的数学本质："所有矩形都满足条件" = "最难满足的矩形满足条件"，

而最难满足的矩形正是 rmin 最大、rmax 最大的那个，用两个运行最大值捕获即可。

线段树的适用场景：这道原题用运行最大值更优（O(1) < O(log n)）。

线段树真正发挥价值是在范围查询变体："第 l 到 r 个保存的矩形能否都放入箱子？"

此时需要对任意区间查询 max_rmin 和 max_rmax，线段树是最合适的数据结构。

面试技巧：主动说"对这道原题运行最大值更优，但如果改成范围查询，线段树是正确选择"，

体现了对数据结构适用场景的深入理解，是加分项。

Q120. Algorithms: Online Interval Merge with Streaming Input

QUESTION

Design a data structure that receives intervals [lower, upper] one at a time (streaming input). After each insertion, all overlapping intervals must be merged. Follow-up: where can the implementation be further optimized?

ANSWER

```
# -- Solution 1: Naive -- sort on every insertion, O(n log n) per call --------

def add_naive(intervals: list[list[int]], lo: int, hi: int) -> list[list[int]]:
    """
    Insert new interval [lo, hi] and merge all overlapping intervals.
    Naive approach: append then re-sort every time.

    Time:  O(n log n) per call
    Space: O(n)
    """
    intervals.append([lo, hi])
    intervals.sort(key=lambda x: x[0])

    merged: list[list[int]] = [intervals[0]]
    for start, end in intervals[1:]:
        if start <= merged[-1][1]:        # overlapping
            merged[-1][1] = max(merged[-1][1], end)
        else:
            merged.append([start, end])

    return merged

# -- Solution 2: Optimised -- binary search, O(log n + k) per call ----------

class IntervalStore:
    """
    Maintains a sorted, merged list of intervals.
    Uses binary search to locate the overlap window for each insertion.

    Time per add(): O(log n + k)
      - log n : two binary searches to find overlap boundaries
      - k     : merging k overlapping intervals (each interval merged once)
    Amortized: O(log n) per add
    Space: O(n)
    """

    def __init__(self) -> None:
        self.intervals: list[list[int]] = []   # always sorted and merged

    def add(self, lo: int, hi: int) -> list[list[int]]:
        """
        Insert interval [lo, hi], merging any overlapping intervals.

        Args:
            lo: lower bound of the new interval (inclusive)
            hi: upper bound of the new interval (inclusive)

        Returns:
            current merged interval list after insertion
        """
        ivs = self.intervals

        # Binary search: find first interval whose end >= lo (may overlap)
        left, right_bound = 0, len(ivs)
```

```
        while left < right_bound:
            mid = (left + right_bound) // 2
            if ivs[mid][1] < lo:
                left = mid + 1
            else:
                right_bound = mid
        left_idx = left   # first potentially overlapping interval

        # Binary search: find first interval whose start > hi (no overlap)
        left, right_bound = left_idx, len(ivs)
        while left < right_bound:
            mid = (left + right_bound) // 2
            if ivs[mid][0] <= hi:
                left = mid + 1
            else:
                right_bound = mid
        right_idx = left  # first interval completely to the right

        if left_idx < right_idx:
            # Merge all intervals in [left_idx, right_idx) with [lo, hi]
            new_lo = min(lo, ivs[left_idx][0])
            new_hi = max(hi, ivs[right_idx - 1][1])
            ivs[left_idx:right_idx] = [[new_lo, new_hi]]   # O(k) slice replace
        else:
            ivs.insert(left_idx, [lo, hi])   # no overlap: insert in sorted order

        return ivs

# -- Usage example ---------------------------------------------------------
# store = IntervalStore()
# store.add(1, 3)    # -> [[1, 3]]
# store.add(6, 9)    # -> [[1, 3], [6, 9]]
# store.add(2, 5)    # -> [[1, 5], [6, 9]]   overlaps [1, 3]
# store.add(5, 6)    # -> [[1, 9]]             bridges both
# store.add(20, 25)  # -> [[1, 9], [20, 25]]

# -- Follow-up optimisations -----------------------------------------------
# 1. list.insert() is O(n); use SortedList (sortedcontainers) for O(log n) insert
# 2. Adjacent byte ranges: change overlap condition to ivs[i][1] >= lo - 1
# 3. Lazy merging: batch inserts, merge periodically (write-heavy workloads)
# 4. Point/range queries: use an interval tree (intervaltree package)
```

解释

核心思路：维护已排序且已合并的区间列表，用两次二分查找定位重叠范围。

第一次 bisect 找"第一个 end >= lo 的区间"（左边界），

第二次 bisect 找"第一个 start > hi 的区间"（右边界），

中间所有 k 个区间一次合并，摊销 O(log n) per insertion。

Follow-up 三层优化：数据结构（SortedList）-> 业务语义（adjacent merge）->

访问模式（lazy merging / interval tree）。

Q121. Algorithms: Event ID Check Queue

QUESTION

An exclusive event starts at time 0. You are given an array of integers arrivals,

where arrivals[i] is the time in seconds (since event start) when person i arrives.

Entry requires an ID check that takes exactly 5 minutes (300 seconds) per person.

If a person arrives and sees more than 10 people currently in the queue (i.e., people who have arrived but not yet finished their ID check), they leave immediately.

Return an array of integers where result[i] is the time (in seconds) when person i finishes their ID check, or their arrival time if they left immediately.

ANSWER

```
def event_id_check(arrivals: list[int]) -> list[int]:
    """
    Simulate an ID-check queue at an exclusive event.
    Each check takes 300 seconds (5 minutes). A person leaves immediately
    if more than 10 people are currently in the queue at their arrival time.

    Args:
        arrivals: list of arrival times in seconds since event start

    Returns:
        list where result[i] = finish time if person i joined the queue,
        or arrival time if they left immediately (queue was full)

    Time:  O(n^2) worst case (queue scan per person); O(n log n) with heap
    Space: O(n)
    """
    CHECK_TIME: int = 300   # 5 minutes in seconds
    queue:  list[int] = []  # finish times of people currently in queue
    result: list[int] = []

    for arrival in arrivals:
        # Remove people whose ID check completed before this person arrived
        queue = [ft for ft in queue if ft > arrival]

        if len(queue) > 10:
            # More than 10 in queue -> person leaves immediately
            result.append(arrival)
        else:
            # Join the queue: start after the last person finishes
            start:  int = max(queue) if queue else arrival
            finish: int = start + CHECK_TIME
            queue.append(finish)
            result.append(finish)

    return result

# -- Example -------------------------------------------------------------
# 12 people all arriving at t=0:
#   Persons 0-10 join (queue grows to 11, still <= 10 at each join point)
#   Person 11 arrives: queue has 11 entries > 10 -> result[11] = 0 (leaves)
```

解释

维护一个 queue 列表，存储当前尚未处理完的人的结束时间。

每次新人到达，先过滤掉 finish_time <= arrival 的人（已处理完）。

若剩余人数 > 10，新人立刻离开，记录到达时间；否则排到队尾，

开始时间为队列中最晚的结束时间（或当前时刻），结束时间加 300 秒。

时间复杂度 O(n^2)，可用最小堆优化至 O(n log n)。

Q122. Algorithms: Count Digits 0, 2, 4 in Range [0, n]

QUESTION

You are given an integer n. Calculate how many times the digits 0, 2, and 4 appear in all integers from 0 to n, inclusive.

ANSWER

```
def count_024(n: int) -> int:
    """
    Count total occurrences of digits 0, 2, and 4 in all integers [0, n].

    Args:
        n: upper bound (inclusive), n >= 0

    Returns:
        total count of digit occurrences of 0, 2, and 4 across [0, n]

    Time:  O(n * d) where d = number of digits in n  (brute force)
    Space: O(1)
    """
    TARGET_DIGITS: frozenset[str] = frozenset({"0", "2", "4"})
    count: int = 0

    for i in range(n + 1):
        for ch in str(i):
            if ch in TARGET_DIGITS:
                count += 1

    return count

# -- Optimised digit-DP for large n (O(log n)) ----------------------------

def count_024_dp(n: int) -> int:
    """
    Count occurrences of digits 0, 2, 4 in [0, n] using digit DP.
    Counts each target digit d independently, then sums.

    Time:  O(log n) -- one pass per digit position per target digit
    Space: O(1)
    """

    def count_digit(n: int, d: int) -> int:
        """Count occurrences of digit d in [0, n]."""
        total:  int = 0
        factor: int = 1

        while factor <= n:
            lower:  int = n % factor
            cur:    int = (n // factor) % 10
            higher: int = n // (factor * 10)

            if d == 0:
                # Avoid leading zeros: subtract one full group
                total += (higher - 1) * factor + (lower + 1 if cur > d else 0)
            elif cur < d:
                total += higher * factor
            elif cur == d:
                total += higher * factor + lower + 1
            else:
                total += (higher + 1) * factor

            factor *= 10
```

```
        return total

    return count_digit(n, 0) + count_digit(n, 2) + count_digit(n, 4)

# -- Example ------------------------------------------------------------
# count_024(10) -> 4  (0:twice in {0,10}, 2:once, 4:once)
# count_024(20) -> 8
```

解释

暴力解枚举 [0, n] 每个数，逐位检查是否为 0/2/4，时间复杂度 O(n * digits)。

面试中若 n <= 10^6 暴力足够。n 更大时用数位 DP：

对每个目标数字 d，计算其在 [0, n] 的每个十进制位上出现的次数，

分高位/当前位/低位三种情况讨论，注意 d=0 时高位不能为 0（避免前导零）。

Q123. Algorithms: Longest Contiguous Same-Character Substring

QUESTION

You are given a string source consisting of lowercase English letters.

Find the longest contiguous substring consisting of the same character.

If there are multiple substrings of the same maximum length, return the

rightmost one. Return a string consisting of the character concatenated

with its count (e.g., "b3" for "bbb").

ANSWER

```
def longest_same_char(s: str) -> str:
    """
    Find the longest contiguous substring of a single repeated character.
    If multiple substrings tie on length, return the RIGHTMOST one.
    Return the character concatenated with its run length, e.g. "b3".

    Args:
        s: non-empty string of lowercase English letters

    Returns:
        string of form "<char><count>", e.g. "b3" for "bbb"

    Time:  O(n)
    Space: O(1)
    """
    if not s:
        return ""

    best_char: str = s[0]
    best_len:  int = 1
    cur_char:  str = s[0]
    cur_len:   int = 1

    for i in range(1, len(s)):
        if s[i] == s[i - 1]:
            cur_len += 1
        else:
            cur_char = s[i]
            cur_len  = 1
```

```
        # >= keeps the RIGHTMOST run when lengths tie
        if cur_len >= best_len:
            best_len  = cur_len
            best_char = cur_char

    return best_char + str(best_len)

# -- Examples ------------------------------------------------------------
# longest_same_char("aabbbccbb") -> "b3"  (bbb is longest)
# longest_same_char("aabbcc")    -> "c2"  (all length-2, rightmost is cc)
# longest_same_char("aaabbaaa")  -> "a3"  (two runs of aaa, take rightmost)
# longest_same_char("abcd")      -> "d1"  (all length-1, rightmost is d)
```

解释

单趟线性扫描，维护当前连续段长度 cur_len 和全局最优 best_len。

关键：使用 >= 而非 > 来更新最优，保证等长时取最右边的子串。

每当字符切换，重置 cur_char 和 cur_len = 1（不要忘记更新 cur_char）。

时间复杂度 O(n)，空间 O(1)。

Q124. Algorithms: Place Lamp to Illuminate Maximum Objects

QUESTION

There are objects placed on a number line, given as a sorted integer array objects with no duplicates. Place a lamp at an integer coordinate c such that it illuminates the maximum number of objects within the range [c - radius, c + radius] (inclusive). The lamp can be placed on any integer coordinate, even one occupied by an object. Return the coordinate c that maximizes illuminated objects. If there are multiple such coordinates, return the smallest one.

ANSWER

```
import bisect

def place_lamp(objects: list[int], radius: int) -> int:
    """
    Place a lamp at an integer coordinate to illuminate the maximum number
    of objects within [c - radius, c + radius]. Return the smallest such
    coordinate if multiple positions achieve the maximum count.

    Key insight: the optimal lamp position always has some object on its
    LEFT boundary (c - radius = objects[i] for some i). Proof: if no
    object sits on the left boundary, shift the lamp left until one does
    without losing any illuminated objects.

    Args:
        objects: sorted list of distinct integer object coordinates
        radius:  illumination radius (inclusive on both ends)

    Returns:
        optimal lamp coordinate (smallest among all maximum-count positions)

    Time:  O(n log n) -- one binary search per object
    Space: O(1)
    """
```

```
    best_coord: int = objects[0] + radius
    best_count: int = 0

    for i in range(len(objects)):
        # Place lamp so its LEFT edge aligns with objects[i]
        lamp_c: int = objects[i] + radius
        lo:     int = lamp_c - radius   # == objects[i]
        hi:     int = lamp_c + radius

        # Count objects in [lo, hi] with binary search
        left:  int = bisect.bisect_left(objects,  lo)
        right: int = bisect.bisect_right(objects, hi)
        count: int = right - left

        # Update best: prefer larger count; break ties by smaller coordinate
        if count > best_count or (count == best_count and lamp_c < best_coord):
            best_count = count
            best_coord = lamp_c

    return best_coord

# -- Example -------------------------------------------------------------
# objects=[1,2,3,10,11], radius=2
#   lamp at 3: covers [1,5] -> illuminates 1,2,3 (count=3) <- winner
#   lamp at 4: covers [2,6] -> illuminates 2,3   (count=2)
#   lamp at 12: covers [10,14] -> illuminates 10,11 (count=2)
# return 3
```

解释

核心结论：最优灯的位置一定满足"某个物体恰好落在灯的左边界"。

反证：若最优位置 c 使得左边界 c-r 左侧没有物体，则灯可以左移，

覆盖物体数不减少，直到某物体碰到左边界为止。

因此枚举每个物体作为左边界，令 lamp_c = objects[i] + radius，

用 bisect 二分统计 [objects[i], objects[i]+2*radius] 内的物体数。

时间复杂度 O(n log n)，空间 O(1)。

Q125. Algorithms: Cyclic Shift Validation

QUESTION

Given an array of distinct integers, determine whether it is a cyclic rotation

of a sorted (ascending) sequence. If yes, return the number of positions it was

right-rotated; if no valid rotation exists, return -1.

A cyclic rotation of [1,2,3,4,5] by 2 produces [4,5,1,2,3].

Example: [3,4,5,1,2] -> shift=2, because right-rotating [1,2,3,4,5] by 2 gives [4,5,1,2,3]...

wait, left-rotating by 2 gives [3,4,5,1,2]. Return 2.

ANSWER

```
def cyclic_shift(arr: list[int]) -> int:
    """
    Determine if arr is a cyclic rotation of a sorted (ascending) sequence.
    If yes, return the number of positions it was right-rotated.
```

```
    If no valid rotation exists, return -1.

    A valid cyclic rotation has at most ONE "breaking point" where
    arr[i] > arr[(i+1) % n]. The shift amount equals n - break_index - 1.

    Args:
        arr: list of distinct integers

    Returns:
        rotation amount (>= 0) if arr is a valid cyclic rotation, else -1

    Time:  O(n)
    Space: O(1)  (at most one break index stored)
    """
    n: int = len(arr)
    if n == 0:
        return 0

    break_index: int = -1   # index where arr[i] > arr[(i+1) % n]
    num_breaks:  int = 0

    for i in range(n):
        if arr[i] > arr[(i + 1) % n]:
            num_breaks  += 1
            break_index  = i
            if num_breaks > 1:
                return -1   # early exit: more than one break -> not valid

    if num_breaks == 0:
        return 0                        # already sorted
    return n - break_index - 1          # right-rotation amount

# -- Examples ------------------------------------------------------------
# cyclic_shift([3,4,5,1,2]) -> 2   (break at index 2; rotate right by 2)
# cyclic_shift([1,2,3,4,5]) -> 0   (no breaks; already sorted)
# cyclic_shift([5,1,2,3,4]) -> 4   (break at index 0)
# cyclic_shift([2,1,3,4,5]) -> -1  (breaks at indices 0 and 4)
```

解释

关键洞察：一个合法的循环有序数组（某排序数组的循环移位）在循环比较相邻元素时，

最多只有一个"下降点"（breaking point）。

扫描数组，比较 arr[i] 和 arr[(i+1)%n]，统计下降点位置：

- 0个下降点：数组本身已排序，shift=0

- 1个下降点在索引 k：shift = n - k - 1（右旋了这么多位）

- 2个及以上：不是任何排序数组的循环移位，返回 -1

时间复杂度 O(n)，空间 O(1)（breaks 列表最多存 2 个元素）。

Q126. Algorithms: Memory Allocator Simulation

QUESTION

Implement a simple memory allocator with a fixed-size memory array of n cells,

initialized to 0 (free). Support two operations:

alloc(size): Find the leftmost contiguous free block of length >= size.

Mark those cells with a new unique ID and return the ID.

If no valid block exists, return -1.

erase(id): Free all cells belonging to the given allocation ID.

Return 0 on success, -1 if the ID does not exist.

ANSWER

```
class Allocator:
    """
    Simple memory allocator over a fixed-size integer array.
    Cells are 0 (free) or a positive allocation ID (occupied).

    Operations:
        alloc(size)  -> int   find leftmost contiguous free block, return ID or -1
        erase(id)    -> int   free the block with given ID, return 0 or -1 if not found
    """

    def __init__(self, n: int) -> None:
        """
        Args:
            n: total number of memory cells
        """
        self.mem:     list[int]             = [0] * n   # 0 = free
        self.blocks:  dict[int, tuple[int, int]] = {}   # id -> (start, length)
        self.next_id: int                   = 1

    def alloc(self, size: int) -> int:
        """
        Find the leftmost contiguous free block of length >= size.
        Mark it with a new unique ID and return that ID.
        Returns -1 if no valid block exists.

        Time:  O(n) scan
        """
        n:   int = len(self.mem)
        i:   int = 0

        while i < n:
            if self.mem[i] != 0:
                i += 1
                continue

            # Measure the length of this free segment
            j: int = i
            while j < n and self.mem[j] == 0:
                j += 1

            if j - i >= size:
                # Sufficient space found: assign a new allocation ID
                alloc_id: int = self.next_id
                self.next_id += 1
                for k in range(i, i + size):
                    self.mem[k] = alloc_id
                self.blocks[alloc_id] = (i, size)
                return alloc_id

            i = j   # skip past this (too-short) free segment

        return -1   # no valid block found

    def erase(self, alloc_id: int) -> int:
        """
        Free all cells belonging to alloc_id.
```

```
        Returns 0 on success, -1 if the ID does not exist.

        Time:  O(size) to zero the cells
        """
        if alloc_id not in self.blocks:
            return -1

        start, length = self.blocks.pop(alloc_id)
        for k in range(start, start + length):
            self.mem[k] = 0
        return 0

# -- Example -----------------------------------------------------------------
# a = Allocator(5)          # [0, 0, 0, 0, 0]
# a.alloc(2)  -> 1          # [1, 1, 0, 0, 0]
# a.alloc(1)  -> 2          # [1, 1, 2, 0, 0]
# a.erase(1)  -> 0          # [0, 0, 2, 0, 0]
# a.alloc(3)  -> -1         # no contiguous block of size 3 available
```

解释

内存用整数数组表示，0 为空闲，非零值为分配ID。

alloc：从左到右扫描，找第一个连续空闲段长度 >= size 的位置，

填入新ID并用 hash map 记录 (start, length) 供后续 erase 使用。

跳过已占用格子和过短的空闲段（直接令 i=j），避免重复扫描，复杂度 O(n)。

erase：O(1) 查找 + O(size) 清零。整体模拟复杂度 O(n) per operation。

扩展：可用线段树/区间树将 alloc 优化到 O(log n)。

Q127. Algorithms: Byte Stream Chunk Merging

QUESTION

Chunks of data arrive as byte range intervals [start, end] (both inclusive).

After each new chunk arrives, return the current merged list of received byte ranges.

Overlapping or adjacent intervals should be merged into one.

Example:

Receive [1,3] -> [[1,3]]

Receive [2,5] -> [[1,5]] (overlaps with [1,3])

Receive [10,12]-> [[1,5],[10,12]]

Receive [6,7] -> [[1,7],[10,12]] (adjacent: 5+1=6)

ANSWER

```
def add_chunk(
    received: list[list[int]],
    new_chunk: list[int]
) -> list[list[int]]:
    """
    Add a new byte-range chunk [start, end] to the received list and merge
    all overlapping or adjacent ranges. Adjacent ranges (e.g. [1,3] and [4,6])
    ARE merged because byte 3 and byte 4 are consecutive.

    Args:
```

```
        received:  current list of merged byte ranges [[start, end], ...]
        new_chunk: new range to add [start, end]

    Returns:
        updated merged list of byte ranges

    Time:  O(n log n) -- dominated by sort; O(n) for the merge scan
    Space: O(n)
    """
    received.append(list(new_chunk))
    received.sort(key=lambda x: x[0])

    merged: list[list[int]] = [received[0]]

    for start, end in received[1:]:
        # Merge if overlapping OR adjacent (start <= prev_end + 1)
        if start <= merged[-1][1] + 1:
            merged[-1][1] = max(merged[-1][1], end)
        else:
            merged.append([start, end])

    return merged

# -- Usage example -------------------------------------------------------
# received: list[list[int]] = []
# received = add_chunk(received, [1, 3])    # -> [[1, 3]]
# received = add_chunk(received, [2, 5])    # -> [[1, 5]]   overlaps [1,3]
# received = add_chunk(received, [10, 12])  # -> [[1, 5], [10, 12]]
# received = add_chunk(received, [6, 7])    # -> [[1, 7], [10, 12]]  adjacent

# -- Optimised: bisect insertion to avoid full sort each time ---------------
# import bisect
# def add_chunk_fast(received, new_chunk):
#     lo, hi = new_chunk
#     starts = [r[0] for r in received]
#     pos = bisect.bisect_left(starts, lo)
#     received.insert(pos, [lo, hi])
#     merged = [received[0]]
#     for seg in received[1:]:
#         if seg[0] <= merged[-1][1] + 1:
#             merged[-1][1] = max(merged[-1][1], seg[1])
#         else:
#             merged.append(seg)
#     return merged
```

解释

经典区间合并问题，每次插入新区间后重新合并。

合并条件：current_start <= prev_end + 1（+1 是因为相邻字节也应合并，如[1,3]和[4,5]合并为[1,5]）。

排序后线性扫描合并：维护结果列表 merged，逐个检查新区间是否与最后一个合并区间重叠/相邻，

是则扩展右边界，否则追加新区间。时间复杂度 O(n log n)（排序主导）。

优化：用 bisect 插入保持有序，省去每次全量排序，摊销到 O(n) per insertion。

Q128. Algorithms: Merge Intervals and Return All Unique Numbers

QUESTION

A function receives an interval [lo, hi] on each call. After multiple calls,

return all unique integers covered by any of the intervals (merged, sorted,

no duplicates).

Example: calls (1,3), (5,7), (2,4) -> [1,2,3,4,5,6,7]

ANSWER

```
class IntervalUnique:
    """
    Accumulates intervals across multiple add() calls.
    get_all() returns sorted list of all unique integers covered.

    Uses the sorted+merge approach: maintain a list of non-overlapping
    intervals in sorted order; each add() merges overlapping ranges.
    """

    def __init__(self) -> None:
        self.intervals: list[list[int]] = []

    def add(self, lo: int, hi: int) -> None:
        """
        Insert interval [lo, hi] and merge with any overlapping intervals.

        Args:
            lo: inclusive lower bound
            hi: inclusive upper bound

        Time:  O(n) per call (scan to find overlap range)
        Space: O(n) total intervals stored
        """
        new_iv: list[int] = [lo, hi]
        merged: list[list[int]] = []
        inserted: bool = False

        for iv in self.intervals:
            if iv[1] < lo - 1:          # completely to the left: keep as-is
                merged.append(iv)
            elif iv[0] > hi + 1:        # completely to the right: flush new, then keep
                if not inserted:
                    merged.append(new_iv)
                    inserted = True
                merged.append(iv)
            else:                        # overlapping or adjacent: absorb into new_iv
                new_iv[0] = min(new_iv[0], iv[0])
                new_iv[1] = max(new_iv[1], iv[1])

        if not inserted:
            merged.append(new_iv)

        self.intervals = merged

    def get_all(self) -> list[int]:
        """
        Return all unique integers covered by any stored interval, sorted.

        Time:  O(N) where N = total integers across all intervals
        Space: O(N)
        """
        result: list[int] = []
        for lo, hi in self.intervals:
            result.extend(range(lo, hi + 1))
        return result
```

```
# -- Usage example --------------------------------------------------------
# store = IntervalUnique()
# store.add(1, 3)   # intervals: [[1,3]]
# store.add(5, 7)   # intervals: [[1,3],[5,7]]
# store.add(2, 4)   # intervals: [[1,4],[5,7]]  (merged [1,3] and [2,4])
# store.get_all()   # -> [1,2,3,4,5,6,7]
#
# store.add(4, 5)   # intervals: [[1,7]]         (adjacent: merge all)
# store.get_all()   # -> [1,2,3,4,5,6,7]

# -- Complexity -----------------------------------------------------------
# add():     O(n) per call, O(log n) possible with bisect (see Q61 for detail)
# get_all(): O(N) where N = total integers covered
# Space:     O(k) intervals stored, always k <= number of add() calls
```

解释

这道题是 Q61（Online Interval Merge）的变体，区别在于最终要输出所有具体整数而不是区间列表。

核心仍是维护已合并的区间列表，add() 每次扫描找到所有重叠/相邻的区间并吸收。

相邻合并条件（iv[1] < lo-1 / iv[0] > hi+1）：整数区间 [1,3] 和 [4,6] 是相邻的，

合并后 [1,6] 包含所有整数，不会遗漏4，这是整数区间和实数区间的关键区别。

get_all() 用 range(lo, hi+1) 展开每个区间，直接得到去重排序的整数列表。

面试技巧：先问清楚是整数区间还是实数区间，整数区间 [1,3]+[4,6] 可以合并，实数不行。

Q129. Algorithms: Clock Chain Parameter Search

QUESTION

A clock passes through three modules in series:

Module A: divides by a_div -> clk_a must be in [10, 1000] MHz

Module B: multiplies by b_mul -> clk_b must be in [900, 1600] MHz

Module C: divides by c_div -> clk_out fixed at 1000 MHz

Given clk_in = 100 MHz and clk_out = 1000 MHz, write a program that

outputs a valid (a_div, b_mul, c_div) combination using randomization.

Note: c_div is fully determined by a_div and b_mul -- derive the formula.

ANSWER

```
# Mathematical derivation:
# clk_a   = clk_in  / a_div
# clk_b   = clk_a   * b_mul  = clk_in * b_mul / a_div
# clk_out = clk_b   / c_div  = clk_in * b_mul / (a_div * c_div)
#
# Solving for c_div:
# c_div = clk_in * b_mul / (a_div * clk_out)
#       = 100 * b_mul / (a_div * 1000)
#       = b_mul / (a_div * 10)
#
# Constraints:
#   a_div in {1..10}          so clk_a = 100/a_div in [10, 1000] MHz
#   b_mul in [9*a_div, 16*a_div]  so clk_b in [900, 1600] MHz
#   c_div must be a positive integer  (b_mul divisible by a_div*10)

import random
```

```
CLK_IN:  int = 100    # MHz
CLK_OUT: int = 1000   # MHz
RATIO:   int = CLK_OUT // CLK_IN   # = 10

def find_clk_params(seed: int = None) -> tuple[int, int, int]:
    """
    Randomly sample a_div and b_mul until a valid integer c_div is found.
    Returns (a_div, b_mul, c_div) satisfying all frequency constraints.

    Time: O(1) expected (valid solutions exist for every a_div)
    """
    if seed is not None:
        random.seed(seed)

    while True:
        # Step 1: randomly pick a_div in [1, 10]
        a_div: int = random.randint(1, 10)

        # Step 2: randomly pick b_mul so clk_b in [900, 1600] MHz
        # clk_b = (CLK_IN / a_div) * b_mul in [900, 1600]
        # b_mul in [9*a_div, 16*a_div]
        b_mul: int = random.randint(9 * a_div, 16 * a_div)

        # Step 3: derive c_div -- must be a positive integer
        # c_div = b_mul / (a_div * RATIO)
        if b_mul % (a_div * RATIO) != 0:
            continue   # not integer -- resample

        c_div: int = b_mul // (a_div * RATIO)
        if c_div < 1:
            continue

        # Verify all constraints
        clk_a: float = CLK_IN / a_div
        clk_b: float = clk_a * b_mul
        clk_out: float = clk_b / c_div

        assert 10   <= clk_a   <= 1000
        assert 900  <= clk_b   <= 1600
        assert abs(clk_out - CLK_OUT) < 0.001

        return a_div, b_mul, c_div

# -- Example output ------------------------------------------------------
# a_div, b_mul, c_div = find_clk_params(seed=42)
# -> a_div=4, b_mul=40, c_div=1
#    clk_a=25MHz, clk_b=1000MHz, clk_out=1000MHz
#
# a_div, b_mul, c_div = find_clk_params(seed=99)
# -> a_div=1, b_mul=10, c_div=1
#    clk_a=100MHz, clk_b=1000MHz, clk_out=1000MHz

# -- Why c_div = 1 in all solutions? -------------------------------------
# c_div = b_mul / (a_div * 10)
# The smallest valid b_mul divisible by (a_div*10) is exactly 10*a_div,
# which is always within [9*a_div, 16*a_div]. This gives c_div=1.
# Higher multiples (20*a_div, 30*a_div) would fall outside b_mul range
# (b_mul <= 16*a_div). So c_div is always 1 for this specific problem.

# -- SystemVerilog equivalent (for hardware design context) --------------
# int a_div, b_mul, c_div;
# forever begin
```

```
//     a_div = $urandom_range(1, 10);
//     b_mul = $urandom_range(9 * a_div, 16 * a_div);
//     if ((b_mul % (a_div * 10)) == 0) begin
//         c_div = b_mul / (a_div * 10);
//         break;
//     end
// end
```

解释

核心是推导 c_div 的公式。由流水线关系：

clk_out = clk_in * b_mul / (a_div * c_div)

整理得：c_div = clk_in * b_mul / (a_div * clk_out) = b_mul / (a_div * 10)

所以 c_div 由 a_div 和 b_mul 完全确定，不需要随机。

随机策略：先随机 a_div（决定 clk_a），再随机 b_mul（决定 clk_b），

检查 c_div 是否为正整数，不是就重新采样。

这道题的隐藏结论：在给定的范围约束下，c_div 恒为1，

因为满足整除条件的最小 b_mul = 10*a_div 刚好在 [9*a_div, 16*a_div] 范围内。

面试中能说出"c_div 由其他两个参数决定，不需要独立随机化"是关键分。

Q130. Algorithms: 100 Switches Toggle -- How Many Are On?

QUESTION

There are 100 switches numbered 1 to 100, all initially OFF.

You make 100 visits:

- Visit 1: toggle every switch that is a multiple of 1 (all of them)

- Visit 2: toggle every switch that is a multiple of 2 (2, 4, 6, ...)

- Visit 3: toggle every switch that is a multiple of 3 (3, 6, 9, ...)

- ...

- Visit i: toggle every switch whose number is a multiple of i

- ...

- Visit 100: toggle only switch 100

After all 100 visits, how many switches are ON?

Write a function to solve this for any N.

ANSWER

```
# ---- Brute force O(N log N) ----------------------------------------

def count_on_switches_brute(n: int) -> tuple[int, list[int]]:
    """
    Simulate all N visits and return (count, list of ON switches).

    Time:  O(N log N)  -- harmonic series: N/1 + N/2 + ... + N/N = N*H(N)
    Space: O(N)
    """
    switches: list[bool] = [False] * (n + 1)   # index 1..N
```

```
    for visit in range(1, n + 1):
        # Toggle all multiples of visit
        for switch in range(visit, n + 1, visit):
            switches[switch] = not switches[switch]

    on: list[int] = [i for i in range(1, n + 1) if switches[i]]
    return len(on), on

# ---- Mathematical O(sqrt(N)) insight --------------------------------
#
# Key observation:
#   Switch i is toggled ONCE for each divisor of i.
#   (Visit d toggles switch i iff d divides i.)
#
#   Final state of switch i:
#     ON  iff toggled an ODD number of times
#         iff i has an ODD number of divisors
#
#   When does a number have an ODD number of divisors?
#     Divisors come in PAIRS: d pairs with (i / d).
#     The pair collapses into ONE when d == i / d, i.e. d == sqrt(i).
#     This only happens when i is a PERFECT SQUARE.
#
#   Therefore:
#     Switch i is ON  iff  i is a perfect square.
#     Answer = count of perfect squares in [1, N] = floor(sqrt(N))

import math

def count_on_switches_math(n: int) -> tuple[int, list[int]]:
    """
    Mathematical solution: switches ON == perfect squares in [1, N].

    Time:  O(sqrt(N))
    Space: O(sqrt(N))  for the result list
    """
    squares: list[int] = [k * k for k in range(1, int(math.isqrt(n)) + 1)]
    return len(squares), squares

# ---- Examples -------------------------------------------------------
# count_on_switches_brute(100) -> (10, [1, 4, 9, 16, 25, 36, 49, 64, 81, 100])
# count_on_switches_math(100)  -> (10, [1, 4, 9, 16, 25, 36, 49, 64, 81, 100])
#
# count_on_switches_math(1000) -> (31, [1, 4, 9, ..., 961]) floor(sqrt(1000))=31
# count_on_switches_math(10)   -> (3,  [1, 4, 9])

# ---- Verification example: switch 36 --------------------------------
# Divisors of 36: 1, 2, 3, 4, 6, 9, 12, 18, 36  -> 9 divisors (ODD) -> ON
# Toggle timeline: toggled at visits 1,2,3,4,6,9,12,18,36
# State:  OFF->ON->OFF->ON->OFF->ON->OFF->ON->OFF->ON  -> final: ON OK
#
# Divisors of 12: 1, 2, 3, 4, 6, 12  -> 6 divisors (EVEN) -> OFF
# Toggle timeline: toggled at visits 1,2,3,4,6,12
# State:  OFF->ON->OFF->ON->OFF->ON->OFF  -> final: OFF OK
```

解释

这道题表面像循环模拟题，核心考察数学洞察力。

每个开关 i 被拨动的次数 = i 的因子个数。

因子成对出现（d 配对 i/d），只有完全平方数的因子个数为奇数（sqrt(i) 自己配自己）。

所以最终亮着的灯 = 100 以内的完全平方数 = {1,4,9,16,25,36,49,64,81,100} = 10 个。

面试中能直接说出"完全平方数"是关键，从 O(N log N) 模拟直接跳到 O(sqrt(N)) 数学解法。

Q131. Algorithms: Sliding Window Maximum Sum

QUESTION

Given an integer array and window size k, return the maximum sum achievable by any contiguous subarray of length k.

ANSWER

```
def max_sliding_window_sum(arr: list[int], k: int) -> int:
    """
    Find maximum sum of any contiguous subarray of length k.

    Args:
        arr: input integer array
        k:   window length (1 <= k <= len(arr))

    Returns:
        maximum sum across all windows of length k

    Time:  O(n) -- one pass with sliding window
    Space: O(1)
    """
    if not arr or k <= 0 or k > len(arr):
        return 0

    # Compute sum of the first window
    window_sum: int = sum(arr[:k])
    best: int       = window_sum

    # Slide: add next element, remove leftmost element
    for i in range(k, len(arr)):
        window_sum += arr[i] - arr[i - k]
        best = max(best, window_sum)

    return best

# -- Examples ------------------------------------------------------------
# max_sliding_window_sum([1,3,-1,-3,5,3,6,7], 3) -> 16  (window [6,7] sum... wait)
# [6,7] is only 2 elements; windows of 3: [1,3,-1]->3, [3,-1,-3]->-1,
# [-1,-3,5]->1, [-3,5,3]->5, [5,3,6]->14, [3,6,7]->16  -> answer: 16
# max_sliding_window_sum([2,1,5,1,3,2], 3) -> 9  (window [5,1,3])
# max_sliding_window_sum([-2,-1,-3], 2)    -> -3 (window [-2,-1])
#
# Time complexity:  O(n) -- each element added and removed exactly once
# Space complexity: O(1) -- only two variables (window_sum, best)
#
# Naive approach: O(n*k) -- recompute full window sum each time
# Sliding window is the canonical O(n) optimization
```

解释

滑动窗口的核心：避免重复计算。

朴素做法每次重新求和 O(k)，总共 O(n*k)。

滑动窗口利用相邻窗口只差一个元素：新窗口 = 旧窗口 + 新元素 - 最左元素，O(1) 更新。

面试时主动说出时间复杂度从 O(n*k) 降到 O(n)，并解释"每个元素只进出窗口各一次"是加分项。

Q132. Algorithms: Index of Second Smallest Element

QUESTION

Given an integer array, return the index of the second smallest

distinct value. If no second smallest exists, return -1.

ANSWER

```
def second_smallest_index(arr: list[int]) -> int:
    """
    Return the index of the second smallest DISTINCT value in arr.
    If all elements are equal, or arr has < 2 distinct values, return -1.

    Args:
        arr: non-empty list of integers

    Returns:
        index of the element with the second smallest distinct value,
        or -1 if no such element exists

    Time:  O(n) -- single pass
    Space: O(1)
    """
    first_val:  float = float("inf")
    second_val: float = float("inf")
    first_idx:  int   = -1
    second_idx: int   = -1

    for i, v in enumerate(arr):
        if v < first_val:
            # New minimum: demote current first to second
            second_val, second_idx = first_val, first_idx
            first_val,  first_idx  = v, i
        elif first_val < v < second_val:
            # New second minimum (strictly between first and current second)
            second_val, second_idx = v, i

    return second_idx

# -- Examples ------------------------------------------------------------
# second_smallest_index([3,1,4,1,5,9,2,6])
#   distinct sorted: [1,2,3,4,5,6,9]; second smallest = 2 at index 6 -> 6
# second_smallest_index([5,2,1,3])
#   sorted: [1,2,3,5]; second smallest = 2 at index 1 -> 1
# second_smallest_index([1,1,1])
#   only one distinct value -> -1
# second_smallest_index([2,1])
#   second smallest = 2 at index 0 -> 0
```

解释

单次遍历维护两个变量：first_val（最小值）和 second_val（严格大于最小值的最小值）。

关键：条件 first_val < v < second_val 保证 second 是严格不同的第二小值，

不会把重复的最小值当作第二小值。

面试追问：如果允许重复（second smallest 可以和最小值相同）？

把条件改为 v <= second_val 并处理等于 first_val 的情况。

Q133. Algorithms: Tournament Elimination Bracket

QUESTION

Simulate a single-elimination tournament. Given N teams, print each round's matchups, winners, and the final champion. Assume N is a power of 2.

The winner of each match is randomly selected.

ANSWER

```
import random

def tournament(teams: list[int], seed: int = 42) -> None:
    """
    Simulate single-elimination tournament.
    Each pair of adjacent teams competes; winner advances.
    For simplicity, higher-numbered team wins each match (deterministic demo).

    Args:
        teams: list of team identifiers (length must be power of 2)
        seed:  random seed for reproducible results

    Time:  O(N)  -- N-1 total matches
    Space: O(N)  -- store current round's teams
    """
    random.seed(seed)
    round_num: int       = 1
    current:   list[int] = teams[:]

    print("=" * 40)
    while len(current) > 1:
        pairs:   list[tuple[int, int]] = [
            (current[i], current[i + 1])
            for i in range(0, len(current), 2)
        ]
        winners: list[int] = [random.choice([a, b]) for a, b in pairs]

        print(f"Round {round_num}: " +
              "  ".join(f"({a},{b})" for a, b in pairs))
        print(f"Winners: {', '.join(map(str, winners))}")
        print("=" * 40)

        current    = winners
        round_num += 1

    print(f"Champion: {current[0]}")

# -- Example output (seed=42, teams=[1,5,2,4,3,8,6,7]) -------------------
# ========================================
# Round 1: (1,5)  (2,4)  (3,8)  (6,7)
# Winners: 5, 4, 8, 7
# ========================================
# Round 2: (5,4)  (8,7)
# Winners: 5, 8
# ========================================
# Round 3: (5,8)
# Winners: 8
# ========================================
# Champion: 8

# -- How to VERIFY this function -----------------------------------------
# Correctness attributes to check:
#   1. Number of rounds = ceil(log2(N))  (for N=8: 3 rounds)
```

```
#   2. Total matches = N-1              (one match eliminates one team)
#   3. Each team appears in at most one match per round (no double booking)
#   4. Winner of each match comes from that match's pair (not external)
#   5. Champion is one of the original N teams
#   6. No team can win after being eliminated
#   7. If N is not a power of 2: byes are correctly handled
#   8. Each round has exactly floor(current_teams / 2) matches
```

解释

锦标赛结构本质是一棵完全二叉树，N 支队伍需要 N-1 场比赛和 log2(N) 轮。

实现核心：每轮把当前队伍数组两两配对，产生获胜者列表，递归直到只剩一支队伍。

验证这个函数的属性（面试追问）：

总轮数 = ceil(log2(N))；总比赛数 = N-1；每轮获胜者数量 = 前一轮的一半；

冠军必须是原始队伍之一；每支队伍在被淘汰后不再出现。

Q134. Algorithms: Move Zeros to End (LC 283)

QUESTION

Given an integer array, move all zeros to the end while preserving

the relative order of non-zero elements. Must be done in-place.

(LeetCode 283)

ANSWER

```
def move_zeroes(nums: list[int]) -> None:
    """
    Move all zeros to end of array, preserve non-zero element order.
    In-place: O(1) extra space.

    Args:
        nums: integer array, modified in-place

    Time:  O(n) -- two-pointer single pass
    Space: O(1)
    """
    # write_pos: next position to place a non-zero element
    write_pos: int = 0

    # Pass 1: compact all non-zero elements to the front
    for i in range(len(nums)):
        if nums[i] != 0:
            nums[write_pos] = nums[i]
            write_pos += 1

    # Pass 2: fill the rest with zeros
    for i in range(write_pos, len(nums)):
        nums[i] = 0

# -- Examples ------------------------------------------------------------
# move_zeroes([0,1,0,3,12]) -> [1,3,12,0,0]
# move_zeroes([0,0,1])      -> [1,0,0]
# move_zeroes([1,2,3])      -> [1,2,3]  (no zeros, unchanged)
# move_zeroes([0,0,0])      -> [0,0,0]  (all zeros, unchanged)
#
# Optimisation: avoid unnecessary writes when write_pos == i
```

```
def move_zeroes_optimised(nums: list[int]) -> None:
    """Same logic but skips self-assignment when no zeros have been seen."""
    write_pos: int = 0
    for i in range(len(nums)):
        if nums[i] != 0:
            if i != write_pos:        # avoid self-assignment
                nums[write_pos] = nums[i]
                nums[i] = 0           # immediately place zero at vacated spot
            write_pos += 1
```

解释

双指针：write_pos 指向下一个非零元素该放的位置，i 扫描整个数组。

第一遍把所有非零元素紧凑到前面，第二遍把剩余位置填零。

优化版：在移动非零元素时直接在原位置写0，合并两遍为一遍，

并跳过 i==write_pos 的情况避免自写（当没有遇到零时 i 和 write_pos 同步前进）。

时间复杂度 O(n)，空间 O(1)，是原地操作的标准双指针模板。

Q135. Algorithms: Array Deduplication -- O(n^2) to O(n) Optimization

QUESTION

Analyze the following Python code (equivalent to a Perl snippet).

What does it do? What is the output for initial_array = [1,2,3,4,5,6,3,4,5,6]?

How can it be optimized?

```
initial_array = [1, 2, 3, 4, 5, 6, 3, 4, 5, 6]
final_array   = []

for num_1 in initial_array:
    matched = 0
    for num_2 in range(len(final_array)):
        if num_1 == final_array[num_2]:
            matched = 1
            break
    if not matched:
        final_array.append(num_1)
```

ANSWER

```
# Function: remove duplicates from initial_array while preserving order.
# Output: final_array = [1, 2, 3, 4, 5, 6]
#
# How it works:
#   For each element num_1, scan the already-built final_array.
#   If num_1 is not already in final_array -> append it.
#   Result: first occurrence of each value is kept, duplicates dropped.
#
# Time complexity: O(n^2) -- for each of n elements, scan up to n elements
# Space complexity: O(n)

# Optimized O(n) version using a set as a lookup structure:

def dedup_preserve_order(arr: list[int]) -> list[int]:
    """
    Remove duplicates preserving original order of first occurrences.
```

```
    Args:
        arr: input list potentially containing duplicates

    Returns:
        list with duplicates removed, preserving order of first occurrence

    Time:  O(n)    -- set lookup is O(1) average
    Space: O(n)    -- set stores seen values
    """
    seen:   set[int]  = set()
    result: list[int] = []

    for x in arr:
        if x not in seen:
            seen.add(x)
            result.append(x)

    return result

# Example:
# dedup_preserve_order([1,2,3,4,5,6,3,4,5,6]) -> [1,2,3,4,5,6]
#
# Why O(n) is better:
#   Original: inner loop scans final_array -> O(n) per element -> O(n^2) total
#   Optimized: set lookup is O(1) -> O(n) total
#   For n=1000: original ~500,000 comparisons vs optimized ~1,000
#
# Python one-liner (order-preserving since Python 3.7+):
# result = list(dict.fromkeys(arr))
```

解释

原始代码每次都遍历 final_array 来查重，相当于用线性搜索代替哈希查找。

优化核心：用 set 记录已出现的值，set 的 in 操作是 O(1)（哈希表实现），

把整体复杂度从 O(n^2) 降到 O(n)。

保序是这道题的关键要求（不能 sorted(set(arr))，那会改变顺序）。

Brain Teasers

Q136. Brain Teaser: 20 Switches, Any Change Toggles the Light

QUESTION

You have 20 switches and 1 light bulb.

Design a circuit such that any time ANY switch changes state

(ON -> OFF or OFF -> ON), the light changes state (ON -> OFF or OFF -> ON).

All 20 switches must individually control the light.

ANSWER

```
// Solution: XOR all 20 switches together
//
```

```
//   LIGHT = S0 XOR S1 XOR S2 XOR ... XOR S19
//
// Why XOR works:
//   XOR outputs 1 when an ODD number of inputs are 1.
//   Toggling ANY single switch flips exactly one input bit.
//   Flipping one bit of an XOR chain ALWAYS flips the output.
//   Proof: if current output = X, toggling Si changes Si from b to (1-b),
//   which changes the XOR result from X to (1-X). QED.
//
// Circuit implementation: XOR tree (log2(20) ~ 4-5 levels)
//
//   Level 1 (10 XOR gates, each 2-input):
//     X01 = S0  XOR S1
//     X23 = S2  XOR S3
//     X45 = S4  XOR S5
//     X67 = S6  XOR S7
//     X89 = S8  XOR S9
//     Xab = S10 XOR S11
//     Xcd = S12 XOR S13
//     Xef = S14 XOR S15
//     Xgh = S16 XOR S17
//     Xij = S18 XOR S19
//
//   Level 2 (5 XOR gates):
//     Y0 = X01 XOR X23
//     Y1 = X45 XOR X67
//     Y2 = X89 XOR Xab
//     Y3 = Xcd XOR Xef
//     Y4 = Xgh XOR Xij
//
//   Level 3 (2 XOR gates):
//     Z0 = Y0 XOR Y1
//     Z1 = Y2 XOR Y3
//
//   Level 4 (1 XOR gate):
//     W = Z0 XOR Z1
//
//   Level 5 (final, with leftover Y4):
//     LIGHT = W XOR Y4
//
// Truth table sample (3 switches for simplicity):
//   S0  S1  S2 | LIGHT = S0 XOR S1 XOR S2
//    0   0   0 |   0
//    1   0   0 |   1   <- S0 toggled: light changes
//    1   1   0 |   0   <- S1 toggled: light changes
//    1   1   1 |   1   <- S2 toggled: light changes
//    0   1   1 |   0   <- S0 toggled: light changes
//
// Alternative implementation (XNOR start):
//   If you want light ON by default (all switches OFF -> light ON):
//   LIGHT = NOT(S0 XOR S1 XOR ... XOR S19)
//   Equivalently: replace the final XOR with XNOR.
//
// Why NOT use simple series/parallel wiring?
//   Series (AND): light only ON when ALL 20 are ON. Wrong.
//   Parallel (OR): toggling OFF when others are ON has no effect. Wrong.
//   XOR is the ONLY Boolean function where every input independently
//   controls the output with equal weight.
```

解释

XOR 的核心性质：改变任意一个输入 bit，输出一定翻转。

这是 XOR 与 AND/OR 最根本的区别：AND 和 OR 在某些情况下改变一个输入不改变输出

（AND：其他输入已有0时改变任何1个输入不影响结果；OR 同理）。

XOR 没有这种"掩蔽"效应，每一个输入对输出都有完全独立的控制权。

实现上用 XOR 树结构（二叉树）可以在 ceil(log2(20)) ≈ 5 级门延迟内完成，

比 20 个开关串联的线性链路更快，且易于扩展。

面试中的考察点：能直接说出"XOR"并解释为什么，而不是从电路图推导。

Q137. Brain Teaser: 7L and 9L Buckets -- Measure 8 Liters

QUESTION

You have an unlimited water source, a 7-liter bucket (B7) and a

9-liter bucket (B9). How do you measure exactly 8 liters of water?

(No markings on the buckets.)

ANSWER

```
// Solution: repeatedly fill the 9L bucket and use the 7L to drain off excess.
// Keep track of (B9, B7) at each step.
//
// Step 1:  Fill B9.                B9=9, B7=0
// Step 2:  Pour B9 -> B7 (7L).     B9=2, B7=7
// Step 3:  Empty B7.               B9=2, B7=0
// Step 4:  Pour B9 -> B7 (2L).    B9=0, B7=2
// Step 5:  Fill B9.                B9=9, B7=2
// Step 6:  Pour B9 -> B7 (5L).    B9=4, B7=7
// Step 7:  Empty B7.               B9=4, B7=0
// Step 8:  Pour B9 -> B7 (4L).    B9=0, B7=4
// Step 9:  Fill B9.                B9=9, B7=4
// Step 10: Pour B9 -> B7 (3L).    B9=6, B7=7
// Step 11: Empty B7.               B9=6, B7=0
// Step 12: Pour B9 -> B7 (6L).    B9=0, B7=6
// Step 13: Fill B9.                B9=9, B7=6
// Step 14: Pour B9 -> B7 (1L).    B9=8, B7=7  <- DONE!
//
// B9 now contains exactly 8 liters.
//
// Mathematical insight:
//   Using Bezout's identity: gcd(7, 9) = 1, so any integer combination
//   7a + 9b can produce any multiple of gcd(7,9) = 1.
//   8 = 9*8 - 7*8 = 9 - 7 + 9 - 7 ... (repeated subtraction)
//   More directly: 8 = 9 - 1 = 9 - (7 - 6) = 9 - 7 + 6 ...
//   The algorithm terminates in at most 7+9-1=15 steps.
```

解释

解题策略：用大桶倒满小桶来"减量"，直到大桶剩余目标值。

gcd(7,9) = 1 保证任何整数（1..16）都可以精确测量出来。

如果 gcd(a,b) 不整除目标值，则无解（例如两个偶数桶无法测出奇数升）。

面试中能快速说出"反复用大桶填满小桶再倒空"的思路，然后列出步骤即可。